科学上网完全指南

从零基础到精通的完整技术教程

来源: https://github.com/ihmily/proxy-guide

全书大纲

[书籍] 科学上网完全指南

[统计] 全书概览

总体结构： 7个部分 + 22章 + 4个附录 **预估字数：** 约60-70万字 **文件数量：** 27个

[目标] 内容特点

- **系统全面** - 从零基础到高级应用的完整知识体系
- **实战导向** - 大量配置示例、命令脚本、实战案例
- **深入浅出** - 原理讲解 + 实践操作，循序渐进
- **注重安全** - 专门章节讲解安全防护和隐私保护
- **合规意识** - 强调法律法规和合理使用边界

✨ 全书亮点

- [完成] **22章完整内容** - 覆盖从入门到精通的全部知识点
- [完成] **数十个配置示例** - 真实可用的配置代码
- [完成] **大量的实用脚本** - 自动化运维和管理脚本
- [完成] **详细的图文说明** - 流程图、对比表、示例图
- [完成] **完整的故障排查流程** - 系统化的问题解决方法
- [完成] **真实的使用场景** - 日常、办公、开发、游戏等
- [完成] **明确的法律提示** - 合规使用和风险防范

第一部分：基础知识篇

第1章 什么是科学上网

- 1.1 互联网访问限制的背景
- 1.2 科学上网的定义与原理
- 1.3 合法性与使用边界
- 1.4 为什么需要科学上网

第2章 网络基础知识

- 2.1 IP地址与DNS解析
- 2.2 HTTP/HTTPS协议基础
- 2.3 TCP/UDP传输协议
- 2.4 代理的概念与分类
- 2.5 加密技术基础

第3章 核心技术原理

- 3.1 代理服务器工作原理
- 3.2 VPN技术详解
- 3.3 隧道技术（Tunneling）
- 3.4 流量混淆与伪装
- 3.5 加密传输机制

第二部分：主流协议与工具篇

第4章 常见代理协议详解

- 4.1 Shadowsocks系列（SS/SSR）
- 4.2 V2Ray/Xray协议族

- VMess协议

- VLESS协议 - Trojan协议

- 4.3 Clash内核与规则
- 4.4 WireGuard现代VPN
- 4.5 其他协议对比

第5章 客户端工具使用指南

- 5.1 Windows平台

- Clash for Windows

- V2RayN - Qv2ray

- 5.2 macOS平台

- ClashX

- Surge - V2RayX

- 5.3 Android平台

- Clash for Android

- V2RayNG - SagerNet

- 5.4 iOS平台

- Shadowrocket

- Surge - Quantumult X

- 5.5 Linux平台配置
- 5.6 路由器端配置

第6章 浏览器与插件方案

- 6.1 浏览器代理设置
- 6.2 SwitchyOmega配置详解
- 6.3 浏览器指纹防护
- 6.4 隐私浏览模式

第三部分：高级配置篇

第7章 订阅与节点管理

- 7.1 订阅链接原理
- 7.2 订阅转换服务
- 7.3 节点测速与选择
- 7.4 负载均衡配置
- 7.5 故障转移策略

第8章 分流规则详解

- 8.1 规则分流基础
- 8.2 域名规则编写
- 8.3 IP-CIDR规则
- 8.4 GeoIP数据库使用
- 8.5 自定义规则集
- 8.6 常用规则推荐

第9章 进阶优化技巧

- 9.1 TCP优化（BBR等）
- 9.2 UDP加速配置
- 9.3 WebSocket+TLS伪装
- 9.4 CDN中转优化
- 9.5 多层代理链
- 9.6 国内外分流最佳实践

第四部分：服务器搭建篇

第10章 VPS选购指南

- 10.1 VPS供应商对比
- 10.2 地域选择策略
- 10.3 配置需求分析
- 10.4 付款方式与隐私保护
- 10.5 IP质量检测

第11章 服务器环境配置

- 11.1 Linux基础操作
- 11.2 SSH安全配置
- 11.3 防火墙设置
- 11.4 系统优化调优
- 11.5 时间同步配置

第12章 一键脚本部署

- 12.1 V2Ray/Xray一键脚本
- 12.2 Trojan一键安装
- 12.3 WireGuard快速部署
- 12.4 脚本安全性分析

第13章 手动搭建教程

- 13.1 V2Ray完整部署流程
- 13.2 Trojan-Go配置详解

- 13.3 Shadowsocks-Rust搭建
- 13.4 域名与证书配置
- 13.5 Web服务器伪装

第14章 多用户管理方案

- 14.1 面板管理工具（X-UI等）
- 14.2 流量统计与限制
- 14.3 用户配置管理
- 14.4 续费与到期提醒

第五部分：安全与隐私篇

第15章 网络安全防护

- 15.1 常见攻击方式
- 15.2 端口伪装技术
- 15.3 流量特征混淆
- 15.4 防止DPI深度检测
- 15.5 服务器安全加固

第16章 隐私保护实践

- 16.1 DNS泄漏防护
- 16.2 WebRTC泄漏预防
- 16.3 IPv6泄漏处理
- 16.4 匿名支付方式
- 16.5 数字足迹清理

第17章 审计与日志管理

- 17.1 访问日志配置
- 17.2 流量分析工具
- 17.3 异常检测机制
- 17.4 日志隐私平衡

第六部分：故障排查篇

第18章 常见问题诊断

- 18.1 无法连接的排查步骤
- 18.2 速度慢的优化方法
- 18.3 频繁断线解决方案
- 18.4 特定网站无法访问
- 18.5 客户端错误代码解析

第19章 网络测试工具

- 19.1 Ping与Traceroute
- 19.2 MTR路由测试
- 19.3 SpeedTest速度测试
- 19.4 端口连通性检测
- 19.5 TLS握手测试

第七部分：最佳实践篇

第20章 不同场景应用

- 20.1 日常浏览场景
- 20.2 流媒体解锁（Netflix、Disney+等）
- 20.3 游戏加速优化
- 20.4 远程办公配置
- 20.5 开发者需求

第21章 成本优化策略

- 21.1 机场服务对比
- 21.2 自建vs购买分析
- 21.3 流量节省技巧
- 21.4 共享方案设计

第22章 合规使用建议

- 22.1 法律法规认知

- 22.2 使用场景边界
- 22.3 风险防范意识
- 22.4 责任与义务

附录

附录A：术语表

- 常用技术术语中英对照

附录B：配置模板

- 常用客户端配置示例
- 服务器配置模板

附录C：资源推荐

- 官方文档链接
- 社区论坛资源
- 工具下载地址

附录D：参考文献

- 技术白皮书
- RFC标准文档

[笔记] 使用说明

适合人群

- [新] **初学者** - 想要了解科学上网的基本原理和使用方法
- [办公] **办公人士** - 需要访问国际资源进行工作
- 👨[电脑] **开发者** - 需要访问GitHub、Stack Overflow等技术平台
- [学生] **学生/研究者** - 需要查阅国外学术资料
- [工具] **进阶用户** - 希望自建服务器或深入优化配置

学习路径

快速上手（1-2天）：

- 第1章：了解基本概念
- 第5章：选择并配置客户端工具
- 第8章：配置基础分流规则

深入学习（1-2周）：

- 第一部分：建立理论基础
- 第二部分：掌握各种工具
- 第三部分：优化配置
- 第六部分：学会排查问题

高级进阶（1个月+）：

- 第四部分：自建服务器
- 第五部分：安全加固
- 第七部分：最佳实践

配套资源

本书包含大量实用资源：

- [列表] 配置模板（见附录B）
- [链接] 工具下载地址（见附录C）
- [书籍] 参考文献（见附录D）
- 💬 术语表（见附录A）

[注意] 重要声明

- **本书内容仅供技术学习和研究使用**
- **使用科学上网工具需遵守当地法律法规**
- **用户对自己的行为承担全部法律责任**
- **作者不承担因使用本书内容产生的任何责任**
- **建议优先使用合法合规的网络访问方式**

版权所有 © 2025 | 技术无罪，使用需责 | 遵纪守法，理性使用

前言

[书籍] 科学上网完全指南

> 一本从零基础到精通的科学上网技术完整教程

![GitHub](https://github.com/ihmily/proxy-guide) ![License](https://github.com/ihmily/proxy-guide/blob/main/LICENSE) ![Version](https://github.com/ihmily/proxy-guide) ![Updated](https://github.com/ihmily/proxy-guide)

[阅读] 关于本书

《科学上网完全指南》是一本系统全面的技术教程，涵盖从基础理论到实战部署的完整知识体系。本书已更新至2025年最新版本，包含Reality、Hysteria 2等最新代理协议，以及现代化客户端工具的使用指南。旨在帮助读者深入理解网络代理技术原理，掌握各种工具的使用方法，并在合法合规的前提下实现高效、安全的网络访问。

✨ 核心特点

- **[目标] 系统全面** - 从零基础到高级应用的完整知识体系
- **[电脑] 实战导向** - 大量配置示例、命令脚本、实战案例
- **[笔记] 深入浅出** - 原理讲解 + 实践操作，循序渐进
- **[加密] 注重安全** - 专门章节讲解安全防护和隐私保护
- ⚖️ **合规意识** - 强调法律法规和合理使用边界

[统计] 内容规模

- **总文件数**：27个
- **正文章节**：22章
- **附录资料**：4个
- **总字数**：约 60-70万字

[新] 2025年重要更新

新增协议与技术

- [亮点] **Reality协议** - 无需域名和证书的TLS伪装技术
- [新] **Hysteria 2** - 基于QUIC的高性能代理协议
- [安全] **Firefox 145反指纹技术** - 浏览器隐私保护最新进展
- [网络] **DNS over QUIC (DoQ)** - 新一代加密DNS标准
- [移动] **Clash Verge Rev** - 替代已停止维护的Clash for Windows
- 🎛️ **3X-UI面板** - 支持Reality协议的管理面板

更新的客户端工具

- **Windows**: Clash Verge Rev（替代CfW）
- **macOS**: Clash Verge Rev（跨平台统一）
- **Android**: Clash Meta、NekoBox（新推荐）
- **Linux**: Clash Meta（功能增强）

隐私保护新内容

- 浏览器指纹防护（Firefox 145、Brave、Tor）
- WebRTC泄漏防护（Manifest V3兼容扩展）
- 加密DNS最佳实践（DoH/DoT/DoQ）
- 美国联邦政府加密DNS强制要求

已废弃工具标注

- [注意] Clash for Windows（2023.11停止维护）
- [注意] Clash for Android原版（2023归档）
- [注意] X-UI原版（建议迁移3X-UI）
- [注意] Qv2ray、V2RayX、SagerNet（已停止维护）

📑 目录结构

```
科学上网指南/
```

```
├── 00-全书大纲.md
│
├── 第一部分-基础知识篇\
│   ├── 第01章-什么是科学上网.md
│   ├── 第02章-网络基础知识.md
│   └── 第03章-核心技术原理.md
│
├── 第二部分-主流协议与工具篇\
│   ├── 第04章-常见代理协议详解.md
│   ├── 第05章-客户端工具使用指南.md
│   └── 第06章-浏览器与插件方案.md
│
├── 第三部分-高级配置篇\
│   ├── 第07章-订阅与节点管理.md
│   ├── 第08章-分流规则详解.md
│   └── 第09章-进阶优化技巧.md
│
├── 第四部分-服务器搭建篇\
│   ├── 第10章-VPS选购指南.md
│   ├── 第11章-服务器环境配置.md
│   ├── 第12章-一键脚本部署.md
│   ├── 第13章-手动搭建教程.md
│   └── 第14章-多用户管理方案.md
│
├── 第五部分-安全与隐私篇\
│   ├── 第15章-网络安全防护.md
│   ├── 第16章-隐私保护实践.md
│   └── 第17章-审计与日志管理.md
│
├── 第六部分-故障排查篇\
│   ├── 第18章-常见问题诊断.md
│   └── 第19章-网络测试工具.md
│
├── 第七部分-最佳实践篇\
│   ├── 第20章-不同场景应用.md
│   ├── 第21章-成本优化策略.md
│   └── 第22章-合规使用建议.md
│
└── 附录\
    ├── 附录A-术语表.md
    ├── 附录B-配置模板.md
    ├── 附录C-资源推荐.md
    └── 附录D-参考文献.md
```

[目标] 适合人群

- [新] **初学者** - 想要了解科学上网的基本原理和使用方法
- [办公] **办公人士** - 需要访问国际资源进行工作
- 👨[电脑] **开发者** - 需要访问GitHub、Stack Overflow等技术平台
- [学生] **学生/研究者** - 需要查阅国外学术资料
- [工具] **进阶用户** - 希望自建服务器或深入优化配置

[新] 快速开始

新手入门路径（1-2天）

- **第01章** - 了解什么是科学上网
- **第05章** - 选择并配置客户端工具
- **第08章** - 配置基础分流规则

深入学习路径（1-2周）

- **第一部分** - 建立理论基础
- **第二部分** - 掌握各种工具
- **第三部分** - 优化配置
- **第六部分** - 学会排查问题

高级进阶路径（1个月+）

- **第四部分** - 自建服务器
- **第五部分** - 安全加固
- **第七部分** - 最佳实践

[提示] 全书亮点

- [完成] **22章完整内容** - 覆盖从入门到精通的全部知识点
- [完成] **数十个配置示例** - 真实可用的配置代码
- [完成] **大量的实用脚本** - 自动化运维和管理脚本
- [完成] **详细的图文说明** - 流程图、对比表、示例图

- [完成] **完整的故障排查流程** - 系统化的问题解决方法
- [完成] **真实的使用场景** - 日常、办公、开发、游戏等
- [完成] **明确的法律提示** - 合规使用和风险防范
- [完成] **隐私保护深度指南** - 浏览器指纹、DNS泄漏等最新防护

[书籍] 章节概览

第一部分：基础知识篇

介绍科学上网的基本概念、网络基础知识和核心技术原理，为后续学习打下坚实基础。

第二部分：主流协议与工具篇

详细讲解 Shadowsocks、V2Ray、Xray、Reality、Hysteria 2、Trojan、Clash 等主流代理协议，以及各平台客户端工具的使用方法。重点介绍Reality协议和Hysteria 2高性能方案。

第三部分：高级配置篇

深入探讨订阅管理、分流规则、性能优化等高级话题，提升使用效率。

第四部分：服务器搭建篇

从 VPS 选购到服务器部署，手把手教你搭建自己的代理服务器。包含Reality协议一键部署、3X-UI面板管理、Hysteria 2配置等最新内容。

第五部分：安全与隐私篇

重点关注网络安全防护、隐私保护、日志管理等安全话题。新增浏览器指纹防护、WebRTC泄漏防护、Firefox 145反追踪技术等最新隐私保护内容。

第六部分：故障排查篇

系统化的故障诊断方法和网络测试工具使用指南。

第七部分：最佳实践篇

针对不同应用场景的最佳实践方案，以及成本优化和合规使用建议。

[工具] 配套资源

本书包含大量实用资源：

- [列表] **配置模板**（附录B）- 即拿即用的配置文件
- [链接] **工具下载**（附录C）- 全平台客户端下载地址
- [书籍] **参考文献**（附录D）- RFC标准、技术文档、开源项目
- 💬 **术语表**（附录A）- 技术术语中英对照

[注意] 重要声明

- **本书内容仅供技术学习和研究使用**
- **使用科学上网工具需遵守当地法律法规**
- **用户对自己的行为承担全部法律责任**
- **作者不承担因使用本书内容产生的任何责任**
- **建议优先使用合法合规的网络访问方式**
- **本书中出现的所有超链接、工具下载地址和外部资源仅在编写时有效，作者不保证其长期可访问性，请读者自行验证**

合规使用原则

- [完成] 用于学习和研究
- [完成] 访问学术资源
- [完成] 工作需要
- [完成] 正常娱乐
- [禁止] 违法犯罪活动
- [禁止] 传播违禁内容
- [禁止] 侵犯他人权益
- [禁止] 未经许可提供服务

[阅读] 如何使用本书

阅读建议

- **按顺序阅读** - 前面的章节是后续内容的基础

- **动手实践** - 每章的实践任务务必亲自操作
- **查阅附录** - 遇到不懂的术语随时查阅附录A
- **使用模板** - 直接使用附录B的配置模板快速上手
- **持续学习** - 技术在不断更新，保持学习态度

实践建议

- [电脑] 准备一台测试设备（虚拟机或旧电脑）
- [笔记] 做好笔记和备份
- 🔍 遇到问题先查看故障排查篇
- [网络] 善用搜索引擎和社区资源
- ⚖️ 时刻保持法律意识

🤝 贡献与反馈

发现错误?

如果您在阅读过程中发现任何错误或不当之处，欢迎：

- 📧 发送邮件反馈
- 💬 在社区讨论
- 🐛 提交问题报告

内容建议

如果您对内容有改进建议或希望看到某些主题的深入讲解，欢迎提出您的想法。

[笔记] 更新日志

Version 2025.1

重大更新：

- [完成] 新增Reality协议完整教程
- [完成] 新增Hysteria 2协议配置指南
- [完成] 新增Firefox 145反指纹技术
- [完成] 更新客户端工具推荐（Clash Verge Rev等）

- [完成] 新增浏览器指纹防护专题
- [完成] 新增WebRTC泄漏防护（Manifest V3兼容）
- [完成] 更新DNS隐私保护（DoH/DoT/DoQ）
- [完成] 标注已废弃工具并推荐替代品
- [完成] 新增3X-UI面板管理教程
- [完成] 更新VPS服务商推荐
- [完成] 更新所有配置模板至最新标准
- [完成] 补充最新安全标准和最佳实践

文档优化：

- 优化专业术语表达，提升可读性
- 更新所有外部链接
- 修正过时技术信息

Version 1.0 (2024年)

- [完成] 完成全部22章正文
- [完成] 完成4个附录
- [完成] 总字数达到55-65万字
- [完成] 包含数百个配置示例和脚本

📄 许可证

本书采用 **CC BY-NC-SA 4.0** 许可协议。

您可以自由地：

- [完成] **分享** - 以任何媒介或格式复制、发行本作品
- [完成] **演绎** - 修改、转换或以本作品为基础进行创作

惟须遵守下列条件：

- [笔记] **署名** - 您必须给出适当的署名，提供指向本许可协议的链接
- [费用] **非商业性使用** - 您不得将本作品用于商业目的
- 🔄 **相同方式共享** - 如果您再混合、转换或者基于本作品进行创作，您必须基于与原先许可协议相同的许可协议分发您贡献的作品

版权所有 © 2024-2025

[亮点] 致谢

感谢所有为开源社区做出贡献的开发者，感谢各协议和工具的维护者，感谢所有技术博主和教程作者。

特别感谢：

- V2Ray / Xray 项目团队
- RPRX (Reality协议作者)
- Hysteria 项目团队
- Clash Meta (mihomo) 开发者
- Clash Verge Rev 维护者
- Shadowsocks 社区
- 所有开源贡献者

[推荐] 如果本书对您有帮助，欢迎 Star 支持！

最后更新：2025年1月 技术无罪，使用需责 | 遵纪守法，理性使用

第一章 什么是科学上网

第1章 什么是科学上网

1.1 互联网访问限制的背景

1.1.1 全球互联网的分割现状

互联网最初的设计理念是开放、自由、互联的全球网络。然而在实际发展过程中，由于政治、文化、安全等多方面因素，互联网逐渐呈现出区域化、分割化的特征。

主要表现形式：

- **防火长城（GFW - Great Firewall）**

- 中国大陆实施的网络审查系统

- 通过DNS污染、IP封锁、深度包检测等技术手段 - 限制访问特定国外网站和服务

- **地理版权限制（Geo-blocking）**

- 流媒体平台基于地理位置限制内容访问

- Netflix、HBO、BBC iPlayer等实施区域内容库 - 游戏服务器的地区锁定

- **企业网络限制**

- 公司内网对外网访问的管控

- 教育机构的网络审查 - 公共WiFi的访问限制

- **其他国家的网络审查**

- 俄罗斯的主权互联网法案

- 伊朗、朝鲜等国的严格网络管控 - 土耳其对社交媒体的阶段性封锁

1.1.2 技术实现手段

网络限制主要通过以下技术手段实现：

DNS劫持与污染

```
用户请求：访问 google.com
```

```
正常流程：DNS服务器返回真实IP（如46.82.174.69）
被污染后：DNS返回错误IP或不可达IP
结果：用户无法访问目标网站
```

IP地址封锁

- 在国际出口处直接封禁特定IP段
- 对已知服务器IP进行黑名单拦截
- 通过路由器过滤丢弃相关数据包

深度包检测（DPI - Deep Packet Inspection）

- 检查数据包的应用层内容
- 识别敏感关键词或特定协议特征
- 主动探测和干扰加密连接

TCP连接重置

- 检测到敏感连接时发送伪造的RST包
- 强制中断双方的TCP连接
- 造成连接异常断开

1.1.3 对普通用户的影响

这些限制给普通用户带来诸多不便：

- **学术研究受阻**：无法访问Google Scholar、arXiv等学术资源
- **技术学习困难**：GitHub、Stack Overflow等开发平台访问受限
- **信息获取单一**：无法浏览多元化的新闻和观点
- **商业活动受限**：跨境电商、外贸从业者沟通不畅
- **娱乐内容缺失**：无法使用YouTube、Instagram等平台
- **软件更新受阻**：许多国际软件无法正常更新

1.2 科学上网的定义与原理

1.2.1 什么是科学上网

科学上网（也称为"翻墙"、"突破网络封锁"）是指通过技术手段绕过网络审查和访问限制，访问被屏蔽的互联网资源的行为。

核心思想：

```
正常访问（被拦截）：

用户 → 防火墙 ✗ → 目标网站



科学上网（绕过限制）：
用户 → 加密隧道 → 境外代理服务器 → 目标网站
        ↑___________↑
        防火墙无法识别和拦截
```

1.2.2 基本工作原理

科学上网的本质是 **流量中转与加密伪装**：

1. 流量中转

- 在境外架设一台代理服务器
- 用户的访问请求先发送到代理服务器
- 代理服务器代替用户访问目标网站
- 将结果返回给用户

2. 加密伪装

- 用户与代理服务器之间的通信进行加密
- 防火墙无法识别加密流量的真实内容
- 伪装成正常的HTTPS、WebSocket等流量
- 避免被特征识别和主动探测

简化流程图：

```
┌──────────┐      加密隧道      ┌──────────────┐      明文请求      ┌──────────┐

│ 用户 │ ──────────────────────► │ 代理服务器 │ ──────────────────► │ 目标网站 │
└──────────┘                    └──────────────┘                    └──────────┘
  ▲                                  │                                  │
  │          加密返回                │          明文响应                │
  └──────────────────────────────────┘◄─────────────────────────────────┘
```

1.2.3 与VPN的区别

很多人将科学上网等同于VPN，实际上两者有明显区别：

对比维度	传统VPN	现代代理工具
设计目的	企业远程办公、加密通信	突破网络封锁
流量处理	全局流量都走VPN	可以分流（部分流量走代理）
加密方式	IPsec、L2TP等标准协议	自定义加密，特征更少
抗封锁能力	较弱，特征明显易被识别	强，持续对抗升级
速度影响	全局代理，速度损失大	智能分流，国内直连速度快
典型代表	OpenVPN、PPTP	Shadowsocks、V2Ray、Trojan

1.2.4 技术演进历程

科学上网技术经历了多代演进：

第一代：传统VPN（2000-2010）

- OpenVPN、PPTP、L2TP
- 协议特征明显，易被识别封锁
- 2012年后逐渐失效

第二代：代理工具（2012-2015）

- Shadowsocks（2012年诞生）
- GoAgent、SSH隧道
- 引入流量加密和混淆概念

第三代：高级协议（2015-2020）

- V2Ray（2015年）
- Trojan（2018年）
- 增加流量伪装、多协议支持

第四代:现代化方案(2020-2024)

- Xray、Clash等新工具
- VLESS、Reality等新协议
- WireGuard等现代VPN协议
- Shadowsocks 2022新版本(固定密钥+AEAD)

- 多层流量伪装技术

第五代:智能化时代(2024至今)

- Hysteria2、TUIC等基于QUIC的高性能协议
- Sing-box等新一代通用代理平台
- vless-vision、Reality-XTLS等极致抗审查协议
- AI智能流量分析与规则优化
- ECH(Encrypted Client Hello)技术普及
- 多路径传输(MPTCP)应用
- 去中心化代理网络探索

1.3 合法性与使用边界

1.3.1 法律法规现状

关于科学上网的合法性问题，需要理性认识：

中国大陆的相关规定：

- **《计算机信息网络国际联网管理暂行规定》**

- 规定必须通过国际出入口信道进行国际联网

- 未经批准不得自行建立或使用其他信道

- **实际执法情况**

- 主要针对提供翻墙服务的商业行为 - 出售VPN服务、搭建翻墙平台会被追责 - 个人使用相对较少被追究 - 但法律风险依然存在

- **近年案例**

- 2017年:广西吴某因销售VPN被判刑 - 2019年:江苏朱某提供翻墙服务获刑 - 2020-2024年:多起经营翻墙服务案件 - 2024年:部分地区加强对商业化VPN服务的打击 - 个人自用仍较少被追究,但法律风险持续存在

国际视角:

- 美国、欧洲等国家:使用代理工具完全合法
- 俄罗斯:自2017年起限制VPN服务,2024年进一步加强封锁
- 伊朗:严格管控VPN使用,2024年加强技术封锁
- 土耳其:间歇性封锁社交媒体和VPN服务
- 朝鲜:完全禁止普通民众访问国际互联网

1.3.2 使用边界与风险

相对安全的使用场景：

- [完成] 访问学术资源（Google Scholar、PubMed）
- [完成] 技术学习（GitHub、Stack Overflow）
- [完成] 外贸工作需求（联系海外客户）
- [完成] 正版软件下载与更新
- [完成] 跨境旅游时访问国内服务

高风险行为：

- [禁止] 提供商业化翻墙服务（出售VPN账号）
- [禁止] 架设公开代理服务器供他人使用
- [禁止] 传播违法违规内容
- [禁止] 进行网络攻击、诈骗等犯罪活动
- [禁止] 在社交媒体上公开宣传翻墙方法

1.3.3 理性使用建议

- **仅用于合法目的**

- 学习、工作、正当娱乐

- 不传播违法内容 - 不参与违法活动

- **低调使用**

- 不在社交平台公开讨论

- 不向他人大量推广 - 不建立商业化服务

- **注意隐私保护**

- 不在翻墙时登录敏感账号

- 使用可信的代理服务 - 定期更换节点和密码

- **技术防护**

- 启用加密DNS

- 使用TLS加密连接 - 避免使用免费公共代理

1.4 为什么需要科学上网

1.4.1 学术与技术需求

学术研究人员：

- 访问国际学术数据库（Web of Science、Scopus）
- 查阅最新论文（arXiv、SSRN）
- 使用协作工具（Google Docs、Overleaf）
- 参加国际学术会议（Zoom、YouTube直播）

技术开发者:

```
场景示例:
  - 访问GitHub:查看开源项目、提交代码、使用Copilot
  - Stack Overflow:解决技术问题
  - Google搜索:获取更准确的技术文档
  - Docker Hub/GHCR:拉取容器镜像
  - npm/PyPI/Maven:下载开发依赖包
  - 阅读官方文档:React、Vue、TensorFlow等
  - ChatGPT/Claude:AI辅助开发
  - Hugging Face:访问AI模型和数据集
  - Vercel/Netlify:部署和托管应用
```

1.4.2 商业与工作场景

跨境电商从业者:

- 管理Facebook、Instagram、TikTok广告投放

- 回复Amazon、eBay、Shopify客户咨询
- 使用Google Analytics 4分析数据
- 通过WhatsApp、Telegram、Discord联系客户
- 管理TikTok Shop、Temu等新兴平台
- 使用AI工具(如ChatGPT)辅助客服和营销

外贸业务人员：

- 收发Gmail邮件
- 使用LinkedIn拓展客户
- 参加线上展会（Zoom会议）
- 查询国际市场行情

设计与创意工作者：

- 访问Dribbble、Behance寻找灵感
- 使用Figma、Canva在线设计
- 下载Adobe Creative Cloud资源
- 观看设计教程（YouTube、Skillshare）

1.4.3 信息获取与视野拓展

多元化信息源：

- 对比不同视角的新闻报道
- 了解国际事件的多方观点
- 访问维基百科完整内容
- 关注国际智库研究报告

知识学习：

- Coursera、edX等在线课程平台
- Khan Academy免费教育资源
- TED演讲视频
- 各类专业技术博客

1.4.4 娱乐与文化交流

流媒体内容:

- Netflix、Disney+、HBO Max、Apple TV+
- YouTube原创内容及YouTube Premium

- Spotify、Apple Music、YouTube Music
- Twitch、Kick游戏直播平台
- OpenAI、Anthropic等AI服务

社交与沟通:

- X(原Twitter)了解实时动态
- Instagram、Threads分享生活
- Reddit参与兴趣社区
- Discord社群与语音聊天
- Bluesky等新兴去中心化社交平台
- Mastodon联邦宇宙社交网络

游戏加速：

- 访问Steam、Epic Games
- 连接国际游戏服务器
- 下载游戏更新补丁
- 参与国际电竞赛事

1.4.5 隐私保护需求

在某些场景下，科学上网也是保护隐私的手段：

- **公共WiFi安全**：通过加密隧道保护数据
- **避免ISP追踪**：防止运营商记录访问历史
- **绕过广告追踪**：减少精准广告推送
- **匿名浏览**：保护个人浏览隐私

本章小结

科学上网是在特定网络环境下，为了访问被限制资源而采用的技术手段。它基于代理服务器和加密通信实现流量中转，经历了多代技术演进。

关键要点：

- 互联网访问限制是客观存在的现实问题
- 科学上网通过加密隧道+流量中转实现突破
- 需要理性认识法律风险，仅用于合法目的
- 学术、技术、商业等领域有广泛的实际需求

在接下来的章节中，我们将深入学习实现科学上网所需的网络基础知识和技术原理。

延伸阅读:

- 互联网自由报告2025
- GFW技术分析报告
- OONI网络审查测量项目
- 《网络审查的历史与技术》相关论文
- Censored Planet项目

思考题:

- 除了本章提到的场景，你还能想到哪些科学上网的合理用途?
- 如何在技术需求和合规使用之间找到平衡?
- 未来网络审查与反审查技术会如何发展?

第二章 网络基础知识

第2章 网络基础知识

在深入学习科学上网技术之前，我们需要掌握一些必要的网络基础知识。这些知识将帮助你理解代理工作原理、排查连接问题，以及优化网络性能。

2.1 IP地址与DNS解析

2.1.1 IP地址基础

什么是IP地址?

IP地址（Internet Protocol Address）是互联网中每个设备的唯一标识，就像现实中的门牌号码。

IPv4地址

```
格式：xxx.xxx.xxx.xxx


示例：192.168.1.1
范围：0.0.0.0 - 255.255.255.255
总数：约43亿个（2^32）
```

IPv4地址分类：

类别	IP范围	用途
A类	1.0.0.0 - 126.255.255.255	大型网络
B类	128.0.0.0 - 191.255.255.255	中型网络
C类	192.0.0.0 - 223.255.255.255	小型网络
私有IP	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	内网使用
回环地址	127.0.0.1	本机测试

IPv6地址

```
格式：xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx


示例：2001:0db8:85a3:0000:0000:8a2e:0370:7334
简写：2001:db8:85a3::8a2e:370:7334
总数：约2^128个
```

公网IP vs 内网IP

```
家庭网络拓扑：


                 公网IP：222.79.44.112
互联网 ◄──────────────────► 路由器（NAT）
                     │
         ┌───────────┴────────────┬──────────────┐
         │             │             │
    电脑A          电脑B           手机C
192.168.1.100  192.168.1.101  192.168.1.102
  （内网IP）      （内网IP）     （内网IP）
```

2.1.2 DNS域名解析系统

DNS的作用

DNS（Domain Name System）将人类易记的域名转换为计算机使用的IP地址。

```
用户输入：www.google.com


↓
DNS查询
↓
返回IP：142.250.77.14
↓
浏览器连接该IP地址
```

DNS解析过程详解：

```
完整DNS查询流程：

  • 用户输入 www.example.com
```

```
  ↓
• 浏览器检查本地DNS缓存

  ↓（未命中）
• 查询操作系统hosts文件

  ↓（未找到）
• 向本地DNS服务器查询（通常是ISP提供）

  ↓
• 本地DNS向根域名服务器查询 "."

  ↓
• 根服务器返回 .com 顶级域名服务器地址

  ↓
• 查询 .com 服务器，获取 example.com 的权威DNS

  ↓
• 查询权威DNS，获取 www.example.com 的IP

  ↓
• 返回结果并缓存（TTL时间内有效）
```

DNS记录类型：

记录类型	说明	示例
A记录	域名指向IPv4地址	example.com → 222.79.44.112
AAAA记录	域名指向IPv6地址	example.com → 2606:2800:220:1:248:...
CNAME	域名别名	www.example.com → example.com
MX记录	邮件服务器	mail.example.com → 优先级10
TXT记录	文本信息	用于域名验证、SPF记录等
NS记录	域名服务器	指定权威DNS服务器

DNS污染问题

在某些网络环境下，DNS查询会被劫持返回错误IP：

```
正常情况:

查询 x.com → 返回 162.159.140.229



DNS污染:
查询 x.com → 被劫持返回 0.0.0.0 或错误IP
                → 用户无法访问
```

解决DNS污染的方法：

- 使用国外公共DNS（8.8.8.8 - Google DNS）
- 使用加密DNS（DoH、DoT）
- 通过代理服务器进行DNS查询
- 修改hosts文件直接指定IP

2.1.3 常用公共DNS服务

DNS提供商	首选DNS	备用DNS	平均响应时间	特点
Cloudflare	1.1.1.1	1.0.0.1	~10ms	全球最快,隐私优先,24小时删除日志
Google	8.8.8.8	8.8.4.4	~20ms	稳定可靠,支持DoH/DoT
Quad9	9.9.9.9	149.112.112.112	~15ms	安全防护,自动拦截恶意域名
OpenDNS	208.67.222.222	208.67.220.220	~18ms	内容过滤,家长控制
阿里DNS	223.5.5.5	223.6.6.6	~8ms(国内)	国内访问最快
腾讯DNS	119.29.29.29	182.254.116.116	~10ms(国内)	国内优化,支持HttpDNS
NextDNS	45.90.28.0	45.90.30.0	~12ms	可定制规则,广告拦截

加密DNS协议:

- **DoH(DNS over HTTPS)**:通过HTTPS加密DNS查询,端口443,主流浏览器已原生支持
- **DoT(DNS over TLS)**:通过TLS加密DNS查询,端口853,Android 9+原生支持
- **DoQ(DNS over QUIC)**:基于QUIC的新一代加密DNS,RFC 9250标准(2022年发布)
- **DoH3(DNS over HTTP/3)**:基于HTTP/3的DNS查询,性能更优
- **DNSCrypt**:独立加密DNS协议,需专用客户端

2.2 HTTP/HTTPS协议基础

2.2.1 HTTP协议详解

HTTP（HyperText Transfer Protocol）超文本传输协议

HTTP是Web的基础协议，定义了客户端和服务器之间的通信规则。

HTTP请求结构：

```
GET /index.html HTTP/1.1


Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Accept-Language: zh-CN,zh;q=0.9
Connection: keep-alive



[请求体（POST请求时包含）]
```

HTTP响应结构:

```
HTTP/1.1 200 OK


Date: Mon, 16 Dec 2025 10:00:00 GMT
Server: nginx/1.25.3
Content-Type: text/html; charset=UTF-8
Content-Length: 1024
Connection: keep-alive
```

```
<!DOCTYPE html>
<html>
<head><title>示例页面</title></head>
...
</html>
```

常见HTTP方法：

方法	说明	特点
GET	获取资源	无请求体、可缓存
POST	提交数据	有请求体、不可缓存
PUT	更新资源	完整替换
DELETE	删除资源	删除指定资源
HEAD	获取头信息	仅返回响应头
OPTIONS	查询支持的方法	CORS预检请求

HTTP状态码：

状态码	含义	示例
1xx	信息性	100 Continue
2xx	成功	200 OK, 201 Created
3xx	重定向	301 永久重定向, 302 临时重定向
4xx	客户端错误	404 Not Found, 403 Forbidden
5xx	服务器错误	500 Internal Server Error, 502 Bad Gateway

2.2.2 HTTPS加密通信

HTTPS = HTTP + TLS

HTTPS通过TLS协议(SSL已废弃)对HTTP通信进行加密,保护数据安全。

TLS版本演进:

- TLS 1.0/1.1:已废弃(2021年起)

- TLS 1.2:当前主流,广泛支持
- TLS 1.3:最新标准(2018年发布),性能更优

加密过程示意:

```
客户端                                服务器


  │                                   │
  │──────── ①ClientHello ──────────────►│
  │   (支持的加密算法列表)                │
  │                                   │
  │◄──────── ②ServerHello ─────────────│
  │   (选择的加密算法+服务器证书)          │
  │                                   │
  │──────────── ③验证证书 ──────────────────────────►│
  │   (检查证书有效性)                    │
  │                                   │
  │◄──────────── ④密钥交换 ──────────────────────────│
  │   (协商加密密钥)                      │
  │                                   │
  │──────────── ⑤加密通信 ──────────────────────────►│
  │   (使用对称加密传输数据)              │
  └──────────────────────────────────────────────────────────┘
```

HTTPS的优势:

- [完成] **加密传输**:数据无法被中间人窃取
- [完成] **身份验证**:确认服务器身份真实性
- [完成] **数据完整性**:防止数据被篡改
- [完成] **SEO优化**:搜索引擎优先排名
- [完成] **浏览器信任**:显示安全锁图标

TLS握手过程:

- 客户端发送支持的TLS版本和加密套件
- 服务器选择加密套件并发送证书
- 客户端验证证书（CA签名、有效期、域名）
- 使用非对称加密协商会话密钥
- 切换到对称加密进行后续通信

2.2.3 证书与CA

数字证书的作用

证书用于证明服务器身份的真实性，包含：

- 域名信息
- 公钥
- 有效期
- CA签名

证书链验证：

```
根证书（Root CA）


   ↓ 签名
中间证书（Intermediate CA）
   ↓ 签名
网站证书（example.com）
```

常见CA机构：

- Let's Encrypt（免费、自动化）
- DigiCert
- GlobalSign
- Comodo

证书类型：

- **DV证书**（域名验证）：仅验证域名所有权
- **OV证书**（组织验证）：验证组织身份
- **EV证书**（扩展验证）：最高级别验证，浏览器显示绿色地址栏

2.3 TCP/UDP传输协议

2.3.1 TCP协议详解

TCP（Transmission Control Protocol）传输控制协议

TCP是面向连接的可靠传输协议，保证数据按序到达。

TCP三次握手建立连接：

```
客户端                    服务器
```

```
  │                      │
  │──── ①SYN ──────────►│
  │  (Seq=X)              │
  │                      │
  │◄── ②SYN+ACK ────────│
  │  (Seq=Y, Ack=X+1)     │
  │                      │
  │──── ③ACK ──────────►│
  │  (Ack=Y+1)            │
  │                      │
  │═══ 连接建立，开始传输 ═══│
```

为什么是三次握手？

- 第一次：客户端发送能力正常
- 第二次：服务器收发能力正常
- 第三次：客户端接收能力正常，防止旧连接请求

TCP四次挥手断开连接：

```
客户端                    服务器


  │                      │
  │──── ①FIN ──────────►│
  │  (我要关闭了)         │
  │                      │
  │◄── ②ACK ────────────│
  │  (知道了，等我发完数据)  │
  │                      │
  │◄── ③FIN ────────────│
  │  (我也发完了)         │
  │                      │
  │──── ④ACK ──────────►│
  │  (好的，再见)         │
  └──────────────────────────────────────┘
```

TCP特性：

- [完成] 面向连接（建立连接后才能通信）
- [完成] 可靠传输（丢包重传、顺序保证）
- [完成] 流量控制（滑动窗口机制）
- [完成] 拥塞控制（慢启动、拥塞避免）

- [禁止] 开销较大（三次握手、ACK确认）

TCP常见端口：

- 80：HTTP
- 443：HTTPS
- 22：SSH
- 21：FTP
- 3306：MySQL

2.3.2 UDP协议详解

UDP（User Datagram Protocol）用户数据报协议

UDP是无连接的不可靠传输协议，追求速度。

UDP通信过程：

```
客户端                    服务器

  │                         │
  │──────── 数据包1 ────────►│
  │──────── 数据包2 ────────►│ （可能丢失）
  │──────── 数据包3 ────────►│
  │                         │
无需建立连接，直接发送
无需等待确认，继续发送
```

UDP特性：

- [完成] 无连接（无需握手，直接发送）
- [完成] 低延迟（无ACK确认等待）
- [完成] 开销小（无连接维护）
- [禁止] 不可靠（可能丢包、乱序）
- [禁止] 无拥塞控制（可能造成网络拥堵）

UDP适用场景：

- [游戏] 在线游戏（实时性要求高）
- 📹 视频直播（丢帧可接受）
- 🎤 语音通话（VoIP）
- [信号] DNS查询（快速响应）

- 🔄 QUIC协议（HTTP/3基础）

TCP vs UDP对比表：

特性	TCP	UDP
连接方式	面向连接	无连接
可靠性	可靠（重传机制）	不可靠
速度	较慢	较快
开销	大	小
顺序保证	是	否
应用场景	文件传输、网页浏览	游戏、视频直播

2.3.3 端口的概念

什么是端口?

端口是网络通信的"门牌号"，用于区分同一主机上的不同服务。

```
IP地址：指向哪台计算机（192.168.2.28）

端口号：指向哪个应用程序（80端口=Web服务器）



完整地址：192.168.2.28:80
```

端口范围：

- **0-1023**：系统端口（需要管理员权限）
- **1024-49151**：注册端口（常用软件）
- **49152-65535**：动态端口（临时分配）

常用端口速查：

```
Web服务：

  80     - HTTP
  443    - HTTPS
 8080  - 备用HTTP
```

```
远程控制:
  22    - SSH
  23    - Telnet
  3389  - Windows远程桌面



数据库:
  3306  - MySQL
  5432  - PostgreSQL
  6379  - Redis
  27017 - MongoDB



代理服务:
  1080  - SOCKS5代理
  8388  - Shadowsocks常用端口
  10808 - Clash默认端口
```

2.4 代理的概念与分类

2.4.1 代理服务器基础

什么是代理?

代理服务器是客户端和目标服务器之间的中间者，代替客户端向目标服务器发起请求。

```
直接访问:


用户 ──────────────────────► 目标网站
  (真实IP暴露)



通过代理访问:
用户 ──────────────► 代理服务器 ──────────────► 目标网站
   (加密)    (代理IP)      (看到的是代理IP)
```

代理的作用:

- **隐藏真实IP**: 保护隐私

- **突破限制**：访问被封锁的网站
- **加速访问**：通过缓存提高速度
- **内容过滤**：企业网络管控
- **负载均衡**：分散流量压力

2.4.2 代理类型分类

按工作层次分类：

1. HTTP代理

- 工作在应用层
- 仅能代理HTTP/HTTPS流量
- 配置简单，使用广泛
- 例如：Squid、Nginx

2. SOCKS代理

- 工作在会话层
- 可代理任何TCP/UDP流量
- 更加通用和灵活
- SOCKS4：仅支持TCP
- SOCKS5：支持TCP/UDP + 身份验证

3. 透明代理

- 用户无感知，自动转发
- 常用于企业网关
- 可能缓存内容

按匿名程度分类：

类型	HTTP头	匿名性	说明
透明代理	`Via: proxy_IP` `X-Forwarded-For: 真实IP`	低	目标服务器知道你使用了代理和真实IP
匿名代理	`Via: proxy_IP` 无真实IP	中	知道你用了代理，但不知道真实IP
高匿代理	无任何代理标识	高	完全无法察觉使用了代理

按部署位置分类：

1. 正向代理（Forward Proxy）

```
企业内网场景：

员工电脑 ──► 公司代理服务器 ──► 互联网
            (统一管理、内容过滤)
```

- 代理客户端访问外网
- 客户端需要配置代理
- 科学上网属于正向代理

2. 反向代理（Reverse Proxy）

```
网站服务器场景：

用户 ──► Nginx反向代理 ──► 后端服务器1
                      ──► 后端服务器2
                      ──► 后端服务器3
```

- 代理服务器接收外部请求
- 分发到内部多台服务器
- 常见：Nginx、HAProxy

2.4.3 科学上网常用代理协议

1. HTTP/HTTPS代理

```
配置示例：

代理地址：proxy.example.com
端口：8080
用户名：user
密码：pass
```

- 简单易用
- 浏览器原生支持

- 易被识别封锁

2. SOCKS5代理

```
特点:

✓ 支持TCP和UDP
✓ 支持身份验证
✓ 协议通用性强
✓ 可代理任何应用
✓ 性能开销小
```

3. Shadowsocks系列

```
优势:

✓ 加密流量,难以检测
✓ 轻量级,速度快
✓ 支持多种加密算法(2022版本强制AEAD)
✓ 开源,生态完善
✓ SIP003插件系统支持
```

4. V2Ray/Xray(VMess/VLESS)

```
特点:

✓ 强大的流量伪装能力
✓ 多种传输方式(TCP/WebSocket/HTTP/2/gRPC/QUIC)
✓ 灵活的路由规则
✓ 抗封锁能力强
✓ Xray性能优于V2Ray
✓ 支持XTLS、Vision等新特性
```

5. Trojan/Trojan-Go

```
设计理念:

✓ 模仿HTTPS流量
✓ 流量特征与正常网站一致
```

```
✓ 难以被主动探测
✓ 配置相对简单
✓ Trojan-Go支持多路复用
```

6. Hysteria2/TUIC(基于QUIC)

```
新兴协议:

✓ 基于QUIC(UDP),拥塞控制优秀
✓ 弱网环境表现优异
✓ 0-RTT握手,延迟更低
✓ 多路复用无队头阻塞
✓ 适合高丢包率网络
```

2.5 加密技术基础

2.5.1 加密的重要性

在科学上网过程中，加密技术保护你的：

- [加密] 访问内容不被窃听
- [加密] 真实目的不被识别
- [加密] 流量特征不被检测
- [加密] 身份隐私不被泄露

2.5.2 对称加密

原理： 加密和解密使用相同的密钥。

```
明文 ──[密钥K加密]──► 密文 ──[密钥K解密]──► 明文
```

常见算法：

算法	密钥长度	特点	应用	状态
AES	128/192/256位	安全性高、速度快	广泛使用,硬件加速	[完成]推荐
ChaCha20	256位	移动端性能优异	Shadowsocks首选	[完成]推荐
XChaCha20	256位	更大nonce空间	新协议采用	[完成]推荐
DES/3DES	56/168位	已被破解	完全淘汰	[禁止]禁用

优点：

- [完成] 加解密速度快
- [完成] 适合大量数据

缺点：

- [禁止] 密钥分发困难
- [禁止] 密钥泄露风险大

2.5.3 非对称加密

原理： 使用公钥加密，私钥解密（或反之）。

```
公钥加密过程：

明文 ──[公钥加密]──► 密文 ──[私钥解密]──► 明文


数字签名过程：
数据 ──[私钥签名]──► 签名 ──[公钥验证]──► 验证通过
```

常见算法：

- **RSA**：最广泛使用，2048位以上安全
- **ECC**（椭圆曲线）：更短密钥达到同等安全性
- **Ed25519**：性能优异的签名算法

应用场景：

- TLS握手中的密钥交换
- 数字签名和身份认证
- SSL证书验证

2.5.4 混合加密（HTTPS的方式）

结合两种加密的优点：

```
TLS连接建立：
  • 服务器发送公钥证书 ──────────► 客户端



  • 客户端生成随机会话密钥

  • 用服务器公钥加密会话密钥 ──► 服务器

  • 服务器用私钥解密获得会话密钥

  • 双方使用会话密钥进行对称加密通信



    (AES加密实际数据传输)
```

2.5.5 哈希算法

特点：

- 单向函数，不可逆
- 输入任意长度，输出固定长度
- 微小改变导致完全不同的结果

常见算法:

```
MD5:    128位输出(已不安全,仅用于文件校验)


SHA-1:  160位输出(已废弃,不应使用)
SHA-256:256位输出(当前推荐标准)
SHA-512:512位输出(更高安全性)
SHA-3:  可变输出(最新标准,2015年)
BLAKE2/BLAKE3: 高性能哈希(现代选择)
```

应用：

- 密码存储（加盐哈希）

- 数据完整性校验
- 数字签名

2.5.6 加密强度对比

科学上网推荐加密方案:

用途	推荐算法	安全性	性能	备注
流量加密	AES-256-GCM	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	CPU硬件加速
流量加密	ChaCha20-Poly1305	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	移动设备首选
流量加密	XChaCha20-Poly1305	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	SS2022新标准
密钥交换	X25519(ECDH)	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	椭圆曲线DH
身份认证	Ed25519	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	现代签名算法
TLS握手	RSA-2048+	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	传统方案

不推荐的加密:

- [禁止] RC4(已被完全破解)
- [禁止] DES/3DES(密钥过短,已淘汰)
- [禁止] MD5用于安全目的(仅可用于文件校验)
- [禁止] SHA-1(已被攻破,禁止使用)
- [禁止] RSA-1024(密钥长度不足)
- [禁止] 任何自创加密算法
- [禁止] Shadowsocks旧式流加密(非AEAD)

本章小结

本章介绍了科学上网所需的网络基础知识:

核心概念：

- **IP与DNS**：互联网寻址的基础，DNS污染是主要封锁手段之一
- **HTTP/HTTPS**：Web通信协议，HTTPS加密是现代网络安全基石
- **TCP/UDP**：传输层协议，科学上网工具灵活运用两者优势
- **代理技术**：流量中转的核心机制，多种协议各有特点
- **加密技术**：保护通信安全，对抗流量检测的关键

知识应用：

- 理解DNS污染才能选择正确的DNS方案
- 掌握TCP/UDP特性才能优化连接性能
- 了解加密原理才能选择安全的配置
- 熟悉代理类型才能排查连接问题

在下一章，我们将学习这些基础知识如何应用于科学上网的核心技术原理。

实践建议：

- 尝试使用 `nslookup` 命令查询域名解析
- 用 `ping` 测试延迟，理解网络往返时间
- 使用浏览器开发者工具观察HTTP请求
- 查看HTTPS网站的证书信息

延伸阅读:

- RFC 791(IP协议规范)
- RFC 793(TCP协议规范)
- RFC 1035(DNS协议规范)
- RFC 8446(TLS 1.3规范,2018)
- RFC 9000(QUIC协议,2021)
- RFC 9250(DNS over QUIC,2022)

第三章 核心技术原理

第3章 核心技术原理

在掌握了网络基础知识后，本章将深入探讨科学上网的核心技术原理。我们将理解代理服务器如何工作、VPN的运行机制，以及现代科学上网工具如何对抗网络封锁。

3.1 代理服务器工作原理

3.1.1 正向代理详解

基本工作流程：

```
完整代理流程：

第1步：客户端配置
┌──────────────────┐
│ 浏览器  │ 配置代理：proxy.example.com:1080
└──────────────────┘



第2步：发起请求
用户想访问：https://www.google.com
↓
浏览器不直接连接Google
↓
而是连接到代理服务器



第3步：代理转发
┌────────────────────┐
│ 代理服务器  │ ← 接收用户请求
└────────────────────┘
      ↓
连接真实的Google服务器
      ↓
┌────────────────────┐
│ Google.com │
└────────────────────┘
```

```
第4步：返回数据
Google响应 → 代理服务器 → 用户浏览器
```

关键技术点：

- **请求改写**

```
原始请求：

GET https://www.google.com/search?q=test HTTP/1.1
Host: www.google.com



通过HTTP代理：
CONNECT www.google.com:443 HTTP/1.1
Host: www.google.com



通过SOCKS5代理：
[SOCKS5握手] → [目标地址] → [数据传输]
```

- **连接维持**

```
用户 ←═══ 加密隧道 ═══→ 代理服务器 ←─── 明文/加密 ───→ 目标网站

    （长连接）                    （按需建立）
```

- **IP地址替换**

```
目标网站看到的访问者：

不是：用户真实IP（如 123.45.67.89）
而是：代理服务器IP（如 140.82.114.26）
```

3.1.2 HTTP代理原理

HTTP CONNECT方法：

用于建立隧道，支持HTTPS流量穿透。

```
客户端请求：

CONNECT www.example.com:443 HTTP/1.1
Host: www.example.com:443
Proxy-Authorization: Basic dXNlcjpwYXNz



代理服务器响应：
HTTP/1.1 200 Connection Established



隧道建立后：
客户端 ←═══════════ TLS加密通道 ═══════════→ 目标服务器
        （代理服务器只转发，不解密）
```

优势与限制：

- [完成] 浏览器原生支持，无需额外软件
- [完成] 配置简单，易于使用
- [禁止] 仅适用于HTTP/HTTPS流量
- [禁止] 协议特征明显，易被识别
- [禁止] 大部分HTTP代理已被封锁

3.1.3 SOCKS5代理原理

SOCKS5握手过程：

```
阶段1：版本协商

客户端 → 服务器：[版本(0x05)，认证方法数量，认证方法列表]
服务器 → 客户端：[版本(0x05)，选择的认证方法]



阶段2：身份认证（如需要）
客户端 → 服务器：[版本(0x01)，用户名长度，用户名，密码长度，密码]
服务器 → 客户端：[版本(0x01)，状态(0x00成功)]
```

```
阶段3：连接请求
客户端 → 服务器：[版本，命令，保留，地址类型，目标地址，目标端口]
服务器 → 客户端：[版本，状态，保留，绑定地址类型，绑定地址，绑定端口]



阶段4：数据传输
客户端 ←──────────── 双向数据流 ────────────→ 服务器
```

SOCKS5支持的命令：

- `CONNECT (0x01)`：建立TCP连接
- `BIND (0x02)`：绑定端口等待连接
- `UDP ASSOCIATE (0x03)`：关联UDP端口

地址类型：

- `IPv4 (0x01)`：4字节IPv4地址
- `域名 (0x03)`：长度+域名字符串
- `IPv6 (0x04)`：16字节IPv6地址

SOCKS5的优势：

- [完成] 协议层次低，可代理任何TCP/UDP流量
- [完成] 支持用户名密码认证
- [完成] 可代理非HTTP协议（FTP、SMTP、游戏等）
- [完成] UDP支持（重要！游戏和语音需要）

3.1.4 本地代理与远程代理

架构模式：

```
模式1：直接连接远程代理


应用程序 ──────────────────────► 远程代理服务器 ──────► 目标网站
            (需要应用支持SOCKS5/HTTP代理)



模式2：本地客户端+远程代理
应用程序 ──► 本地客户端 ──(加密)──► 远程代理 ──► 目标网站
  (系统代理)  (127.0.0.1:1080)      (境外服务器)
               Clash/V2RayN等
```

本地客户端的作用：

- **协议转换**：将标准SOCKS5/HTTP转换为加密协议
- **流量加密**：使用Shadowsocks/VMess等加密
- **规则分流**：智能判断哪些走代理，哪些直连
- **负载均衡**：多个节点间切换和分配
- **用户界面**：提供图形化配置和管理

3.2 VPN技术详解

3.2.1 VPN基本概念

VPN（Virtual Private Network）虚拟专用网络

VPN在公共网络上建立加密隧道，创建虚拟的专用网络连接。

```
传统专线：


公司总部 ═══ 物理专线 ═══ 分公司
          （昂贵、固定）



VPN方案：
公司总部 ═══ 互联网 ═══ 分公司
          ╚══ 加密隧道 ══╝
         （便宜、灵活、安全）
```

VPN隧道示意：

```
原始数据包：


 ┌──────────┬──────────┬──────────────┐
│ IP头 │ TCP头 │ 应用数据 │
 └──────────┴──────────┴──────────────┘



VPN封装后：
 ┌──────────┬──────────┬──────────────────────────────────────────────┐
│ 新IP头 │ VPN协议 │ [加密的(原IP头+TCP头+数据)]      │
```

```
└──────────────────┴──────────────────┴──────────────────────────────────────┘
    ↑                    ↑
到VPN服务器          加密隧道内容
```

3.2.2 VPN与代理的区别

工作层次对比：

```
OSI七层模型：

应用层  ──────────────  HTTP代理工作在这里
表示层
会话层  ──────────────  SOCKS代理工作在这里
传输层  ──────────────  部分VPN工作在这里(如SSL VPN)
网络层  ──────────────  大多数VPN工作在这里(如IPsec)
数据链路层
物理层
```

详细对比表：

维度	VPN	代理
流量范围	全局（所有网络流量）	应用级（仅配置的应用）
配置方式	系统级（虚拟网卡）	应用级（浏览器/软件）
透明度	对应用完全透明	需要应用支持
DNS处理	全部通过VPN	可能泄露到本地DNS
加密范围	整个数据包	应用层数据
性能开销	较大（全局处理）	较小（按需处理）
灵活性	低（全局模式）	高（可分流）

使用场景建议：

```
选择VPN：
```

```
✓ 远程办公访问企业内网
✓ 需要保护所有网络流量
✓ 使用不支持代理的软件
✓ 在公共WiFi保护隐私



选择代理:
✓ 科学上网（国内外分流）
✓ 只需代理特定应用
✓ 追求更快的速度
✓ 需要灵活的规则配置
```

3.2.3 常见VPN协议

1. PPTP（Point-to-Point Tunneling Protocol）

```
年代: 1990s


加密: MPPE (RC4)
安全性: [推荐] (已被破解，不推荐)
速度: [推荐][推荐][推荐][推荐][推荐]
状态: 已淘汰
```

2. L2TP/IPsec

```
年代: 2000s


加密: IPsec (AES)
安全性: [推荐][推荐][推荐]
速度: [推荐][推荐][推荐]
特点: 双层封装，开销大
状态: 仍在使用但逐渐被淘汰
```

3. OpenVPN

```
年代:2001年至今


加密:OpenSSL库(AES-256)
安全性:[推荐][推荐][推荐][推荐][推荐]
```

```
速度:[推荐][推荐][推荐]
特点:开源、灵活、可靠
状态:仍在使用,但逐渐被更快的协议替代
```

4. WireGuard

```
年代:2020年正式发布


加密:ChaCha20-Poly1305、Curve25519
安全性:[推荐][推荐][推荐][推荐][推荐]
速度:[推荐][推荐][推荐][推荐][推荐]
特点:代码简洁(~4000行)、性能极佳
状态:2024年成为主流首选
```

5. IKEv2/IPsec

```
年代:2005年发布,持续更新


加密:AES-256
安全性:[推荐][推荐][推荐][推荐]
速度:[推荐][推荐][推荐][推荐]
特点:移动端稳定,自动重连
状态:Apple设备原生支持,企业常用
```

协议对比:

协议	加密强度	吞吐量	连接延迟	配置难度	抗封锁	推荐度
PPTP	低	快	低	简单	差	[禁止] 已废弃
L2TP/IPsec	中	中等	中	中等	差	[注意] 逐步淘汰
OpenVPN	高	0.65-0.78Gbps	8-12ms	复杂	一般	[完成] 企业可用
IKEv2/IPsec	高	~4Gbps	2-5ms	中等	一般	[完成] 移动设备优选
WireGuard	高	7.88Gbps	1-3ms	简单	一般	[完成][完成] 性能最佳
Shadowsocks	高	快	低	简单	强	[完成][完成] 经典方案
Xray/V2Ray	高	6.8Gbps	2-4ms	中等	很强	[完成][完成][完成] 主流首选
Hysteria2	高	7.5Gbps	<5ms	中等	强	[完成][完成][完成] 弱网优选
Sing-box	高	优秀	低	中等	很强	[完成][完成][完成] 新兴热门

3.2.4 WireGuard现代VPN

为什么WireGuard是现代之选：

代码简洁：

```
代码行数对比：

OpenVPN：~70,000行
IPsec：~400,000行
WireGuard：~4,000行
↓
更少的代码 = 更少的漏洞 = 更高的安全性
```

性能实测数据(10Gbps网络环境)：

```
协议吞吐量对比：
```

```
WireGuard:     7.88 Gbps  (接近线速,CPU占用低)
OpenVPN:       0.65-0.78 Gbps  (受限于加密开销)
IPsec/IKEv2:   ~4 Gbps  (内核级处理)
Hysteria2:     ~7.5 Gbps  (QUIC协议优势)
Xray(XTLS):    ~6.8 Gbps  (Vision流控优化)



连接建立时间:
WireGuard:     0.5-1秒
OpenVPN:       3-8秒
IKEv2:         1-2秒



内存占用:
WireGuard:     5-15 MB
OpenVPN:       50-100 MB
```

核心特性:

- **现代密码学**:ChaCha20-Poly1305、Curve25519、BLAKE2s哈希
- **无状态握手**:类似SSH的公钥认证,无需复杂PKI基础设施
- **连接迁移**:IP地址或端口变化时自动无缝切换(移动网络友好)
- **极简配置**:配置文件通常仅10-20行,类似SSH密钥对
- **内核集成**:Linux 5.6+内核原生支持,Windows/macOS/iOS/Android全平台可用
- **极小代码**:仅约4000行代码,便于审计,漏洞更少

配置示例:

```
[Interface]


PrivateKey = <客户端私钥>
Address = 10.0.0.2/24
DNS = 1.1.1.1, 8.8.8.8
MTU = 1420



[Peer]
PublicKey = <服务器公钥>
Endpoint = server.example.com:51820
AllowedIPs = 0.0.0.0/0, ::/0
PersistentKeepalive = 25
```

3.3 隧道技术（Tunneling）

3.3.1 隧道的概念

什么是网络隧道?

隧道技术将一种网络协议封装在另一种协议中传输，就像在公路下挖地道一样。

```
现实类比:

地面交通受限 → 挖掘地下隧道 → 在隧道内自由通行



网络隧道:
网络访问受限 → 建立加密隧道 → 在隧道内传输数据
```

隧道封装原理:

```
原始数据:

┌──────────────────────┐
│ 应用数据        │
└──────────────────────┘



加上传输层头:
┌──────────┬──────────────────────┐
│ TCP  │ 应用数据        │
└──────────┴──────────────────────┘



加上网络层头:
┌──────────┬──────────┬──────────────────────┐
│ IP  │ TCP  │ 应用数据        │
└──────────┴──────────┴──────────────────────┘
```

```
隧道封装（整体加密后再封装）：
┌──────────────┬──────────────┬──────────────────────────────────┐
│ 外层IP │ 隧道协议 │ 加密(IP+TCP+应用数据)    │
└──────────────┴──────────────┴──────────────────────────────────┘
```

3.3.2 常见隧道类型

1. IP隧道（IP-in-IP）

```
用途：IPv6 over IPv4、站点间VPN

原理：将整个IP包封装在另一个IP包中
应用：企业多站点互联
```

2. GRE隧道

```
GRE (Generic Routing Encapsulation)

特点：可封装多种协议
用途：路由器之间的隧道
限制：不加密（需配合IPsec）
```

3. SSL/TLS隧道

```
原理：在TLS连接内传输其他协议

优势：伪装成HTTPS流量
应用：Trojan、V2Ray的TLS模式
```

4. SSH隧道

```
本地端口转发：

ssh -L 本地端口:目标地址:目标端口 用户@SSH服务器
```

```
远程端口转发:
ssh -R 远程端口:本地地址:本地端口 用户@SSH服务器



动态端口转发（SOCKS代理）:
ssh -D 1080 用户@SSH服务器
```

3.3.3 科学上网中的隧道应用

Shadowsocks隧道：

```
客户端                                服务器


  │                                    │
  │  ──────  ①连接SS服务器 ──────────────────────────►  │
  │      (建立加密隧道)                  │
  │                                    │
  │  ──────  ②发送加密请求 ──────────────────────────►  │
  │       [加密(目标地址+数据)]            │
  │                                    │
  │                                    │──── ③解密请求
  │                                    │    访问真实目标
  │                                    │
  │  ◄──────  ④返回加密响应 ──────────────────────────  │
  │       [加密(目标响应数据)]            │
  │                                    │
  │  ──────  ⑤本地解密显示                    │
```

V2Ray多层隧道：

```
复杂场景: WebSocket + TLS + CDN


用户 → V2Ray客户端
         ↓ (VMess协议加密)
      WebSocket连接
       ↓ (套上TLS加密)
      HTTPS流量
       ↓ (通过CDN)
      Cloudflare CDN
       ↓
```

```
        V2Ray服务器
         ↓
       目标网站



防火墙看到的：正常的HTTPS网站访问
实际传输的：代理流量
```

3.4 流量混淆与伪装

3.4.1 为什么需要流量伪装

深度包检测(DPI)的威胁:

```
防火墙的多维度检测手段：

• 端口与协议特征：

  ✗ 非443端口的加密流量 → 可疑

  ✗ 大量同端口连接但协议特征不符 → 标记

• 流量统计特征：

  ✗ 熵值分析(加密数据随机性检测)

  ✗ 包长度分布模式识别
  ✗ 发包间隔时序分析
  ✗ 流量突发模式异常

• TLS指纹识别：

  ✗ JA3/JA4指纹库匹配(Client Hello特征)

  ✗ 服务器证书链分析
  ✗ SNI(Server Name Indication)域名检查
  ✗ ALPN协议协商异常

• 主动探测攻击：
```

```
    ✗ 模拟客户端连接可疑服务器

    ✗ 证书重放攻击检测
    ✗ 无效数据包响应测试
    ✗ 时间关联分析

  • 行为模式分析:

    ✗ 连接时长统计异常

    ✗ 流量方向性分析
    ✗ 连接频率模式识别
    ✗ 机器学习模型检测(近年新增)
```

3.4.2 流量混淆技术

1. 简单混淆（http_simple, tls1.2_ticket_auth等）

```
原始Shadowsocks流量:

[随机数据...] (虽然加密但有特征)



混淆后:
[伪造的HTTP头]
GET / HTTP/1.1
Host: www.bing.com
[...实际加密数据...]



防火墙看到: 像是访问Bing的HTTP流量
```

2. TLS伪装

```
真正的HTTPS连接特征:

Client Hello (TLS) → Server Hello → Certificate → ...
```

```
伪装的TLS：
模仿Client Hello结构
伪造Server Name Indication (SNI)
使用真实网站域名作为掩护



结果：看起来像在访问正常HTTPS网站
```

3. WebSocket伪装

```
WebSocket协议特点：
  • 通过HTTP升级建立



  • 长连接,双向通信


  • 广泛用于聊天、推送、在线协作





伪装方案：
HTTP请求升级到WebSocket
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Version: 13
↓
在WebSocket通道内传输代理数据
↓
防火墙认为是正常的Web应用
```

4. gRPC伪装

```
gRPC特点：
  • 基于HTTP/2的RPC框架
```

```
  • Google开发,微服务架构广泛采用

  • 正常业务流量,防火墙难以区分

  • Kubernetes、云原生应用大量使用




Xray gRPC模式:
使用gRPC作为传输层
流量特征与正常gRPC API完全一致
支持multiplexing多路复用
内置flow control流控机制
```

5. QUIC/HTTP3伪装

```
QUIC技术优势:
  • UDP协议基础,绕过TCP针对性封锁



  • 内置1-RTT/0-RTT握手,首次连接<5ms

  • 内建加密,无明文握手阶段

  • 连接迁移,网络切换不断线

  • HTTP/3正式标准(RFC 9114, 2022年)




Hysteria2/TUIC实现:
完全基于QUIC协议构建
伪装成普通HTTP/3流量
拥塞控制采用BBR/Brutal算法
弱网丢包率20%仍可稳定使用
测试吞吐量可达7.5Gbps
```

3.4.3 CDN中转技术

CDN加速+隐藏真实IP：

```
传统方案问题：

用户 ──────────► VPS服务器IP ──────► 目标网站
            (IP暴露，易被封)



使用CDN：
用户 ──► Cloudflare CDN ──► 源站服务器 ──► 目标网站
      (大量CDN节点)   (真实IP隐藏)



优势：
✓ 真实服务器IP不暴露
✓ CDN的IP太多，难以全部封锁
✓ 流量看起来是访问CDN的正常网站
✓ CDN本身提供加速
```

WebSocket + TLS + CDN组合：

```
V2Ray配置示例：

{
  "streamSettings": {
    "network": "ws",           // 使用WebSocket
    "security": "tls",         // 启用TLS加密
    "tlsSettings": {
      "serverName": "yoursite.com"
    },
    "wsSettings": {
      "path": "/websocket"     // WebSocket路径
    }
  }
}
```

3.4.4 协议伪装案例

Trojan协议的伪装策略：

```
Trojan设计哲学:

"如果流量看起来完全像HTTPS，就无法被识别"



实现方式:

  • 使用真实的TLS证书



  • 在真实Web服务器上运行

  • 流量与正常HTTPS完全相同

  • 通过密码区分真实用户和探测




流程:
防火墙探测 ──► Trojan服务器
              │
        密码错误?
              ↓
        返回真实网站内容
        (防火墙认为是正常网站)



真实用户 ──► 密码正确 ──► 建立代理连接
```

VMess + WebSocket + TLS + Nginx(经典方案):

```
server {

    listen 443 ssl http2;
    server_name yoursite.com;
```

```
    ssl_certificate /path/to/fullchain.pem;
    ssl_certificate_key /path/to/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;



    location / {
       # 正常网站内容
        root /var/www/html;
        index index.html;
   }



    location /secret-path {
        # WebSocket代理到V2Ray
        proxy_pass http://127.0.0.1:10000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
   }
}



结果:


 • 访问首页:看到真实网站



 • 访问/secret-path:进入V2Ray隧道


 • 防火墙无法区分
```

Reality协议(最新抗审查技术):

```
Reality设计理念:
```

"借用"真实网站的TLS指纹
完全模拟目标网站的握手过程
无需自己的域名和证书

技术原理:

- 客户端发起连接到Reality服务器
- 握手时借用指定网站(如microsoft.com)的证书
- 对外呈现与目标网站完全相同的TLS特征
- 通过隐藏字段识别真实用户
- 非法探测只能看到真实网站的响应

实测优势(2023年发布以来):
✓ 无需购买域名和SSL证书
✓ TLS指纹100%真实
✓ 抗主动探测能力极强
✓ 配置相对简单
✓ 已在多国成功抵御封锁
✓ 支持xtls-rprx-vision流控

局限性:

- 依赖目标网站稳定性
- 需选择合适的"掩护"网站

3.5 加密传输机制

3.5.1 端到端加密

加密的层次：

```
多层加密保护：


应用层：HTTPS/TLS
  ↓ [加密应用数据]
代理层：Shadowsocks/VMess
  ↓ [再次加密]
传输层：TCP/WebSocket
  ↓
最终传输：双重加密的数据流



即使代理层被破解，应用层仍然安全
```

Shadowsocks加密流程：

```
发送端：


明文数据 → [加密算法 + 密码] → 密文 → 发送
           ↓
        AES-256-GCM
        ChaCha20-Poly1305



接收端：
接收 → 密文 → [解密算法 + 密码] → 明文数据
```

3.5.2 认证加密（AEAD）

什么是AEAD?

AEAD (Authenticated Encryption with Associated Data) = 加密 + 完整性验证

```
传统加密问题：
```

```
密文可能被篡改，接收方无法察觉



AEAD解决方案：
加密的同时生成认证标签（MAC）
┌──────────────────┬──────────────────┐
│ 密文数据 │ 认证标签 │
└──────────────────┴──────────────────┘



接收方：

  • 验证标签是否匹配

  • 如果被篡改，标签验证失败

  • 拒绝解密，保证安全
```

推荐的AEAD算法:

算法	密钥长度	性能	安全性	推荐场景	状态
AES-128-GCM	128位	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	通用,硬件加速	[完成]
AES-256-GCM	256位	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	更高安全需求	[完成][完成]
ChaCha20-Poly1305	256位	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	移动设备首选	[完成][完成]
XChaCha20-Poly1305	256位	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	更大nonce空间	[完成][完成]

3.5.3 密钥派生与管理

密钥派生函数（KDF）：

```
从简单密码生成强密钥:

用户密码: "mypassword123"
   ↓ [PBKDF2/HKDF/bcrypt]
   ↓ (加盐 + 多次哈希)
   ↓
强密钥: 0x8a3f2b9c1e5d7f... (256位随机)



防止:
✗ 弱密码直接用作密钥
✗ 彩虹表攻击
✗ 暴力破解
```

Shadowsocks密钥派生:

```
主密码 → EVP_BytesToKey → 加密密钥 + IV

         (MD5多次迭代)
```

V2Ray UUID机制:

```
用户ID:550e8400-e29b-41d4-a716-446655440000

        (UUID v4,随机生成)



作用:

  • 用户身份标识



  • 防止重放攻击(结合时间戳)

  • 流量统计和管理

  • 多用户隔离
```

Shadowsocks 2022版本(2022年发布):

```
重大改进:
  • 采用预共享密钥(PSK)体系
  • 强制使用AEAD加密(AES-GCM/ChaCha20-Poly1305)
  • 内置完整的重放攻击防护
  • 使用Blake3高性能哈希算法
  • 消除TCP重放过滤问题
  • UDP流量增加重放保护

推荐加密方法:
  • 2022-blake3-aes-128-gcm
  • 2022-blake3-aes-256-gcm
  • 2022-blake3-chacha20-poly1305

密钥生成:
openssl rand -base64 32
密钥格式: 2022-blake3-aes-256-gcm:YourBase64Key==
```

```
相比旧版优势：
✓ 消除已知安全漏洞
✓ 性能显著提升
✓ 完整的重放保护
✓ 更规范的密钥管理
```

3.5.4 防重放攻击

重放攻击原理：

```
攻击者：
  • 截获合法用户的加密数据包
  • 不解密，直接重新发送
  • 冒充合法用户访问

结果：即使加密也不安全
```

防御机制：

1. 时间戳验证

```
数据包结构：

┌──────────────┬──────────────┬──────────────┐
│ 时间戳       │ 随机数       │ 实际数据     │
└──────────────┴──────────────┴──────────────┘

服务器验证：
if (|收到的时间戳 - 当前时间| > 30秒) {
    拒绝连接; // 可能是重放攻击
}
```

2. Nonce（一次性随机数）

```
每个数据包使用唯一的Nonce


服务器记录已使用的Nonce
重复的Nonce → 检测到重放 → 拒绝
```

3. 序列号机制

```
TCP序列号：


每个包有递增的序列号
接收方检查序列号连续性
乱序/重复 → 丢弃
```

3.5.5 完美前向保密（PFS）

什么是PFS?

Perfect Forward Secrecy：即使长期密钥泄露，过去的通信仍然安全。

```
没有PFS：


长期密钥泄露 → 所有历史通信被解密



有PFS：
长期密钥仅用于身份认证
实际加密使用临时会话密钥
会话结束后密钥销毁
→ 历史通信无法解密
```

实现方式：

```
TLS 1.3 握手（支持PFS）：
  • 客户端生成临时密钥对（ephemeral key）
```

```
  • 服务器生成临时密钥对

  • 通过ECDHE交换公钥

  • 各自计算会话密钥

  • 使用会话密钥加密通信

  • 连接结束销毁临时密钥



即使服务器私钥泄露，
攻击者仍无法解密过去的会话
```

本章小结

本章深入探讨了科学上网的核心技术原理：

关键技术点：

- **代理服务器**：通过流量中转和IP替换实现访问
- **VPN隧道**：在网络层建立加密通道，保护所有流量
- **隧道技术**：协议封装，在受限网络中建立自由通道
- **流量伪装**：对抗DPI检测，模仿正常HTTPS流量
- **加密机制**：多层加密+认证，确保通信安全

技术演进趋势:

```
简单代理 → 加密代理 → 流量伪装 → 行为模拟 → 智能对抗

（易封）   （特征检测）  （主动探测）  （难以区分）（持续博弈）
```

前沿技术方向:

- **QUIC协议普及**:Hysteria2、TUIC等高性能实现,测试吞吐量7.5Gbps+
- **Reality技术**:借用真实TLS指纹,无需证书,伊朗等地实测有效
- **ECH(Encrypted Client Hello)**:加密SNI字段,RFC 9180标准推进中
- **多路径传输**:MPTCP同时使用多条网络路径,提升稳定性
- **智能流量模拟**:机器学习生成真实用户行为模式
- **去中心化方案**:Tor、I2P等匿名网络技术应用
- **WebTransport**:基于HTTP/3的新一代传输API

核心理念：

- **加密保安全**：防窃听、防篡改
- **伪装防检测**：模仿正常流量
- **灵活保可用**：多协议、多节点

在下一部分，我们将学习具体的代理协议和客户端工具的使用方法。

实践思考:

- 为什么Trojan要运行在真实网站上?
- CDN中转如何同时实现加速和隐藏IP?
- AEAD相比传统加密有什么优势?
- 如何选择合适的加密算法?
- Reality协议相比传统TLS伪装有何优势?
- 为什么QUIC协议在弱网环境下表现更好?

延伸阅读:

- WireGuard白皮书(2020)
- Shadowsocks 2022版本设计文档
- Xray-core完整配置指南
- TLS 1.3协议规范(RFC 8446)
- QUIC协议规范(RFC 9000)
- Reality协议技术文档(2023)
- Hysteria2设计理念与实现

第四章 常见代理协议详解

第4章 常见代理协议详解

在了解了科学上网的基础原理后，本章将深入介绍主流的代理协议。这些协议是实现科学上网的核心技术，各有特点和适用场景。

4.1 Shadowsocks系列（SS/SSR）

4.1.1 Shadowsocks诞生背景

历史时刻：

- **2012年**：@clowwindy 在GitHub发布Shadowsocks
- **设计理念**：轻量、快速、安全、易部署
- **核心思想**：SOCKS5代理 + 流量加密

为什么叫Shadowsocks?

```
SOCKS协议 + 影子(Shadow) = Shadowsocks
```

Shadowsocks：在SOCKS代理的阴影下，隐藏真实流量

4.1.2 Shadowsocks工作原理

完整通信流程：

```
客户端                                                    服务器


  │                                                        │
  │──────────────────── ①本地应用发起SOCKS5请求 ──────────────────────────►│
  │         (如访问google.com)                              │
  │                                                        │
  │──────────── ②SS客户端加密请求 ──────────────────────────────────────► │
  │           [加密(目标地址 + 数据)]                        │
  │           使用预设密码和加密算法                          │
  │                                                       │
  │                                                              │── ③SS服务器解密
  │                                                       │     获取真实目标地址
  │                                                       │
```

```
    │                                                │──────── ④连接目标网站
    │                                                         │    (google.com)
    │                                                │
    │◄──────────────────── ⑤返回加密响应 ──────────────────────────────────────────
    │            [加密(网站返回数据)]                    │
    │                                                │
    │──────────────── ⑥本地解密，返回给应用                         │
```

数据包结构：

```
Shadowsocks数据包格式：


┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
│ IV/Salt          │ 目标地址类型      │ 目标地址         │ 加密的数据       │
│ (随机)           │ (1字节)           │ (变长)           │                  │
└──────────────────┴──────────────────┴──────────────────┴──────────────────┘



目标地址类型：
0x01 = IPv4地址 (4字节)
0x03 = 域名 (1字节长度 + 域名)
0x04 = IPv6地址 (16字节)



加密方式：
使用密码派生密钥
每个连接使用唯一的IV（初始化向量）
```

4.1.3 加密方法详解

支持的加密算法：

加密方法	密钥长度	安全性	速度	推荐度
aes-128-gcm	128位	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[完成] 推荐
aes-256-gcm	256位	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[完成] 推荐
chacha20-ietf-poly1305	256位	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[完成][完成] 强烈推荐
aes-256-cfb	256位	[推荐][推荐][推荐]	[推荐][推荐][推荐]	[注意] 旧版本
rc4-md5	128位	[推荐]	[推荐][推荐][推荐][推荐][推荐]	[禁止] 已淘汰

AEAD加密方法（推荐）：

```
AEAD = Authenticated Encryption with Associated Data

认证加密，防篡改



为什么推荐：
✓ 加密 + 认证，双重保护
✓ 防止中间人攻击
✓ 检测数据是否被篡改
✓ 现代密码学标准



推荐选择：


  • chacha20-ietf-poly1305 (移动端优先)



  • aes-256-gcm (桌面端优先，CPU支持AES-NI)


  • aes-128-gcm (性能和安全平衡)
```

4.1.4 ShadowsocksR (SSR)

SSR的增强功能：

ShadowsocksR是Shadowsocks的分支版本，增加了混淆和协议插件。

主要特性：

- **协议插件（Protocol）**

```
作用：改变数据包的协议特征

常用协议：

  • origin：原版SS协议



  • auth_aes128_md5：增加认证

  • auth_aes128_sha1：更强的认证

  • auth_chain_a/b：抗重放攻击




推荐：auth_aes128_sha1 或 auth_chain_a
```

- **混淆插件（Obfs）**

```
作用：伪装流量特征

常用混淆：

  • plain：无混淆



  • http_simple：伪装成HTTP流量

  • http_post：伪装成HTTP POST请求
```

```
  • tls1.2_ticket_auth：伪装成TLS流量




推荐: tls1.2_ticket_auth
```

SSR配置示例：

```
{

  "server": "example.com",
  "server_port": 8388,
  "password": "your_password",
  "method": "chacha20-ietf-poly1305",
  "protocol": "auth_aes128_sha1",
  "protocol_param": "",
  "obfs": "tls1.2_ticket_auth",
  "obfs_param": "cloudflare.com"
}
```

混淆参数说明：

```
obfs_param: "cloudflare.com"


↓
伪装成访问cloudflare.com的TLS连接
防火墙看到: 正常HTTPS流量
实际传输: 代理数据
```

4.1.5 SS vs SSR对比

特性	Shadowsocks	ShadowsocksR
开发者	clowwindy	breakwa11
发布时间	2012年	2015年
协议纯净度	简洁	复杂（增加协议层）
混淆能力	基础	强（协议+混淆）
抗封锁	中	强
性能	优秀	良好（有额外开销）
维护状态	活跃	较少更新
推荐场景	一般环境	封锁严重地区

选择建议：

```
选择Shadowsocks：

✓ 网络环境较好
✓ 追求极致性能
✓ 使用AEAD加密方法
✓ 配合CDN等其他手段



选择ShadowsocksR：
✓ 网络封锁严重
✓ SS经常被断连
✓ 需要流量混淆
✓ 配合协议和混淆插件
```

4.1.6 实战配置案例

服务器端配置（以ss-rust为例）：

```
{

  "server": "0.0.0.0",
  "server_port": 8388,
```

```
  "password": "myStrongPassword123!",
  "method": "chacha20-ietf-poly1305",
  "timeout": 300,
  "fast_open": true,
  "mode": "tcp_and_udp"
}
```

客户端配置：

```
{

  "server": "your-server-ip",
  "server_port": 8388,
  "local_address": "127.0.0.1",
  "local_port": 1080,
  "password": "myStrongPassword123!",
  "method": "chacha20-ietf-poly1305",
  "timeout": 300
}
```

启动命令：

```
# 服务器端

ss-server -c config.json

# 客户端



ss-local -c config.json

# 后台运行



nohup ss-server -c config.json > ss.log 2>&1 &
```

4.1.7 Shadowsocks插件系统

常用插件：

1. simple-obfs（简单混淆）

```
# 服务器端

ss-server -c config.json --plugin obfs-server --plugin-opts "obfs=tls"

# 客户端


ss-local -c config.json --plugin obfs-local --plugin-opts "obfs=tls;obfs-host=cloudflare.com"
```

2. v2ray-plugin（V2Ray混淆）

```
{

  "plugin": "v2ray-plugin",
  "plugin_opts": "server;tls;host=yoursite.com;path=/v2ray"
}
```

3. kcptun（KCP加速）

```
# 用于优化高丢包网络
# 将TCP流量转换为UDP，提高速度
```

4.2 V2Ray/Xray协议族

4.2.1 V2Ray项目简介

V2Ray的定位：

- 不仅是代理工具，更是一个平台

- 模块化设计，灵活组合
- 支持多种协议和传输方式
- 强大的路由功能

核心架构：

```
┌──────────────────────────────────────────────────────────────────────────┐


│          V2Ray Core                 │
│                                 │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐  │
│   │  入站   │   │   路由  │   │ 出站   │  │
│  │ Inbound│→│Routing │→│Outbound│ │
│   └──────────────┘   └──────────────┘   └──────────────┘  │
│      ▲          │          │      │
│      │          ▼          ▼      │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐  │
│   │ 协议层  │   │ 规则库  │   │ 协议层  │   │
│  │ VMess  │  │ 域名/IP │  │ Freedom│ │
│  │ VLESS  │  │ GeoIP  │  │ VMess  │ │
│  │ Socks  │  │ 自定义  │  │ Socks  │ │
│   └──────────────┘   └──────────────┘   └──────────────┘  │
│                                 │
│   ┌──────────────────────────────────────────────┐    │
│   │   传输层 (StreamSettings)     │    │
│  │   TCP / WebSocket / HTTP/2   │    │
│  │   mKCP / QUIC / gRPC         │    │
│   └──────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────────┘
```

Xray的关系：

```
V2Ray (原项目)


    ↓
Project X (新团队)
    ↓
Xray-core (性能增强版)



Xray特点:
✓ 完全兼容V2Ray配置
```

```
✓ 性能优化（更快）
✓ 新协议支持（VLESS、XTLS）
✓ 更活跃的开发
```

4.2.2 VMess协议

VMess特点：

- V2Ray原创协议
- 基于时间的动态认证
- 强加密，支持多种加密方式
- 内置混淆和伪装

VMess数据包结构：

```
┌──────────────┬──────────────┬──────────────┬──────────────┐
│ 认证信息 │ 指令部分 │ 数据部分 │ 校验和 │
└──────────────┴──────────────┴──────────────┴──────────────┘

认证信息：
  • 时间戳（Unix time）
  • UUID（用户ID）
  • 随机值

→ 防重放攻击

指令部分：
  • 目标地址
  • 端口
```

```
  • 命令类型（TCP/UDP）




数据部分：

  • 实际传输数据


  • 使用AES加密
```

VMess配置示例：

```
{

  "inbounds": [{
    "port": 10086,
    "protocol": "vmess",
    "settings": {
      "clients": [{
        "id": "UUID-HERE",
        "alterId": 0
     }]
    },
    "streamSettings": {
      "network": "tcp"
    }
  }],
  "outbounds": [{
    "protocol": "freedom"
  }]
}
```

alterId参数说明：

```
alterId: 额外ID数量
```

```
alterId = 0 (推荐)：
✓ 不生成额外ID
✓ 更安全（减少攻击面）
✓ 新版推荐设置



alterId = 64/100 (旧版)：
✗ 生成额外ID混淆
✗ 已过时
✗ 可能有安全隐患
```

4.2.3 VLESS协议

VLESS设计理念：

- VMess的轻量化版本
- 去除加密层（由TLS负责）
- 更少的协议开销
- 更快的传输速度

[注意] 重要提示：

```
警告：VMess协议直连模式在2025年面临更强的检测压力


建议：优先使用VLESS，特别是配合Reality技术
推荐：VLESS + Reality 被评为2025年最佳组合
```

VLESS vs VMess：

特性	VMess	VLESS
加密	内置加密	依赖TLS
性能	良好	更优
协议开销	较多	极少
时间校验	严格（±90s）	无
复杂度	较高	简单
抗检测能力	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]
推荐场景	不使用TLS时	配合TLS/Reality使用

VLESS配置示例：

```
{

  "inbounds": [{
    "port": 443,
    "protocol": "vless",
    "settings": {
      "clients": [{
        "id": "UUID-HERE",
        "level": 0
     }],
      "decryption": "none"
   },
    "streamSettings": {
      "network": "tcp",
      "security": "tls",
      "tlsSettings": {
        "certificates": [{
          "certificateFile": "/path/to/cert.pem",
          "keyFile": "/path/to/key.pem"
       }]
     }
   }
  }]
}
```

VLESS + XTLS：

```
XTLS = 极致的TLS优化

原理:
当内层已经是TLS加密时（如HTTPS）
外层TLS不再加密，直接转发
↓
避免双重加密的性能损失
↓
速度提升20%-40%



配置:
"security": "xtls",
"xtlsSettings": {
  "alpn": ["h2", "http/1.1"]
}
```

VLESS + Reality（推荐）：

```
Reality = 新一代伪装技术

核心优势:
✓ 完美模仿真实TLS指纹
✓ 无需购买域名和证书
✓ 抗主动探测能力极强
✓ GFW难以通过TLS特征识别



工作原理:


  • 使用真实网站的TLS指纹



  • 客户端认证通过 → 建立代理连接


  • 认证失败 → 转发到真实网站


  • 防火墙无法区分真假流量
```

为什么是最佳选择：

- 能够有效抵抗基于TLS特征的检测
- GFW的检测技术不断升级，Reality应对能力最强
- 配置相对简单，性能优秀
- 适合需要高安全性和稳定性的用户

配置示例：

```
{
  "protocol": "vless",
  "settings": {
    "clients": [{
      "id": "UUID-HERE",
      "flow": "xtls-rprx-vision"
    }],
    "decryption": "none"
  },
  "streamSettings": {
    "network": "tcp",
    "security": "reality",
    "realitySettings": {
      "dest": "www.microsoft.com:443",
      "serverNames": ["www.microsoft.com"],
      "privateKey": "YOUR_PRIVATE_KEY",
      "shortIds": [""]
    }
  }
}
```

推荐目标网站（dest）：

- www.microsoft.com

```
  • www.apple.com

  • www.cloudflare.com

  • www.amazon.com


选择访问量大、信誉好的网站
```

4.2.4 Trojan协议

Trojan设计哲学： > "最好的伪装就是不被发现"

核心思想：

```
不是创造新的混淆方法

而是完全模仿真实的HTTPS流量



Trojan服务器 = 真实的Web服务器
    ↓
防火墙探测时：看到真实网站
真实用户连接：通过密码验证进入代理



结果：无法区分，无法封锁
```

Trojan工作流程：

```
客户端连接 ──► Trojan服务器

                  │
            验证密码hash
                  │
          ┌─────────────────┴─────────────────┐
          │                   │
    密码正确            密码错误
          │                   │
```

```
    建立代理连接       返回真实网站内容
         │                  │
    访问目标网站       (伪装成普通访问)
         │                  │
     ◄────┘                  │
                             │
                    防火墙无法识别
```

Trojan数据包格式：

```
┌──────────────────────┬──────┬──────────────┬──────┬──────────────┐

│ 密码SHA224  │ CRLF │ 目标地址  │ CRLF │ 数据     │
└──────────────────────┴──────┴──────────────┴──────┴──────────────┘
     56字节    2字节             2字节



外层：TLS加密
内层：Trojan协议



防火墙看到：标准TLS 1.2/1.3流量
实际传输：代理数据
```

Trojan配置示例：

```
{

  "run_type": "server",
  "local_addr": "0.0.0.0",
  "local_port": 443,
  "remote_addr": "127.0.0.1",
  "remote_port": 80,
  "password": [
    "your_password_here"
  ],
  "ssl": {
    "cert": "/path/to/fullchain.pem",
    "key": "/path/to/privkey.pem",
    "sni": "yoursite.com"
  }
}
```

Trojan-Go增强版：

```
Trojan-Go特性:


✓ 完全兼容原版Trojan
✓ 支持WebSocket
✓ 支持Shadowsocks混合
✓ CDN友好
✓ 多路复用（Mux）
✓ 更好的性能
```

4.2.5 传输方式详解

V2Ray支持多种传输方式（Transport），可灵活组合。

1. TCP传输

```
{

  "network": "tcp",
  "tcpSettings": {
    "header": {
      "type": "none"  // 或 "http" 伪装
    }
  }
}



特点:
✓ 最简单、最直接
✓ 延迟低
✗ 容易被识别
```

2. WebSocket (WS)

```
{

  "network": "ws",
  "wsSettings": {
    "path": "/v2ray",
```

```
    "headers": {
      "Host": "yoursite.com"
    }
  }
}



特点:
✓ CDN友好（可过Cloudflare）
✓ 伪装成Web应用
✓ 可配合Nginx
✗ 性能略有损失
```

3. HTTP/2

```
{

  "network": "h2",
  "httpSettings": {
    "host": ["yoursite.com"],
    "path": "/v2ray"
  }
}



特点:
✓ 多路复用
✓ 流量特征像真实HTTPS
✓ 性能优秀
✗ 必须配合TLS
```

4. mKCP

```
{

  "network": "mkcp",
  "kcpSettings": {
    "mtu": 1350,
    "tti": 50,
    "uplinkCapacity": 12,
    "downlinkCapacity": 100,
    "congestion": true,
```

```
    "header": {
      "type": "wechat-video"  // 伪装成微信视频
    }
  }
}



特点:
✓ 基于UDP
✓ 高丢包环境表现好
✓ 可伪装（wechat-video/dtls/wireguard）
✗ 流量消耗大（约2-3倍）
```

5. QUIC

```
{


  "network": "quic",
  "quicSettings": {
    "security": "aes-128-gcm",
    "key": "your_key",
    "header": {
      "type": "dtls"
    }
  }
}



特点:
✓ 0-RTT连接
✓ 移动网络友好
✓ HTTP/3基础协议
✗ 部分地区UDP受限
```

6. gRPC

```
{


  "network": "grpc",
  "grpcSettings": {
    "serviceName": "your_service_name",
```

```
    "multiMode": true
  }
}



特点:
✓ Google的RPC框架
✓ 流量特征真实
✓ 支持多路复用
✓ 性能优秀
```

传输方式选择建议：

```
场景推荐:

常规使用:
TCP + TLS (简单稳定)



通过CDN：
WebSocket + TLS (Cloudflare友好)



高速传输:
gRPC + TLS 或 HTTP/2 + TLS



丢包环境:
mKCP (游戏、移动网络)



极致隐蔽:
WebSocket + TLS + CDN + 真实网站
```

4.2.6 Hysteria协议

Hysteria简介：

Hysteria是基于QUIC协议的高性能代理协议，专为不稳定网络环境优化。

核心特点：

```
基于QUIC：


✓ 0-RTT连接建立
✓ 多路复用
✓ 连接迁移（IP变化不断连）
✓ 拥塞控制优化



性能优势:
✓ 高丢包环境表现优异
✓ 大流量传输速度快
✓ 移动网络友好
✓ 适合视频流媒体
```

Hysteria vs 传统协议：

特性	Hysteria	Shadowsocks	VMess
传输协议	QUIC (UDP)	TCP	TCP
弱网表现	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]
速度	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
流量消耗	较高	正常	正常
抗检测	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]

配置示例：

```
// 服务器配置


{
  "listen": ":443",
  "protocol": "udp",
  "cert": "/path/to/cert.pem",
  "key": "/path/to/key.pem",
  "up_mbps": 100,
  "down_mbps": 100,
  "auth": {
    "mode": "password",
    "config": {
      "password": "your_password"
```

```
    }
  }
}



// 客户端配置
{
  "server": "your-server.com:443",
  "protocol": "udp",
  "up_mbps": 50,
  "down_mbps": 100,
  "auth_str": "your_password",
  "socks5": {
    "listen": "127.0.0.1:1080"
  }
}
```

适用场景：

```
推荐使用：


✓ 网络不稳定（高丢包、高延迟）
✓ 大文件传输
✓ 4K视频流媒体
✓ 移动网络（4G/5G频繁切换）
✓ 游戏加速



注意事项：
[注意] 长时间大流量可能被GFW针对
[注意] 流量消耗约为正常的1.5-2倍
[注意] 部分地区UDP受限
[注意] 需要服务器支持高带宽
```

使用建议：

```
主力方案：VLESS + Reality（稳定隐蔽）


备用方案：Hysteria（速度优先）
组合策略：根据场景智能切换
```

```
场景切换：

  • 日常浏览 → VLESS + Reality



  • 视频下载 → Hysteria

  • 游戏加速 → Hysteria

  • 敏感时期 → VLESS + Reality
```

4.3 Clash内核与规则

4.3.1 Clash简介

Clash是一个基于规则的多协议代理客户端。

Clash家族：

```
Clash (原版)

  ├── Clash Premium (闭源增强版)
  │      ✓ TUN模式
  │     ✓ 规则增强
  │     ✓ 脚本支持
  │
  └── Clash.Meta (开源增强版)
       ✓ 完全开源
       ✓ 更多协议支持
       ✓ 性能优化
```

支持的协议：

- Shadowsocks / ShadowsocksR
- VMess / VLESS
- Trojan / Trojan-Go

- Snell
- SOCKS5 / HTTP(S)
- WireGuard (Meta)

4.3.2 Clash配置文件结构

基本配置框架：

```
# Clash配置文件 (config.yaml)
# 端口配置
port: 7890              # HTTP代理端口
socks-port: 7891        # SOCKS5代理端口
mixed-port: 7892        # 混合端口
allow-lan: false        # 允许局域网连接
bind-address: "*"       # 绑定地址
mode: rule              # 运行模式: rule/global/direct
log-level: info         # 日志级别

# DNS配置
dns:
  enable: true
  listen: 0.0.0.0:53
  enhanced-mode: fake-ip
  nameserver:
    - 223.5.5.5
    - 119.29.29.29
  fallback:
    - 8.8.8.8
    - 1.1.1.1

# 代理节点
```

```
proxies:
  - name: "节点1"
    type: ss
    server: server1.com
    port: 8388
    cipher: chacha20-ietf-poly1305
    password: "password"



  - name: "节点2"
    type: vmess
    server: server2.com
    port: 443
    uuid: your-uuid
    alterId: 0
    cipher: auto
    tls: true
```

代理组

```
proxy-groups:
  - name: "[新] 代理选择"
    type: select
    proxies:
     - "♻️ 自动选择"
     - "节点1"
     - "节点2"



  - name: "♻️ 自动选择"
    type: url-test
    proxies:
     - "节点1"
     - "节点2"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
```

规则

```
rules:
  - DOMAIN-SUFFIX,google.com,[新] 代理选择
  - DOMAIN-KEYWORD,youtube,[新] 代理选择
  - GEOIP,CN,DIRECT
  - MATCH,[新] 代理选择
```

4.3.3 代理组类型详解

1. select（手动选择）

```
proxy-groups:


  - name: "[新] 节点选择"
    type: select
    proxies:
     - "香港节点"
     - "美国节点"
     - "日本节点"



用途：手动切换节点
场景：看不同地区Netflix、游戏切换
```

2. url-test（自动选择）

```
proxy-groups:


  - name: "♻️ 自动选择"
    type: url-test
    proxies:
     - "节点A"
     - "节点B"
     - "节点C"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    tolerance: 150
```

```
原理:
定期测试所有节点延迟
自动选择最快的节点
延迟差<tolerance时不切换（避免频繁切换）
```

3. fallback（故障转移）

```
proxy-groups:


  - name: "[网络] 故障转移"
    type: fallback
    proxies:
     - "主节点"
     - "备用节点1"
     - "备用节点2"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300



原理:
按顺序测试节点
使用第一个可用的节点
主节点故障时自动切换到备用
主节点恢复后自动切回
```

4. load-balance（负载均衡）

```
proxy-groups:


  - name: "⚖️ 负载均衡"
    type: load-balance
    proxies:
     - "节点1"
     - "节点2"
     - "节点3"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    strategy: consistent-hashing  # 或 round-robin
```

```
策略:
consistent-hashing: 同一域名总是走同一节点（保持会话）
round-robin: 轮流使用各节点（分散负载）
```

4.3.4 规则类型详解

规则语法：

```
规则类型,匹配内容,策略

示例:
DOMAIN-SUFFIX,google.com,Proxy
```

常用规则类型：

```
# 1. 域名规则
  - DOMAIN,www.google.com,Proxy          # 完整匹配


  - DOMAIN-SUFFIX,google.com,Proxy       # 后缀匹配（含子域名）

  - DOMAIN-KEYWORD,google,Proxy          # 关键词匹配


# 2. IP规则


  - IP-CIDR,192.168.0.0/16,DIRECT        # IP段


  - IP-CIDR6,2001:db8::/32,DIRECT        # IPv6段

  - GEOIP,CN,DIRECT                       # 地理位置（中国IP直连）
```

3. 端口规则

```
- DST-PORT,80,DIRECT                    # 目标端口
- SRC-PORT,7777,DIRECT                  # 源端口
```

4. 进程规则（Clash Premium）

```
- PROCESS-NAME,chrome.exe,Proxy         # Windows
- PROCESS-NAME,telegram,Proxy           # Linux/Mac
```

5. 规则集

```
- RULE-SET,google,Proxy                 # 引用规则集
```

6. 最终规则

```
- MATCH,Proxy                           # 兜底规则（匹配所有）
```

规则集（Rule Providers）：

```
rule-providers:


  google:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/google.txt"
    path: ./ruleset/google.yaml
    interval: 86400



rules:
  - RULE-SET,google,Proxy
```

4.3.5 实用配置案例

完整的分流配置：

```
proxy-groups:


  - name: "[新] 节点选择"
    type: select
    proxies:
     - "♻️ 自动选择"
     - "🇭🇰 香港节点"
     - "🇺🇸 美国节点"
     - "🇯🇵 日本节点"
      - "DIRECT"



  - name: "♻️ 自动选择"
    type: url-test
    use:
      - all
    url: 'http://www.gstatic.com/generate_204'
    interval: 300



  - name: "🎬 流媒体"
    type: select
    proxies:
```

```
      - "🇭🇰 香港节点"
      - "🇺🇸 美国节点"
      - "🇯🇵 日本节点"



  - name: "📲 Telegram"
    type: select
    proxies:
      - "[新] 节点选择"
      - "🇭🇰 香港节点"



  - name: "🍎 Apple"
    type: select
    proxies:
      - "DIRECT"
      - "[新] 节点选择"



  - name: "[目标] 国内直连"
    type: select
    proxies:
      - "DIRECT"



rules:
  # 本地网络
  - DOMAIN-SUFFIX,local,DIRECT
  - IP-CIDR,127.0.0.0/8,DIRECT
  - IP-CIDR,192.168.0.0/16,DIRECT



  # Telegram
  - DOMAIN-SUFFIX,t.me,📲 Telegram
  - DOMAIN-SUFFIX,telegram.org,📲 Telegram
  - IP-CIDR,91.108.0.0/16,📲 Telegram
  - IP-CIDR,149.154.160.0/20,📲 Telegram



  # 流媒体
  - DOMAIN-SUFFIX,netflix.com,🎬 流媒体
  - DOMAIN-SUFFIX,youtube.com,🎬 流媒体
  - DOMAIN-KEYWORD,disney,🎬 流媒体
```

```
  # Apple服务
  - DOMAIN-SUFFIX,apple.com,🍎 Apple
  - DOMAIN-SUFFIX,icloud.com,🍎 Apple



 # 国内直连
  - DOMAIN-SUFFIX,cn,[目标] 国内直连
  - GEOIP,CN,[目标] 国内直连



 # 兜底规则
  - MATCH,[新] 节点选择
```

4.4 WireGuard现代VPN

4.4.1 WireGuard优势

在第3章我们简要介绍了WireGuard，这里深入探讨其在科学上网中的应用。

性能对比（实测数据）：

```
测试环境: 1Gbps带宽, 100ms延迟

OpenVPN (TCP):     ~200 Mbps
OpenVPN (UDP):     ~300 Mbps
IPsec/IKEv2:       ~450 Mbps
WireGuard:         ~950 Mbps



CPU占用:
OpenVPN:  15-20%
WireGuard: 3-5%
```

4.4.2 WireGuard配置实战

服务器配置 (wg0.conf)：

```
[Interface]
```

服务器私钥

```
PrivateKey = SERVER_PRIVATE_KEY
```

服务器虚拟IP

```
Address = 10.0.0.1/24
```

监听端口

```
ListenPort = 51820
```

启动后执行的命令（配置NAT）

```
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o eth0 -j MASQU
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o eth0 -j MAS
```

客户端1

```
[Peer]
PublicKey = CLIENT1_PUBLIC_KEY
AllowedIPs = 10.0.0.2/32
```

客户端2

```
[Peer]
PublicKey = CLIENT2_PUBLIC_KEY
AllowedIPs = 10.0.0.3/32
```

客户端配置：

```
[Interface]
# 客户端私钥



PrivateKey = CLIENT_PRIVATE_KEY

# 客户端虚拟IP



Address = 10.0.0.2/24

# 客户端DNS



DNS = 1.1.1.1, 8.8.8.8



[Peer]

# 服务器公钥



PublicKey = SERVER_PUBLIC_KEY
```

```
# 服务器地址和端口

Endpoint = your-server-ip:51820

# 允许的IP（0.0.0.0/0表示全局流量）

AllowedIPs = 0.0.0.0/0

# 保持连接（NAT穿透）

PersistentKeepalive = 25
```

生成密钥对：

```
# 生成私钥

wg genkey > privatekey

# 从私钥生成公钥

cat privatekey | wg pubkey > publickey

# 查看密钥
```

```
cat privatekey
cat publickey
```

启动服务：

```
# 启动WireGuard

wg-quick up wg0

# 查看状态


wg show

# 停止


wg-quick down wg0

# 开机自启


systemctl enable wg-quick@wg0
```

4.4.3 分流配置（部分流量走WireGuard）

仅代理国外流量：

```
[Interface]
```

```
PrivateKey = CLIENT_PRIVATE_KEY
Address = 10.0.0.2/24
DNS = 1.1.1.1

# 路由表编号

Table = 12345

[Peer]
PublicKey = SERVER_PUBLIC_KEY
Endpoint = your-server-ip:51820

# 仅代理国外IP

AllowedIPs = 0.0.0.0/1, 128.0.0.0/1
PersistentKeepalive = 25

# 添加自定义路由

PostUp = ip rule add table 12345
PostUp = ip route add default via 10.0.0.1 table 12345
PostDown = ip rule delete table 12345
```

4.4.4 WireGuard与科学上网

优势：

- [完成] 速度快，延迟低
- [完成] 电池友好（移动设备）
- [完成] 自动重连（IP变化）

- [完成] 代码简洁安全

劣势：

- [禁止] 流量特征明显（易识别为VPN）
- [禁止] 不支持混淆伪装
- [禁止] 在某些地区可能被封锁

适用场景：

```
推荐使用：


✓ 网络环境宽松地区
✓ 追求极致性能
✓ 移动设备（省电）
✓ 游戏加速（低延迟）



不推荐：
✗ GFW严格地区
✗ 需要高度隐蔽性
✗ ISP限制VPN协议
```

4.5 其他协议对比

4.5.1 协议综合对比

协议	速度	安全性	隐蔽性	配置难度	推荐度	备注
VLESS+Reality	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[完成][完成][完成] **最推荐**	最佳选择
Trojan	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[完成][完成] **强烈推荐**	极致伪装
Hysteria	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[完成][完成] **特定场景推荐**	弱网环境优选
VLESS	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[完成][完成] **强烈推荐**	配合TLS使用
VMess	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]	[注意]易被检测
Shadowsocks	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[注意]直连易识别
ShadowsocksR	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐]	更新较少
WireGuard	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐]	特征明显

4.5.2 选择建议

新手入门：

```
推荐顺序：

  • 购买机场服务（支持Clash订阅）



  • 自建Trojan（一键脚本）

  • 自建VLESS + Reality（最佳隐蔽性）





[注意] 重要提醒：
避免使用纯Shadowsocks或VMess直连
优先选择带混淆/伪装的方案
```

追求性能：

```
最佳组合：

VLESS + Reality + gRPC
或
Hysteria（高带宽场景）



性能排序：

  • Hysteria（弱网环境）



  • VLESS + XTLS（一般环境）

  • WireGuard（网络环境允许时）
```

追求隐蔽：

```
最佳方案：

首选：VLESS + Reality（无需域名证书）
次选：Trojan + 真实网站（需域名证书）
备选：VMess + WebSocket + TLS + CDN



隐蔽性排序：

  • VLESS + Reality [推荐][推荐][推荐][推荐][推荐]



  • Trojan + 真实网站 [推荐][推荐][推荐][推荐][推荐]

  • 其他协议 + 混淆 [推荐][推荐][推荐]
```

综合方案：

```
主力协议：VLESS + Reality（日常使用）

备用协议：Hysteria（高速场景）
应急备份：Shadowsocks + 插件混淆
客户端：Clash Meta / sing-box（规则分流）



多协议策略：

  • 平时：VLESS + Reality（稳定隐蔽）



  • 看视频：Hysteria（高速传输）

  • 敏感期：多协议轮换使用
```

4.5.3 协议组合策略

多协议部署：

```
单台VPS部署多个协议：


443端口：Trojan (HTTPS伪装)
    ↓
Nginx反向代理
    ├─► /trojan → Trojan服务
    ├─► /vmess  → V2Ray VMess
    ├─► /vless  → V2Ray VLESS
    └─► /      → 真实网站



优势：
✓ 灵活切换
✓ 互为备份
✓ 资源利用最大化
```

本章小结

本章详细介绍了主流代理协议的原理和配置：

核心要点：

- **Shadowsocks**：轻量快速，适合入门
- **V2Ray/Xray**：功能强大，灵活组合
- **Trojan**：极致伪装，难以检测
- **Clash**：规则分流，智能代理
- **WireGuard**：现代VPN，性能卓越

协议选择原则：

- 网络环境宽松 → WireGuard / Shadowsocks
- 封锁严重地区 → Trojan / VMess+WS+TLS
- 追求便利性 → 机场服务 + Clash

- 技术爱好者 → 自建V2Ray/Xray

最佳实践：

```
客户端：Clash（规则管理）


主协议：Trojan / VLESS（隐蔽性）
备用：Shadowsocks（轻量备份）
传输：WebSocket + TLS + CDN（稳定性）
```

下一章我们将学习各平台客户端工具的具体使用方法。

实践任务：

- 尝试配置一个Shadowsocks服务器
- 理解Clash配置文件结构
- 对比不同协议的速度和稳定性
- 编写自己的分流规则

延伸阅读：

- Shadowsocks协议规范
- V2Ray配置指南
- Trojan协议文档
- WireGuard白皮书

第五章 客户端工具使用指南

第5章 客户端工具使用指南

掌握了代理协议的原理后，本章将介绍各个平台上常用的客户端工具。这些工具将协议转化为易用的图形界面，让科学上网变得简单。

5.1 Windows平台

5.1.1 Clash 客户端与替代方案

[注意] 提醒：

```
Clash for Windows (CFW) 已停止维护!

状态:
[禁止] 项目仓库已删除
[禁止] 不再接受更新
[禁止] 可能存在安全隐患

影响:

  • 旧版本仍可使用，但不推荐

  • 无法获得新功能和安全补丁

  • 建议尽快迁移到替代方案
```

推荐替代方案：

1. Clash Verge（首选推荐）

软件介绍：

- CFW的完美替代品
- 基于Clash Meta内核
- 支持Windows/Linux/macOS
- 开源免费，持续维护

下载与安装：

- 官方地址：https://github.com/clash-verge-rev/clash-verge-rev
- 下载文件：Clash.Verge_x.x.x_x64-setup.exe

优势：

- ✓ 界面美观，操作类似CFW
- ✓ 内置中文语言
- ✓ 支持TUN模式
- ✓ 支持脚本增强
- ✓ 自动更新订阅

基本使用：

```
1. 安装后首次启动，会自动下载内核
  • 订阅 → 新建 → 粘贴订阅链接
  • 点击更新订阅
  • 代理 → 开启系统代理
  • 选择代理模式（规则/全局/直连）
  • 选择节点
[完成] 完全兼容CFW的配置文件！
```

2. FlClash（轻量化选择）

特点：

```
✓ Material You设计风格


✓ 跨平台（Windows/Android/Linux）
✓ 支持Shadowsocks、VMess、Trojan等
✓ 界面简洁易用
```

下载地址：

- https://github.com/chen08209/FlClash

3. Mihomo Party（功能增强版）

特点：

```
✓ 基于Mihomo内核


✓ 支持多种协议
✓ 规则管理强大
✓ 性能优秀
```

下载地址：

- https://github.com/pompurin404/mihomo-party

4. NekoRay / sing-box（进阶方案）

NekoRay：

- ✓ Qt框架，界面友好
- ✓ 支持V2Ray/Xray/sing-box
- ✓ 跨平台
- 下载：https://github.com/MatsuriDayo/nekoray

sing-box：

- ✓ 通用代理平台
- ✓ 支持所有主流协议
- ✓ 高性能

- 下载：https://github.com/SagerNet/sing-box

CFW迁移指南：

```
如果您正在使用CFW：
  • 导出配置
    - 备份订阅链接
    - 导出配置文件（如果需要）
  • 安装替代客户端
    - 推荐：Clash Verge
    - 备选：FlClash / Mihomo Party
  • 导入配置
    - 直接粘贴订阅链接
    - 或导入配置文件
  • 测试连接
    - 验证代理是否正常工作
  • 卸载CFW（可选）
```

以下为CFW旧版本使用说明（仅供参考，不推荐继续使用）

<details> <summary>点击查看Clash for Windows旧版说明</summary>

<h3>Clash for Windows (CFW) - 已停止维护</h3>

软件介绍： <ul> <li>基于Clash和Electron开发</li> <li>图形化界面，易于使用</li> <li>支持多种协议和订阅</li> <li>曾是Windows平台最受欢迎的客户端</li> </ul>

下载与安装： <pre><code>[注意] 警告：官方仓库已删除，请使用替代方案！

旧版下载（不推荐）：

- 从第三方镜像站下载（有安全风险）
- 或使用替代品

</code></pre>

配置方法参考：

步骤1：导入订阅 <ol> <li>获取订阅链接（从机场或自建服务器）</li> <li>打开CFW → Profiles → 粘贴订阅链接</li> <li>点击Download下载配置</li> <li>配置自动加载</li> </ol>

步骤2：启用系统代理 <ol> <li>主界面 → System Proxy开关打开</li> <li>选择代理模式： <ul> <li>Rule（规则模式）：推荐，智能分流</li> <li>Global（全局模式）：所有流量走代理</li> <li>Direct（直连模式）：不走代理</li> </ul> </li> </ol>

步骤3：选择节点 <ol> <li>Proxies标签页</li> <li>在"[新] 节点选择"中选择具体节点</li> <li>或选择"♻️ 自动选择"让系统自动选最快节点</li> </ol>

高级功能：

TUN模式（接管所有流量）： <ul> <li>适用场景： <ul> <li>✓ 游戏加速（不支持系统代理的应用）</li> <li>✓ 全局接管网络</li> <li>✓ 避免DNS泄漏</li> </ul> </li> </ul> 开启方式：
 Settings → General → TUN Mode → 启用
 需要管理员权限运行

Mixin配置（混合配置）： <pre><code class="language-yaml"># 在Mixin标签页添加自定义配置

会与订阅配置合并

mixin: dns: enable: true enhanced-mode: fake-ip nameserver: - 223.5.5.5 - 119.29.29.29 fallback: - 8.8.8.8 - 1.1.1.1 </code></pre>

常见问题： <pre><code>问题1：启动后无法上网 解决：检查系统代理是否正确设置 检查防火墙是否拦截

问题2：订阅更新失败 解决：检查订阅链接是否有效 尝试使用VPN/其他节点更新

问题3：速度慢 解决：切换不同节点 使用延迟测试选择最快节点 检查是否开启了不必要的规则 </code></pre>

故障排查提示： 以上问题适用于大多数Clash系列客户端。

</details>

5.1.2 V2RayN

软件特点：

- 专注于V2Ray/Xray协议
- 轻量级，资源占用少
- 配置灵活，功能强大
- 适合高级用户

下载安装：

- 官方地址：https://github.com/2dust/v2rayN
- 下载：v2rayN-windows-64.zip（含v2ray-core）或 v2rayN.zip（需单独下载core）
- 解压后直接运行 v2rayN.exe

配置方法：

方法1：订阅导入

```
1. 订阅 → 订阅设置
   - 添加订阅地址
   - 确定 → 更新订阅
   - 服务器列表自动导入
```

方法2：手动添加

```
1. 服务器 → 添加VMess/VLESS/Shadowsocks服务器
   - 填写服务器参数：
```

```
  - 地址（域名或IP）

  - 端口
  - 用户ID（UUID）
  - 加密方式
  - 传输协议（TCP/WebSocket等）

• 确定保存
```

方法3：扫描二维码/导入链接

```
1. 服务器 → 从剪贴板导入批量URL
• 或使用扫码功能（需要摄像头）
```

使用技巧：

路由规则设置：

```
设置 → 路由设置

预设方案：

• 绕过大陆：国内直连，国外代理（推荐）

• 全局：所有流量走代理

• 绕过局域网及大陆：更精细的分流

自定义规则：

• 可以添加自定义域名/IP规则
```

```
  • 支持导入外部规则文件
```

测速与排序：

```
选中多个服务器 → 右键 → 测试服务器延迟（Tcping）

服务器 → 按延迟排序
选择延迟最低的节点
```

核心类型切换：

```
设置 → Core类型设置

选项：

  • v2fly (v2ray-core)：稳定版


  • Xray-core：性能增强版（推荐）

  • SagerNet：移动端核心



[完成] 推荐使用 Xray-core
```

5.1.3 Qv2ray

软件特点：

- 跨平台（Windows/Linux/macOS）
- 模块化设计
- 插件系统

- 高度可定制

下载安装：

- 官方地址：https://github.com/Qv2ray/Qv2ray
- 下载 Qv2ray 安装包和 v2ray-core
- 首次启动需要配置 core 路径

核心配置：

1. 首选项设置

```
首选项 → 常规设置

入站设置：

  • SOCKS端口：10808

  • HTTP端口：10809

  • 监听地址：127.0.0.1

连接设置：

  • 启用统计

  • 启用自动连接
```

2. 添加服务器

```
方式1：订阅
```

```
分组 → 订阅管理 → 添加订阅
输入订阅地址 → 更新订阅



方式2: 手动配置
新建 → 手动输入 → 选择协议类型
填写服务器参数



方式3: 导入
导入 → 从链接导入 / 从JSON导入
```

插件推荐:

```
QvPlugin-Trojan：Trojan协议支持


QvPlugin-SS：Shadowsocks支持
QvPlugin-NaiveProxy：NaiveProxy支持
QvPlugin-Command：自定义命令



安装: 首选项 → 插件 → 浏览插件目录
将插件文件放入后重启
```

5.1.4 Windows平台对比

特性	Clash Verge	FlClash	V2RayN	Qv2ray
易用性	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]
界面美观	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
协议支持	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]
规则功能	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
资源占用	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
维护状态	[完成] 活跃	[完成] 活跃	[完成] 活跃	[注意] 较少
适合人群	新手/一般用户	新手/追求简洁	中级用户	高级用户

推荐选择：

```
首选推荐：Clash Verge
  • CFW的完美替代
  • 界面友好，上手简单
  • 规则强大，自动分流
  • 订阅管理方便
  • 持续更新维护
```

```
简洁推荐: FlClash

  • 界面美观现代

  • 跨平台支持

  • 轻量快速


进阶推荐: V2RayN

  • 轻量快速

  • V2Ray全功能支持

  • 配置灵活


极客推荐: Qv2ray

  • 跨平台

  • 插件扩展

  • 高度定制
```

```
[注意] 重要：不再推荐Clash for Windows，请选择上述替代方案！
```

5.2 macOS平台

5.2.1 ClashX / ClashX Pro

软件特点：

- macOS原生应用
- 基于Clash核心
- 菜单栏常驻，操作便捷
- Pro版本支持TUN模式

下载安装：

```
ClashX：https://github.com/yichengchen/clashX

ClashX Pro：https://install.appcenter.ms/users/clashx/apps/clashx-pro/distribution_groups/publi



安装后需要授权：
系统偏好设置 → 安全性与隐私 → 允许
```

基本使用：

导入配置：

```
方法1：订阅

菜单栏ClashX图标 → 配置 → 托管配置 → 管理
添加订阅URL → 更新



方法2：本地文件
配置 → 打开配置文件夹
将config.yaml复制到该文件夹
重新加载配置
```

设置代理：

```
菜单栏ClashX → 设置为系统代理（勾选）

代理模式：

  • 规则判断：根据规则自动分流（推荐）

  • 全局连接：所有流量走代理

  • 直接连接：不使用代理

  • 脚本：使用JavaScript脚本判断
```

增强模式（ClashX Pro）：

```
菜单栏 → 增强模式

功能：
✓ TUN设备，接管所有流量
✓ 不需要系统代理设置
✓ 支持UDP转发
✓ 真正的透明代理

使用场景：

  • 游戏加速

  • 非HTTP应用

  • 完全接管网络
```

实时日志：

```
菜单栏 → 控制台

显示：

  • 连接日志



  • 规则匹配情况

  • DNS查询

  • 流量统计




调试必备功能
```

5.2.2 Surge

软件特点：

- macOS/iOS平台的高端工具
- 功能极其强大
- 价格较高（$49.99-$99.99）
- 专业用户首选

核心功能：

1. 强大的规则引擎

支持规则类型：

- DOMAIN/DOMAIN-SUFFIX/DOMAIN-KEYWORD
- IP-CIDR/IP-CIDR6/GEOIP
- USER-AGENT/URL-REGEX
- PROCESS-NAME（进程匹配）

- RULE-SET（规则集）
- SCRIPT（JavaScript脚本）

2. 模块系统

模块作用：

- 重写规则（广告屏蔽）
- MITM解密HTTPS
- 脚本增强
- 可在线订阅模块

```
# 示例：去广告模块

[Rule]
DOMAIN-SUFFIX,ad.example.com,REJECT
URL-REGEX,^https?://.*(ad|ads)/.*,REJECT
```

3. 脚本功能

```
// 示例：修改响应

let body = $response.body;
body = body.replace(/ads/g, '');
$done({body});
```

用途：

- 修改请求/响应
- 自动签到
- 去除广告
- 数据提取

配置示例：

```
[General]

loglevel = notify
dns-server = 223.5.5.5, 119.29.29.29
skip-proxy = 127.0.0.1, 192.168.0.0/16, localhost
```

```
[Proxy]
Proxy1 = ss, server.com, 8388, encrypt-method=chacha20-ietf-poly1305, password=pwd
Proxy2 = vmess, server.com, 443, username=uuid, tls=true



[Proxy Group]
Proxy = select, Proxy1, Proxy2, Auto
Auto = url-test, Proxy1, Proxy2, url=http://www.gstatic.com/generate_204



[Rule]
DOMAIN-SUFFIX,google.com,Proxy
DOMAIN-KEYWORD,youtube,Proxy
GEOIP,CN,DIRECT
FINAL,Proxy
```

5.2.3 V2RayX / V2RayU

V2RayX：

- 老牌V2Ray客户端
- 功能基础但稳定
- 适合简单使用

V2RayU：

- V2RayX的增强版
- 界面更美观
- 功能更丰富
- 支持订阅

下载地址：

- V2RayX：https://github.com/Cenmrev/V2RayX
- V2RayU：https://github.com/yanue/V2rayU

基本使用：

```
1. 菜单栏图标 → Configure
   - 添加服务器或导入订阅
```

```
• 选择服务器
• Load Core（加载内核）
• PAC模式或全局模式
```

5.2.4 macOS平台推荐

需求	推荐工具	理由
一般使用	ClashX	免费、易用、功能够用
游戏/全局	ClashX Pro	TUN模式，性能好
专业需求	Surge	功能最强，脚本支持
V2Ray专用	V2RayU	轻量，V2Ray全功能

5.3 Android平台

5.3.1 Clash for Android (CFA)

软件特点：

- 基于Clash内核
- 开源免费
- 支持规则和订阅
- Android平台最流行

下载安装：

```
官方：https://github.com/Kr328/ClashForAndroid

下载：cfa-x.x.x-premium-universal-release.apk
```

```
或通过Google Play搜索"Clash for Android"
```

配置步骤：

1. 导入配置

```
方法1：URL导入

配置 → 新配置 → URL → 输入订阅链接 → 保存



方法2：文件导入
配置 → 新配置 → 文件 → 选择config.yaml



方法3：扫码导入（如果订阅提供二维码）
```

2. 启动代理

```
首页 → 点击开关启动

首次使用需要：

  • 同意VPN连接请求



  • 授予网络权限
```

3. 选择节点

```
代理 → 选择代理组
  • 手动选择具体节点
```

```
  • 或使用自动选择/负载均衡
```

高级设置：

访问控制：

```
设置 → 网络 → 访问控制

可以设置：

  • 仅代理指定应用（白名单模式）

  • 不代理指定应用（黑名单模式）

示例：仅代理Chrome和Telegram
勾选"访问控制模式"
选择"允许选中应用"
勾选Chrome、Telegram
```

DNS设置：

```
设置 → 网络 → DNS

推荐设置：
DNS模式：fake-ip（性能最好）
或 redir-host（兼容性好）

自定义DNS服务器：
223.5.5.5
119.29.29.29
```

5.3.2 V2RayNG

软件特点：

- V2Ray官方推荐客户端
- 支持所有V2Ray协议
- 轻量快速
- 免费开源

下载安装：

```
GitHub：https://github.com/2dust/v2rayNG

Google Play：搜索"v2rayNG"



下载v2rayNG_x.x.x_arm64-v8a.apk
```

使用教程：

添加服务器：

```
方法1：扫描二维码

首页 → 右上角 + → 扫描二维码



方法2：从剪贴板导入
复制vmess://或vless://链接
打开应用自动识别
或手动点击 + → 从剪贴板导入



方法3：订阅
右上角 ≡ → 订阅设置 → + 添加
输入订阅URL → 确定
返回首页下拉更新订阅
```

启动连接：

```
选择服务器 → 点击右下角V图标

授权VPN连接
连接成功后状态栏显示钥匙图标
```

路由设置：

```
左侧菜单 → 设置 → 路由设置

预定义规则：

  • 绕过局域网及大陆地址

  • 绕过大陆地址

  • 绕过局域网

  • 全局

自定义：
可以添加自定义域名和IP规则
```

分应用代理：

```
设置 → 分应用代理

允许/禁止应用列表：
选择需要代理的应用
其他应用直连

适合：
```

```
  • 国内应用直连（微信、支付宝）


  • 国外应用代理（Google、Twitter）
```

5.3.3 SagerNet

软件特点：

- 基于V2Ray/Xray
- 支持多种协议
- 插件系统
- 性能优化

下载安装：

```
GitHub：https://github.com/SagerNet/SagerNet

Google Play：搜索"SagerNet"
```

特色功能：

1. 多种核心支持

```
内置核心：
  • V2Ray


  • Xray

  • Trojan-Go

  • NaiveProxy

  • Brook
```

```
通过插件支持:

  • Shadowsocks

  • ShadowsocksR

  • WireGuard
```

2. 路由规则

```
设置 → 路由

预设方案:

  • 绕过中国大陆

  • 仅代理中国大陆

  • 全局

自定义规则:
支持domain/geoip/cidr等
```

3. 配置格式

```
支持导入：
  • V2Ray JSON
  • Clash YAML
  • Shadowsocks URL
  • 订阅链接
```

5.3.4 Android平台对比

特性	Clash for Android	V2RayNG	SagerNet
易用性	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]
规则功能	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
协议支持	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]
性能	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
更新频率	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]

推荐：

```
一般用户：Clash for Android
  • 规则强大，自动分流
  • 界面美观，易于使用
```

```
技术用户：V2RayNG

  • 协议全面


  • 配置灵活

  • 性能优秀




折中选择：SagerNet

  • 功能丰富


  • 多核心支持

  • 插件扩展
```

5.4 iOS平台

5.4.1 Shadowrocket (小火箭)

软件特点：

- iOS平台最受欢迎
- 功能强大全面
- 规则丰富
- 需付费购买（$2.99）

购买与下载：

```
美区App Store搜索：Shadowrocket


需要美区Apple ID账号



购买注意：
[注意] 从官方App Store购买
[注意] 不要使用共享账号（有风险）
[注意] 认准开发者：Shadow Launch Technology Limited
```

配置方法：

1. 添加服务器

```
方法1：扫描二维码


首页 → 右上角 + → 类型：扫描二维码



方法2：手动添加
首页 → 右上角 + → 类型：选择协议
填写服务器信息



方法3：URL导入
复制ss://、vmess://等链接
打开应用自动识别



方法4：订阅
首页 → 右上角 + → 类型：Subscribe
粘贴订阅URL → 保存
下拉更新订阅
```

2. 启用代理

```
选择服务器（打钩）
```

```
首页顶部开关 → 打开
允许VPN配置
连接成功
```

规则配置：

使用远程规则：

```
配置 → 右上角 + → 添加配置

推荐规则:
https://raw.githubusercontent.com/Loyalsoldier/surge-rules/release/ruleset/proxy.txt



功能:

  • 自动分流国内外流量



  • 广告屏蔽

  • 隐私保护
```

自定义规则：

```
配置 → 编辑纯文本配置

[Rule]

# 直连规则


DOMAIN-SUFFIX,baidu.com,DIRECT
DOMAIN-SUFFIX,qq.com,DIRECT
```

```
# 代理规则

DOMAIN-SUFFIX,google.com,PROXY
DOMAIN-SUFFIX,youtube.com,PROXY
DOMAIN-KEYWORD,facebook,PROXY

# IP规则

IP-CIDR,192.168.0.0/16,DIRECT
GEOIP,CN,DIRECT

# 兜底

FINAL,PROXY
```

实用功能：

按需连接：

```
设置 → 按需连接

可设置：

  • WiFi下自动断开（家里网络）

  • 移动网络自动连接

  • 特定WiFi自动连接
```

全局路由：

```
设置 → 全局路由

选项：

  • 配置：使用规则分流

  • 代理：全局代理

  • 直连：全部直连

  • 场景：自动切换
```

5.4.2 Surge

软件特点：

- iOS/macOS平台最强大工具
- 专业级网络调试
- 价格较高（$49.99起）
- 适合专业用户

功能特性：

1. MITM（中间人抓包）

```
可以：
  • 解密HTTPS流量

  • 修改请求/响应

  • 调试API接口
```

```
  • 去除应用广告




设置:
Surge → MITM → 安装证书
设置 → 通用 → 关于 → 证书信任设置 → 启用
```

2. 模块系统

```
模块示例:

#!name=去广告模块
#!desc=屏蔽常见广告



[Rule]
DOMAIN-SUFFIX,ad.example.com,REJECT
URL-REGEX,^https?://.*(ad|ads).*,REJECT



[URL Rewrite]
^https?://api\.example\.com/ad - reject



使用:
配置 → 模块 → 安装新模块
输入模块URL或本地文件
```

3. 脚本支持

```
// 示例: 自动签到脚本

let url = "https://api.example.com/checkin";
$httpClient.post(url, (error, response, data) => {
  if (!error) {
    console.log("签到成功");
    $notification.post("签到", "", "已完成每日签到");
```

```
  }
  $done();
});
```

5.4.3 Quantumult X (圈X)

软件特点：

- 功能强大
- 规则灵活
- 重写功能
- 价格适中（$7.99）

核心功能：

1. 策略组

```
[policy]
# 手动选择

static=节点选择, 香港节点, 美国节点, 自动选择

# 自动选择（延迟最低）

url-latency-benchmark=自动选择, server-tag-regex=.*, check-interval=600

# 故障转移

available=故障转移, 节点1, 节点2, 节点3
```

2. 分流规则

```
[filter_remote]
# 远程规则


https://example.com/proxy.list, tag=代理规则, enabled=true


[filter_local]

# 本地规则


host-suffix, google.com, proxy
host-keyword, youtube, proxy
ip-cidr, 192.168.0.0/16, direct
geoip, cn, direct
final, proxy
```

3. 重写规则

```
[rewrite_remote]
# 去广告


https://example.com/adblock.conf, tag=去广告, enabled=true


[rewrite_local]

# 自定义重写


```

```
^https?://api\.example\.com/ad url reject-dict
^https?://.*\.jpg$ url reject-img
```

5.4.4 iOS平台对比

特性	Shadowrocket	Surge	Quantumult X
价格	$2.99	$49.99+	$7.99
易用性	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
功能	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]
性能	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]
适合人群	一般用户	专业用户	进阶用户

推荐：

```
新手/一般用户：Shadowrocket
  • 价格最低
  • 功能够用
  • 简单易上手

专业用户/开发者：Surge
  • 功能最强
```

```
  • 调试利器

  • MITM抓包




进阶用户: Quantumult X

  • 功能强大


  • 性价比高

  • 重写灵活
```

5.5 Linux平台配置

5.5.1 命令行工具

Shadowsocks-libev:

```
# Ubuntu/Debian安装

sudo apt install shadowsocks-libev

# 配置文件: /etc/shadowsocks-libev/config.json



{
  "server": "server_ip",
  "server_port": 8388,
  "local_address": "127.0.0.1",
  "local_port": 1080,
```

```
  "password": "your_password",
  "method": "chacha20-ietf-poly1305"
}

# 启动



ss-local -c /etc/shadowsocks-libev/config.json

# 开机自启



sudo systemctl enable shadowsocks-libev-local@config
sudo systemctl start shadowsocks-libev-local@config
```

V2Ray：

```
# 安装脚本

bash <(curl -L https://raw.githubusercontent.com/v2fly/fhs-install-v2ray/master/install-releas

# 配置文件：/usr/local/etc/v2ray/config.json

# 启动



sudo systemctl start v2ray

# 开机自启
```

```
sudo systemctl enable v2ray
```

Clash（已经停止维护）：

```
# 下载Clash（此地址已经无法访问）

wget https://github.com/Dreamacro/clash/releases/download/v1.x.x/clash-linux-amd64.tar.gz
tar -zxvf clash-linux-amd64.tar.gz
sudo mv clash /usr/local/bin/

# 下载配置

mkdir ~/.config/clash
wget -O ~/.config/clash/config.yaml "订阅链接"

# 运行

clash -d ~/.config/clash

# 设置系统代理

export http_proxy=http://127.0.0.1:7890
export https_proxy=http://127.0.0.1:7890
```

5.5.2 图形化工具

Qv2ray（推荐）：

```
# Arch Linux
```

```
sudo pacman -S qv2ray

# Ubuntu/Debian（AppImage）



wget https://github.com/Qv2ray/Qv2ray/releases/download/v2.x.x/Qv2ray-v2.x.x-linux-x64.AppImag
chmod +x Qv2ray-*.AppImage
./Qv2ray-*.AppImage
```

Clash for Windows（通过Wine）：

```
# 安装Wine

sudo apt install wine

# 运行Windows版CFW



wine Clash.for.Windows.exe
```

5.5.3 浏览器代理设置

系统级代理：

```
# 临时设置

export http_proxy=http://127.0.0.1:1080
export https_proxy=http://127.0.0.1:1080
export all_proxy=socks5://127.0.0.1:1080

# 永久设置（添加到~/.bashrc或~/.zshrc）
```

```
echo 'export http_proxy=http://127.0.0.1:1080' >> ~/.bashrc
echo 'export https_proxy=http://127.0.0.1:1080' >> ~/.bashrc
source ~/.bashrc
```

使用proxychains：

```
# 安装

sudo apt install proxychains

# 配置：/etc/proxychains.conf



socks5 127.0.0.1 1080

# 使用



proxychains curl google.com
proxychains firefox
```

5.6 路由器端配置

5.6.1 支持的路由器固件

OpenWrt：

- 开源路由器固件
- 插件丰富
- 最广泛支持

Merlin（梅林）：

- 基于华硕官方固件

- 稳定性好
- 适合华硕路由器

Padavan（老毛子）：

- 针对MT7620/MT7621芯片
- 内置Shadowsocks
- 适合小米、斐讯等

5.6.2 OpenWrt安装代理

安装Shadowsocks：

```
# 更新软件包

opkg update

# 安装shadowsocks-libev



opkg install shadowsocks-libev-ss-redir
opkg install shadowsocks-libev-ss-tunnel

# 安装luci界面



opkg install luci-app-shadowsocks-libev
```

安装Clash：

```
# 安装OpenClash插件
# 通过luci界面上传ipk包或在线安装



opkg install luci-app-openclash
```

```
访问：路由器IP → 服务 → OpenClash
```

配置方法：

```
1. 导入订阅链接
   • 设置运行模式（全局/规则/直连）
   • 配置DNS（推荐使用国内外分流DNS）
   • 启用代理
   • 连接路由器WiFi的所有设备自动代理
```

5.6.3 路由器代理优势

优点：

```
✓ 所有设备自动翻墙（手机、电脑、电视、游戏机）
✓ 无需每台设备单独配置
✓ 智能分流（国内直连，国外代理）
✓ 全家共享，一次配置永久使用
✓ 支持不能安装软件的设备（智能电视、PS5等）
```

缺点：

```
✗ 需要刷机（有风险）
✗ 配置较复杂
✗ 性能受路由器硬件限制
✗ 故障影响全家网络
```

本章小结

本章详细介绍了各平台的客户端工具使用方法：

平台推荐：

平台	推荐工具	备选方案	适合人群
Windows	Clash Verge	FlClash / V2RayN	新手、一般用户
macOS	ClashX Pro / Clash Verge	Surge / V2RayU	一般/专业用户
Android	Clash Meta for Android	V2RayNG / SagerNet	大多数用户
iOS	Shadowrocket	Surge / Quantumult X	性价比之选
Linux	Clash Verge / Qv2ray	v2rayA / Clash	命令行/图形界面
路由器	OpenClash	Shadowsocks	全家使用

重要说明：

```
生态系统变化：
  • Clash for Windows 已停止维护

    建议迁移至 Clash Verge 等活跃维护的项目
  • Clash Meta 成为新一代主流内核

    提供更好的性能和更广泛的协议支持
  • sing-box 等新兴通用代理平台崭露头角

    支持几乎所有主流代理协议
  • 移动端生态持续活跃

    各平台客户端保持积极更新迭代
```

使用建议：

```
入门：选择图形化客户端 + 订阅服务
```

```
进阶：学习规则配置，优化分流
高级：路由器部署，全家共享
```

下一章我们将学习浏览器代理配置和隐私保护技巧。

实践任务：

- 在你的平台上安装推荐的客户端
- 导入订阅或配置服务器
- 测试连接和访问效果
- 尝试自定义分流规则

第六章 浏览器与插件方案

第6章 浏览器与插件方案

除了系统级的代理客户端，浏览器插件也是实现科学上网的重要方式。本章将介绍浏览器代理设置和常用插件的使用方法。

6.1 浏览器代理设置

6.1.1 Chrome浏览器代理

系统代理模式：

Chrome默认使用系统代理设置。

```
Windows：

设置 → 系统 → 代理
手动代理设置 → 输入代理服务器地址和端口



macOS：
系统偏好设置 → 网络 → 高级 → 代理
勾选"网页代理(HTTP)"和"安全网页代理(HTTPS)"
输入代理服务器：127.0.0.1:7890



Linux：
设置 → 网络 → 网络代理 → 手动
HTTP代理：127.0.0.1:7890
HTTPS代理：127.0.0.1:7890
```

命令行启动（临时代理）：

```
# Windows

chrome.exe --proxy-server="socks5://127.0.0.1:1080"
```

```
macOS
/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome --proxy-server="socks5://127.0.
Linux
google-chrome --proxy-server="socks5://127.0.0.1:1080"
参数说明:
--proxy-server="http://127.0.0.1:7890"    # HTTP代理
--proxy-server="socks5://127.0.0.1:1080"  # SOCKS5代理
```

快捷方式设置:

```
Windows:
  • 右键Chrome快捷方式 → 属性
  • 目标栏添加: --proxy-server="socks5://127.0.0.1:1080"
    完整示例: "C:\...\chrome.exe" --proxy-server="socks5://127.0.0.1:1080"
  • 应用 → 确定
```

```
优势：
✓ 仅浏览器走代理
✓ 不影响系统其他应用
✓ 方便切换（创建多个快捷方式）
```

6.1.2 Firefox浏览器代理

内置代理设置：

Firefox有独立的代理设置，不使用系统代理。

```
设置步骤：
  • 菜单 → 设置 → 常规



  • 滚动到底部"网络设置" → 设置

  • 选择"手动代理配置"

  • 配置代理：



    HTTP代理：127.0.0.1  端口：7890

    SOCKS主机：127.0.0.1  端口：1080
    SOCKS v5：勾选

  • 勾选"使用SOCKS v5时代理DNS查询"（重要！）



  • 不代理：localhost, 127.0.0.1

  • 确定
```

配置选项说明：

选项	说明	推荐
不使用代理	直接连接	国内网络
自动检测代理	自动探测PAC	不推荐
使用系统代理	跟随系统设置	推荐（配合系统代理）
手动代理配置	手动指定代理	推荐（精确控制）
自动代理配置URL	使用PAC文件	高级用法

关键设置：

```
必须勾选：

☑ 使用SOCKS v5时代理DNS查询
☑ 为所有协议使用相同代理


作用：

  • 防止DNS泄漏


  • 统一代理配置
```

6.1.3 Edge浏览器代理

Edge基于Chromium，代理设置与Chrome类似。

```
方式1：使用系统代理

设置 → 系统和性能 → 打开计算机的代理设置


方式2：命令行启动
msedge.exe --proxy-server="socks5://127.0.0.1:1080"
```

```
方式3：使用扩展插件（见6.2节）
```

6.1.4 Safari浏览器代理

Safari使用系统代理设置。

```
macOS：

系统偏好设置 → 网络 → 高级 → 代理



配置：
☑ 网页代理(HTTP)
   服务器：127.0.0.1  端口：7890
☑ 安全网页代理(HTTPS)
   服务器：127.0.0.1  端口：7890
☑ SOCKS代理
   服务器：127.0.0.1  端口：1080



要忽略这些主机与域的代理设置：
*.local, 169.254/16, localhost, 127.0.0.1
```

6.2 SwitchyOmega配置详解

[注意] 重要安全警告：

```
重要安全提示：

近期，SwitchyOmega被曝存在安全风险事件：

  • 开发者遭遇钓鱼攻击



  • 插件被注入恶意代码

  • 可能窃取用户私钥和敏感信息
```

```
安全建议：
[完成] 确保使用最新安全版本
[完成] 仅从官方Chrome应用商店下载
[完成] 定期检查插件权限
[完成] 考虑使用替代方案：ZeroOmega



推荐替代：ZeroOmega


  • 兼容Manifest V3



  • 更好的安全性


  • 持续维护


  • 下载：https://github.com/zero-peak/ZeroOmega
```

6.2.1 SwitchyOmega简介

SwitchyOmega是什么？

- Chrome/Edge/Firefox扩展插件
- 管理和切换多个代理配置
- 支持自动切换规则
- 开源免费

安装方法：

```
方案A：SwitchyOmega（传统方案）


Chrome/Edge：


  • Chrome网上应用店搜索"Proxy SwitchyOmega"
```

- 或访问: https://chrome.google.com/webstore/detail/padekgcemlokbadohgkifijomclgjgif
- 添加到Chrome

[注意] 安全提示:

- 仅从官方应用商店安装
- 安装后检查权限设置
- 定期更新到最新版本

Firefox：

- Firefox附加组件市场搜索"SwitchyOmega"
- 或访问: https://addons.mozilla.org/zh-CN/firefox/addon/switchyomega/
- 添加到Firefox

方案B：ZeroOmega（推荐方案）

```
优势:
✓ 兼容Manifest V3
✓ 更好的安全性
✓ 持续维护
✓ 功能与SwitchyOmega类似



Chrome/Edge:


  • 访问: https://chromewebstore.google.com/detail/pfnededegaaopdmhkdmcofjmoldfiped



  • 搜索"Proxy SwitchyOmega 3 (ZeroOmega)"

  • 添加到浏览器





配置方法与SwitchyOmega完全相同!
```

6.2.2 基础配置教程

情景模式配置:

1. 新建代理情景模式

```
步骤:
  • 点击SwitchyOmega图标 → 选项



  • 左侧"新建情景模式"

  • 情景模式名称: proxy (或任意名称)

  • 类型: 代理服务器

  • 创建
```

```
配置：
代理协议：SOCKS5
代理服务器：127.0.0.1
代理端口：1080
（根据实际代理软件端口填写）



或使用HTTP代理：
代理协议：HTTP
代理服务器：127.0.0.1
代理端口：7890
```

2. 配置自动切换模式

```
步骤：

  • 左侧"auto switch"（自动切换）



  • 切换规则：





规则列表规则：


  • 条件类型：规则列表



  • 规则列表格式：AutoProxy


  • 规则列表网址：
```

```
  https://raw.githubusercontent.com/gfwlist/gfwlist/master/gfwlist.txt
  • 情景模式：proxy




  • 立即更新情景模式




默认情景模式：直接连接



规则列表规则：proxy
```

规则列表URL推荐：

```
GFWList（被墙网站列表）：

https://raw.githubusercontent.com/gfwlist/gfwlist/master/gfwlist.txt



Loyalsoldier规则（推荐）：
https://raw.githubusercontent.com/Loyalsoldier/v2ray-rules-dat/release/gfwlist.txt



CHNRoute（中国IP列表，反向规则）：
https://raw.githubusercontent.com/Loyalsoldier/v2ray-rules-dat/release/direct-list.txt
```

6.2.3 高级规则配置

自定义切换规则：

```
添加条件规则：
  • 域名通配符：


    条件类型：域名通配符
```

```
  条件设置: *.google.com
  情景模式: proxy


• 主机名正则:


  条件类型: 主机正则表达式


  条件设置: .*\.google\.com$
  情景模式: proxy


• 网址通配符:


  条件类型: 网址通配符


  条件设置: ://www.youtube.com/
  情景模式: proxy


• 网址正则:


  条件类型: 网址正则表达式


  条件设置: ^https?://.\.facebook\.com/.
  情景模式: proxy
```

实用规则示例:

```
代理国外网站:


*.google.com          → proxy
*.youtube.com         → proxy
*.facebook.com        → proxy
*.twitter.com         → proxy
*.instagram.com       → proxy
*.github.com          → proxy



直连国内网站:
*.baidu.com           → 直接连接
```

```
*.qq.com              → 直接连接
*.taobao.com          → 直接连接
*.jd.com              → 直接连接



本地地址直连:
127.0.0.1            → 直接连接
localhost              → 直接连接
192.168.*            → 直接连接
```

6.2.4 情景模式类型

1. 代理服务器

```
用途: 基础代理配置


配置: 单一代理服务器
适合: 简单使用场景
```

2. 自动切换

```
用途: 基于规则自动选择情景模式


配置: 规则列表 + 条件规则
适合: 日常使用（推荐）
```

3. PAC情景模式

```
用途: 使用PAC脚本


配置: JavaScript函数
适合: 高级用户



示例PAC脚本:
function FindProxyForURL(url, host) {
  if (shExpMatch(host, "*.google.com")) {
    return "SOCKS5 127.0.0.1:1080";
  }
```

```
  if (shExpMatch(host, "*.youtube.com")) {
    return "SOCKS5 127.0.0.1:1080";
  }
  return "DIRECT";
}
```

4. 虚拟情景模式

```
用途：组合其他情景模式


配置：按顺序尝试多个代理
适合：故障转移场景
```

6.2.5 快速切换技巧

使用方法：

```
点击浏览器工具栏SwitchyOmega图标：


选项：
○ 直接连接          # 不使用代理
○ 系统代理          # 使用系统代理设置
○ proxy             # 使用配置的代理
○ auto switch       # 自动切换模式（推荐）



快捷键：
可在选项中设置快捷键快速切换
```

指示器颜色：

```
🔵 蓝色：使用代理


⚪ 灰色：直接连接
🟢 绿色：自动切换（当前直连）
🟠 橙色：自动切换（当前代理）
```

6.2.6 导入导出配置

导出配置：

```
选项 → 导入/导出 → 生成备份文件


下载.bak或.json文件
保存配置，便于迁移
```

导入配置：

```
选项 → 导入/导出 → 从备份文件恢复


选择之前导出的文件
一键恢复所有配置
```

在线恢复（推荐）：

```
将配置文件上传到GitHub Gist


获取Raw链接
任何设备上：
选项 → 导入/导出 → 在线恢复
输入链接 → 恢复
```

6.2.7 安全使用建议

防范插件风险：

```
1. 从官方渠道下载：

   ✓ 仅使用Chrome/Firefox官方应用商店
   [禁止] 避免第三方网站下载


 • 谨慎授予权限：


   ✓ 安装时仔细阅读权限说明
```

```
  ✓ 定期检查已安装插件的权限
  [注意] 过度权限请求需警惕


• 定期检查更新:


  ✓ 保持插件为最新版本


  ✓ 关注安全通告
  ✓ 发现异常立即移除


• 使用监控工具:


  ✓ 使用浏览器开发者工具监控网络请求


  ✓ 定期扫描系统查找后门


• 如果发现插件被感染:


  [禁止] 立即移除插件


  [禁止] 更改所有相关账户密码
  [禁止] 扫描系统查找恶意软件
  [禁止] 监测账户活动
```

开发者建议:

```
1. 加强OAuth访问控制
  • 定期审计插件代码



  • 实时监测插件安全性


  • 使用双因素认证保护账户
```

替代方案推荐:

```
ZeroOmega：

✓ SwitchyOmega的分支版本
✓ 兼容Manifest V3
✓ 更好的安全性
✓ 持续维护
✓ 功能与SwitchyOmega完全相同



下载：
https://chromewebstore.google.com/detail/pfnededegaaopdmhkdmcofjmoldfiped
```

6.3 浏览器指纹防护

6.3.1 什么是浏览器指纹

浏览器指纹定义：

网站通过收集浏览器特征来识别和追踪用户，即使不使用Cookie。

收集的信息：

```
基础信息：
  • User-Agent（浏览器版本）



  • 屏幕分辨率

  • 时区和语言

  • 操作系统

  • 安装的字体





高级信息：
```

```
• Canvas指纹（图形渲染特征）



• WebGL指纹（GPU信息）

• AudioContext指纹（音频处理）

• WebRTC（真实IP泄漏）

• 插件列表

• 硬件特征
```

测试网站：

```
https://browserleaks.com/

https://amiunique.org/
https://coveryourtracks.eff.org/



可以查看你的浏览器泄露了哪些信息
```

6.3.2 防护方法

浏览器扩展：

1. uBlock Origin（广告拦截）

```
功能：
  • 拦截广告和追踪器



  • 屏蔽恶意网站
```

```
  • 减少指纹采集




安装:
Chrome/Firefox应用商店搜索"uBlock Origin"
```

2. Privacy Badger（隐私獾）

```
功能:
  • 自动学习并屏蔽追踪器



  • 保护隐私

  • 防止指纹识别




由EFF开发，值得信赖
```

3. Canvas Defender

```
功能:
  • 防止Canvas指纹追踪



  • 添加随机噪声

  • 保护隐私
```

```
使用:
安装后自动工作，无需配置
```

4. WebRTC Leak Shield

```
功能:
  • 防止WebRTC泄露真实IP
  • 保护代理效果
配置:
安装后选择"Disable non-proxied UDP"
```

6.3.3 浏览器设置优化

Firefox隐私设置：

```
1. 增强型跟踪保护:
   设置 → 隐私与安全 → 增强型跟踪保护 → 严格
  • Cookie设置:
   仅限同站Cookie
   或选择"删除Cookie和网站数据（Firefox关闭时）"
  • 阻止指纹识别:
   about:config
   privacy.resistFingerprinting = true
```

```
  • 禁用WebRTC：

    about:config

    media.peerconnection.enabled = false
```

Chrome隐私设置：

```
1. 预加载网页：

   设置 → 隐私和安全 → Cookie和其他网站数据
   关闭"预加载网页，以便实现更快速的浏览和搜索"

  • 不跟踪：

   发送"不跟踪"请求（效果有限）
  • 第三方Cookie：

   阻止第三方Cookie
  • 使用扩展插件：

   依赖上述提到的扩展
```

6.3.4 专业隐私浏览器

Tor Browser（洋葱浏览器）：

```
特点：

✓ 最强隐私保护
✓ 多层加密
✓ IP匿名化
✓ 防指纹追踪


下载：
https://www.torproject.org/
```

```
使用场景:

  • 极度注重隐私

  • 访问暗网

  • 绕过严格审查

缺点:
✗ 速度较慢
✗ 部分网站可能屏蔽Tor
```

Brave浏览器:

```
特点:

✓ 内置广告拦截
✓ 默认HTTPS
✓ 防指纹追踪
✓ 速度快

下载:
https://brave.com/

优势:

  • 基于Chromium,兼容性好

  • 默认隐私保护
```

```
  • 内置Tor模式
```

Firefox Focus（移动端）：

```
特点:

✓ 自动删除浏览历史
✓ 广告拦截
✓ 追踪保护
✓ 极简设计



适用:
iOS/Android
```

6.4 隐私浏览模式

6.4.1 无痕模式/隐私模式

各浏览器开启方式：

```
Chrome：

Ctrl + Shift + N (Windows/Linux)
Cmd + Shift + N (macOS)



Firefox：
Ctrl + Shift + P (Windows/Linux)
Cmd + Shift + P (macOS)



Edge：
Ctrl + Shift + N
```

```
Safari：
Cmd + Shift + N (macOS)
```

隐私模式的作用：

```
✓ 不保存浏览历史

✓ 不保存Cookie（关闭后清除）
✓ 不保存表单数据
✓ 不保存搜索记录



✗ 不能隐藏IP地址
✗ 不能防止ISP追踪
✗ 不能绕过网络限制
✗ 仍然可能被指纹识别
```

常见误解：

```
[禁止] 误解：隐私模式可以匿名上网

[完成] 事实：只是不在本地保存记录



[禁止] 误解：ISP看不到我的访问
[完成] 事实：ISP仍能看到所有流量



[禁止] 误解：网站无法追踪我
[完成] 事实：仍可通过指纹、IP追踪
```

6.4.2 容器标签（Firefox）

Firefox容器功能：

```
安装：
```

```
Firefox Multi-Account Containers扩展



功能:

  • 不同容器互相隔离



  • Cookie不共享

  • 可用于多账号登录

  • 提高隐私保护




使用场景:
容器1: 工作账号
容器2: 个人账号
容器3: 购物
容器4: 社交媒体



优势:
✓ 防止跨站追踪
✓ 多账号同时登录
✓ 隔离不同网站
```

配置示例:

```
创建容器:

  • 工作 (蓝色)



  • 个人 (绿色)

  • 购物 (橙色)
```

```
  • 社交（粉色）




自动打开规则：
google.com → 工作容器
facebook.com → 社交容器
taobao.com → 购物容器
```

6.4.3 搜索引擎隐私

隐私友好的搜索引擎：

DuckDuckGo：

```
网址：https://duckduckgo.com


特点：
✓ 不追踪用户
✓ 不记录搜索历史
✓ 不个性化结果
✓ 加密连接



设置为默认：
浏览器设置 → 搜索引擎 → DuckDuckGo
```

Startpage：

```
网址：https://www.startpage.com


特点：
✓ 使用Google搜索结果
✓ 但不追踪用户
✓ 代理访问网站
✓ 保护隐私
```

```
优势:

  • 搜索质量高（Google结果）

  • 隐私保护好
```

Searx（自托管）：

```
特点:

✓ 开源搜索引擎
✓ 聚合多个搜索源
✓ 可自己部署
✓ 完全掌控

公共实例:
https://searx.space/
```

6.4.4 HTTPS Everywhere

扩展功能：

```
自动将HTTP升级为HTTPS

保护传输安全
防止中间人攻击

由EFF开发
```

安装与使用：

```
Chrome/Firefox应用商店搜索"HTTPS Everywhere"
```

```
功能:

  • 自动检测HTTPS支持

  • 强制使用HTTPS

  • 保护隐私和安全

注意:
现代浏览器已内置类似功能
但该扩展提供更强制的保护
```

6.4.5 综合隐私方案

推荐配置组合:

基础配置（所有用户）:

```
✓ 浏览器: Firefox / Brave

✓ 扩展:
  - uBlock Origin（广告拦截）
  - Privacy Badger（追踪保护）
  - HTTPS Everywhere（强制HTTPS）
✓ 搜索引擎: DuckDuckGo
✓ DNS：加密DNS（DoH）
```

进阶配置（注重隐私）:

```
✓ 浏览器: Firefox + 严格隐私设置

✓ 扩展:
  - uBlock Origin
  - Privacy Badger
  - Canvas Defender
```

```
  - WebRTC Leak Shield
  - Decentraleyes（本地CDN）
✓ 容器标签：网站隔离
✓ 定期清除：Cookie和缓存
```

极致配置（最高隐私）：

```
✓ 浏览器：Tor Browser


✓ 代理：多层代理
✓ 虚拟机：隔离环境
✓ VPN + Tor：双重保护
✓ 定期更换：浏览器配置文件
```

本章小结

本章介绍了浏览器代理配置和隐私保护方案：

核心要点：

- **浏览器代理**：

- Chrome/Edge：使用系统代理或SwitchyOmega

- Firefox：独立代理设置 - Safari：系统代理

- **SwitchyOmega**：

- 自动切换规则

- 多情景模式 - 灵活配置

- **指纹防护**：

- 浏览器扩展保护

- 隐私浏览器选择 - 设置优化

- **隐私模式**：

- 无痕浏览

- 容器隔离 - 隐私搜索引擎

最佳实践：

```
日常使用：

浏览器 + SwitchyOmega + 基础隐私扩展



注重隐私：
Firefox + 容器 + 严格隐私设置



极致需求：
Tor Browser + VPN + 虚拟机
```

注意事项：

```
[注意] 隐私保护要适度，过度配置可能影响体验

[注意] 不同场景选择合适的方案
[注意] 定期更新扩展和浏览器
[注意] 了解各工具的局限性
```

至此，第二部分"主流协议与工具篇"全部完成！

实践任务：

- 安装并配置SwitchyOmega
- 测试浏览器指纹泄露情况
- 尝试使用隐私友好的搜索引擎
- 配置Firefox容器标签

延伸阅读：

- EFF隐私保护指南
- 浏览器指纹追踪技术
- Tor网络原理
- 现代隐私保护技术

第七章 订阅与节点管理

第7章 订阅与节点管理

订阅功能是现代科学上网工具的核心特性，它让用户无需手动配置即可快速获取和更新节点。本章将深入讲解订阅机制和节点管理技巧。

7.1 订阅链接原理

7.1.1 什么是订阅

订阅定义：

订阅是一个URL链接，访问后返回包含多个代理节点配置的文本内容。客户端定期访问这个链接来获取最新的节点信息。

```
订阅工作流程：

用户 ──────────► 订阅链接 ──────────► 服务器
      ①请求            ②返回节点列表



客户端解析节点列表
          ↓
自动添加/更新节点
          ↓
用户可直接使用
```

7.1.2 订阅链接格式

Base64编码的节点列表：

```
原始内容（多个节点URL）：


ss://YWVzLTI1Ni1nY206cGFzc3dvcmQ@server1.com:8388#节点1
vmess://ewogICJ2IjogIjIiL...#节点2
trojan://password@server3.com:443#节点3
```

```
Base64编码后:
c3M6Ly9ZV1Z6TFRJMU5pMW5ZMjA2Y0dGemMzZHZjbVFA...



订阅链接:
https://example.com/sub?token=abc123
访问后返回Base64编码的节点列表
```

标准格式示例：

Shadowsocks订阅格式：

```
ss://method:password@server:port#remark

示例:
ss://YWVzLTI1Ni1nY206bXlwYXNzd29yZA@server.com:8388#香港节点



解码后:
method = aes-256-gcm
password = mypassword
server = server.com
port = 8388
remark = 香港节点
```

VMess订阅格式：

```
{

  "v": "2",
  "ps": "节点备注",
  "add": "server.com",
  "port": "443",
  "id": "uuid-here",
  "aid": "0",
  "net": "ws",
  "type": "none",
  "host": "server.com",
  "path": "/path",
  "tls": "tls"
}
```

```
编码为: vmess://base64(上述JSON)
```

Clash订阅格式（YAML）：

```
proxies:

  - name: "节点1"
    type: ss
    server: server1.com
    port: 8388
    cipher: aes-256-gcm
    password: password1



  - name: "节点2"
    type: vmess
    server: server2.com
    port: 443
    uuid: your-uuid
    alterId: 0
    cipher: auto
    tls: true
    network: ws
    ws-opts:
      path: /path
      headers:
        Host: server2.com



proxy-groups:
  - name: "[新] 节点选择"
    type: select
    proxies:
     - "节点1"
     - "节点2"
```

7.1.3 订阅更新机制

自动更新流程：

```
客户端启动


    ↓
检查订阅更新时间
    ↓
距离上次更新 > 设定间隔?
    ↓ 是
发送HTTP GET请求到订阅链接
    ↓
接收服务器响应
    ↓
解析订阅内容（Base64解码/YAML解析）
    ↓
对比现有节点列表
    ↓
┌──────────────────────────┬──────────────────────────┐
│ 新增节点：添加   │ 删除节点：移除│
│ 更新节点：覆盖   │ 保留节点：不变│
└──────────────────────────┴──────────────────────────┘
    ↓
更新本地配置文件
    ↓
记录更新时间
```

更新策略：

```
手动更新：

用户主动点击"更新订阅"按钮



定时更新：
每24小时自动更新一次



启动更新：
每次启动客户端时检查更新



智能更新：
检测到节点全部失效时自动更新
```

7.1.4 订阅链接安全性

潜在风险：

```
1. 订阅链接泄露

    ✗ 他人可能盗用你的订阅
    ✗ 导致流量被消耗
    ✗ 服务商可能封禁账号

  • 中间人攻击

    ✗ HTTP订阅可能被劫持

    ✗ 返回恶意节点配置
    ✗ 窃取用户流量

  • 隐私泄露

    ✗ 订阅请求暴露真实IP

    ✗ 服务商可追踪用户
```

安全建议：

```
✓ 使用HTTPS订阅链接

  确保传输加密，防止劫持



✓ 不要公开分享订阅链接
  订阅链接 = 账号密码



✓ 定期更换订阅链接
  如果怀疑泄露，及时重置
```

```
✓ 使用订阅时连接代理
  通过现有节点更新订阅，隐藏真实IP



✓ 验证节点来源
  只使用信任的订阅源
```

7.2 订阅转换服务

7.2.1 为什么需要订阅转换

问题场景：

```
机场提供的订阅格式：Shadowsocks通用订阅

你的客户端需要：Clash配置格式



不兼容！
    ↓
需要订阅转换服务
```

转换需求：

原始格式	目标格式	场景
SS/SSR/V2Ray通用订阅	Clash配置	使用Clash客户端
Clash配置	Quantumult X	iOS用户
V2Ray订阅	Surge配置	Surge用户
单一协议	混合配置	多协议支持

7.2.2 在线转换服务

[注意] 重要隐私提示：

```
使用在线转换服务的风险：
```

```
[注意] 订阅链接包含所有节点信息
[注意] 第三方服务可能记录你的订阅
[注意] 不可信的服务可能窃取节点



安全建议:
✓ 优先选择自建转换服务（见7.2.3）
✓ 使用知名、可信的在线服务
✓ 避免使用不明来源的转换后端
✓ 定期更换订阅链接和密码
```

常用转换服务：

1. sub-web（推荐）

```
在线服务:

https://sub.xeton.dev/
https://api.dler.io/
https://sub.id/



功能:
✓ 支持多种格式互转
✓ 自定义规则
✓ 节点过滤
✓ 重命名节点
```

使用方法：

```
1. 选择客户端类型（如Clash）
  • 粘贴订阅链接



  • 选择后端地址

  • 高级设置（可选）:
```

```
  - 包含节点：香港|美国

  - 排除节点：过期|测试
  - 启用UDP
  - 启用TFO


• 生成订阅链接



• 复制链接到客户端使用
```

2. ACL4SSR在线转换

```
网址：https://acl4ssr-sub.github.io/

特点：
✓ 预设多种规则配置
✓ 广告拦截规则
✓ 流媒体解锁规则
✓ 国内外分流



规则类型：

• ACL_默认版



• ACL_无测速版

• ACL_去广告版

• ACL_全分组版

• ACL_精简版
```

7.2.3 自建转换服务

subconverter后端：

```
# Docker部署（推荐）

docker run -d \
  --name subconverter \
  -p 25500:25500 \
  tindy2013/subconverter:latest

# 访问：http://localhost:25500/
```

配置文件示例（pref.ini）：

```
[common]

api_mode=true
api_access_token=your_secret_token


[node_pref]

# UDP支持


udp_flag=true

# TCP Fast Open


tfo_flag=true
```

跳过证书验证

```
skip_cert_verify_flag=false
```

```
[managed_config]
```

配置更新间隔（秒）

```
config_update_interval=86400
```

配置更新严格模式

```
config_update_strict=false
```

```
[surge_external_proxy]
```

Surge外部代理

```
surge_ssr_path=/usr/bin/ssr-local
```

```
[template]
```

模板路径

```
template_path=templates
```

API使用示例：

```
转换请求：


https://your-domain.com/sub?
  target=clash                          # 目标格式
  &url=https://sub-url.com/link         # 原始订阅
  &config=ACL4SSR_Online_Full.ini       # 规则配置
  &emoji=true                           # 添加国旗emoji
  &list=false                           # 输出为配置文件
  &udp=true                             # 启用UDP
  &tfo=false                            # 禁用TFO
  &expand=true                          # 展开规则
  &scv=false                            # 关闭证书验证
  &fdn=false                            # 过滤非法节点
  &sort=false                           # 不排序节点



参数说明：
target: clash, surge, quan, quanx, loon, ss, ssr, v2ray等
config: 规则配置文件名
```

7.2.4 订阅转换进阶

节点过滤规则：

```
包含关键词（include）：


include=香港|HK|美国|US|日本|JP



示例URL：
&include=香港|美国



排除关键词（exclude）：
exclude=过期|到期|剩余
```

```
示例URL：
&exclude=过期|测试
```

节点重命名：

```
rename规则:

@node_name@新节点名称



示例:
将所有包含"香港"的节点重命名为"HK"
&rename=香港@HK



正则替换:
&rename=/香港\d+/HK$1/
```

自定义规则：

```
# 自定义规则文件 custom.ini

[custom]
clash_rule_base=https://example.com/base.yaml

# 添加自定义规则



custom_rules=
  DOMAIN-SUFFIX,openai.com,[新] 节点选择
  DOMAIN-SUFFIX,claude.ai,[新] 节点选择
  DOMAIN-SUFFIX,notion.so,[新] 节点选择

# 自定义代理组
```

```
custom_proxy_groups=
  🤖 AI服务select(香港|美国)
  🎬 流媒体select(美国|日本)
```

7.3 节点测速与选择

7.3.1 测速原理

延迟测试（Ping/Tcping）：

```
原理:

客户端 ──► ICMP Echo Request ──► 服务器
         ◄── ICMP Echo Reply  ◄──
             ↑
        测量往返时间 (RTT)



TCP Ping：
客户端 ──► TCP SYN ──► 服务器:端口
         ◄── SYN+ACK ◄──
             ↑
        测量TCP握手时间



结果:
延迟 < 50ms     ：优秀 🟢
延迟 50-100ms  ：良好 🟡
延迟 100-200ms：一般 🟠
延迟 > 200ms    ：较差 🔴
超时           ： 不可用 ⚫
```

HTTP测速：

```
原理:

访问特定URL，测量响应时间
```

```
常用测速地址:
http://www.gstatic.com/generate_204
http://www.google.com/generate_204
http://cp.cloudflare.com/



优势:
✓ 更接近实际使用体验
✓ 可以验证节点可用性
✓ 检测HTTP/HTTPS是否正常
```

带宽测速:

```
原理:

下载测试文件，计算下载速度



测速过程:

  • 连接节点



  • 下载固定大小文件

  • 计算传输速率

  • 显示速度结果





测速文件:
小文件: 1MB（快速测试）
中文件: 10MB（常规测试）
大文件: 100MB（精确测试）
```

```
注意:
[注意] 消耗流量
[注意] 测速时间较长
[注意] 受服务器带宽限制
```

7.3.2 客户端测速功能

Clash测速:

```
# config.yaml配置

proxy-groups:
  - name: "♻️ 自动选择"
    type: url-test
    proxies:
     - "节点1"
     - "节点2"
     - "节点3"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300          # 测速间隔(秒)
    tolerance: 150         # 容差(ms)
    lazy: true            # 懒加载模式



测速逻辑:


  • 每300秒测试一次所有节点



  • 选择延迟最低的节点


  • 延迟差 < 150ms时不切换(避免频繁切换)


  • lazy=true时只在使用时才测速
```

V2RayN测速:

```
操作步骤:
  • 选中要测速的节点（可多选）



  • 右键 → 测试服务器延迟



    - 真连接延迟（TCPing）

    - 测速（下载速度）

  • 查看结果:

    - 延迟显示在节点名称后

    - 可按延迟排序



快捷键:
Ctrl+A：全选
Ctrl+点击: 多选
```

Shadowrocket测速：

```
延迟测试:

主界面下拉刷新 → 自动测试所有节点延迟



批量测速:
连通性测试 → 选择多个节点 → 测速



自动选择:
设置 → 延迟测试方法 → URL测试
设置测速URL和间隔
```

7.3.3 命令行测速工具

LiteSpeedTest：

```
# 安装

wget https://github.com/xxf098/LiteSpeedTest/releases/download/v0.x.x/lite-linux-amd64
chmod +x lite-linux-amd64

# 测速订阅链接


./lite-linux-amd64 \
  --config ./config.json \
  --test https://your-subscription-url

# 输出结果


节点名称              延迟    速度
香港-01              15ms    125MB/s
美国-03              180ms   98MB/s
日本-02              45ms    156MB/s
```

配置文件示例：

```
{

  "thread": 4,
  "speedtestMode": "pingonly",
  "pingMethod": "googleping",
  "sortMethod": "speed",
  "outputMode": "2"
}
```

stairspeedtest：

```
# Docker运行

docker run -d \
  -p 65500:65500 \
  tindy2013/stairspeedtest-reborn

# Web界面访问


http://localhost:65500


功能:
✓ 批量测速
✓ 生成测速报告
✓ 支持多种协议
✓ 导出结果
```

7.3.4 节点选择策略

基于用途选择:

```
日常浏览:

优先级: 稳定性 > 速度 > 延迟
推荐: 本地区节点（香港/日本/新加坡）



看视频/流媒体:
优先级: 速度 > 稳定性 > 延迟
推荐: 高带宽节点，原生IP
流媒体解锁: Netflix需美国/日本原生IP



下载/上传:
优先级: 带宽 > 稳定性 > 延迟
推荐: 不限流量的高速节点
```

```
游戏加速：
优先级：延迟 > 稳定性 > 速度
推荐：本地区低延迟节点（<50ms）



匿名浏览：
优先级：隐私 > 稳定性 > 其他
推荐：远程节点，多层代理
```

地区选择建议：

```
中国大陆用户：


首选：
🇭🇰 香港：延迟低（5-30ms），速度快
🇯🇵 日本：延迟低（30-60ms），带宽大
🇸🇬 新加坡：延迟中等（50-80ms），稳定



次选：
🇺🇸 美国西海岸：延迟中（120-180ms），流媒体好
🇰🇷 韩国：延迟低（30-60ms），游戏好



不推荐：
🇪🇺 欧洲：延迟高（200-300ms）
🇦🇺 澳大利亚：延迟极高（300+ms）
```

时间段选择：

```
高峰期（晚上8-12点）：


✗ 热门节点拥挤
✓ 选择冷门地区节点
✓ 使用负载均衡
```

```
低峰期（凌晨/白天）：
✓ 所有节点速度较好
✓ 适合大文件下载
```

7.4 负载均衡配置

7.4.1 负载均衡原理

什么是负载均衡?

将流量分散到多个节点，提高整体性能和可靠性。

```
传统单节点：

所有流量 ──────► 单个节点 ──────► 互联网
                (容易拥堵)



负载均衡：
        ┌──► 节点1 ──┐
流量 ──┼──► 节点2 ──┼──► 互联网
        └──► 节点3 ──┘
    (分散负载，提高速度)
```

负载均衡策略：

1. 轮询（Round Robin）

```
原理：

请求1 → 节点A
请求2 → 节点B
请求3 → 节点C
请求4 → 节点A（循环）



特点：
✓ 简单公平
✓ 平均分配
✗ 不考虑节点性能差异
```

2. 随机（Random）

```
原理：

每个请求随机选择一个节点



特点：
✓ 实现简单
✓ 分布均匀（大量请求时）
✗ 短期内可能不均
```

3. 最少连接（Least Connections）

```
原理：

选择当前连接数最少的节点



特点：
✓ 智能分配
✓ 适应节点性能差异
✓ 避免某个节点过载
```

4. 一致性哈希（Consistent Hashing）

```
原理：

根据请求特征（如域名）计算哈希值
相同特征总是选择相同节点



特点：
✓ 保持会话连续性
✓ 同一网站走同一节点
✓ 适合需要保持登录状态的场景
```

```
应用：
同一个网站的多个请求走同一节点
避免频繁切换导致需要重新登录
```

7.4.2 Clash负载均衡配置

基础配置：

```
proxy-groups:


  - name: "⚖️ 负载均衡"
    type: load-balance
    proxies:
     - "香港-01"
     - "香港-02"
     - "香港-03"
     - "日本-01"
     - "日本-02"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    strategy: consistent-hashing  # 策略选择
```

策略详解：

```
# 1. 一致性哈希（推荐）


strategy: consistent-hashing

# 同一域名走同一节点


# 适合：日常浏览、保持登录


# 2. 轮询
```

```
strategy: round-robin

# 依次轮流使用节点

# 适合：下载、流媒体
```

高级配置：

```
proxy-groups:

  # 按地区分组负载均衡
  - name: "⚖️ 香港负载均衡"
    type: load-balance
    use:
      - provider1
    filter: "香港|HK|Hong Kong"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    strategy: consistent-hashing



  - name: "⚖️ 美国负载均衡"
    type: load-balance
    use:
      - provider1
    filter: "美国|US|United States"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    strategy: round-robin



  # 主选择组
  - name: "[新] 节点选择"
    type: select
    proxies:
      - "⚖️ 香港负载均衡"
      - "⚖️ 美国负载均衡"
      - "♻️ 自动选择"
```

7.4.3 V2Ray负载均衡配置

V2Ray outbounds配置：

```
{


  "outbounds": [
    {
       "tag": "balancer",
       "protocol": "freedom",
       "settings": {}
    },
    {
       "tag": "proxy1",
       "protocol": "vmess",
       "settings": { ... }
    },
    {
       "tag": "proxy2",
       "protocol": "vmess",
       "settings": { ... }
    },
    {
       "tag": "proxy3",
       "protocol": "vmess",
       "settings": { ... }
    }
  ],
  "routing": {
    "rules": [
      {
         "type": "field",
         "outboundTag": "balancer",
         "balancerTag": "load-balancer"
      }
    ],
    "balancers": [
      {
         "tag": "load-balancer",
         "selector": ["proxy1", "proxy2", "proxy3"],
         "strategy": {
           "type": "random"  // 或 "leastPing"
        }
      }
```

```
    ]
  }
}
```

策略类型：

```
// 1. 随机选择

"strategy": {
  "type": "random"
}



// 2. 最低延迟
"strategy": {
  "type": "leastPing"
}



// 3. 轮询（需要额外配置）
"strategy": {
  "type": "roundRobin"
}
```

7.4.4 负载均衡最佳实践

场景推荐：

```
下载大文件：


策略：轮询（round-robin）
节点：3-5个高速节点
原因：分散负载，充分利用带宽



日常浏览：
策略：一致性哈希（consistent-hashing）
节点：同地区节点（如全部香港）
原因：保持会话，避免验证码
```

```
看流媒体：
策略：手动选择（不用负载均衡）
节点：单个原生IP节点
原因：避免IP变化导致检测



多人共享：
策略：最少连接（least-connections）
节点：多个不同地区节点
原因：智能分配，适应不同使用场景
```

注意事项：

```
[注意] 避免混用不同地区节点

原因：可能导致访问异常（IP频繁变化）



[注意] 节点数量适中（3-5个）
过少：负载均衡效果不明显
过多：管理复杂，故障率增加



[注意] 定期测速
确保所有节点状态良好
及时剔除失效节点



[注意] 监控流量分布
检查是否真正实现负载均衡
调整策略和节点配置
```

7.5 故障转移策略

7.5.1 故障转移原理

Failover机制：

```
主节点正常：
```

```
流量 ──────► 主节点 ──────► 互联网
             (优先使用)



主节点故障:
流量 ──────✗ 主节点(失败)
      ↓
      └─────► 备用节点1 ──────► 互联网
             (自动切换)



备用节点1也故障:
流量 ──────► 备用节点2 ──────► 互联网
             (继续尝试下一个)
```

故障检测方式:

```
1. 超时检测

   连接超时 > 5秒 → 判定故障

 • HTTP状态码

   返回4xx/5xx → 判定故障

   返回200 → 节点正常

 • 定期探测

   每N秒访问测试URL

   失败次数 > 阈值 → 故障

 • 心跳检测

   定期发送心跳包

   无响应 → 故障
```

7.5.2 Clash故障转移配置

基础配置：

```
proxy-groups:


  - name: "[网络] 故障转移"
    type: fallback
    proxies:
     - "香港-主节点"
     - "香港-备用1"
     - "香港-备用2"
     - "日本-备用"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300        # 健康检查间隔（秒）
    timeout: 2000        # 超时时间（毫秒）
```

工作流程：

```
1. 定期检查（每300秒）


   ├── 测试 "香港-主节点"
   ├── 测试 "香港-备用1"
   ├── 测试 "香港-备用2"
   └── 测试 "日本-备用"


 • 选择节点


   ├── 优先使用列表中第一个可用节点


   ├── 主节点正常 → 使用主节点
   ├── 主节点故障 → 使用备用1
   └── 备用1故障 → 使用备用2


 • 自动恢复


   └── 主节点恢复后自动切回
```

分级备份配置：

```
proxy-groups:

  # 第一层：地区故障转移
  - name: "🇭🇰 香港故障转移"
    type: fallback
    proxies:
      - "香港-01"
      - "香港-02"
      - "香港-03"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300



  - name: "🇯🇵 日本故障转移"
    type: fallback
    proxies:
      - "日本-01"
      - "日本-02"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300



  # 第二层：跨地区故障转移
  - name: "[网络] 全局故障转移"
    type: fallback
    proxies:
      - "🇭🇰 香港故障转移"
      - "🇯🇵 日本故障转移"
      - "🇺🇸 美国故障转移"
    url: 'http://www.gstatic.com/generate_204'
    interval: 600
```

7.5.3 综合策略配置

组合使用：自动选择 + 故障转移

```
proxy-groups:

  # 香港节点自动选择（选最快）
  - name: "🇭🇰 HK-Auto"
    type: url-test
    proxies:
      - "香港-01"
```

```
    - "香港-02"
    - "香港-03"
  url: 'http://www.gstatic.com/generate_204'
  interval: 300



# 日本节点自动选择
- name: "🇯🇵 JP-Auto"
  type: url-test
  proxies:
    - "日本-01"
    - "日本-02"
  url: 'http://www.gstatic.com/generate_204'
  interval: 300



# 地区级故障转移
- name: "[网络] 主代理组"
  type: fallback
  proxies:
    - "🇭🇰 HK-Auto"   # 优先香港最快节点
    - "🇯🇵 JP-Auto"   # 备用日本最快节点
    - "🇺🇸 US-01"     # 最后备用
  url: 'http://www.gstatic.com/generate_204'
  interval: 600
```

完整实战配置：

```
proxy-groups:


# 主选择组（用户手动选择）
- name: "[新] 节点选择"
  type: select
  proxies:
    - "♻️ 自动选择"
    - "[网络] 故障转移"
    - "⚖️ 负载均衡"
    - "🇭🇰 香港节点"
    - "🇯🇵 日本节点"
    - "🇺🇸 美国节点"



# 自动选择（全局最快）
- name: "♻️ 自动选择"
```

```
    type: url-test
    use:
      - all-proxies
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    tolerance: 50



  # 故障转移（高可用）
  - name: "[网络] 故障转移"
    type: fallback
    proxies:
     - "🇭🇰 香港自动"
     - "🇯🇵 日本自动"
     - "🇺🇸 美国自动"
    url: 'http://www.gstatic.com/generate_204'
    interval: 600



  # 负载均衡（高性能）
  - name: "⚖️ 负载均衡"
    type: load-balance
    use:
      - all-proxies
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    strategy: consistent-hashing



  # 香港节点组
  - name: "🇭🇰 香港节点"
    type: select
    proxies:
     - "🇭🇰 香港自动"
     - "🇭🇰 香港故障转移"
     - "🇭🇰 香港负载均衡"



  - name: "🇭🇰 香港自动"
    type: url-test
    use:
      - all-proxies
    filter: "香港|HK|Hong Kong"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
```

```
  - name: "🇭🇰 香港故障转移"
    type: fallback
    use:
      - all-proxies
    filter: "香港|HK|Hong Kong"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300



  - name: "🇭🇰 香港负载均衡"
    type: load-balance
    use:
      - all-proxies
    filter: "香港|HK|Hong Kong"
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    strategy: consistent-hashing



proxy-providers:
  all-proxies:
    type: http
    url: "your-subscription-url"
    interval: 86400
    path: ./proxies/all.yaml
    health-check:
      enable: true
      interval: 600
      url: http://www.gstatic.com/generate_204
```

7.5.4 故障转移最佳实践

配置原则：

```
1. 分层设计

   节点级 → 地区级 → 全局级


 • 合理间隔
```

```
    节点检查：300-600秒

    地区检查：600-1200秒
    避免过于频繁（浪费流量）

  • 超时设置

    正常节点：< 1000ms

    慢速节点：1000-3000ms
    超时阈值：2000-5000ms

  • 备份充足

    至少2-3个备用节点

    覆盖不同地区
    避免单点故障
```

监控与维护：

```
定期检查：

✓ 查看节点健康状态
✓ 分析故障转移记录
✓ 优化节点顺序


故障处理：
✓ 记录故障时间和原因
✓ 联系服务商解决
✓ 及时更新订阅


性能优化：
✓ 根据使用情况调整优先级
✓ 移除长期不可用节点
✓ 添加更稳定的备用节点
```

本章小结

本章深入讲解了订阅与节点管理的核心知识：

核心要点：

- **订阅机制**：

- 订阅链接格式与原理

- 自动更新机制 - 安全性注意事项

- **订阅转换**：

- 格式互转需求

- 在线转换服务 - 自建转换后端

- **节点测速**：

- 延迟、HTTP、带宽测速

- 客户端测速功能 - 节点选择策略

- **负载均衡**：

- 轮询、随机、一致性哈希

- Clash/V2Ray配置 - 场景化应用

- **故障转移**：

- Failover机制

- 分层备份设计 - 自动恢复策略

实用配置建议：

```
日常使用：


[新] 节点选择
   ├── ♻️ 自动选择（默认，选最快）
   ├── [网络] 故障转移（高可用）
   └── ⚖️ 负载均衡（高性能）



专业配置：
分地区组织
   ├── 每个地区：自动选择 + 故障转移
   ├── 跨地区：故障转移
   └── 特定用途：负载均衡（下载）或手动选择（流媒体）
```

下一章我们将学习分流规则的详细编写方法。

实践任务：

- 配置订阅并尝试转换格式
- 测试所有节点延迟和速度
- 设置负载均衡组
- 配置多层故障转移
- 观察节点切换行为

第八章 分流规则详解

第8章 分流规则详解

分流规则是科学上网的核心功能，它能智能地决定哪些流量走代理，哪些直连。本章将深入讲解规则的编写和应用。

8.1 规则分流基础

8.1.1 什么是分流

分流定义：

根据预设规则，自动判断网络请求是走代理还是直连。

```
传统全局代理：

所有流量 ──────► 代理服务器 ──────► 互联网
              （国内网站也走代理，浪费流量，速度慢）


智能分流：
国内流量 ──────► 直接连接 ──────► 国内网站（快）


国外流量 ──────► 代理服务器 ──────► 国外网站（翻墙）
```

分流的优势：

```
✓ 节省流量

  国内网站直连，不消耗代理流量


✓ 提高速度
  国内网站直连速度更快
```

```
✓ 降低延迟
  避免不必要的绕路



✓ 避免异常
  银行/支付等敏感网站直连更安全



✓ 提升体验
  该快的快，该稳的稳
```

8.1.2 分流工作原理

请求处理流程：

```
应用发起请求：访问 www.example.com


        ↓
进入代理客户端
        ↓
┌──────────────────────────────┐
│   规则匹配引擎      │
└──────────────────────────────┘
        ↓
   按顺序匹配规则
        ↓
┌──────────────────────────────────────────────────────┐
│ 规则1: DOMAIN,www.example.com,[新]代理  │ ← 匹配成功!
│ 规则2: DOMAIN-SUFFIX,cn,DIRECT        │
│ 规则3: GEOIP,CN,DIRECT                │
│ 规则4: MATCH,[新]代理                 │
└──────────────────────────────────────────────────────┘
        ↓
   执行对应策略
        ↓
   通过代理访问
```

匹配原则：

```
1. 从上到下顺序匹配
```

```
  第一条匹配的规则生效
  后续规则不再检查


• 精确匹配优先


  DOMAIN > DOMAIN-SUFFIX > DOMAIN-KEYWORD
• 特殊规则在前


  具体规则 → 通用规则 → 兜底规则
• 性能考虑


  常用规则放前面


  复杂规则放后面
```

8.1.3 规则类型概览

Clash规则类型：

规则类型	说明	示例
DOMAIN	完整域名匹配	`DOMAIN,www.google.com,[新]代理`
DOMAIN-SUFFIX	域名后缀匹配	`DOMAIN-SUFFIX,google.com,[新]代理`
DOMAIN-KEYWORD	域名关键词	`DOMAIN-KEYWORD,google,[新]代理`
IP-CIDR	IP地址段	`IP-CIDR,192.168.0.0/16,DIRECT`
IP-CIDR6	IPv6地址段	`IP-CIDR6,2001:db8::/32,DIRECT`
GEOIP	地理位置	`GEOIP,CN,DIRECT`
SRC-IP-CIDR	源IP地址	`SRC-IP-CIDR,192.168.1.0/24,DIRECT`
SRC-PORT	源端口	`SRC-PORT,7777,DIRECT`
DST-PORT	目标端口	`DST-PORT,80,DIRECT`
PROCESS-NAME	进程名称	`PROCESS-NAME,chrome.exe,[新]代理`
RULE-SET	规则集	`RULE-SET,google,[新]代理`
MATCH	兜底规则	`MATCH,[新]代理`

8.1.4 策略类型

常见策略：

```
DIRECT            # 直接连接（不走代理）


REJECT            # 拒绝连接（用于广告屏蔽）
[新]代理          # 走指定代理组
Proxy             # 走代理
♻️自动选择       # 自动选择最快节点
🇭🇰香港          # 指定地区节点组
```

8.2 域名规则编写

8.2.1 DOMAIN（完整域名匹配）

语法：

```
DOMAIN,域名,策略
```

示例：

```
rules:

  # 精确匹配
  - DOMAIN,www.google.com,[新]代理
  - DOMAIN,api.openai.com,[新]代理
  - DOMAIN,www.baidu.com,DIRECT



  # 只匹配完全相同的域名
  - DOMAIN,google.com,[新]代理
  # ✓ 匹配：google.com
  # ✗ 不匹配：www.google.com
  # ✗ 不匹配：mail.google.com
```

适用场景：

```
✓ 需要精确控制的域名


✓ 避免误匹配
✓ 优先级最高的特殊规则



示例：
某个子域名直连，其他子域名代理：


  • DOMAIN,cn.example.com,DIRECT        # 国内版直连



  • DOMAIN-SUFFIX,example.com,[新]代理    # 其他都代理
```

8.2.2 DOMAIN-SUFFIX（域名后缀匹配）

语法：

```
DOMAIN-SUFFIX,后缀,策略
```

示例：

```
rules:

  # 匹配域名及所有子域名
  - DOMAIN-SUFFIX,google.com,[新]代理
  # ✓ 匹配: google.com
  # ✓ 匹配: www.google.com
  # ✓ 匹配: mail.google.com
  # ✓ 匹配: drive.google.com
  # ✗ 不匹配: google.co.uk



  # 国内网站直连
  - DOMAIN-SUFFIX,baidu.com,DIRECT
  - DOMAIN-SUFFIX,qq.com,DIRECT
  - DOMAIN-SUFFIX,taobao.com,DIRECT
  - DOMAIN-SUFFIX,jd.com,DIRECT



  # 国外服务代理
  - DOMAIN-SUFFIX,youtube.com,[新]代理
  - DOMAIN-SUFFIX,facebook.com,[新]代理
  - DOMAIN-SUFFIX,twitter.com,[新]代理
  - DOMAIN-SUFFIX,instagram.com,[新]代理
```

通配符技巧：

```
# 顶级域名规则
  • DOMAIN-SUFFIX,cn,DIRECT              # 所有.cn域名直连



  • DOMAIN-SUFFIX,gov.cn,DIRECT          # 政府网站直连
```

常见场景

```
• DOMAIN-SUFFIX,apple.com,DIRECT       # Apple服务


• DOMAIN-SUFFIX,icloud.com,DIRECT      # iCloud

• DOMAIN-SUFFIX,microsoft.com,DIRECT   # 微软（可选）
```

8.2.3 DOMAIN-KEYWORD（域名关键词匹配）

语法：

```
DOMAIN-KEYWORD,关键词,策略
```

示例：

```
rules:

  # 包含关键词即匹配
  - DOMAIN-KEYWORD,google,[新]代理
  # ✓ 匹配: google.com
  # ✓ 匹配: www.google.com
  # ✓ 匹配: google.co.uk
  # ✓ 匹配: google.com.hk
  # ✓ 匹配: mygooglesite.com （注意: 也会匹配）



  # 广告屏蔽
  - DOMAIN-KEYWORD,ad,REJECT
  - DOMAIN-KEYWORD,ads,REJECT
  - DOMAIN-KEYWORD,analytics,REJECT
  - DOMAIN-KEYWORD,tracker,REJECT
```

```
  # 特定服务
  - DOMAIN-KEYWORD,spotify,🎵音乐
  - DOMAIN-KEYWORD,netflix,🎬影视
  - DOMAIN-KEYWORD,github,[电脑]开发
```

注意事项：

```
[注意] 容易误匹配

DOMAIN-KEYWORD,china,DIRECT
可能误匹配：

  • chinasupplier.com（想代理）

  • porcelain-china.com（瓷器网站）

建议：
✓ 使用更具体的关键词
✓ 关键词规则放在后面
✓ 优先使用DOMAIN-SUFFIX
```

8.2.4 域名规则实战

Google全家桶：

```
rules:

  # Google搜索
  - DOMAIN-SUFFIX,google.com,[新]代理
  - DOMAIN-SUFFIX,google.com.hk,[新]代理
  - DOMAIN-SUFFIX,google.co.jp,[新]代理
  - DOMAIN-SUFFIX,googleapis.com,[新]代理
  - DOMAIN-SUFFIX,googleusercontent.com,[新]代理
```

```
  # Google服务
  - DOMAIN-SUFFIX,gmail.com,[新]代理
  - DOMAIN-SUFFIX,gstatic.com,[新]代理
  - DOMAIN-SUFFIX,youtube.com,[新]代理
  - DOMAIN-SUFFIX,ytimg.com,[新]代理
  - DOMAIN-SUFFIX,googlevideo.com,[新]代理



 # 或使用关键词（简化但可能误匹配）
  - DOMAIN-KEYWORD,google,[新]代理
```

社交媒体：

```
rules:

  # Facebook
  - DOMAIN-SUFFIX,facebook.com,[新]代理
  - DOMAIN-SUFFIX,fb.com,[新]代理
  - DOMAIN-SUFFIX,fbcdn.net,[新]代理



  # Twitter/X
  - DOMAIN-SUFFIX,twitter.com,[新]代理
  - DOMAIN-SUFFIX,x.com,[新]代理
  - DOMAIN-SUFFIX,twimg.com,[新]代理
  - DOMAIN-SUFFIX,t.co,[新]代理



  # Instagram
  - DOMAIN-SUFFIX,instagram.com,[新]代理
  - DOMAIN-SUFFIX,cdninstagram.com,[新]代理



  # Telegram
  - DOMAIN-SUFFIX,telegram.org,[新]代理
  - DOMAIN-SUFFIX,t.me,[新]代理
```

国内网站直连：

```
rules:
```

```
# 电商
- DOMAIN-SUFFIX,taobao.com,DIRECT
- DOMAIN-SUFFIX,tmall.com,DIRECT
- DOMAIN-SUFFIX,jd.com,DIRECT
- DOMAIN-SUFFIX,pinduoduo.com,DIRECT



# 视频
- DOMAIN-SUFFIX,bilibili.com,DIRECT
- DOMAIN-SUFFIX,iqiyi.com,DIRECT
- DOMAIN-SUFFIX,youku.com,DIRECT
- DOMAIN-SUFFIX,qq.com,DIRECT



# 社交
- DOMAIN-SUFFIX,weixin.qq.com,DIRECT
- DOMAIN-SUFFIX,wechat.com,DIRECT
- DOMAIN-SUFFIX,weibo.com,DIRECT



# 搜索
- DOMAIN-SUFFIX,baidu.com,DIRECT
- DOMAIN-SUFFIX,so.com,DIRECT
- DOMAIN-SUFFIX,sogou.com,DIRECT
```

8.3 IP-CIDR规则

8.3.1 IP-CIDR基础

CIDR表示法：

```
IP地址/前缀长度


示例：
192.168.1.0/24
  ↓        ↓
 基地址   前24位是网络部分
```

```
表示范围:
192.168.1.0 - 192.168.1.255
共256个IP地址
```

常用CIDR:

CIDR	掩码	地址数	说明
/32	255.255.255.255	1	单个IP
/24	255.255.255.0	256	C类网络
/16	255.255.0.0	65536	B类网络
/8	255.0.0.0	16777216	A类网络

8.3.2 IP-CIDR规则编写

语法:

```
IP-CIDR,IP地址/前缀,策略,no-resolve
```

基础示例:

```
rules:

  # 局域网直连
  - IP-CIDR,10.0.0.0/8,DIRECT,no-resolve
  - IP-CIDR,172.16.0.0/12,DIRECT,no-resolve
  - IP-CIDR,192.168.0.0/16,DIRECT,no-resolve



  # 本地回环
  - IP-CIDR,127.0.0.0/8,DIRECT,no-resolve



  # 特定服务IP
  - IP-CIDR,1.1.1.1/32,[新]代理,no-resolve     # Cloudflare DNS
  - IP-CIDR,8.8.8.8/32,[新]代理,no-resolve     # Google DNS
```

no-resolve参数：

```
作用：跳过DNS解析，直接匹配IP


不加no-resolve：
域名请求 → 先DNS解析 → 获取IP → 匹配规则
         (可能泄露DNS查询)



加no-resolve：
域名请求 → 跳过此规则（只匹配已知IP的请求）
IP请求 → 直接匹配



建议：
✓ IP-CIDR规则都加no-resolve
✓ 避免不必要的DNS解析
✓ 防止DNS泄露
```

8.3.3 IPv6规则

IP-CIDR6语法：

```
rules:


  # IPv6局域网
  - IP-CIDR6,fe80::/10,DIRECT,no-resolve
  - IP-CIDR6,fd00::/8,DIRECT,no-resolve



  # IPv6环回
  - IP-CIDR6,::1/128,DIRECT,no-resolve



  # Cloudflare IPv6
  - IP-CIDR6,2606:4700::/32,[新]代理,no-resolve



  # Google IPv6
  - IP-CIDR6,2001:4860::/32,[新]代理,no-resolve
```

IPv6常用段：

```
fe80::/10     - 本地链路地址


fc00::/7      - 唯一本地地址
2000::/3     - 全球单播地址
ff00::/8      - 组播地址
::1/128      - 环回地址
::/128       - 未指定地址
```

8.3.4 实用IP规则

Telegram IP段：

```
rules:


  # Telegram IPv4
  - IP-CIDR,91.108.4.0/22,📲Telegram,no-resolve
  - IP-CIDR,91.108.8.0/22,📲Telegram,no-resolve
  - IP-CIDR,91.108.12.0/22,📲Telegram,no-resolve
  - IP-CIDR,91.108.16.0/22,📲Telegram,no-resolve
  - IP-CIDR,91.108.56.0/22,📲Telegram,no-resolve
  - IP-CIDR,149.154.160.0/20,📲Telegram,no-resolve



  # Telegram IPv6
  - IP-CIDR6,2001:b28:f23d::/48,📲Telegram,no-resolve
  - IP-CIDR6,2001:b28:f23f::/48,📲Telegram,no-resolve
  - IP-CIDR6,2001:67c:4e8::/48,📲Telegram,no-resolve
```

保留地址直连：

```
rules:


  # IPv4保留地址
  - IP-CIDR,0.0.0.0/8,DIRECT,no-resolve
  - IP-CIDR,10.0.0.0/8,DIRECT,no-resolve
  - IP-CIDR,100.64.0.0/10,DIRECT,no-resolve
  - IP-CIDR,127.0.0.0/8,DIRECT,no-resolve
```

```
  - IP-CIDR,169.254.0.0/16,DIRECT,no-resolve
  - IP-CIDR,172.16.0.0/12,DIRECT,no-resolve
  - IP-CIDR,192.0.0.0/24,DIRECT,no-resolve
  - IP-CIDR,192.0.2.0/24,DIRECT,no-resolve
  - IP-CIDR,192.168.0.0/16,DIRECT,no-resolve
  - IP-CIDR,198.18.0.0/15,DIRECT,no-resolve
  - IP-CIDR,198.51.100.0/24,DIRECT,no-resolve
  - IP-CIDR,203.0.113.0/24,DIRECT,no-resolve
  - IP-CIDR,224.0.0.0/4,DIRECT,no-resolve
  - IP-CIDR,240.0.0.0/4,DIRECT,no-resolve
```

8.4 GeoIP数据库使用

8.4.1 GeoIP原理

什么是GeoIP?

GeoIP数据库记录了IP地址与地理位置的对应关系。

```
IP地址: 123.56.78.90


  ↓ 查询GeoIP数据库
国家: CN (中国)
  ↓ 应用规则
策略: DIRECT (直连)
```

8.4.2 GEOIP规则

语法:

```
GEOIP,国家代码,策略,no-resolve
```

示例:

```
rules:


  # 中国IP直连
  - GEOIP,CN,DIRECT,no-resolve
```

```
# 特定国家代理
- GEOIP,US,🇺🇸美国节点,no-resolve
- GEOIP,JP,🇯🇵日本节点,no-resolve
- GEOIP,HK,🇭🇰香港节点,no-resolve



# 局域网直连
- GEOIP,LAN,DIRECT,no-resolve



# 私有地址
- GEOIP,PRIVATE,DIRECT,no-resolve
```

国家代码（ISO 3166-1 alpha-2）：

```
CN - 中国大陆


HK - 香港
TW - 台湾
US - 美国
JP - 日本
KR - 韩国
SG - 新加坡
GB - 英国
DE - 德国
FR - 法国
```

8.4.3 GeoIP数据库来源

Clash使用的GeoIP数据库：

```
# Country.mmdb文件位置


Windows: %USERPROFILE%\.config\clash\Country.mmdb
macOS: ~/.config/clash/Country.mmdb
Linux: ~/.config/clash/Country.mmdb
```

```
# 数据库来源


  • MaxMind GeoLite2（官方）

    https://github.com/Dreamacro/maxmind-geoip

  • Loyalsoldier版（推荐）

    https://github.com/Loyalsoldier/geoip

  特点:
  ✓ 包含中国IP段
  ✓ 定期更新
  ✓ 针对中国大陆优化
```

手动更新GeoIP数据库：

```
# 下载Loyalsoldier版本
# Windows PowerShell



Invoke-WebRequest -Uri "https://github.com/Loyalsoldier/geoip/releases/latest/download/Country

# Linux/macOS



wget -O ~/.config/clash/Country.mmdb https://github.com/Loyalsoldier/geoip/releases/latest/dow

# 或使用curl



curl -L -o ~/.config/clash/Country.mmdb https://github.com/Loyalsoldier/geoip/releases/latest/
```

Clash配置自动更新：

```
# config.yaml中配置（Clash.Meta支持）

geodata-mode: true
geox-url:
  geoip: "https://cdn.jsdelivr.net/gh/Loyalsoldier/geoip@release/Country.mmdb"
  geosite: "https://cdn.jsdelivr.net/gh/Loyalsoldier/v2ray-rules-dat@release/geosite.dat"
```

8.4.4 GeoIP vs IP-CIDR

对比：

维度	GEOIP	IP-CIDR
便利性	简单（一条规则）	复杂（需列举所有IP段）
准确性	依赖数据库	完全精确
维护	需定期更新数据库	规则相对固定
性能	需查询数据库	直接匹配
覆盖范围	全国所有IP	指定IP段

使用建议：

```
推荐GEOIP：

✓ 国内IP直连（GEOIP,CN,DIRECT）
✓ 按国家分流（流媒体）
✓ 简化配置



推荐IP-CIDR：
✓ 特定服务（Telegram、iCloud等）
✓ 局域网地址
✓ 精确控制
```

8.5 自定义规则集

8.5.1 规则集（Rule Providers）

什么是规则集?

将大量相关规则组织成单独的文件，通过引用方式使用。

```
优势:


✓ 模块化管理
✓ 便于维护
✓ 可在线更新
✓ 多配置复用
```

Clash规则集配置:

```
rule-providers:


  # Google规则集
  google:
    type: http
    behavior: domain        # domain/ipcidr/classical
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/google.txt"
    path: ./ruleset/google.yaml
    interval: 86400        # 更新间隔（秒）



  # Apple规则集
  apple:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/apple.txt"
    path: ./ruleset/apple.yaml
    interval: 86400



  # 中国IP规则集
  cnip:
    type: http
    behavior: ipcidr
```

```
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/cncidr.txt"
    path: ./ruleset/cnip.yaml
    interval: 86400



  # 广告屏蔽规则集
  reject:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/reject.txt"
    path: ./ruleset/reject.yaml
    interval: 86400



rules:
  # 使用规则集
  - RULE-SET,google,[新]代理
  - RULE-SET,apple,🍎Apple
  - RULE-SET,reject,REJECT
  - RULE-SET,cnip,DIRECT
  - MATCH,[新]代理
```

8.5.2 规则集类型

behavior类型：

1. domain（域名规则）

```
# 规则文件内容（google.txt）


payload:
  - 'google.com'
  - 'googleapis.com'
  - 'googleusercontent.com'
  - '+.youtube.com'        # +. 表示包含子域名
  - 'gmail.com'

# 等效于


  • DOMAIN-SUFFIX,google.com,[新]代理
```

```
  • DOMAIN-SUFFIX,googleapis.com,[新]代理
```

2. ipcidr（IP规则）

```
# 规则文件内容（cnip.txt）

payload:
  - '1.0.1.0/24'
  - '1.0.2.0/23'
  - '1.0.8.0/21'
  - '1.0.32.0/19'

# 等效于

  • IP-CIDR,1.0.1.0/24,DIRECT,no-resolve

  • IP-CIDR,1.0.2.0/23,DIRECT,no-resolve
```

3. classical（经典规则）

```
# 规则文件内容（custom.txt）

payload:
  - DOMAIN-SUFFIX,google.com
  - DOMAIN,www.apple.com
  - DOMAIN-KEYWORD,facebook
  - IP-CIDR,192.168.0.0/16,no-resolve
  - GEOIP,CN
```

支持混合多种规则类型

8.5.3 常用规则集推荐

Loyalsoldier规则集（推荐）：

```
rule-providers:

  reject:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/reject.txt"
    path: ./ruleset/reject.yaml
    interval: 86400



  icloud:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/icloud.txt"
    path: ./ruleset/icloud.yaml
    interval: 86400



  apple:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/apple.txt"
    path: ./ruleset/apple.yaml
    interval: 86400



  google:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/google.txt"
    path: ./ruleset/google.yaml
    interval: 86400
```

```
  proxy:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/proxy.txt"
    path: ./ruleset/proxy.yaml
    interval: 86400



  direct:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/direct.txt"
    path: ./ruleset/direct.yaml
    interval: 86400



  private:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/private.txt"
    path: ./ruleset/private.yaml
    interval: 86400



  gfw:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/gfw.txt"
    path: ./ruleset/gfw.yaml
    interval: 86400



  telegramcidr:
    type: http
    behavior: ipcidr
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/telegramcidr.txt"
    path: ./ruleset/telegramcidr.yaml
    interval: 86400



  cncidr:
    type: http
    behavior: ipcidr
```

```
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/cncidr.txt"
    path: ./ruleset/cncidr.yaml
    interval: 86400



  lancidr:
    type: http
    behavior: ipcidr
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/lancidr.txt"
    path: ./ruleset/lancidr.yaml
    interval: 86400
```

8.5.4 自建规则集

创建本地规则集：

```
# 1. 创建规则文件: ./ruleset/custom.yaml


payload:
  # AI服务
  - 'openai.com'
  - 'anthropic.com'
  - 'claude.ai'
  - 'bard.google.com'



  # 开发工具
  - 'github.com'
  - 'gitlab.com'
  - 'stackoverflow.com'
  - 'npmjs.com'



  # 设计资源
  - 'figma.com'
  - 'dribbble.com'
  - 'behance.net'

# 2. 在config.yaml中引用
```

```
rule-providers:
  custom:
    type: file
    behavior: domain
    path: ./ruleset/custom.yaml



rules:
  - RULE-SET,custom,[新]代理
```

创建在线规则集：

```
# 1. 将规则文件上传到GitHub/Gitee
# 2. 获取Raw链接
# 3. 配置使用



rule-providers:
  my-rules:
    type: http
    behavior: domain
    url: "https://raw.githubusercontent.com/username/repo/main/rules.txt"
    path: ./ruleset/my-rules.yaml
    interval: 86400
```

8.6 常用规则推荐

8.6.1 完整规则配置示例

```
# Clash完整规则配置


proxy-groups:
  - name: "[新] 节点选择"
    type: select
```

```
    proxies:
     - "♻️ 自动选择"
     - "🇭🇰 香港节点"
     - "🇺🇸 美国节点"
      - "DIRECT"



  - name: "📲 Telegram"
    type: select
    proxies:
     - "[新] 节点选择"
     - "🇭🇰 香港节点"
     - "🇸🇬 新加坡节点"



  - name: "🎬 流媒体"
    type: select
    proxies:
     - "🇺🇸 美国节点"
     - "🇯🇵 日本节点"
     - "🇭🇰 香港节点"



  - name: "🍎 Apple"
    type: select
    proxies:
      - "DIRECT"
     - "[新] 节点选择"



  - name: "[目标] 全球直连"
    type: select
    proxies:
      - "DIRECT"
     - "[新] 节点选择"



rule-providers:
  reject:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/reject.txt"
    path: ./ruleset/reject.yaml
    interval: 86400
```

```yaml
  icloud:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/icloud.txt"
    path: ./ruleset/icloud.yaml
    interval: 86400


  apple:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/apple.txt"
    path: ./ruleset/apple.yaml
    interval: 86400


  google:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/google.txt"
    path: ./ruleset/google.yaml
    interval: 86400


  proxy:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/proxy.txt"
    path: ./ruleset/proxy.yaml
    interval: 86400


  direct:
    type: http
    behavior: domain
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/direct.txt"
    path: ./ruleset/direct.yaml
    interval: 86400


  telegramcidr:
    type: http
```

```
    behavior: ipcidr
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/telegramcidr.txt"
    path: ./ruleset/telegramcidr.yaml
    interval: 86400



  cncidr:
    type: http
    behavior: ipcidr
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/cncidr.txt"
    path: ./ruleset/cncidr.yaml
    interval: 86400



  lancidr:
    type: http
    behavior: ipcidr
    url: "https://cdn.jsdelivr.net/gh/Loyalsoldier/clash-rules@release/lancidr.txt"
    path: ./ruleset/lancidr.yaml
    interval: 86400



rules:
  # 本地网络直连
  - DOMAIN-SUFFIX,local,DIRECT
  - DOMAIN,clash.razord.top,DIRECT
  - DOMAIN,yacd.haishan.me,DIRECT



  # 广告屏蔽
  - RULE-SET,reject,REJECT



  # Telegram
  - RULE-SET,telegramcidr,📲 Telegram
  - DOMAIN-SUFFIX,t.me,📲 Telegram
  - DOMAIN-SUFFIX,telegram.org,📲 Telegram



  # Apple服务
  - RULE-SET,icloud,🍎 Apple
  - RULE-SET,apple,🍎 Apple
```

```
  # Google服务
  - RULE-SET,google,[新] 节点选择



  # 代理列表
  - RULE-SET,proxy,[新] 节点选择



  # 国内网站直连
  - RULE-SET,direct,[目标] 全球直连



  # 局域网
  - RULE-SET,lancidr,DIRECT,no-resolve



  # 中国IP直连
  - RULE-SET,cncidr,[目标] 全球直连,no-resolve
  - GEOIP,CN,[目标] 全球直连,no-resolve



  # 兜底规则
  - MATCH,[新] 节点选择
```

8.6.2 简化版规则

适合新手：

```
rules:

  # 局域网直连
  - DOMAIN-SUFFIX,local,DIRECT
  - IP-CIDR,127.0.0.0/8,DIRECT,no-resolve
  - IP-CIDR,192.168.0.0/16,DIRECT,no-resolve



  # 常用国外网站代理
  - DOMAIN-SUFFIX,google.com,[新]代理
  - DOMAIN-SUFFIX,youtube.com,[新]代理
  - DOMAIN-SUFFIX,facebook.com,[新]代理
```

```
  - DOMAIN-SUFFIX,twitter.com,[新]代理
  - DOMAIN-SUFFIX,instagram.com,[新]代理
  - DOMAIN-SUFFIX,github.com,[新]代理



  # 常用国内网站直连
  - DOMAIN-SUFFIX,baidu.com,DIRECT
  - DOMAIN-SUFFIX,qq.com,DIRECT
  - DOMAIN-SUFFIX,taobao.com,DIRECT
  - DOMAIN-SUFFIX,jd.com,DIRECT
  - DOMAIN-SUFFIX,bilibili.com,DIRECT



  # 中国IP直连
  - GEOIP,CN,DIRECT,no-resolve



  # 其他走代理
  - MATCH,[新]代理
```

8.6.3 规则调试技巧

查看规则匹配日志：

```
# Clash配置

log-level: info    # 或debug（更详细）

# 查看日志文件



Windows: %USERPROFILE%\.config\clash\logs\
macOS/Linux: ~/.config/clash/logs/

# 日志内容示例
```

```
[INFO] [DNS] www.google.com --> 142.250.185.46
[INFO] [Rule] DOMAIN-SUFFIX,google.com -> [新]代理
```

测试规则匹配：

```
Clash Dashboard (YACD)：
  • 打开 http://clash.razord.top



  • 连接到Clash API


  • 查看Connections（连接）


  • 观察每个连接匹配的规则
```

常见问题排查：

```
问题1：规则不生效

检查：
✓ 规则顺序（是否被前面的规则拦截）
✓ 语法正确性
✓ 代理组名称是否匹配
✓ 重启Clash



问题2：规则集无法下载
检查：
✓ URL是否正确
✓ 网络连接（可能需要代理下载）
✓ 使用CDN加速链接



问题3：部分网站仍然直连/代理错误
检查：
✓ DNS设置
```

```
✓ 是否有缓存
✓ 规则优先级
✓ 使用DOMAIN精确匹配特殊处理
```

本章小结

本章详细讲解了分流规则的编写和应用：

核心要点：

- **规则基础**：

- 分流原理和优势

- 规则匹配机制 - 规则类型概览

- **域名规则**：

- DOMAIN（精确匹配）

- DOMAIN-SUFFIX（后缀匹配） - DOMAIN-KEYWORD（关键词匹配）

- **IP规则**：

- IP-CIDR语法和应用

- CIDR表示法 - IPv6规则

- **GeoIP**：

- 地理位置匹配

- GeoIP数据库更新 - 与IP-CIDR对比

- **规则集**：

- Rule Providers使用

- 在线/本地规则集 - 自建规则集

- **实战配置**：

- 完整规则示例

- 简化版配置 - 调试技巧

规则编写原则：

```
1. 精确优先
```

```
  DOMAIN > DOMAIN-SUFFIX > DOMAIN-KEYWORD


• 特殊在前


  具体规则 → 通用规则 → 兜底规则
• 性能考虑


  常用规则前置


  使用规则集减少规则数量


• 维护方便


  模块化管理


  使用在线规则集自动更新
```

下一章我们将学习进阶优化技巧，进一步提升代理性能。

实践任务：

- 编写自己的分流规则
- 配置规则集并测试
- 观察规则匹配日志
- 优化规则顺序和性能
- 创建自定义规则集

第九章 进阶优化技巧

第9章 进阶优化技巧

在掌握了基本配置后，本章将介绍各种进阶优化技巧，帮助你获得更快、更稳定的科学上网体验。

9.1 TCP优化（BBR等）

9.1.1 TCP拥塞控制算法

什么是拥塞控制?

TCP拥塞控制算法决定了数据传输的速度和效率。

```
网络拥堵时:

传统算法 → 大幅降速 → 恢复慢
BBR算法 → 智能调节 → 保持高速
```

常见算法对比:

算法	年代	特点	性能
Reno	1990	经典算法	[推荐][推荐]
CUBIC	2005	Linux默认	[推荐][推荐][推荐]
BBR	2016	Google开发	[推荐][推荐][推荐][推荐][推荐]
BBRv2	2020	BBR改进版	[推荐][推荐][推荐][推荐][推荐]

9.1.2 BBR算法原理

BBR (Bottleneck Bandwidth and RTT):

```
传统算法:
```

```
丢包 → 认为拥堵 → 降速
      (可能误判，如WiFi干扰导致的丢包)



BBR算法：
持续探测：
  ├── 测量瓶颈带宽
  ├── 测量往返时延
  └── 智能调节发送速率



优势：
✓ 高丢包环境仍保持高速
✓ 跨国长距离连接提升明显
✓ 充分利用带宽
```

性能提升：

```
典型场景：国内→美国VPS

CUBIC算法：
下载速度：~5 MB/s
延迟：200ms
丢包率：1%



BBR算法：
下载速度：~15-20 MB/s（提升3-4倍）
延迟：180ms
丢包率：1%（相同条件）



注：实际提升取决于网络环境
```

9.1.3 开启BBR（Linux服务器）

检查当前算法：

```
# 查看可用算法
```

```
sysctl net.ipv4.tcp_available_congestion_control

# 查看当前使用算法


sysctl net.ipv4.tcp_congestion_control

# 查看内核版本（BBR需要4.9+）


uname -r
```

启用BBR：

```
# 方法1：临时启用（重启失效）

sudo sysctl -w net.core.default_qdisc=fq
sudo sysctl -w net.ipv4.tcp_congestion_control=bbr

# 方法2：永久启用

# 编辑sysctl配置


sudo nano /etc/sysctl.conf

# 添加以下内容
```

```
net.core.default_qdisc=fq
net.ipv4.tcp_congestion_control=bbr

# 保存后应用


sudo sysctl -p

# 验证


sysctl net.ipv4.tcp_congestion_control

# 输出: net.ipv4.tcp_congestion_control = bbr


lsmod | grep bbr

# 如果显示tcp_bbr，说明BBR已加载
```

一键脚本：

```
# 秋水逸冰BBR一键脚本

wget --no-check-certificate https://github.com/teddysun/across/raw/master/bbr.sh
chmod +x bbr.sh
./bbr.sh

# 脚本会：
```

```
# 1. 检查内核版本

# 2. 升级内核（如需要）

# 3. 启用BBR

# 4. 重启系统

# 重启后验证

uname -r                                    # 查看内核版本
sysctl net.ipv4.tcp_congestion_control    # 查看BBR状态
```

9.1.4 BBR Plus / BBR2 / 锐速

BBR Plus（魔改BBR）：

```
# 特点:

✓ 基于BBR改进
✓ 更激进的策略
✓ 某些场景速度更快
[注意] 可能增加丢包

# 安装（需谨慎）
```

```
wget "https://github.com/chiakge/Linux-NetSpeed/raw/master/tcp.sh"
chmod +x tcp.sh
./tcp.sh

# 选择：安装 BBR Plus
```

BBRv2（官方改进版）：

```
# 需要Linux 5.10+内核
# Debian 11/Ubuntu 20.04+默认支持

# 启用BBRv2



sudo sysctl -w net.ipv4.tcp_congestion_control=bbr2
echo "net.ipv4.tcp_congestion_control=bbr2" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

BBRv3（最新版本）：

```
# 特点:

✓ 进一步改进公平性
✓ 优化拥塞控制算法
✓ 更好的多流性能
✓ 减少缓冲区膨胀

# 需要Linux 5.18+内核
```

```
# Debian 12/Ubuntu 22.04+支持

# 检查是否支持

sysctl net.ipv4.tcp_available_congestion_control

# 启用BBRv3（如果支持）

sudo sysctl -w net.ipv4.tcp_congestion_control=bbr
echo "net.ipv4.tcp_congestion_control=bbr" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# 注意：较新内核中的bbr可能已是v3版本

# 查看实际版本需检查内核源码
```

版本选择建议：

```
推荐方案：

• 生产环境：BBR v1（最稳定，5.4+内核）

• 追求性能：BBRv2/v3（需较新内核）

• 不确定时：使用系统默认的BBR版本
```

```
避免:


  • BBR Plus等第三方魔改版（可能不稳定）



  • 频繁切换版本（影响连接稳定性）
```

锐速（LotServer）：

```
特点:


✓ 商业加速软件
✓ 单边加速（仅服务器端）
✓ 对丢包环境优化好
[注意] 闭源，有风险
[注意] 部分内核不兼容



不推荐使用（BBR已足够）
```

9.1.5 客户端TCP优化

Windows优化：

```
# 以管理员运行PowerShell
# 1. 启用TCP窗口自动调节



netsh interface tcp set global autotuninglevel=normal

# 2. 启用直接缓存访问
```

```
netsh interface tcp set global dca=enabled

# 3. 启用接收端合并


netsh interface tcp set global rsc=enabled

# 4. 查看当前设置


netsh interface tcp show global
```

macOS优化：

```
# 增大接收缓冲区

sudo sysctl -w net.inet.tcp.recvspace=65536
sudo sysctl -w net.inet.tcp.sendspace=65536

# 启用TCP窗口缩放


sudo sysctl -w net.inet.tcp.rfc1323=1
```

Linux优化：

```
# 编辑 /etc/sysctl.conf

sudo nano /etc/sysctl.conf
```

```
# 添加以下优化参数

# TCP优化

net.ipv4.tcp_fastopen=3
net.ipv4.tcp_slow_start_after_idle=0
net.ipv4.tcp_no_metrics_save=1

# 缓冲区优化

net.core.rmem_max=67108864
net.core.wmem_max=67108864
net.core.netdev_max_backlog=250000
net.core.somaxconn=4096
net.ipv4.tcp_rmem=4096 87380 67108864
net.ipv4.tcp_wmem=4096 65536 67108864
net.ipv4.tcp_mem=67108864 67108864 67108864

# 应用设置

sudo sysctl -p
```

9.2 UDP加速配置

9.2.1 为什么需要UDP

UDP的重要性：

```
TCP协议：
```

```
✓ 可靠传输
✓ 顺序保证
✗ 延迟高
✗ 效率低



UDP协议:
✓ 低延迟
✓ 效率高
✗ 不保证送达
✗ 无序传输



适用场景:
UDP必需:


  • 视频通话（Zoom、Teams）



  • 在线游戏


  • 语音通话


  • 直播推流




TCP即可:


  • 网页浏览



  • 文件下载


  • 邮件收发
```

9.2.2 代理协议UDP支持

Shadowsocks UDP：

```
{

  "server": "your-server.com",
  "server_port": 8388,
  "password": "your-password",
  "method": "chacha20-ietf-poly1305",
  "mode": "tcp_and_udp",    # 关键：同时支持TCP和UDP
  "fast_open": true
}
```

V2Ray UDP：

```
{

  "inbounds": [{
    "port": 1080,
    "protocol": "socks",
    "settings": {
      "udp": true    # 启用UDP
   }
  }],
  "outbounds": [{
    "protocol": "vmess",
    "settings": {
      "vnext": [{
        "address": "server.com",
        "port": 443,
        "users": [{
          "id": "uuid"
       }]
     }]
   },
    "streamSettings": {
      "network": "tcp"    # 传输协议
   }
  }]
}
```

Clash UDP：

```
# 全局UDP设置

mixed-port: 7890
allow-lan: false
mode: rule
log-level: info
ipv6: false
enable-process: true

# 节点UDP支持



proxies:
  - name: "节点1"
    type: ss
    server: server.com
    port: 8388
    cipher: chacha20-ietf-poly1305
    password: password
    udp: true    # 启用UDP



  - name: "节点2"
    type: vmess
    server: server.com
    port: 443
    uuid: your-uuid
    alterId: 0
    cipher: auto
    tls: true
    network: ws
    udp: true    # 启用UDP
```

9.2.3 QUIC协议

QUIC (Quick UDP Internet Connections)：

```
QUIC特点:
```

```
✓ 基于UDP
✓ 0-RTT连接建立
✓ 多路复用
✓ 内置加密
✓ 连接迁移（IP变化不断线）



应用:


  • HTTP/3基于QUIC



  • Google服务广泛使用


  • Chrome优先使用QUIC
```

V2Ray QUIC传输：

```
{

  "streamSettings": {
    "network": "quic",
    "quicSettings": {
      "security": "aes-128-gcm",
      "key": "your-key-here",
      "header": {
        "type": "dtls"    # 或 "wechat-video"
     }
    },
    "security": "none"
  }
}
```

优势与限制：

```
优势:
```

```
✓ 移动网络表现优秀
✓ 快速建立连接
✓ 抗丢包能力强



限制:
✗ 部分ISP限制UDP
✗ 某些防火墙阻止UDP
✗ NAT环境可能有问题



建议:


  • WiFi/移动网络: 推荐



  • 企业网络: 可能被限制


  • 家庭宽带: 通常可用
```

9.2.4 Hysteria 2协议

Hysteria 2 - 基于QUIC的高性能代理:

```
核心优势:


✓ 基于QUIC协议, 原生UDP支持
✓ 自适应拥塞控制算法
✓ 多路复用, 单连接高效
✓ 抗丢包能力极强
✓ 移动网络表现优异



适用场景:


  • 高延迟网络环境
```

```
  • 弱网环境（丢包率高）

  • 移动网络（4G/5G）

  • 需要高速上传的场景
```

Hysteria 2服务端配置：

```
# /etc/hysteria/config.yaml

listen: :443



tls:
  cert: /path/to/cert.pem
  key: /path/to/key.pem



auth:
  type: password
  password: your-strong-password



masquerade:
  type: proxy
  proxy:
    url: https://www.bing.com
    rewriteHost: true



bandwidth:
  up: 100 mbps    # 服务器上行带宽
  down: 100 mbps  # 服务器下行带宽



quic:
  initStreamReceiveWindow: 8388608
```

```
  maxStreamReceiveWindow: 8388608
  initConnReceiveWindow: 20971520
  maxConnReceiveWindow: 20971520
```

Hysteria 2客户端配置：

```
server: your-server.com:443

auth: your-strong-password



tls:
  sni: your-server.com
  insecure: false



bandwidth:
  up: 50 mbps     # 客户端上行带宽
  down: 200 mbps  # 客户端下行带宽



socks5:
  listen: 127.0.0.1:1080



http:
  listen: 127.0.0.1:8080
```

Hysteria 2优化建议：

```
带宽设置技巧：

• 设置为实际带宽的80-90%



• 上行带宽尤其重要（影响ACK包）


• 可以通过测速工具确定实际带宽
```

```
性能调优:

  • initStreamReceiveWindow：初始流窗口

  - 默认8MB，高带宽可调至16MB
  • maxConnReceiveWindow：最大连接窗口

  - 默认20MB，高延迟可调至40MB

伪装配置:

  • 使用masquerade伪装成正常HTTPS网站



  • 推荐伪装为大型网站（如Bing、必应）

  • 配置rewriteHost确保完美伪装




与其他协议对比:
Hysteria 2 vs QUIC：
✓ 更激进的拥塞控制
✓ 专为代理场景优化
✓ 弱网环境下速度更快



Hysteria 2 vs mKCP：
✓ 更成熟的协议栈
✓ 更好的带宽利用率
✓ 更强的抗检测能力
```

9.2.5 UDP加速实战

mKCP（KCP over UDP）：

```
{

  "streamSettings": {
    "network": "mkcp",
    "kcpSettings": {
      "mtu": 1350,
      "tti": 50,
      "uplinkCapacity": 12,      # 上行带宽（MB）
      "downlinkCapacity": 100,   # 下行带宽（MB）
      "congestion": true,        # 拥塞控制
      "readBufferSize": 2,
      "writeBufferSize": 2,
      "header": {
        "type": "wechat-video"   # 伪装类型
     }
    }
  }
}
```

mKCP参数优化：

```
{

  "kcpSettings": {
    // 基础参数
     "mtu": 1350,           # 最大传输单元（不建议修改）
     "tti": 50,             # 传输时间间隔，越小越快但流量越大（10-100）



    // 带宽设置（关键）
     "uplinkCapacity": 5,   # 你的上行带宽（MB/s）
     "downlinkCapacity": 20,# 你的下行带宽（MB/s）



    // 拥塞控制
     "congestion": true,    # 启用拥塞控制（推荐）



    // 缓冲区
     "readBufferSize": 2,   # 读取缓冲区（MB）
     "writeBufferSize": 2,  # 写入缓冲区（MB）
```

```
    // 伪装
     "header": {
       "type": "wechat-video"  # 伪装成微信视频通话
      // 其他选项: none, srtp, utp, dtls, wireguard
    }
  }
}



场景推荐:



游戏加速:
tti = 10-20 (低延迟)
congestion = false (最大速度)



视频通话:
tti = 20-30 (平衡)
congestion = true (稳定)



日常使用:
tti = 50 (默认)
congestion = true (省流量)
```

9.3 WebSocket+TLS伪装

9.3.1 为什么使用WebSocket+TLS

优势:

```
1. CDN友好

   可以通过Cloudflare等CDN
   隐藏真实服务器IP


 • 流量特征
```

```
    看起来像正常HTTPS网站流量

    难以被识别和屏蔽

  • 端口复用

    与网站共用443端口

    降低被检测的可能性

  • 兼容性好

    几乎所有网络环境都支持

    企业网络、公共WiFi通常放行
```

9.3.2 V2Ray WebSocket+TLS配置

服务器端配置：

```
{

  "inbounds": [{
    "port": 10000,
    "listen": "127.0.0.1",
    "protocol": "vmess",
    "settings": {
      "clients": [{
        "id": "your-uuid-here",
        "alterId": 0
     }]
   },
    "streamSettings": {
      "network": "ws",
      "wsSettings": {
        "path": "/v2ray"    # WebSocket路径
     }
   }
  }],
  "outbounds": [{
```

```
    "protocol": "freedom"
  }]
}
```

Nginx反向代理配置：

```
server {

    listen 443 ssl http2;
    server_name your-domain.com;



    # SSL证书配置
    ssl_certificate /path/to/fullchain.pem;
    ssl_certificate_key /path/to/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;



   # 网站根目录（伪装）
    root /var/www/html;
    index index.html;



   # 正常网站访问
    location / {
        try_files $uri $uri/ =404;
   }



    # V2Ray WebSocket代理
    location /v2ray {
        if ($http_upgrade != "websocket") {
            return 404;
       }
        proxy_redirect off;
        proxy_pass http://127.0.0.1:10000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

HTTP自动跳转HTTPS

```
server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$server_name$request_uri;
}
```

客户端配置：

```
{

  "outbounds": [{
    "protocol": "vmess",
    "settings": {
      "vnext": [{
        "address": "your-domain.com",
        "port": 443,
        "users": [{
          "id": "your-uuid-here",
          "alterId": 0
       }]
     }]
   },
    "streamSettings": {
      "network": "ws",
      "security": "tls",
      "tlsSettings": {
        "serverName": "your-domain.com",
        "allowInsecure": false
     },
      "wsSettings": {
        "path": "/v2ray",
        "headers": {
          "Host": "your-domain.com"
       }
     }
```

```
    }
  }]
}
```

9.3.3 使用Caddy简化配置

Caddy自动HTTPS：

```
# Caddyfile配置

your-domain.com {
    # 正常网站
     root * /var/www/html
     file_server



     # V2Ray反向代理
     @v2ray {
         path /v2ray
         header Connection Upgrade
         header Upgrade websocket
    }
     reverse_proxy @v2ray localhost:10000
}
```

Caddy自动：

✓ 申请Let's Encrypt证书

✓ 自动续期

✓ 强制HTTPS

```
✓ HTTP/2支持
```

启动Caddy：

```
# 安装Caddy

curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o /usr
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee /etc/apt/s
sudo apt update
sudo apt install caddy

编辑Caddyfile


sudo nano /etc/caddy/Caddyfile

启动Caddy


sudo systemctl start caddy
sudo systemctl enable caddy

查看状态


sudo systemctl status caddy
```

9.4 CDN中转优化

9.4.1 CDN中转原理

工作流程：

```
传统直连：

用户 ──────────► VPS服务器 ──────────► 互联网
      直接连接     (IP容易被墙)



CDN中转：
用户 ──────────► Cloudflare CDN ──────► 源站VPS ──────► 互联网
      HTTPS          (隐藏真实IP)         代理流量



优势：
✓ 隐藏真实服务器IP
✓ Cloudflare全球节点就近接入
✓ 免费SSL证书
✓ DDoS防护
✓ 流量特征像正常HTTPS
```

9.4.2 Cloudflare CDN配置

前提条件：

```
✓ 一个域名


✓ WebSocket+TLS配置的V2Ray
✓ Cloudflare账号（免费）
```

配置步骤：

```
1. 添加域名到Cloudflare
```

```
  - 登录Cloudflare
  - 添加站点 → 输入域名
  - 选择Free计划
  - 修改域名DNS服务器到Cloudflare


• DNS设置


  - 添加A记录


    名称: cdn（或其他子域名）
    内容: 你的VPS IP
    代理状态: 已代理（橙色云朵）


• SSL/TLS设置


  - 加密模式: 完全（严格）


  - 边缘证书: 自动HTTPS重写（开启）


• 网络设置


  - WebSocket：开启


  - HTTP/2：开启
  - gRPC：开启（如果用gRPC）


• 速度优化


  - 自动压缩: 开启


  - Brotli：开启
  - HTTP/3 (QUIC)：开启
```

客户端配置：

```
{
```

```
  "outbounds": [{
    "protocol": "vmess",
    "settings": {
      "vnext": [{
        "address": "cdn.your-domain.com",    # 使用CDN域名
        "port": 443,
        "users": [{
          "id": "your-uuid"
       }]
     }]
   },
    "streamSettings": {
      "network": "ws",
      "security": "tls",
      "tlsSettings": {
        "serverName": "cdn.your-domain.com"
     },
      "wsSettings": {
        "path": "/v2ray",
        "headers": {
          "Host": "cdn.your-domain.com"
       }
     }
   }
  }]
}
```

9.4.3 CDN优选IP

为什么需要优选IP?

```
问题:


Cloudflare分配的CDN节点可能:
✗ 延迟高
✗ 速度慢
✗ 丢包严重



解决:
通过优选IP找到最快的CDN节点
```

CloudflareSpeedTest工具：

```
# Windows下载

https://github.com/XIU2/CloudflareSpeedTest/releases

# 运行测速



CloudflareST.exe

# 参数说明



CloudflareST.exe \
  -n 200              # 测速线程数
  -t 4                # 延迟测速次数
  -dn 10              # 下载测速数量
  -url https://...    # 测速文件URL

# 等待测试完成，获得最优IP列表
```

使用优选IP：

```
// V2Ray配置

{
  "outbounds": [{
    "protocol": "vmess",
    "settings": {
      "vnext": [{
        "address": "104.18.32.167",    # 使用优选的IP
        "port": 443,
        "users": [{
          "id": "your-uuid"
       }]
     }]
```

```
    },
     "streamSettings": {
       "network": "ws",
       "security": "tls",
       "tlsSettings": {
         "serverName": "cdn.your-domain.com"   # Host仍使用域名
      },
       "wsSettings": {
         "path": "/v2ray",
         "headers": {
           "Host": "cdn.your-domain.com"       # Host使用域名
        }
      }
    }
  }]
}
```

推荐优选IP（供参考）：

```
常用IP段（仅供参考）：


电信：
104.16.160.0/24    # 洛杉矶
104.17.0.0/16
162.159.0.0/16
172.64.0.0/24      # 旧金山



联通：
104.16.0.0/16
108.162.236.0/24   # 美国
104.20.157.0/24    # 日本



移动：
172.64.32.0/24     # 香港
172.64.0.0/16
141.101.115.0/24
104.28.14.0/24     # 新加坡



重要提示：
[注意] IP段会变化，上述仅为参考
[注意] 不同地区、不同运营商结果差异大
```

```
✓ 必须自己测试找到最优IP
✓ 建议每腧1-2周重新测试
✓ 不同时段最优IP可能不同
```

高级优化技巧：

```
1. 区域锁定IP技巧：
  • 使用Cloudflare Workers设置固定落地IP
  • 可解锁特定区域服务（如ChatGPT、Netflix）
  • 参考项目：github.com/yonggekkk/Cloudflare-vless-trojan
  • 智能路由：
  • 为不同目标配置不同CDN节点
  • 美国服务使用美国节点
  • 亚洲服务使用亚洲节点
  • 多个IP轮询：
  • 配置多个优选IP
  • 自动故障转移
  • 提高可用性
```

```
  • 测速工具选择:

推荐工具:
  • CloudflareSpeedTest:https://github.com/XIU2/CloudflareSpeedTest



  • 支持Windows/Linux/macOS

  • 自动导出最优IP列表

  • 可自定义测试参数
```

9.4.4 gRPC over CDN

gRPC优势:

```
相比WebSocket:

✓ HTTP/2多路复用
✓ 更好的性能
✓ 更真实的流量特征
✓ Cloudflare原生支持
```

V2Ray gRPC配置:

```
// 服务器

{
  "inbounds": [{
    "port": 10000,
    "listen": "127.0.0.1",
    "protocol": "vmess",
    "settings": {
      "clients": [{
        "id": "your-uuid"
     }]
   },
```

```
    "streamSettings": {
      "network": "grpc",
      "grpcSettings": {
        "serviceName": "your-service-name",
        "multiMode": true    # 多路复用
     }
   }
  }]
}



// Nginx配置
server {
    listen 443 ssl http2;
    server_name your-domain.com;



    ssl_certificate /path/to/cert;
    ssl_certificate_key /path/to/key;



    location / {
        root /var/www/html;
   }



    location /your-service-name {
        grpc_pass grpc://127.0.0.1:10000;
        grpc_set_header X-Real-IP $remote_addr;
   }
}



// 客户端
{
  "outbounds": [{
    "protocol": "vmess",
    "settings": {
      "vnext": [{
        "address": "cdn.your-domain.com",
        "port": 443,
        "users": [{"id": "your-uuid"}]
     }]
   },
    "streamSettings": {
```

```
      "network": "grpc",
      "security": "tls",
      "tlsSettings": {
        "serverName": "cdn.your-domain.com"
     },
      "grpcSettings": {
        "serviceName": "your-service-name",
        "multiMode": true
     }
    }
  }]
}
```

9.5 多层代理链

9.5.1 代理链概念

什么是代理链?

```
单层代理:

用户 → 代理1 → 目标网站



两层代理:
用户 → 代理1 → 代理2 → 目标网站



三层代理:
用户 → 代理1 → 代理2 → 代理3 → 目标网站



用途:
✓ 增强隐私（多跳）
✓ 突破多层封锁
✓ 地区跳转
✗ 速度降低
✗ 稳定性下降
```

9.5.2 V2Ray代理链配置

场景：国内VPS中转 + 国外VPS

```
// 客户端配置


{
  "outbounds": [
    {
       "tag": "proxy1",
       "protocol": "vmess",
       "settings": {
         "vnext": [{
           "address": "china-vps.com",    # 国内中转VPS
           "port": 443,
           "users": [{"id": "uuid1"}]
        }]
      },
       "streamSettings": {
         "network": "ws",
         "security": "tls"
      },
       "proxySettings": {
         "tag": "proxy2"    # 通过proxy2连接
      }
    },
    {
       "tag": "proxy2",
       "protocol": "vmess",
       "settings": {
         "vnext": [{
           "address": "foreign-vps.com",  # 国外VPS
           "port": 443,
           "users": [{"id": "uuid2"}]
        }]
      },
       "streamSettings": {
         "network": "ws",
         "security": "tls"
      }
    }
  ]
}
```

9.5.3 IPLC/IEPL专线

什么是专线?

```
IPLC (International Private Leased Circuit):

国际专线，点对点直连



特点:
✓ 不经过GFW
✓ 速度极快
✓ 延迟稳定
✓ 不限流量（通常）
✗ 价格昂贵



使用场景:

  • 企业跨国通信



  • 游戏主播

  • 外贸从业者

  • 高要求用户
```

专线+中转方案:

```
本地 → 国内中转 → IPLC专线 → 香港/日本 → 互联网

优势:
✓ 国内段速度快
✓ 国际段不过墙
✓ 综合体验最佳
```

```
成本:
约￥50-200/月（100-500Mbps）
```

9.6 国内外分流最佳实践

9.6.1 完整分流方案

Clash完整配置示例：

```
# 端口配置

mixed-port: 7890
allow-lan: false
bind-address: "*"
mode: rule
log-level: info
ipv6: false
external-controller: 127.0.0.1:9090
external-ui: folder

# DNS配置



dns:
  enable: true
  listen: 0.0.0.0:53
  ipv6: false
  enhanced-mode: fake-ip
  fake-ip-range: 198.18.0.1/16
  fake-ip-filter:
    - '*.lan'
    - localhost.ptlogin2.qq.com
  nameserver:
   - 223.5.5.5
   - 119.29.29.29
  fallback:
    - https://1.1.1.1/dns-query
    - https://8.8.8.8/dns-query
  fallback-filter:
```

```
    geoip: true
    ipcidr:
     - 240.0.0.0/4
```

代理组

```
proxy-groups:
  - name: "[新] 节点选择"
    type: select
    proxies:
     - "♻️ 自动选择"
     - "🇭🇰 香港节点"
     - "🇺🇸 美国节点"
     - "🇯🇵 日本节点"
     - "🇸🇬 新加坡节点"
      - "DIRECT"


  - name: "♻️ 自动选择"
    type: url-test
    url: http://www.gstatic.com/generate_204
    interval: 300
    tolerance: 50
    proxies:
     - "🇭🇰 香港节点"
     - "🇺🇸 美国节点"
     - "🇯🇵 日本节点"


  - name: "📲 Telegram"
    type: select
    proxies:
     - "[新] 节点选择"
     - "🇭🇰 香港节点"
     - "🇸🇬 新加坡节点"


  - name: "🎬 流媒体"
    type: select
    proxies:
```

```
    - "🇺🇸 美国节点"
    - "🇯🇵 日本节点"
    - "🇭🇰 香港节点"



  - name: "🍎 Apple"
    type: select
    proxies:
      - "DIRECT"
    - "[新] 节点选择"



  - name: "Ⓜ Microsoft"
    type: select
    proxies:
      - "DIRECT"
    - "[新] 节点选择"



  - name: "[游戏] 游戏平台"
    type: select
    proxies:
      - "DIRECT"
    - "[新] 节点选择"



  - name: "[目标] 全球直连"
    type: select
    proxies:
      - "DIRECT"
    - "[新] 节点选择"



  - name: "🛑 广告拦截"
    type: select
    proxies:
      - "REJECT"
      - "DIRECT"



  - name: "🐟 漏网之鱼"
    type: select
```

```
    proxies:
     - "[新] 节点选择"
      - "DIRECT"
```

规则

```
rules:
  # 本地网络
  - DOMAIN-SUFFIX,local,DIRECT
  - IP-CIDR,127.0.0.0/8,DIRECT
  - IP-CIDR,172.16.0.0/12,DIRECT
  - IP-CIDR,192.168.0.0/16,DIRECT
  - IP-CIDR,10.0.0.0/8,DIRECT



  # 广告拦截
  - DOMAIN-KEYWORD,adservice,🛑 广告拦截
  - DOMAIN-KEYWORD,analytics,🛑 广告拦截
  - DOMAIN-SUFFIX,googlesyndication.com,🛑 广告拦截



  # Telegram
  - DOMAIN-SUFFIX,t.me,📲 Telegram
  - DOMAIN-SUFFIX,telegram.org,📲 Telegram
  - IP-CIDR,91.108.4.0/22,📲 Telegram,no-resolve
  - IP-CIDR,91.108.56.0/22,📲 Telegram,no-resolve
  - IP-CIDR,149.154.160.0/20,📲 Telegram,no-resolve



  # 流媒体
  - DOMAIN-SUFFIX,netflix.com,🎬 流媒体
  - DOMAIN-SUFFIX,youtube.com,🎬 流媒体
  - DOMAIN-KEYWORD,youtube,🎬 流媒体
  - DOMAIN-SUFFIX,hulu.com,🎬 流媒体
  - DOMAIN-SUFFIX,hbo.com,🎬 流媒体
  - DOMAIN-SUFFIX,disney.com,🎬 流媒体
```

```
# Apple
- DOMAIN-SUFFIX,apple.com,🍎 Apple
- DOMAIN-SUFFIX,icloud.com,🍎 Apple
- DOMAIN-SUFFIX,apple-dns.net,🍎 Apple



# Microsoft
- DOMAIN-SUFFIX,microsoft.com,Ⓜ️ Microsoft
- DOMAIN-SUFFIX,windows.com,Ⓜ️ Microsoft
- DOMAIN-SUFFIX,office.com,Ⓜ️ Microsoft



# 游戏平台
- DOMAIN-SUFFIX,steam-chat.com,[游戏] 游戏平台
- DOMAIN-SUFFIX,epicgames.com,[游戏] 游戏平台



# Google
- DOMAIN-SUFFIX,google.com,[新] 节点选择
- DOMAIN-SUFFIX,googleapis.com,[新] 节点选择
- DOMAIN-SUFFIX,gstatic.com,[新] 节点选择



# 国外服务
- DOMAIN-SUFFIX,github.com,[新] 节点选择
- DOMAIN-SUFFIX,githubusercontent.com,[新] 节点选择
- DOMAIN-SUFFIX,twitter.com,[新] 节点选择
- DOMAIN-SUFFIX,facebook.com,[新] 节点选择
- DOMAIN-SUFFIX,instagram.com,[新] 节点选择



# 国内服务
- DOMAIN-SUFFIX,cn,[目标] 全球直连
- DOMAIN-KEYWORD,baidu,[目标] 全球直连
- DOMAIN-KEYWORD,taobao,[目标] 全球直连
- DOMAIN-KEYWORD,alipay,[目标] 全球直连
- DOMAIN-SUFFIX,qq.com,[目标] 全球直连
- DOMAIN-SUFFIX,bilibili.com,[目标] 全球直连



# GeoIP
- GEOIP,CN,[目标] 全球直连
```

```
  # 兜底规则
  - MATCH,🐟 漏网之鱼
```

9.6.2 性能优化建议

配置优化：

```
# 1. 使用fake-ip模式（提升速度）

dns:
  enhanced-mode: fake-ip

# 2. 启用规则集缓存

# 规则集会自动缓存在本地

# 3. 减少不必要的规则

# 使用规则集代替大量单条规则

# 4. 优化代理组

# 自动选择组不要包含太多节点（建议<10个）
```

系统优化：

```
1. 关闭不必要的杀毒软件实时保护
  • 排除代理软件目录
```

- 使用SSD存储配置文件
- 定期清理日志文件
- 保持软件更新

本章小结

本章介绍了多种进阶优化技巧：

核心要点：

- **TCP优化**：

- BBR算法原理和部署

- 客户端TCP参数优化 - 性能提升显著

- **UDP加速**：

- UDP协议重要性

- mKCP/QUIC配置 - 参数调优

- **WebSocket+TLS**：

- 流量伪装

- Nginx/Caddy反向代理 - 端口复用

- **CDN中转**：

- Cloudflare CDN配置

- 优选IP技术 - gRPC传输

- **多层代理**：

- 代理链配置

- IPLC专线 - 场景应用

- **分流实践**：

- 完整配置方案

- 性能优化 - 最佳实践

优化原则：

```
1. 根据需求选择

   不是所有优化都适合所有人

 • 循序渐进

   先基础配置 → 再逐步优化
 • 测试验证

   每次优化后测试效果
 • 权衡取舍

   速度 vs 隐蔽性

   性能 vs 稳定性

 • 持续学习

   技术不断更新

   保持关注新方案
```

至此，第三部分"高级配置篇"全部完成！

实践任务：

- 在VPS上启用BBR
- 配置WebSocket+TLS+CDN
- 测试优选IP效果
- 优化分流规则
- 对比优化前后性能

延伸阅读：

- BBR算法论文
- HTTP/3与QUIC
- Cloudflare Workers
- 网络性能测试方法

第十章 VPS选购指南

第10章 VPS选购指南

自建代理服务器需要租用VPS（Virtual Private Server，虚拟专用服务器）。本章将指导你如何选择合适的VPS供应商和配置。

10.1 VPS供应商对比

10.1.1 主流VPS供应商

国际大厂：

1. Vultr

```
优势：

✓ 全球18+机房
✓ 按小时计费，随时销毁
✓ 支持支付宝/微信支付
✓ 新用户送$100（有条件）
✓ 开通快速（1分钟）


配置与价格：
$6/月  - 1核 1G 内存 25G SSD 1TB流量
$12/月 - 1核 2G 内存 55G SSD 2TB流量
$24/月 - 2核 4G 内存 80G SSD 3TB流量


推荐机房：

  • 东京（延迟低，但易被墙）


  • 洛杉矶（性价比高）

  • 新加坡（稳定）
```

```
官网: https://www.vultr.com
```

2. DigitalOcean

```
优势:

✓ 老牌稳定
✓ 文档详细
✓ 社区活跃
✓ 界面友好
✗ 价格稍贵
✗ 需信用卡验证


配置与价格:
$6/月  - 1核 1G 内存 25G SSD 1TB流量
$12/月 - 1核 2G 内存 50G SSD 2TB流量
$24/月 - 2核 4G 内存 80G SSD 4TB流量


推荐机房:

  • 旧金山


  • 新加坡

  • 纽约


官网: https://www.digitalocean.com
```

3. Linode

```
优势:


✓ 性能优秀
✓ 稳定可靠
✓ 客服响应快
✓ 免费备份



配置与价格:
$5/月  - 1核 1G 内存 25G SSD 1TB流量
$10/月 - 1核 2G 内存 50G SSD 2TB流量
$20/月 - 2核 4G 内存 80G SSD 4TB流量



推荐机房:


  • 东京



  • 弗里蒙特


  • 新加坡




官网: https://www.linode.com
```

4. Bandwagon（搬瓦工）

```
特点:


✓ 针对中国优化
✓ CN2 GIA线路
✓ 支持支付宝
✗ 价格较贵
✗ 经常缺货
```

```
配置与价格:
$49.99/年 - 1核 1G 内存 20G SSD 1TB流量（CN2）
$169.99/年 - 1核 1G 内存 20G SSD 1TB流量（CN2 GIA）



推荐机房:


  • 洛杉矶 CN2 GIA




官网: https://bandwagonhost.com
```

5. Hostwinds

```
优势:


✓ 不限流量
✓ 支持支付宝
✓ 性价比高
✓ 可随时换IP（$3/次）



配置与价格:
$4.99/月 - 1核 1G 内存 30G SSD 无限流量
$8.99/月 - 1核 2G 内存 50G SSD 无限流量



推荐机房:


  • 西雅图



  • 达拉斯
```

```
官网: https://www.hostwinds.com
```

6. AWS Lightsail

```
优势:

✓ 亚马逊出品
✓ 稳定可靠
✓ 与AWS生态集成
✗ 配置复杂
✗ 价格不透明



配置与价格:
$3.5/月 - 1核 512M 内存 20G SSD 1TB流量
$5/月   - 1核 1G 内存 40G SSD 2TB流量
$10/月  - 1核 2G 内存 60G SSD 3TB流量



推荐机房:

  • 东京



  • 首尔

  • 新加坡



官网: https://aws.amazon.com/lightsail/
```

10.1.2 供应商选择建议

对比表:

供应商	价格	性能	稳定性	易用性	支付方式	推荐度
Vultr	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	支付宝/信用卡	[推荐][推荐][推荐][推荐][推荐]
DigitalOcean	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	信用卡	[推荐][推荐][推荐][推荐]
Linode	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	信用卡/PayPal	[推荐][推荐][推荐][推荐]
搬瓦工	[推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	支付宝	[推荐][推荐][推荐]
Hostwinds	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]	支付宝/信用卡	[推荐][推荐][推荐][推荐]
AWS	[推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐]	信用卡	[推荐][推荐][推荐]

场景推荐：

```
新手入门：

首选：Vultr
理由：支付方便、开通快、按小时计费、易上手



追求性能：
首选：Linode / DigitalOcean
理由：性能稳定、口碑好、文档齐全
```

```
追求性价比：
首选：Hostwinds
理由：不限流量、价格低、可换IP



中国优化：
首选：搬瓦工 CN2 GIA
理由：专线直连、速度快、延迟低
注意：价格贵、经常缺货



企业级需求：
首选：AWS Lightsail
理由：稳定可靠、可扩展性强
```

10.1.3 注册与购买流程

Vultr注册购买示例：

```
步骤1：注册账号

  • 访问 https://www.vultr.com



  • 点击 Create Account

  • 填写邮箱和密码

  • 验证邮箱





步骤2：充值


  • 登录后台
```

```
  • Billing → Make Payment

  • 选择支付方式:



    - Alipay（支付宝）

    - WeChat Pay（微信支付）
    - Credit Card（信用卡）
    - PayPal

  • 充值金额（建议$10起）




步骤3: 部署服务器

  • Products → Deploy New Server



  • 选择服务器类型: Cloud Compute

  • 选择机房位置（Server Location）



    推荐: Tokyo、Los Angeles、Singapore
  • 选择操作系统（Server Type）

    推荐: Ubuntu 20.04 LTS / Debian 11
  • 选择配置（Server Size）

    推荐: $6/月（1GB内存）
  • 禁用IPv6（可选）



  • Server Hostname & Label（自定义名称）

  • Deploy Now（立即部署）
```

```
步骤4：等待开通

  • 通常1-3分钟完成

  • 状态变为Running即可使用

  • 记录IP地址、用户名、密码
```

注意事项：

```
[注意] IP被墙问题
  • 新开VPS的IP可能已被墙

  • 开通后立即测试连通性

  • 如果被墙，立即删除重新开通

  • Vultr可随时删除服务器，按小时计费


[注意] 账号安全

  • 使用强密码

  • 启用两步验证（2FA）
```

```
  • 不要分享账号信息




[注意] 付款安全

  • 优先使用支付宝/微信


  • 信用卡注意盗刷风险

  • 定期检查账单
```

10.2 地域选择策略

10.2.1 地理位置与延迟

距离与延迟关系：

```
物理定律：

光速 ≈ 300,000 km/s
光纤中实际速度 ≈ 200,000 km/s



理论延迟计算：
北京 → 东京（约2100km）
理论延迟 = 2100 / 200,000 × 2 ≈ 21ms



实际延迟（包含路由跳转）：
北京 → 东京：30-60ms
北京 → 洛杉矶：150-200ms
北京 → 纽约：200-250ms
北京 → 欧洲：250-350ms
```

中国大陆用户延迟参考：

机房位置	延迟范围	适用场景	推荐度
香港	5-30ms	游戏、实时通信	[推荐][推荐][推荐][推荐][推荐]
日本	30-80ms	日常使用、游戏	[推荐][推荐][推荐][推荐][推荐]
韩国	30-80ms	游戏、流媒体	[推荐][推荐][推荐][推荐]
新加坡	50-100ms	平衡之选	[推荐][推荐][推荐][推荐][推荐]
美西	130-180ms	日常使用	[推荐][推荐][推荐][推荐]
美东	180-230ms	流媒体	[推荐][推荐][推荐]
欧洲	250-350ms	特定服务	[推荐][推荐]

10.2.2 线路类型

电信/联通/移动线路区别：

普通线路（163骨干网）：

特点：

- 价格便宜
- 高峰期拥堵
- 绕路严重
- 丢包率高

适合：

- 预算有限

```
  • 低峰期使用

  • 非高要求用户
```

CN2线路：

```
CN2 GT（Global Transit）：
  • 部分路由走CN2

  • 价格适中

  • 高峰期表现尚可

CN2 GIA（Global Internet Access）：

  • 全程CN2网络

  • 价格昂贵

  • 速度快、延迟低

  • 高峰期稳定

对比：
普通线路：$5-10/月
CN2 GT：   $10-20/月
CN2 GIA：  $30-100/月
```

BGP多线：

```
特点：


✓ 自动选择最优路由
✓ 电信/联通/移动都快
✓ 智能切换
✗ 价格较贵



推荐：
搬瓦工 CN2 GIA
```

10.2.3 机房推荐

按需求选择：

游戏加速：

```
首选：香港、日本、韩国


延迟要求：< 100ms
推荐机房：


  • Vultr 东京



  • Linode 东京


  • Vultr 新加坡
```

日常使用：

```
首选：日本、新加坡、美西


延迟要求：< 200ms
推荐机房：
```

```
  • Vultr 东京/洛杉矶



  • DigitalOcean 旧金山

  • Linode 弗里蒙特
```

流媒体解锁：

```
Netflix：美国原生IP

Disney+：美国、日本
HBO Max：美国



推荐机房:

  • DigitalOcean 纽约



  • Vultr 洛杉矶

  • AWS 美西
```

特定服务：

```
ChatGPT/OpenAI：
  • 支持大部分美国、欧洲、日本IP



  • 避免: 香港、印度、俄罗斯
```

```
Google：

  • 任何地区都可以

  • 推荐：新加坡、日本

GitHub：

  • 任何地区都可以

  • 推荐：美国、新加坡
```

10.2.4 IP被墙检测

测试方法：

```
# 本地ping测试

ping your-vps-ip

# 如果超时，可能被墙

# 进一步确认：
```

```
# 1. 使用在线工具

https://www.ping.pe/your-vps-ip
https://tools.ipip.net/ping.php

# 2. 使用国外服务器测试

# 如果国外能ping通，国内不通 = 被墙

# 3. TCP端口测试

telnet your-vps-ip 22

# 4. HTTP测试

curl -I http://your-vps-ip
```

IP被墙后的处理：

```
方案1：更换IP（推荐）

Vultr/DigitalOcean：

  • 删除当前服务器

  • 重新部署
```

- 免费获得新IP

Hostwinds：

- 付费更换IP（$3/次）

方案2：使用CDN中转

- 配置CloudFlare CDN

- 隐藏真实IP
- 参考第9章CDN配置

方案3：换服务商/机房

- 选择被墙概率低的机房

- 避免热门IP段

10.3 配置需求分析

10.3.1 个人使用配置

单人使用：

```
基础配置（$5-6/月）：

CPU：1核
内存：1GB
硬盘：20-25GB SSD
流量：500GB-1TB/月
带宽：1Gbps



适合：
✓ 日常浏览
✓ 偶尔看视频
✓ 轻度使用



性能：

  • 同时连接：1-2设备



  • 视频：720P流畅

  • 下载：~5-10MB/s
```

重度使用（$10-12/月）：

```
推荐配置：

CPU：1-2核
内存：2GB
硬盘：50GB SSD
流量：2TB/月
带宽：1Gbps
```

```
适合:
✓ 高频使用
✓ 看高清视频
✓ 大文件下载


性能:

  • 同时连接: 3-5设备


  • 视频: 1080P流畅

  • 下载: ~10-20MB/s
```

10.3.2 多人共享配置

2-5人共享:

```
推荐配置（$20/月）:

CPU：2核
内存: 4GB
硬盘: 80GB SSD
流量: 3-4TB/月
带宽: 1Gbps


管理:

  • 使用X-UI面板


  • 设置流量限制

  • 分配不同端口
```

5-10人共享：

```
推荐配置（$40/月）：


CPU：4核
内存：8GB
硬盘：160GB SSD
流量：5-6TB/月
带宽：1Gbps



注意：
[注意] 合理分配流量
[注意] 设置限速策略
[注意] 监控使用情况
[注意] 注意服务商TOS
```

10.3.3 流量计算

流量消耗参考：

```
网页浏览：


普通网页：~1-3MB/页
图片多的网页：~5-10MB/页
估算：100页/天 × 5MB ≈ 500MB/天 ≈ 15GB/月



视频观看：
YouTube 720P：~1.5GB/小时
YouTube 1080P：~3GB/小时
Netflix HD：~3GB/小时
Netflix 4K：~7GB/小时
估算：2小时/天 × 3GB ≈ 6GB/天 ≈ 180GB/月



文件下载：
软件/游戏：偶尔10-50GB
估算：20GB/月
```

```
总计：
轻度：15 + 60 + 0 ≈ 75GB/月（500GB套餐足够）
中度：15 + 180 + 10 ≈ 205GB/月（500GB-1TB）
重度：30 + 360 + 50 ≈ 440GB/月（1TB+）
```

安全余量：

```
建议选择流量为实际需求的2-3倍：

实际需求：200GB/月
推荐选择：500GB-1TB套餐



原因：
✓ 避免超额费用
✓ 预留升级空间
✓ 应对突发需求
```

10.3.4 性能测试

VPS性能测试脚本：

```
# 综合性能测试（推荐）

wget -qO- bench.sh | bash

# 或


curl -Lso- bench.sh | bash

# 测试内容：


  - CPU性能
```

- 内存性能
- 硬盘I/O速度
- 网络速度（全球多节点）

Superbench脚本

```
wget -qO- git.io/superbench.sh | bash
```

或

```
curl -Lso- git.io/superbench.sh | bash
```

测试内容更详细：

- 系统信息
- CPU性能（单核/多核）
- 内存测试
- 硬盘测试（顺序/随机读写）
- 网络测试（上传/下载）

解读测试结果：

```
CPU性能:

单核跑分 > 1000: 优秀
单核跑分 500-1000: 良好
单核跑分 < 500: 较弱



硬盘I/O：
> 500MB/s：SSD，优秀
100-500MB/s：SSD，良好
< 100MB/s：HDD，不推荐



网络速度（中国测试节点）：
> 50MB/s：优秀
20-50MB/s：良好
10-20MB/s：一般
< 10MB/s：较差
```

10.4 付款方式与隐私保护

10.4.1 付款方式对比

支付宝/微信支付：

```
优势:

✓ 方便快捷
✓ 国人友好
✓ 无需信用卡
✓ 汇率透明



劣势:
✗ 实名认证
✗ 可追溯
✗ 部分商家不支持
```

```
隐私等级: [推荐][推荐]


支持商家:

  • Vultr ✓



  • Hostwinds ✓

  • 搬瓦工 ✓

  • DigitalOcean ✗

  • Linode ✗
```

信用卡:

```
优势:

✓ 广泛接受
✓ 自动续费
✓ 有消费保护



劣势:
✗ 需要验证
✗ 可能被盗刷
✗ 有额度限制



隐私等级: [推荐][推荐]
```

```
注意事项：
[注意] 使用虚拟信用卡
[注意] 设置消费提醒
[注意] 定期检查账单
```

PayPal：

```
优势：

✓ 国际通用
✓ 买家保护
✓ 相对安全


劣势：
✗ 需绑定银行卡/信用卡
✗ 可能冻结账户
✗ 有手续费


隐私等级：[推荐][推荐][推荐]


支持商家：

  • 几乎所有国际商家
```

加密货币：

```
优势：

✓ 匿名性强
✓ 不可追溯
✓ 去中心化


劣势：
✗ 操作复杂
```

```
✗ 汇率波动
✗ 支持商家少
✗ 不可退款



隐私等级：[推荐][推荐][推荐][推荐][推荐]



支持商家：


  • 部分小众VPS商家



  • 需要寻找
```

10.4.2 隐私保护建议

注册信息：

```
建议做法：

✓ 使用临时邮箱（注册后绑定常用邮箱）
✓ 不填写真实姓名（如允许）
✓ 地址使用生成器
✓ 使用VPN/代理注册



临时邮箱推荐：


  • https://temp-mail.org



  • https://10minutemail.com


  • https://guerrillamail.com
```

```
地址生成器：

  • https://www.fakeaddressgenerator.com
```

付款隐私：

```
降低追溯性：
  • 使用虚拟信用卡
    - Privacy.com（美国）
    - Revolut（欧洲）
  • 使用礼品卡
    - Amazon礼品卡
    - Google Play礼品卡（部分商家）
  • 使用加密货币
    - Bitcoin（追溯性较强）
    - Monero（隐私性强）

注意：
[注意] 不要过度追求匿名
[注意] 平衡隐私与便利性
[注意] 遵守当地法律
```

10.4.3 账号安全

密码安全：

```
强密码原则：

✓ 至少16位
✓ 包含大小写字母、数字、符号
✓ 不使用个人信息
✓ 每个服务使用不同密码



密码管理器推荐：

  • Bitwarden（开源、免费）



  • 1Password（功能强大）

  • KeePass（本地存储）





示例强密码：
Kx9#mP2$vL4@nQ8!wR5
```

两步验证（2FA）：

```
启用2FA：
  • VPS后台 → Security Settings



  • Enable Two-Factor Authentication

  • 使用认证器应用扫描二维码

  • 保存备用码
```

```
认证器应用推荐:

  • Google Authenticator

  • Microsoft Authenticator

  • Authy（支持多设备同步）

重要:
[注意] 备份恢复码
[注意] 多设备备份
[注意] 不要截图保存
```

10.5 IP质量检测

10.5.1 IP类型识别

住宅IP vs 数据中心IP：

```
住宅IP（Residential IP）：
  • 来自ISP分配给家庭用户

  • 信誉度高

  • 不易被封

  • 价格昂贵

  • VPS很少提供
```

```
数据中心IP（Datacenter IP）：

  • 来自数据中心

  • 信誉度一般

  • 容易被识别

  • 价格便宜

  • VPS常用




检测工具:
https://ipinfo.io/your-ip
查看 "org" 字段，如果是ISP名称 = 住宅IP
```

10.5.2 IP黑名单检测

检测IP是否被列入黑名单：

```
在线检测工具:
  • MXToolbox

    https://mxtoolbox.com/blacklists.aspx
  • MultiRBL

    http://multirbl.valli.org/
  • WhatIsMyIPAddress

    https://whatismyipaddress.com/blacklist-check
```

```
检测内容:

  • Spamhaus

  • Barracuda

  • SORBS

  • 等100+黑名单数据库




判断:
✓ Clean：干净，可用
[注意] Listed：被列入黑名单，可能影响邮件等服务
```

10.5.3 流媒体解锁检测

Netflix解锁检测：

```
# 使用脚本检测

bash <(curl -L -s check.unlock.media)

# 或 ═══════════════════════════════════════════════════════════

wget -O nf https://github.com/sjlleo/netflix-verify/releases/download/v3.1.0/nf_linux_amd64 &&

# 结果解读： ═══════════════════════════════════════════════════
```

```
✓ 原生IP：完全解锁Netflix
[注意] 非原生IP：可能无法观看
✗ 不支持: 无法使用Netflix

# 手动测试



curl -I https://www.netflix.com

# 查看返回的地区信息
```

其他流媒体：

```
# Disney+ 检测

curl -I https://www.disneyplus.com

# YouTube Premium检测

# 登录后查看是否能使用Premium功能

# HBO Max检测



curl -I https://play.hbomax.com
```

10.5.4 端口封锁检测

常用端口检测：

```
# 在VPS上运行（需先安装nc）

apt install netcat -y

# 监听测试端口


nc -l 1234

# 在本地测试连通性


telnet your-vps-ip 1234

# 或


nc -zv your-vps-ip 1234

# 常见被封端口：


25    - SMTP（几乎都被封）
465   - SMTPS
587   - 邮件提交
1194  - OpenVPN
1723  - PPTP VPN
```

在线端口检测：

```
工具：
```

```
https://www.yougetsignal.com/tools/open-ports/
https://canyouseeme.org/



测试常用代理端口:
443  - HTTPS（推荐）
80   - HTTP
8080 - 备用HTTP
1080 - SOCKS
```

10.5.5 IP质量综合评分

IP质量评估清单：

```
✓ Ping延迟


  < 100ms：优秀
  100-200ms：良好
  > 200ms：一般



✓ 丢包率
  0%: 完美
  < 1%: 优秀
  1-5%: 可接受
  > 5%: 较差



✓ 路由跳数
  < 15跳: 优秀
  15-25跳: 一般
  > 25跳: 绕路严重



✓ IP类型
  住宅IP：[推荐][推荐][推荐][推荐][推荐]
  数据中心IP：[推荐][推荐][推荐]
```

```
✓ 黑名单状态
  Clean：[推荐][推荐][推荐][推荐][推荐]
  Listed：[推荐][推荐]



✓ 流媒体解锁
  Netflix原生: [推荐][推荐][推荐][推荐][推荐]
  部分解锁: [推荐][推荐][推荐]
  不解锁: [推荐]



✓ 端口开放
  全部开放: [推荐][推荐][推荐][推荐][推荐]
  部分限制: [推荐][推荐][推荐]
```

综合测试脚本：

```
# 融合怪测试（推荐）


bash <(curl -L -s https://raw.githubusercontent.com/spiritLHLS/ecs/main/ecs.sh)

# 测试项目：


  - 系统信息



  - CPU性能

  - 内存测试

  - 硬盘速度

  - 网络测速

  - 流媒体解锁

  - IP质量检测
```

- 路由跟踪

等待测试完成（约5-10分钟）

获得详细的VPS性能报告

本章小结

本章详细介绍了VPS选购的各个方面：

核心要点：

- **供应商选择**：

- Vultr：新手友好、支付方便

- DigitalOcean/Linode：性能稳定 - 搬瓦工：中国优化、CN2线路 - Hostwinds：性价比高、不限流量

- **地域选择**：

- 游戏：香港、日本（低延迟）

- 日常：日本、新加坡、美西 - 流媒体：美国原生IP - 特定服务：按需选择

- **配置需求**：

- 个人使用：1GB内存，500GB-1TB流量

- 重度使用：2GB内存，2TB流量 - 多人共享：按人数倍增

- **付款隐私**：

- 支付宝/微信：方便但可追溯

- 信用卡：广泛接受 - PayPal：国际通用 - 加密货币：匿名性强

- **IP质量**：

- 延迟测试

- 黑名单检测 - 流媒体解锁 - 端口开放情况

选购建议：

```
新手推荐方案：


供应商：Vultr
机房：东京/洛杉矶/新加坡
配置：$6/月（1GB内存、1TB流量）
付款：支付宝
理由：简单、便宜、够用



进阶方案：
供应商：搬瓦工
机房：洛杉矶 CN2 GIA
配置：$50/年起
付款：支付宝
理由：速度快、稳定、针对中国优化



预算充足：
供应商：Linode / DigitalOcean
机房：按需选择
配置：$10-20/月
付款：信用卡/PayPal
理由：性能好、稳定性高、服务优质
```

下一章我们将学习服务器环境配置。

实践任务：

- 注册VPS供应商账号
- 测试不同机房延迟
- 购买并开通VPS
- 进行IP质量检测
- 记录服务器信息

> **第十一章 服务器环境配置**

第11章 服务器环境配置

购买VPS后，需要进行基础环境配置。本章将介绍Linux基础操作和服务器安全加固。

11.1 Linux基础操作

11.1.1 SSH连接服务器

Windows连接方式：

```
# 方法1：使用PowerShell（Windows 10+）

ssh root@your-vps-ip

# 方法2：使用PuTTY

# 下载：https://www.putty.org/

# 填写：

# Host Name: your-vps-ip

# Port: 22

# 
```

```
# Connection Type: SSH

# 点击Open

# 方法3：使用Xshell（推荐）

# 下载：https://www.xshell.com/zh/free-for-home-sc

# 新建会话 → 输入IP → 连接
```

macOS/Linux连接：

```
# 终端直接连接

ssh root@your-vps-ip

# 首次连接会提示：



The authenticity of host 'xxx' can't be established.
Are you sure you want to continue connecting (yes/no)?

# 输入 yes

# 输入密码（VPS提供的root密码）
```

SSH密钥登录（推荐）：

```
# 本地生成密钥对

ssh-keygen -t rsa -b 4096 -C "your_email@example.com"

# 按回车使用默认路径 ~/.ssh/id_rsa

# 设置密码短语（可选，建议设置）

# 将公钥复制到服务器

ssh-copy-id root@your-vps-ip

# 或手动复制

cat ~/.ssh/id_rsa.pub

# 复制输出内容

# 在服务器上：

mkdir -p ~/.ssh
echo "粘贴的公钥内容" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
chmod 700 ~/.ssh
```

```
# 之后连接无需密码

ssh root@your-vps-ip
```

11.1.2 系统信息查看

查看系统版本：

```
# 查看发行版
cat /etc/os-release

# 查看内核版本
uname -r

# 查看系统架构
uname -m

# x86_64 = 64位
# aarch64 = ARM 64位

# 查看主机名
```

```
hostname

# 查看系统时间



date
timedatectl
```

查看硬件信息：

```
# CPU信息

cat /proc/cpuinfo | grep "model name" | head -1
lscpu

# 内存信息



free -h
cat /proc/meminfo | grep MemTotal

# 硬盘信息



df -h
lsblk

# 网络接口



ip addr
ifconfig
```

```
# 查看进程
ps aux
top    # 实时查看
htop   # 更友好的界面（需安装）
```

11.1.3 包管理器使用

Debian/Ubuntu（apt）：

```
# 更新软件包列表
apt update
# 升级所有软件包
apt upgrade -y
# 安装软件
apt install wget curl git vim -y
# 搜索软件
apt search nginx
```

```
# 卸载软件

apt remove nginx
apt purge nginx    # 删除配置文件
apt autoremove     # 删除不需要的依赖

# 查看已安装软件

apt list --installed
dpkg -l
```

CentOS/RHEL（yum/dnf）：

```
# 更新

yum update -y

# CentOS 8+使用dnf

dnf update -y

# 安装软件

yum install wget curl git vim -y

# 搜索
```

```
yum search nginx

# 卸载


yum remove nginx

# 清理缓存


yum clean all
```

11.1.4 文件操作

基础命令：

```
# 列出文件

ls          # 基本列表
ls -l       # 详细信息
ls -lh      # 人性化大小显示
ls -a       # 显示隐藏文件
ll          # ls -l的别名

# 切换目录


cd /path/to/dir
cd ~        # 回到home目录
cd ..       # 上一级目录
cd -        # 回到之前的目录
```

查看当前路径

```
pwd
```

创建目录

```
mkdir dirname
mkdir -p /path/to/nested/dir    # 创建多级目录
```

创建文件

```
touch filename
echo "content" > filename        # 覆盖写入
echo "content" >> filename       # 追加写入
```

复制

```
cp source dest
cp -r dir1 dir2                  # 递归复制目录
```

移动/重命名

```
mv old new
mv file /path/to/dest/
```

删除

```
rm file
rm -r dir                       # 递归删除目录
rm -rf dir                      # 强制递归删除（危险！）
```

查看文件内容

```
cat file                        # 全部内容
less file                       # 分页查看
head file                       # 前10行
tail file                       # 后10行
tail -f file                    # 实时查看（日志）
```

文件权限：

```
# 查看权限

ls -l filename

-rw-r--r-- 1 root root 1234 Dec 15 10:00 filena

 ↑↑↑  ↑↑↑  ↑↑↑

 所有者 组 其他

权限说明：
```

```
# r = 读(4)

# w = 写(2)

# x = 执行(1)

# 修改权限

chmod 755 filename     # rwxr-xr-x
chmod 644 filename     # rw-r--r--
chmod +x script.sh     # 添加执行权限

# 修改所有者

chown user:group filename
chown -R user:group directory
```

查找文件：

```
# 按名称查找

find /path -name "filename"
find . -name "*.log"

# 按类型查找
```

```
find /path -type f    # 文件
find /path -type d    # 目录

# 按大小查找

find /path -size +100M    # 大于100MB

# 查找并删除

find /path -name "*.tmp" -delete

# 快速查找（需先建立索引）

locate filename
updatedb    # 更新索引数据库
```

11.1.5 系统服务管理

systemd服务管理：

```
# 启动服务

systemctl start servicename

# 停止服务

systemctl stop servicename
```

重启服务

```
systemctl restart servicename
```

重新加载配置

```
systemctl reload servicename
```

查看状态

```
systemctl status servicename
```

开机自启

```
systemctl enable servicename
```

禁用自启

```
systemctl disable servicename
```

查看所有服务

```
systemctl list-units --type=service
```

查看日志

```
journalctl -u servicename
journalctl -u servicename -f    # 实时查看
```

11.2 SSH安全配置

11.2.1 修改SSH端口

为什么要修改端口?

默认端口22:

- 被大量扫描
- 暴力破解尝试频繁
- 日志充满攻击记录

修改到其他端口:

- 减少扫描攻击
- 降低日志噪音
- 提高安全性

修改步骤:

```
# 1. 编辑SSH配置

vi /etc/ssh/sshd_config

# 2. 找到并修改（或添加）


Port 22

# 改为：


Port 2222    # 或其他端口（1024-65535）

# 3. 重启SSH服务


systemctl restart sshd

# 4. 测试新端口（不要关闭当前连接！）

# 新开终端测试


ssh -p 2222 root@your-vps-ip

# 5. 确认可以连接后，关闭旧连接
```

注意事项：

```
[注意] 修改前备份配置

cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak



[注意] 防火墙放行新端口
firewall-cmd --permanent --add-port=2222/tcp
firewall-cmd --reload

# 或ufw



ufw allow 2222/tcp
ufw reload



[注意] 保持一个SSH连接
修改后用新端口测试，成功后再关闭旧连接
```

11.2.2 禁用root登录

创建普通用户：

```
# 创建新用户

adduser username

# 设置密码



passwd username

# 添加到sudo组（Debian/Ubuntu）
```

```
usermod -aG sudo username

# CentOS/RHEL

usermod -aG wheel username

# 切换到新用户测试

su - username
sudo ls /root    # 测试sudo权限
```

禁用root登录：

```
# 编辑SSH配置

vi /etc/ssh/sshd_config

# 修改

PermitRootLogin yes

# 改为

PermitRootLogin no
```

```
# 重启SSH

systemctl restart sshd

# 之后只能用普通用户登录

ssh username@your-vps-ip

# 需要root权限时使用sudo
```

11.2.3 密钥认证

禁用密码登录：

```
# 前提：已配置SSH密钥登录并测试成功
# 编辑SSH配置

vi /etc/ssh/sshd_config

# 修改以下选项

PasswordAuthentication yes

# 改为
```

```
PasswordAuthentication no

# 同时确保密钥认证已启用


PubkeyAuthentication yes

# 重启SSH


systemctl restart sshd
```

SSH配置最佳实践：

```
# /etc/ssh/sshd_config 推荐配置
# 端口设置


Port 2222

# 协议版本


Protocol 2

# 禁用root登录


PermitRootLogin no
```

密钥认证

```
PubkeyAuthentication yes
```

禁用密码登录

```
PasswordAuthentication no
ChallengeResponseAuthentication no
```

禁用空密码

```
PermitEmptyPasswords no
```

禁用X11转发

```
X11Forwarding no
```

最大认证尝试

```
MaxAuthTries 3
```

登录超时

```
LoginGraceTime 30
```

```
# 允许的用户（可选）
AllowUsers username
# 客户端活动检测
ClientAliveInterval 300
ClientAliveCountMax 2
```

11.2.4 Fail2Ban防暴力破解

安装Fail2Ban：

```
# Debian/Ubuntu
apt install fail2ban -y
# CentOS/RHEL
yum install epel-release -y
yum install fail2ban -y
# 启动服务
systemctl start fail2ban
systemctl enable fail2ban
```

配置Fail2Ban：

```
# 复制配置文件

cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local

# 编辑配置

vi /etc/fail2ban/jail.local

# 修改以下部分

[DEFAULT]

# 封禁时间（秒）

bantime = 3600

# 观察时间窗口

findtime = 600

# 最大失败次数

maxretry = 5
```

忽略的IP（自己的IP）

```
ignoreip = 127.0.0.1/8 your-home-ip
```

```
[sshd]
enabled = true
port = 2222    # 你的SSH端口
logpath = /var/log/auth.log
maxretry = 3
```

重启服务

```
systemctl restart fail2ban
```

查看状态

```
fail2ban-client status
fail2ban-client status sshd
```

查看被封禁的IP

```
fail2ban-client get sshd banned
```

手动解封IP

```
fail2ban-client set sshd unbanip xxx.xxx.xxx.xxx
```

11.3 防火墙设置

11.3.1 UFW防火墙（Ubuntu/Debian）

基础配置：

```
# 安装UFW

apt install ufw -y

# 默认策略


ufw default deny incoming    # 拒绝所有入站
ufw default allow outgoing   # 允许所有出站

# 允许SSH（修改后的端口）


ufw allow 2222/tcp

# 允许HTTP/HTTPS


ufw allow 80/tcp
ufw allow 443/tcp

# 允许代理端口（根据实际配置）
```

```
ufw allow 8388/tcp    # Shadowsocks
ufw allow 10086/tcp   # V2Ray

# 启用防火墙

ufw enable

# 查看状态

ufw status verbose

# 查看规则编号

ufw status numbered

# 删除规则

ufw delete 3    # 删除第3条规则
ufw delete allow 80/tcp
```

高级规则：

```
# 允许特定IP访问特定端口

ufw allow from 1.2.3.4 to any port 22
```

```
# 允许IP段

ufw allow from 192.168.1.0/24

# 拒绝特定IP

ufw deny from 1.2.3.4

# 限制连接速率（防DDoS）

ufw limit 22/tcp

# 删除所有规则

ufw reset
```

11.3.2 Firewalld（CentOS/RHEL）

基础配置：

```
# 安装firewalld

yum install firewalld -y

# 启动服务
```

```
systemctl start firewalld
systemctl enable firewalld
```

查看状态

```
firewall-cmd --state
firewall-cmd --list-all
```

允许服务

```
firewall-cmd --permanent --add-service=ssh
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service=https
```

允许端口

```
firewall-cmd --permanent --add-port=2222/tcp
firewall-cmd --permanent --add-port=8388/tcp
```

重新加载

```
firewall-cmd --reload
```

查看开放的端口

```
firewall-cmd --list-ports
```

```
# 删除规则

firewall-cmd --permanent --remove-port=8388/tcp
firewall-cmd --reload
```

11.3.3 iptables规则

基础规则：

```
# 查看当前规则

iptables -L -n -v

# 清空所有规则（慎用！）

iptables -F

# 允许本地回环

iptables -A INPUT -i lo -j ACCEPT

# 允许已建立的连接

iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# 允许SSH
```

```
iptables -A INPUT -p tcp --dport 2222 -j ACCEPT
```

允许HTTP/HTTPS

```
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

允许代理端口

```
iptables -A INPUT -p tcp --dport 8388 -j ACCEPT
```

拒绝其他所有入站

```
iptables -P INPUT DROP
```

保存规则

Debian/Ubuntu

```
iptables-save > /etc/iptables/rules.v4
```

CentOS/RHEL

```
service iptables save
```

11.4 系统优化调优

11.4.1 BBR加速

参考第9章内容：

```
# 检查内核版本

uname -r

# 需要4.9+内核

# 启用BBR

echo "net.core.default_qdisc=fq" >> /etc/sysctl.conf
echo "net.ipv4.tcp_congestion_control=bbr" >> /etc/sysctl.conf

# 应用设置

sysctl -p

# 验证

sysctl net.ipv4.tcp_congestion_control
lsmod | grep bbr
```

11.4.2 系统参数优化

网络参数优化：

```
# 编辑sysctl配置

vi /etc/sysctl.conf

# 添加以下内容

# 文件描述符限制

fs.file-max = 51200

# 网络核心设置

net.core.rmem_max = 67108864
net.core.wmem_max = 67108864
net.core.netdev_max_backlog = 250000
net.core.somaxconn = 4096

# TCP设置

net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 30
net.ipv4.tcp_keepalive_time = 1200
net.ipv4.tcp_max_syn_backlog = 8192
net.ipv4.tcp_max_tw_buckets = 5000
```

```
# TCP内存

net.ipv4.tcp_rmem = 4096 87380 67108864
net.ipv4.tcp_wmem = 4096 65536 67108864
net.ipv4.tcp_mem = 25600 51200 102400

# TCP Fast Open

net.ipv4.tcp_fastopen = 3

# 应用配置

sysctl -p
```

文件描述符限制：

```
# 查看当前限制

ulimit -n

# 临时修改

ulimit -n 51200

# 永久修改
```

```
vi /etc/security/limits.conf

# 添加


* soft nofile 51200
* hard nofile 51200

# 重新登录生效
```

11.4.3 禁用IPv6（可选）

如果不需要IPv6：

```
# 编辑sysctl配置

vi /etc/sysctl.conf

# 添加


net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv6.conf.lo.disable_ipv6 = 1

# 应用


sysctl -p
```

```
# 验证

ip addr | grep inet6

# 应该没有输出
```

11.5 时间同步配置

11.5.1 为什么需要时间同步

时间同步的重要性：

```
很多代理协议依赖时间验证：

VMess协议：

  • 客户端和服务器时间差 > 90秒

  • 连接失败

TLS证书：

  • 时间不准确

  • 证书验证失败
```

```
日志分析：

  • 时间混乱

  • 无法追踪问题
```

11.5.2 配置NTP时间同步

使用systemd-timesyncd：

```
# 查看时间状态

timedatectl status

# 启用NTP同步

timedatectl set-ntp true

# 设置时区

timedatectl set-timezone Asia/Shanghai

# 查看可用时区

timedatectl list-timezones | grep Asia
```

```
# 检查同步状态

timedatectl timesync-status
```

使用chrony（推荐）：

```
# 安装chrony
# Debian/Ubuntu

apt install chrony -y

# CentOS/RHEL

yum install chrony -y

# 编辑配置

vi /etc/chrony/chrony.conf

# 修改NTP服务器

# 注释掉默认服务器
```

```
# 添加国内NTP服务器

server ntp.aliyun.com iburst
server ntp1.aliyun.com iburst
server time1.cloud.tencent.com iburst
server time2.cloud.tencent.com iburst

# 重启服务

systemctl restart chronyd
systemctl enable chronyd

# 查看同步状态

chronyc tracking
chronyc sources -v

# 强制同步

chronyc makestep
```

手动同步（临时）：

```
# 使用ntpdate

apt install ntpdate -y
ntpdate ntp.aliyun.com
```

```
# 或使用date命令

date -s "2025-12-15 10:00:00"
```

本章小结

本章介绍了服务器环境配置的基础知识：

核心要点：

- **Linux基础**：

- SSH连接方法

- 文件和目录操作 - 软件包管理 - 系统服务管理

- **SSH安全**：

- 修改默认端口

- 禁用root登录 - 密钥认证 - Fail2Ban防暴力破解

- **防火墙**：

- UFW（Ubuntu/Debian）

- Firewalld（CentOS/RHEL） - iptables基础规则

- **系统优化**：

- BBR加速

- 网络参数调优 - 文件描述符限制

- **时间同步**：

- NTP配置

- chrony使用 - 时区设置

安全配置清单：

```
必做：
```

```
✓ 修改SSH端口
✓ 禁用密码登录（使用密钥）
✓ 配置防火墙
✓ 安装Fail2Ban
✓ 启用BBR
✓ 配置时间同步



建议：
✓ 禁用root登录
✓ 系统参数优化
✓ 定期更新系统
✓ 配置日志监控



可选：


  • 禁用IPv6



  • 自定义SSH欢迎信息


  • 配置自动备份
```

下一章我们将学习使用一键脚本快速部署代理服务。

实践任务：

- 完成SSH安全配置
- 配置防火墙规则
- 启用BBR加速
- 配置时间同步
- 验证所有配置生效

第十二章 一键脚本部署

第12章 一键脚本部署

一键脚本大大简化了代理服务的部署过程，适合新手快速搭建。本章将介绍常用的一键脚本及使用方法。

12.1 V2Ray/Xray一键脚本

12.1.1 mack-a脚本（推荐）

脚本特点：

```
优势：

✓ 功能全面
✓ 支持多协议
✓ 支持Reality协议
✓ 自动配置伪装
✓ 自动申请证书
✓ 定期更新维护
✓ 中文友好



支持协议：

  • VLESS + Reality（最新推荐）



  • VLESS + TCP + TLS

  • VLESS + TCP + XTLS

  • VLESS + WS + TLS

  • VLESS + gRPC + TLS
```

```
• Trojan + TCP + TLS

• Trojan + gRPC + TLS

• VMess + WS + TLS

• Hysteria 2
```

安装步骤：

```
# 1. 确保系统已更新

apt update && apt upgrade -y

## 2. 安装必要工具


apt install curl wget sudo -y

## 3. 运行一键脚本


wget -P /root -N --no-check-certificate "https://raw.githubusercontent.com/mack-a/v2ray-agent/

## 4. 选择安装选项

## 菜单会自动显示，按数字选择：
```

1. 安装

2. 任意组合安装

...

推荐选择：

VLESS + Reality（无需域名，2025最佳选择）

或

VLESS + TCP + TLS / XTLS（性能最好）

或

VLESS + gRPC + TLS（CDN友好）

使用流程：

```
步骤1：准备域名
  • 域名已解析到服务器IP
```

```
  • A记录: yourdomain.com → your-vps-ip




步骤2: 运行脚本

  • 选择安装类型


  • 输入域名

  • 输入邮箱（申请证书用）




步骤3: 自动化处理

  • 安装Nginx


  • 申请Let's Encrypt证书

  • 配置V2Ray/Xray

  • 配置伪装网站




步骤4: 获取配置

  • 脚本会显示客户端配置


  • 扫描二维码或复制链接
```

```
  • 导入到客户端




步骤5：测试连接

  • 客户端导入配置

  • 测试连接
```

管理命令：

```
# 再次运行脚本进入管理菜单

/root/install.sh

# 常用选项：

# 1. 查看账号

# 2. 查看日志

# 3. 更新

# 4. 卸载
```

5. 更换伪装站

6. 更新证书

12.1.2 233boy脚本

安装V2Ray：

```
# 运行脚本

bash <(curl -s -L https://git.io/v2ray.sh)

或



wget -N https://raw.githubusercontent.com/233boy/v2ray/master/install.sh && bash install.sh

选择协议

推荐：

1. TCP

2. WebSocket + TLS（需要域名）
```

3. HTTP/2（需要域名）

如果选择TLS，需要：

1. 输入域名

2. 自动申请证书

3. 配置Nginx

安装完成后自动生成：

- 配置信息

- vmess://链接

- 二维码

管理命令：

```
# 查看配置

v2ray info
```

查看二维码

```
v2ray qr
```

修改配置

```
v2ray config
```

查看状态

```
v2ray status
```

启动/停止/重启

```
v2ray start
v2ray stop
v2ray restart
```

更新

```
v2ray update
```

卸载

```
v2ray uninstall
```

12.1.3 Xray一键脚本

安装Xray-core：

```
# 官方安装脚本

bash -c "$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ instal

# 或指定版本



bash -c "$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ instal

# 安装目录：

# 配置文件：/usr/local/etc/xray/config.json

# 日志目录：/var/log/xray/

# 执行文件：/usr/local/bin/xray

# 手动编辑配置



vi /usr/local/etc/xray/config.json
```

```
# 启动服务

systemctl start xray
systemctl enable xray

# 查看状态

systemctl status xray

# 查看日志

journalctl -u xray -f
```

配置示例：

```
{

  "log": {
    "loglevel": "warning"
  },
  "inbounds": [{
    "port": 443,
    "protocol": "vless",
    "settings": {
      "clients": [{
        "id": "your-uuid-here",
        "flow": "xtls-rprx-vision"
     }],
      "decryption": "none"
   },
    "streamSettings": {
      "network": "tcp",
      "security": "tls",
      "tlsSettings": {
```

```
        "alpn": ["h2", "http/1.1"],
        "certificates": [{
          "certificateFile": "/path/to/fullchain.pem",
          "keyFile": "/path/to/privkey.pem"
       }]
      }
    }
  }],
  "outbounds": [{
    "protocol": "freedom"
  }]
}
```

12.2 Trojan一键安装

12.2.1 Trojan-Go一键脚本

Jrohy脚本：

```
# 运行安装脚本

bash <(curl -sL https://raw.githubusercontent.com/phlinhng/v2ray-tcp-tls-web/main/install.sh)

或


wget -N --no-check-certificate https://raw.githubusercontent.com/mark-tech/trojan-wiz/main/ins

安装流程：

1. 选择域名或IP

2. 输入域名（如果选择域名）
```

3. 选择是否配置CDN

4. 自动申请证书

5. 配置Trojan-Go

6. 配置伪装网站

安装完成显示：

- 服务器地址

- 端口

- 密码

- 客户端配置

Trojan-Panel面板：

```
# 安装面板（带Web管理界面）

bash <(curl -sL https://raw.githubusercontent.com/trojan-gfw/trojan-panel/master/install-ubunt
```

访问面板

https://your-domain:8888

默认用户名：admin

默认密码：admin（首次登录需修改）

面板功能：

- 用户管理

- 流量统计

- 节点管理

- 系统监控

12.2.2 Trojan-GFW原版安装

使用Docker安装：

```
# 安装Docker
```

```
curl -fsSL https://get.docker.com | bash
```

创建配置目录

```
mkdir -p /etc/trojan
```

创建配置文件

```
vi /etc/trojan/config.json
```

配置内容：

```
{
  "run_type": "server",
  "local_addr": "0.0.0.0",
  "local_port": 443,
  "remote_addr": "127.0.0.1",
  "remote_port": 80,
  "password": ["your_password_here"],
  "log_level": 1,
  "ssl": {
    "cert": "/etc/trojan/fullchain.pem",
    "key": "/etc/trojan/privkey.pem",
    "sni": "your-domain.com"
  }
}
```

运行Trojan容器

```
docker run -d \
  --name trojan \
```

```
  --restart always \
  -p 443:443 \
  -v /etc/trojan:/etc/trojan \
  trojangfw/trojan

# 查看日志



docker logs trojan
```

12.3 WireGuard快速部署

12.3.1 WireGuard一键脚本

Angristan脚本（推荐）：

```
# 下载并运行脚本

curl -O https://raw.githubusercontent.com/angristan/wireguard-install/master/wireguard-install
chmod +x wireguard-install.sh
./wireguard-install.sh

# 安装流程：

# 1. 自动安装WireGuard

# 2. 生成服务器密钥对

# 3. 配置网络接口
```

```
# 4. 创建第一个客户端

# 输入配置:

# - 客户端名称

# - 内网IP地址

# - 是否启用NAT（选择yes）

# 完成后生成:

# - 配置文件: /root/wg0-client-xxx.conf

# - 二维码（移动端扫描）
```

手动配置:

```
# 安装WireGuard
# Ubuntu

apt install wireguard -y
```

CentOS

```
yum install elrepo-release epel-release
yum install kmod-wireguard wireguard-tools
```

生成密钥对

```
cd /etc/wireguard
umask 077
wg genkey | tee server_privatekey | wg pubkey > server_publickey
wg genkey | tee client_privatekey | wg pubkey > client_publickey
```

创建服务器配置

```
vi /etc/wireguard/wg0.conf
```

```
[Interface]
PrivateKey = <server_privatekey内容>
Address = 10.0.0.1/24
ListenPort = 51820
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o eth0 -j MASQU
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o eth0 -j MAS
```

```
[Peer]
PublicKey = <client_publickey内容>
AllowedIPs = 10.0.0.2/32
```

启动WireGuard

```
wg-quick up wg0
systemctl enable wg-quick@wg0
```

查看状态

```
wg show
```

客户端配置：

```
[Interface]

PrivateKey = <client_privatekey内容>
Address = 10.0.0.2/24
DNS = 1.1.1.1



[Peer]
PublicKey = <server_publickey内容>
Endpoint = your-server-ip:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

12.3.2 WireGuard-UI面板

使用Docker安装：

```
# 运行WireGuard-UI

docker run -d \
  --name wgui \
  --cap-add NET_ADMIN \
  --cap-add SYS_MODULE \
  -e WGUI_USERNAME=admin \
  -e WGUI_PASSWORD=password \
  -e WG_CONF_DIR=/etc/wireguard \
  -e WG_INTERFACE_NAME=wg0 \
  -p 5000:5000 \
```

```
  -p 51820:51820/udp \
  -v /etc/wireguard:/etc/wireguard \
  -v /lib/modules:/lib/modules \
  ngoduykhanh/wireguard-ui

# 访问Web界面


http://your-server-ip:5000

# 用户名：admin

# 密码：password
```

12.4 脚本安全性分析

12.4.1 脚本安全风险

潜在风险：

```
1. 代码注入

   恶意脚本可能：
   ✗ 安装后门
   ✗ 窃取密钥
   ✗ 修改系统配置
   ✗ 上传数据到攻击者服务器


 • 权限滥用

   脚本通常需要root权限

   ✗ 可以执行任何操作
   ✗ 修改关键系统文件
```

```
• 依赖风险

  脚本会下载第三方组件

    ✗ 下载地址被劫持
    ✗ 组件本身有漏洞

• 更新风险

  自动更新功能

    ✗ 可能引入恶意代码
```

12.4.2 使用脚本的安全建议

使用前检查：

```
# 1. 查看脚本源码

curl -s URL | less

# 仔细阅读代码

# 检查是否有可疑操作

# 2. 检查脚本来源

# 优先使用：
```

✓ GitHub官方仓库

✓ 知名开发者

✓ Star数多、更新活跃的项目

✓ 有详细文档和说明

避免使用：

✗ 来源不明的脚本

✗ 无法查看源码

✗ 长期不更新的项目

✗ 无人维护的fork

3. 使用沙箱环境测试

先在测试VPS上运行

确认安全后再用于正式环境

推荐的安全脚本：

```
V2Ray/Xray：

✓ mack-a脚本 - GitHub Star 3000+
✓ 233boy脚本 - 老牌脚本，使用广泛
✓ Xray官方安装脚本



Trojan：
✓ Trojan-Panel官方脚本
✓ Jrohy维护的脚本



WireGuard：
✓ Angristan脚本 - Star 3000+
✓ 官方安装包



判断标准:

  • GitHub Star数 > 1000



  • 最近更新时间 < 6个月

  • Issues有人回复

  • 代码可读性好

  • 有详细文档
```

12.4.3 手动安装 vs 一键脚本

对比：

维度	一键脚本	手动安装
难度	[推荐] 简单	[推荐][推荐][推荐][推荐] 困难
速度	[推荐][推荐][推荐][推荐][推荐] 快	[推荐][推荐] 慢
可控性	[推荐][推荐] 低	[推荐][推荐][推荐][推荐][推荐] 高
安全性	[推荐][推荐][推荐] 取决于脚本	[推荐][推荐][推荐][推荐][推荐] 最安全
定制性	[推荐][推荐] 有限	[推荐][推荐][推荐][推荐][推荐] 完全自由
维护	[推荐][推荐][推荐][推荐] 脚本自动	[推荐][推荐] 需要自己

选择建议：

```
推荐使用一键脚本：


✓ 新手用户
✓ 快速部署
✓ 个人使用
✓ 不需要特殊配置



推荐手动安装：
✓ 有Linux经验
✓ 追求安全
✓ 需要定制配置
✓ 企业级部署
✓ 学习目的
```

12.4.4 脚本使用最佳实践

安全使用流程：

```
# 1. 创建快照/备份
在VPS控制面板创建快照
```

或使用rsync备份重要数据

2. 下载脚本到本地查看

```
wget URL -O script.sh
cat script.sh | less
```

检查可疑代码

3. 在测试环境运行

使用廉价VPS测试

观察脚本行为

4. 限制脚本权限（如果可能）

某些操作不需要完整root权限

```
chmod +x script.sh
sudo -u limited-user ./script.sh
```

5. 记录安装过程

```
script install.log
bash script.sh
exit
```

日志保存在install.log

6. 安装后检查

查看新增的服务

```
systemctl list-units --type=service | grep -i v2ray
```

查看开放的端口

```
netstat -tulnp
```

查看定时任务

```
crontab -l
```

检查启动项

```
ls /etc/systemd/system/
```

7. 定期更新

使用脚本的更新功能

或关注项目更新动态

本章小结

本章介绍了常用的一键脚本及使用方法：

核心要点：

- **V2Ray/Xray脚本**：

- mack-a：功能全面，支持多协议

- 233boy：简单易用，老牌稳定 - Xray官方：轻量级，适合DIY

- **Trojan脚本**：

- Trojan-Go：性能优秀

- Trojan-Panel：Web管理界面 - 官方版本：简洁稳定

- **WireGuard脚本**：

- Angristan：简单易用

- WireGuard-UI：图形化管理 - 手动配置：完全掌控

- **安全性**：

- 检查脚本来源

- 查看源码 - 沙箱测试 - 定期更新

使用建议：

```
新手推荐：
```

```
mack-a V2Ray/Xray脚本
理由：全自动、傻瓜式、功能完整



进阶用户：
官方安装脚本 + 手动配置
理由：可控、安全、灵活



企业使用：
完全手动部署
理由：安全、可审计、可定制
```

注意事项：

```
[注意] 一键脚本虽然方便，但要注意：
  • 仅使用知名、可信的脚本



  • 定期检查更新

  • 不要在生产环境盲目使用

  • 了解脚本的基本操作

  • 保留手动配置的学习计划
```

下一章我们将学习手动部署的完整流程。

实践任务：

- 使用mack-a脚本部署V2Ray
- 配置域名和TLS证书
- 测试客户端连接
- 尝试Trojan部署

- 对比不同协议性能

> **第十三章 手动搭建教程**

第13章 手动搭建教程

手动搭建虽然复杂，但能完全掌控配置细节，理解工作原理。本章详细介绍手动部署流程。

13.1 V2Ray完整部署流程

13.1.1 安装V2Ray-core

下载安装：

```
# 方法1：官方安装脚本

bash <(curl -L https://raw.githubusercontent.com/v2fly/fhs-install-v2ray/master/install-releas

# 方法2：手动下载安装

# 访问：https://github.com/v2fly/v2ray-core/releas

# 下载对应架构的版本

# 查看系统架构



uname -m

# x86_64 → amd64
```

```
# aarch64 → arm64

# 下载并解压（以v5.2.0为例）

cd /tmp
wget https://github.com/v2fly/v2ray-core/releases/download/v5.2.0/v2ray-linux-64.zip
unzip v2ray-linux-64.zip
sudo mv v2ray /usr/local/bin/
sudo mv v2ctl /usr/local/bin/
sudo chmod +x /usr/local/bin/v2ray
sudo chmod +x /usr/local/bin/v2ctl

# 创建配置目录

sudo mkdir -p /usr/local/etc/v2ray
sudo mkdir -p /var/log/v2ray

# 测试安装

v2ray version
```

创建系统服务：

```
# 创建systemd服务文件

sudo vi /etc/systemd/system/v2ray.service

# 添加以下内容
```

```
[Unit]
Description=V2Ray Service
Documentation=https://www.v2fly.org/
After=network.target nss-lookup.target



[Service]
User=root
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
NoNewPrivileges=true
ExecStart=/usr/local/bin/v2ray run -config /usr/local/etc/v2ray/config.json
Restart=on-failure
RestartPreventExitStatus=23



[Install]
WantedBy=multi-user.target
```

重新加载systemd

```
sudo systemctl daemon-reload
```

启动V2Ray

```
sudo systemctl start v2ray
sudo systemctl enable v2ray
```

查看状态

```
sudo systemctl status v2ray
```

13.1.2 配置VMess + WebSocket + TLS

完整配置流程：

步骤1：准备域名和证书 （参见13.4节）

步骤2：创建V2Ray配置

```
sudo vi /usr/local/etc/v2ray/config.json
```

```
{

  "log": {
    "access": "/var/log/v2ray/access.log",
    "error": "/var/log/v2ray/error.log",
    "loglevel": "warning"
  },
  "inbounds": [{
    "port": 10000,
    "listen": "127.0.0.1",
    "protocol": "vmess",
    "settings": {
      "clients": [{
        "id": "生成UUID",
        "alterId": 0,
        "email": "client@example.com"
     }]
    },
    "streamSettings": {
      "network": "ws",
      "wsSettings": {
        "path": "/v2ray"
     }
    }
  }],
  "outbounds": [{
    "protocol": "freedom",
    "settings": {}
  },{
    "protocol": "blackhole",
    "settings": {},
    "tag": "blocked"
  }],
  "routing": {
    "rules": [{
      "type": "field",
      "ip": ["geoip:private"],
```

```
      "outboundTag": "blocked"
    }]
  }
}
```

生成UUID：

```
# 在线生成

cat /proc/sys/kernel/random/uuid

# 或使用工具



apt install uuid-runtime
uuidgen
```

步骤3：配置Nginx反向代理

```
sudo vi /etc/nginx/sites-available/v2ray
```

```
server {

    listen 80;
    server_name your-domain.com;
    return 301 https://$server_name$request_uri;
}



server {
    listen 443 ssl http2;
    server_name your-domain.com;



    ssl_certificate /path/to/fullchain.pem;
    ssl_certificate_key /path/to/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
```

```
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;



    root /var/www/html;
    index index.html;



    location / {
        try_files $uri $uri/ =404;
   }



    location /v2ray {
        if ($http_upgrade != "websocket") {
            return 404;
       }
        proxy_redirect off;
        proxy_pass http://127.0.0.1:10000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 300s;
   }
}
```

```
# 启用站点

sudo ln -s /etc/nginx/sites-available/v2ray /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

步骤4：测试配置

```
# 重启V2Ray
```

```
sudo systemctl restart v2ray

# 查看日志


tail -f /var/log/v2ray/error.log

# 测试WebSocket连接


curl -H "Connection: Upgrade" \
     -H "Upgrade: websocket" \
     -H "Host: your-domain.com" \
     https://your-domain.com/v2ray
```

13.1.3 VLESS + XTLS配置

配置文件：

```
{

  "log": {
    "loglevel": "warning"
  },
  "inbounds": [{
    "port": 443,
    "protocol": "vless",
    "settings": {
      "clients": [{
        "id": "your-uuid-here",
        "flow": "xtls-rprx-vision",
        "level": 0,
        "email": "user@example.com"
     }],
      "decryption": "none",
      "fallbacks": [{
        "dest": 80
     }]
```

```
    },
     "streamSettings": {
       "network": "tcp",
       "security": "tls",
       "tlsSettings": {
         "alpn": ["h2", "http/1.1"],
         "certificates": [{
           "certificateFile": "/path/to/fullchain.pem",
           "keyFile": "/path/to/privkey.pem"
        }]
      }
    }
  }],
  "outbounds": [{
    "protocol": "freedom"
  }]
}
```

fallbacks说明：

```
// 当不是VLESS流量时，转发到其他服务

"fallbacks": [
  {
    "dest": 80,              // 转发到端口80（Nginx）
    "xver": 1                // 传递真实IP
  },
  {
    "path": "/vmess",        // 特定路径
    "dest": 10001,           // 转发到VMess
    "xver": 1
  }
]
```

13.2 Trojan-Go配置详解

13.2.1 安装Trojan-Go

```
# 下载最新版本
```

```
cd /tmp
wget https://github.com/p4gefau1t/trojan-go/releases/download/v0.10.6/trojan-go-linux-amd64.zi
unzip trojan-go-linux-amd64.zip

# 安装到系统



sudo mv trojan-go /usr/local/bin/
sudo chmod +x /usr/local/bin/trojan-go

# 创建配置目录



sudo mkdir -p /etc/trojan-go
sudo mkdir -p /var/log/trojan-go
```

13.2.2 配置Trojan-Go服务端

配置文件：

```
sudo vi /etc/trojan-go/config.json
```

```
{

  "run_type": "server",
  "local_addr": "0.0.0.0",
  "local_port": 443,
  "remote_addr": "127.0.0.1",
  "remote_port": 80,
  "password": [
    "your_strong_password_here"
  ],
  "log_level": 1,
  "log_file": "/var/log/trojan-go/trojan.log",
  "ssl": {
    "cert": "/path/to/fullchain.pem",
    "key": "/path/to/privkey.pem",
```

```
    "sni": "your-domain.com",
    "alpn": [
      "http/1.1"
    ],
    "reuse_session": true,
    "session_ticket": false,
    "plain_http_response": ""
  },
  "tcp": {
    "prefer_ipv4": true,
    "no_delay": true,
    "keep_alive": true
  },
  "websocket": {
    "enabled": false
  },
  "router": {
    "enabled": false
  }
}
```

创建系统服务：

```
sudo vi /etc/systemd/system/trojan-go.service
```

```
[Unit]

Description=Trojan-Go Service
Documentation=https://p4gefau1t.github.io/trojan-go/
After=network.target nss-lookup.target



[Service]
Type=simple
User=root
ExecStart=/usr/local/bin/trojan-go -config /etc/trojan-go/config.json
Restart=on-failure
RestartSec=10s



[Install]
WantedBy=multi-user.target
```

```
# 启动服务

sudo systemctl daemon-reload
sudo systemctl start trojan-go
sudo systemctl enable trojan-go

# 查看状态



sudo systemctl status trojan-go
```

13.2.3 Trojan + Nginx伪装

Nginx配置：

```
server {

    listen 80;
    server_name your-domain.com;



    root /var/www/html;
    index index.html;



    location / {
        try_files $uri $uri/ =404;
   }
}
```

工作流程：

```
客户端连接 → Trojan-Go (443端口)
```

```
                ↓
          验证密码
                ↓
        ┌──────────────────────┬──────────────────────┐
    密码正确              密码错误
       ↓                     ↓
    代理流量          转发到Nginx(80端口)
       ↓                     ↓
    目标网站           返回真实网站
```

13.3 Shadowsocks-Rust搭建

13.3.1 安装SS-Rust

```
# 方法1：使用包管理器
# Debian/Ubuntu
wget https://github.com/shadowsocks/shadowsocks-rust/releases/download/v1.15.3/shadowsocks-v1.
tar xf shadowsocks-v1.15.3.x86_64-unknown-linux-gnu.tar.xz
# 安装
sudo mv ss* /usr/local/bin/
sudo chmod +x /usr/local/bin/ss*
```

13.3.2 配置文件

```
sudo mkdir -p /etc/shadowsocks

sudo vi /etc/shadowsocks/config.json
```

```
{

  "server": "0.0.0.0",
  "server_port": 8388,
  "password": "your_strong_password",
  "method": "chacha20-ietf-poly1305",
  "mode": "tcp_and_udp",
  "fast_open": true,
  "no_delay": true
}
```

创建系统服务：

```
sudo vi /etc/systemd/system/shadowsocks.service
```

```
[Unit]

Description=Shadowsocks-Rust Server
After=network.target



[Service]
Type=simple
User=root
ExecStart=/usr/local/bin/ssserver -c /etc/shadowsocks/config.json
Restart=on-failure



[Install]
WantedBy=multi-user.target
```

```
# 启动服务

sudo systemctl daemon-reload
sudo systemctl start shadowsocks
sudo systemctl enable shadowsocks
```

13.4 域名与证书配置

13.4.1 域名解析

DNS配置：

```
登录域名注册商控制面板


添加A记录：
类型：A
名称：@（或your-subdomain）
值：your-vps-ip
TTL：600（10分钟）



等待DNS生效（几分钟到几小时）



验证：
ping your-domain.com
nslookup your-domain.com
```

13.4.2 使用Let's Encrypt证书

certbot安装：

```
# Debian/Ubuntu


sudo apt install certbot python3-certbot-nginx -y


# CentOS


sudo yum install certbot python3-certbot-nginx -y
```

申请证书：

```
# 使用Nginx插件（推荐）

sudo certbot --nginx -d your-domain.com

# 或standalone模式（需要停止占用80端口的服务）



sudo certbot certonly --standalone -d your-domain.com

# 根据提示：

# 1. 输入邮箱

# 2. 同意服务条款

# 3. 是否接收邮件

# 4. 等待验证

# 证书位置：

# 证书：/etc/letsencrypt/live/your-domain.com/full

# 私钥：/etc/letsencrypt/live/your-domain.com/priv
```

自动续期：

```
# 测试续期

sudo certbot renew --dry-run

# certbot会自动配置定时任务

# 查看定时任务

sudo systemctl list-timers | grep certbot

# 或手动添加

sudo crontab -e

# 每天凌晨2点检查并续期

0 2   * /usr/bin/certbot renew --quiet
```

13.4.3 acme.sh申请证书（替代方案）

```
# 安装acme.sh

curl https://get.acme.sh | sh
source ~/.bashrc
```

```
# HTTP方式申请

acme.sh --issue -d your-domain.com -w /var/www/html

# DNS方式（推荐）

# 需要配置DNS API

export Ali_Key="your_ali_key"
export Ali_Secret="your_ali_secret"
acme.sh --issue --dns dns_ali -d your-domain.com

# 安装证书

acme.sh --install-cert -d your-domain.com \
  --key-file /etc/v2ray/privkey.pem \
  --fullchain-file /etc/v2ray/fullchain.pem \
  --reloadcmd "systemctl restart v2ray"

# 自动续期已配置，无需额外操作
```

13.5 Web服务器伪装

13.5.1 部署伪装网站

方法1：静态网站

```
# 下载模板
```

```
cd /var/www/html
sudo wget https://github.com/username/template/archive/master.zip
sudo unzip master.zip
sudo rm master.zip

# 或使用现成的HTML模板

# https://html5up.net/

# https://templated.co/
```

方法2：WordPress博客

```
# 安装LAMP环境

sudo apt install apache2 mysql-server php php-mysql -y

# 下载WordPress



cd /tmp
wget https://wordpress.org/latest.tar.gz
tar xzf latest.tar.gz
sudo mv wordpress/* /var/www/html/

# 配置数据库



sudo mysql -u root -p
CREATE DATABASE wordpress;
CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'password';
```

```
GRANT ALL ON wordpress.* TO 'wpuser'@'localhost';
FLUSH PRIVILEGES;
EXIT;

# 访问网站完成安装



http://your-domain.com
```

方法3：镜像网站

```
# 使用HTTrack镜像网站

sudo apt install httrack -y
httrack https://example.com -O /var/www/html
```

13.5.2 Nginx配置优化

```
server {

    listen 443 ssl http2;
    server_name your-domain.com;



    # SSL配置
    ssl_certificate /path/to/fullchain.pem;
    ssl_certificate_key /path/to/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512:ECDHE-RSA-AES256-GCM-SHA
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;
    ssl_stapling on;
    ssl_stapling_verify on;



   # 安全头
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

```
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;



   # 网站配置
    root /var/www/html;
    index index.html index.php;



    location / {
        try_files $uri $uri/ =404;
   }



   # 代理路径
    location /secret-path {
        # V2Ray/Trojan配置
   }
}
```

本章小结

本章详细介绍了手动搭建代理服务的完整流程：

核心要点：

- **V2Ray部署**：

- 核心安装与配置

- VMess + WebSocket + TLS - VLESS + XTLS - Nginx反向代理

- **Trojan部署**：

- Trojan-Go安装

- 配置文件详解 - Nginx伪装网站

- **Shadowsocks**：

- SS-Rust安装

- 配置优化 - 系统服务

- **域名证书**:

- DNS解析配置

- Let's Encrypt申请 - acme.sh使用 - 自动续期

- **伪装网站**:

- 静态网站部署

- WordPress安装 - 网站镜像 - Nginx优化

手动搭建优势:

```
✓ 完全掌控配置


✓ 理解工作原理
✓ 安全性最高
✓ 定制性最强
✓ 便于调试维护
```

注意事项:

```
[注意] 域名必须解析到服务器


[注意] 证书路径正确
[注意] 防火墙开放端口
[注意] 定期更新证书
[注意] 监控服务状态
```

下一章我们将学习多用户管理方案。

实践任务:

- 手动部署V2Ray服务
- 配置域名和证书
- 部署伪装网站
- 测试客户端连接
- 对比手动与脚本部署

> **第十四章 多用户管理方案**

第14章 多用户管理方案

当需要为多人提供代理服务时，需要专门的管理工具。本章介绍主流的多用户管理面板和方案。

14.1 面板管理工具（X-UI等）

14.1.1 X-UI面板

X-UI简介：

```
X-UI特点：


✓ Web图形化管理界面
✓ 支持多协议（VMess/VLESS/Trojan/Shadowsocks）
✓ 流量统计
✓ 用户管理
✓ 定时重启
✓ 系统监控
✓ 中文界面
```

安装X-UI：

```
# 一键安装脚本


bash <(curl -Ls https://raw.githubusercontent.com/vaxilu/x-ui/master/install.sh)

# 安装过程：


# 1. 选择端口（默认54321）
```

```
2. 设置管理员账号

3. 设置管理员密码

4. 自动安装并启动

访问面板


http://your-server-ip:54321

用户名：设置的用户名

密码：设置的密码
```

X-UI管理命令：

```
# 启动面板

x-ui start

停止面板


x-ui stop
```

重启面板

```
x-ui restart
```

查看状态

```
x-ui status
```

查看日志

```
x-ui log
```

更新面板

```
x-ui update
```

修改配置

```
x-ui setting
```

卸载

```
x-ui uninstall
```

14.1.2 X-UI使用教程

添加用户：

```
步骤1：登录面板

访问 http://your-ip:54321



步骤2：添加入站
面板 → Inbounds → + 添加入站



配置项：

  • 备注：用户昵称（如：user001）



  • 协议：选择协议类型（VLESS/VMess/Trojan）

  • 监听IP：0.0.0.0

  • 端口：随机或指定（建议1000-60000）

  • 流量限制：设置月流量（如50GB）

  • 到期时间：设置有效期

  • 传输配置：



    - TCP（直连）

    - WebSocket（CDN友好）
    - gRPC（推荐）



步骤3：配置TLS（如需要）
```

```
  • 启用TLS：开



  • 域名: your-domain.com

  • 证书路径: /path/to/cert

  • 密钥路径: /path/to/key





步骤4: 添加用户

  • UUID：自动生成或手动输入



  • Email：用户标识

  • 流量: 单用户流量限制（可选）





步骤5: 保存配置
点击保存，系统自动重载配置
```

导出客户端配置：

```
方法1: 扫描二维码
  • 点击用户后面的二维码图标



  • 手机扫描导入
```

```
方法2：复制链接

  • 点击复制链接按钮

  • 发送给用户

方法3：导出配置文件

  • 导出JSON配置

  • 用户手动导入
```

14.1.3 3X-UI（推荐）

3X-UI特点：

```
3X-UI是X-UI的增强版，基于Xray-core构建

核心优势：
✓ 现代化Web管理界面
✓ 支持所有Xray协议
✓ 支持Reality协议（无需域名）
✓ 多用户管理
✓ 流量统计和限制
✓ 系统状态监控
✓ 一键申请SSL证书
✓ 订阅链接生成
✓ Telegram机器人通知
✓ 定时重启任务
✓ 支持多管理员
```

```
支持协议:

  • VLESS (TCP/WS/gRPC/Reality)


  • VMess (TCP/WS/gRPC)

  • Trojan (TCP/WS/gRPC)

  • Shadowsocks

  • Dokodemo-door

  • Socks

  • HTTP
```

安装3X-UI:

```
# 一键安装最新版本

bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)

# 安装指定版本


bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) v2.3.0

# 安装过程:
```

```
1. 选择语言（中文/英文）

2. 设置管理员用户名（默认：admin）

3. 设置管理员密码（默认：admin）

4. 设置面板端口（默认：2053）

5. 自动安装完成

安装完成后：

访问地址：http://your-ip:2053

用户名：你设置的用户名

密码：你设置的密码

首次登录强制修改密码
```

3X-UI管理命令：

```
# 查看状态
```

```
x-ui status
```

启动/停止/重启

```
x-ui start
x-ui stop
x-ui restart
```

查看配置信息

```
x-ui settings
```

修改用户名和密码

```
x-ui username
x-ui password
```

修改面板端口

```
x-ui port
```

更新面板

```
x-ui update
```

卸载

```
x-ui uninstall
```

3X-UI配置Reality节点：

```
Reality配置步骤：
  • 登录3X-UI面板

    访问：http://your-ip:2053

    输入管理员账号密码

  • 添加Reality入站

    点击"入站列表" → "+添加入站"
  • 基本设置：

    - 备注：Reality-用户名

    - 协议：选择 VLESS
    - 监听IP：0.0.0.0
    - 端口：443（推荐）
    - 传输协议：TCP
     - TLS：选择 Reality

  • Reality设置：

     - Dest（目标网站）：www.microsoft.com:443

       （或其他大型网站）
     - ServerNames：www.microsoft.com
     - PrivateKey：点击“生成”按钮
     - ShortIds：点击“生成”按钮

  • 用户配置：
```

```
  - 添加用户：点击“+”

   - Email：用户标识（如user01@example.com）
   - UUID：自动生成
   - Flow：xtls-rprx-vision
  - 流量限制：设置月流量（如50GB）
  - 到期时间：设置有效期


• 保存并启用


  - 点击保存


  - 点击二维码图标生成分享链接
  - 用户扫码或复制链接导入客户端
```

Reality目标网站推荐：

```
选择原则：

• 必须是HTTPS网站（端口443）




• 选择知名大型网站


• 优先选择与VPS同地区的网站




美国区域 VPS：


• www.microsoft.com:443



• www.cisco.com:443


• www.apple.com:443
```

```
  • www.cloudflare.com:443




亚洲区域 VPS：

  • www.samsung.com:443


  • www.yahoo.co.jp:443

  • www.ntt.com:443




欧洲区域 VPS：

  • www.bmw.com:443


  • www.siemens.com:443

  • www.logitech.com:443
```

流量管理功能：

```
在入站配置中设置：
  •  总流量限制

    - 点击编辑入站

    - 设置总流量（如100GB）
    - 0表示不限制
```

```
• 过期时间
  - 设置到期日期
  - 过期后自动禁用
• 单用户限制
  - 编辑用户设置
  - 设置单用户流量限制
• IP连接限制
  - 防止账号共享
  - 设置最大同时连接数
```

14.1.4 V2Ray-UI/V2-UI

V2-UI面板：

```
# 安装
bash <(curl -Ls https://raw.githubusercontent.com/sprov065/v2-ui/master/install.sh)
# 功能：
• V2Ray多协议支持
• 用户管理
• 流量统计
```

```
  • 系统状态监控


# 访问


http://your-ip:65432
```

14.1.5 面板安全建议

加强面板安全：

```
# 1. 修改默认端口
# 面板设置 → 端口 → 改为非常见端口

# 2. 设置强密码

# 至少16位，包含大小写字母、数字、符号

# 3. 启用IP白名单

# 仅允许特定IP访问面板

# 4. 使用防火墙
```

```
ufw allow from your-ip to any port 54321

# 5. 启用HTTPS

# 面板设置 → SSL → 上传证书

# 6. 定期备份

# 面板设置 → 备份 → 下载备份文件

# 7. 限制登录尝试

# 防止暴力破解
```

14.2 流量统计与限制

14.2.1 流量统计功能

X-UI流量统计：

```
统计维度：
  • 总流量：所有用户总计

  • 用户流量：单个用户使用情况

  • 实时流量：当前上传/下载速度
```

```
  • 历史流量：按日/月统计




查看方式：

  • 面板首页：总览


  • Inbounds列表：每个用户的流量

  • 流量统计页面：详细图表

  • 导出报表：Excel/CSV格式
```

流量重置：

```
自动重置：
  • 设置 → 流量设置 → 重置周期


  • 选项：每月1号/自定义日期




手动重置：

  • 用户管理 → 选择用户 → 重置流量


  • 或批量重置所有用户
```

14.2.2 流量限制策略

限制方式：

```
1. 总流量限制

   设置用户月流量（如50GB）
   超出后自动禁用

 • 速度限制

   限制上传/下载速度

   防止单用户占用全部带宽

 • 连接数限制

   限制同时连接数

   防止账号共享

 • IP数量限制

   限制使用的IP数量

   防止多人使用
```

X-UI流量限制配置：

```
添加/编辑用户时设置：

总流量：

 • 输入流量大小（GB）
```

- 0表示不限制
- 超出后自动禁用

到期时间：

- 设置有效期
- 过期后自动禁用

限速（需配置）：

- 上传速度：MB/s
- 下载速度：MB/s

连接数：

- 最大同时连接数

14.2.3 流量监控与告警

Telegram机器人通知：

```
# 3X-UI支持Telegram机器人

配置步骤:

  • 创建Telegram Bot

    - 联系 @BotFather

    - 发送 /newbot
    - 设置bot名称
    - 获取Token

  • 获取Chat ID

    - 联系你的bot

    - 发送任意消息
    - 访问: https://api.telegram.org/bot<token>/getUpdates
    - 查找 "chat":{"id":数字}

  • 面板配置

    - 设置 → Telegram设置

    - 输入Bot Token
    - 输入Chat ID
    - 保存

  • 通知类型

    - 用户流量告警 (80%/90%/100%)

    - 到期提醒
    - 系统告警
    - 登录通知
```

邮件通知:

```
配置SMTP：
  • SMTP服务器
  • 端口（通常587）
  • 用户名
  • 密码
  • 发件人地址

通知事件：
  • 流量即将用完
  • 账号即将到期
  • 新用户添加
  • 系统告警
```

14.3 用户配置管理

14.3.1 批量用户管理

批量添加用户：

```
方法1：面板批量添加
  • 用户管理 → 批量添加
```

```
  • 设置用户数量

  • 统一配置:



  - 流量限制

  - 到期时间
  - 协议类型

  • 自动生成UUID



方法2: 导入用户列表

  • 准备CSV文件

  格式: 用户名,流量(GB),到期时间
  • 面板导入



方法3: API批量创建

  • 使用X-UI API



  • 编写脚本批量创建
```

批量修改配置:

```
选择多个用户 → 批量操作:
  • 重置流量
```

```
  • 延长时间

  • 修改限速

  • 启用/禁用

  • 删除用户
```

14.3.2 用户分组管理

按需求分组：

```
免费用户组：
  • 流量：10GB/月

  • 速度：5MB/s

  • 有效期：1个月


标准用户组：

  • 流量：50GB/月

  • 速度：50MB/s

  • 有效期：1个月
```

```
VIP用户组:

  • 流量: 500GB/月



  • 速度: 不限速

  • 有效期: 3个月




创建模板:
面板设置 → 用户模板

  • 保存常用配置



  • 新建用户时快速应用
```

14.3.3 配置导出与备份

导出用户数据:

```
# X-UI数据库位置

/etc/x-ui/x-ui.db

# 备份数据库


cp /etc/x-ui/x-ui.db /root/x-ui-backup-$(date +%Y%m%d).db
```

```
# 定期自动备份

crontab -e

# 每天凌晨3点备份

0 3  * cp /etc/x-ui/x-ui.db /root/backups/x-ui-$(date +\%Y\%m\%d).db

# 保留最近7天备份

find /root/backups/ -name "x-ui-*.db" -mtime +7 -delete
```

导出用户配置：

```
面板功能：
  • 用户列表 → 导出所有

  • 格式：JSON/CSV

  • 包含：

    - 用户信息

    - 流量统计
    - 配置参数
    - 订阅链接
```

用途：

- 迁移到新服务器
- 数据分析
- 用户分发

14.4 续费与到期提醒

14.4.1 自动续费系统

续费流程：

流程设计：

用户端：

- 用户访问续费页面
- 选择套餐（1月/3月/1年）
- 支付费用
- 系统自动延长

管理端：

- 收到支付通知

```
• 验证支付

• 更新用户配置:

  - 延长到期时间

  - 重置流量

• 发送确认通知
```

支付集成:

```
支付方式选择:
• 手动确认

  - 用户转账

  - 发送截图
  - 管理员手动确认

• 支付API

  - 支付宝

  - 微信支付
  - PayPal
  - 加密货币

• 卡密系统

  - 生成激活码

  - 用户输入激活
  - 自动续期
```

14.4.2 到期提醒机制

提醒时间节点：

```
提醒计划：

到期前7天：

  • 第一次提醒

  • 邮件/Telegram通知

  • 提示续费链接

到期前3天：

  • 第二次提醒

  • 强调即将到期

到期前1天：

  • 最后提醒

  • 紧急通知
```

到期当天：

- 账号禁用
- 发送到期通知

到期后7天：

- 最终通知
- 账号将被删除（可选）

通知模板：

```
邮件模板：

主题：【重要】您的账号将于3天后到期

尊敬的用户，

您的账号信息：
用户名：{username}
到期时间：{expire_date}
```

```
剩余时间：3天
剩余流量：{remain_traffic}GB



为了不影响您的使用，请及时续费。



续费链接：{renew_url}



如有问题，请联系客服。

---



Telegram通知：



[注意] 到期提醒



用户：{username}
到期：{expire_date}
剩余：3天



点击续费：{renew_url}
```

14.4.3 卡密系统实现

卡密生成：

```
# Python示例：生成卡密

import random
import string
```

```
def generate_card(length=16):
    """生成激活码"""
    chars = string.ascii_uppercase + string.digits
    card = ''.join(random.choice(chars) for _ in range(length))
    # 格式化: XXXX-XXXX-XXXX-XXXX
    return '-'.join([card[i:i+4] for i in range(0, len(card), 4)])
```

批量生成

```
cards = [generate_card() for _ in range(100)]
```

保存到数据库

包含:

- 卡密

- 面值（天数/流量）

- 状态（未使用/已使用）

- 使用时间

- 使用用户

卡密使用：

```
用户端：
  • 登录面板/客户端
  • 输入卡密
  • 点击激活

系统处理：
  • 验证卡密有效性
  • 检查是否已使用
  • 更新用户信息：
    - 延长到期时间
    - 增加流量
  • 标记卡密已使用
  • 记录使用日志
```

14.4.4 自动化运维脚本

定期清理脚本：

```
#!/bin/bash
# cleanup.sh - 清理过期用户

# X-UI数据库

DB_FILE="/etc/x-ui/x-ui.db"

# 获取当前时间戳

NOW=$(date +%s)

# 查询过期用户（SQL示例）

sqlite3 $DB_FILE "SELECT id,username FROM users WHERE expire_time < $NOW"

# 删除过期超过30天的用户

sqlite3 $DB_FILE "DELETE FROM users WHERE expire_time < $NOW - 2592000"

# 重启服务

x-ui restart
```

```
# 发送通知

echo "清理完成: $(date)" | mail -s "定期清理报告" admin@example.com
```

流量重置脚本:

```
#!/bin/bash
# reset_traffic.sh - 每月重置流量

# 重置所有用户流量

sqlite3 /etc/x-ui/x-ui.db "UPDATE users SET used_traffic = 0"

# 记录日志

echo "$(date): 流量已重置" >> /var/log/x-ui/reset.log

# 发送通知

curl -X POST "https://api.telegram.org/bot<token>/sendMessage" \
  -d "chat_id=<chat_id>" \
  -d "text=月度流量已重置"
```

定时任务配置:

```
crontab -e
```

```
# 每天凌晨3点清理过期用户
0 3   * /root/scripts/cleanup.sh

# 每月1号凌晨重置流量
0 0 1   /root/scripts/reset_traffic.sh

# 每天检查到期用户并提醒
0 9   * /root/scripts/expire_reminder.sh
```

本章小结

本章介绍了多用户管理的完整方案：

核心要点：

- **管理面板**：

- X-UI：功能全面、易用

- 3X-UI：现代化、功能强大 - V2-UI：简洁实用

- **流量管理**：

- 统计分析

- 限制策略 - 监控告警 - Telegram通知

- **用户管理**：

- 批量操作

- 分组管理 - 导出备份 - 配置模板

- **续费系统**:

- 自动续费

- 到期提醒 - 卡密系统 - 支付集成

最佳实践:

```
安全建议:

✓ 修改默认端口
✓ 设置强密码
✓ 启用IP白名单
✓ 使用HTTPS
✓ 定期备份


管理建议:
✓ 合理分配流量
✓ 设置到期时间
✓ 启用自动提醒
✓ 定期清理过期用户
✓ 监控系统资源


运营建议:
✓ 提供多种套餐
✓ 自动化续费流程
✓ 及时客户服务
✓ 定期系统维护
```

至此，第四部分"服务器搭建篇"全部完成!

实践任务:

- 安装X-UI面板
- 添加测试用户
- 配置流量限制
- 设置到期提醒
- 导出用户配置

- 创建自动化脚本

第十五章 网络安全防护

第15章 网络安全防护

安全是科学上网的核心问题。本章将介绍如何防范各种攻击，保护服务器和隐私安全。

15.1 常见攻击方式

15.1.1 主动探测攻击

GFW主动探测原理：

```
探测流程：
  • 流量特征识别

    GFW检测到可疑流量特征

    ↓

  • 记录服务器IP和端口

    ↓
  • 主动连接探测

    GFW主动连接该IP:端口

    ↓

  • 发送探测包

    模拟客户端握手

    ↓

  • 分析响应
```

```
    如果响应符合代理协议特征

    ↓

  • 封锁IP

    将IP加入黑名单

典型被探测的协议:

  • Shadowsocks（早期版本）



  • OpenVPN

  • PPTP/L2TP

  • 未混淆的V2Ray
```

防御措施:

```
1. 使用TLS加密

    外层流量完全符合HTTPS标准
     GFW无法区分是否为代理


  • 实现回落机制（Fallback）

     VLESS配置示例:

    {
       "fallbacks": [{
         "dest": 80  // 非法连接转发到Web服务器
      }]
    }
```

```
  探测者看到：正常的网站
  真实用户：正确认证后使用代理


• 密码/密钥认证


  Trojan协议：


  - 密码错误 → 返回真实网站内容
  - 密码正确 → 建立代理连接


• 时间窗口验证


  VMess协议：


  - 服务器时间差 > 90秒 → 拒绝连接
  - 防止重放探测


• 流量混淆


  使用WebSocket/gRPC等真实协议


  完全模仿正常应用流量
```

15.1.2 DDoS攻击

DDoS攻击类型：

```
1. SYN Flood


   大量SYN包耗尽连接资源


• UDP Flood


  大量UDP包占满带宽
• HTTP Flood
```

```
  应用层攻击，模拟大量HTTP请求
- 反射放大攻击

  利用DNS/NTP等服务放大流量
```

防御方案：

```
# 1. 启用SYN Cookies

echo "net.ipv4.tcp_syncookies = 1" >> /etc/sysctl.conf
sysctl -p

## 2. 限制连接数



iptables -A INPUT -p tcp --syn -m limit --limit 1/s -j ACCEPT

## 3. 使用Fail2Ban

## 自动封禁异常IP（见第11章）

## 4. CloudFlare CDN

## - DDoS防护

## - 隐藏真实IP

## - 自动防御
```

```
# 5. 流量清洗服务

# - 大型DDoS攻击时

# - 使用专业防护服务
```

CloudFlare防护配置：

```
安全等级设置：

  • 登录CloudFlare



  • 防火墙 → 设置

  • 安全级别：High

  • 质询通过期：30分钟

  • 启用：



    - Browser Integrity Check

    - Email Obfuscation
    - IP Geolocation Header
    - Always Use HTTPS



防火墙规则：


  • 创建规则
```

```
  • 条件:



    - 威胁分数 > 10

    - 或 国家 = 已知攻击来源


  • 动作: 质询 (Managed Challenge)
```

15.1.3 暴力破解攻击

SSH暴力破解:

```
# 查看SSH登录失败记录

grep "Failed password" /var/log/auth.log | tail -20

# 统计攻击IP



grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr

# 防御措施（已在第11章介绍）:

# 1. 修改SSH端口

# 2. 禁用密码登录

# 3. 使用密钥认证
```

4. 部署Fail2Ban

代理密码破解：

```
攻击方式:
  • 字典攻击



  • 暴力枚举

  • 弱密码猜测




防御:

  • 使用强密码

    - 至少32位随机字符

    - 包含大小写、数字、符号

  • 使用UUID而非密码

    V2Ray/Xray：

    - UUID：xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
    - 极难破解

  • 限制失败次数

    某些面板支持:

    - 错误3次 → 临时封禁IP
```

```
• 定期更换密码

  - 每月更换一次

  - 自动化脚本更新
```

15.1.4 中间人攻击（MITM）

攻击原理：

```
正常通信：

客户端 ←── 加密通道 ──→ 服务器



中间人攻击：
客户端 ←── 伪造证书 ──→ 攻击者 ←── 真实连接 ──→ 服务器
          (解密查看)



攻击手段：

• DNS劫持

  返回攻击者的IP地址
• ARP欺骗

  局域网中冒充网关
• 伪造证书

  使用自签名证书
```

防御措施：

```
1. 证书固定（Certificate Pinning）
```

```
  客户端配置:
  {
     "tlsSettings": {
       "serverName": "your-domain.com",
       "allowInsecure": false,  // 不接受不安全证书
       "fingerprint": "sha256:..."  // 证书指纹
    }
  }


• 使用正规CA证书


  - Let's Encrypt


  - 验证证书链完整性


• 启用HSTS


  Nginx配置:


  add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;


• 验证域名


  确保连接的是正确域名


  检查证书CN/SAN字段


• 使用加密DNS


  - DoH (DNS over HTTPS)


  - DoT (DNS over TLS)
  防止DNS劫持
```

15.1.5 流量分析攻击

威胁模型:

```
攻击者能力:

  • 观察流量

    - 包大小

    - 时间间隔
    - 传输模式

  • 统计分析

    - 流量指纹识别

    - 网站指纹识别
    - 行为模式分析


即使加密，仍可能推断:

  • 访问了哪些网站


  • 观看了哪些视频

  • 通信模式
```

防御策略:

```
1. 填充（Padding）

   添加随机数据填充
   统一数据包大小

  • 流量混淆

    - 发送假流量
```

```
  - 随机延迟
  - 随机包大小


• 多路复用（Mux）

  多个连接共享一个通道

  混淆流量模式


• 使用Tor

  多层代理

  难以追踪


• VPN over Tor over VPN

  极致隐私保护

  但速度很慢
```

15.2 端口伪装技术

15.2.1 端口选择策略

常见端口及特点：

```
标准端口：

22   - SSH（容易被扫描）
80   - HTTP（明文，不推荐）
443  - HTTPS（推荐，最常用）
8080 - 备用HTTP（常被扫描）
```

```
推荐端口:
443  - HTTPS标准端口
     ✓ 最不容易被限制
     ✓ 流量混在网页中



2053 - CloudFlare CDN
2083 - cPanel HTTPS
2087 - WHM HTTPS
2096 - cPanel Webmail



8443 - HTTPS备用
     ✓ 常见但不如443安全



随机高端口:
10000-60000
✓ 不易被扫描
✗ 可能被某些防火墙限制
```

15.2.2 端口复用技术

Nginx端口复用:

```
# 443端口同时服务网站和代理

server {
    listen 443 ssl http2;
    server_name your-domain.com;



    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;



   # 正常网站流量
    location / {
```

```
        root /var/www/html;
        index index.html;
    }



    # 代理流量（特定路径）
     location /secret-v2ray-path {
        if ($http_upgrade != "websocket") {
            return 404;
       }
        proxy_pass http://127.0.0.1:10000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

VLESS Fallback端口复用：

```
{


  "inbounds": [{
    "port": 443,
    "protocol": "vless",
    "settings": {
      "clients": [...],
      "decryption": "none",
      "fallbacks": [
       {
         // 默认流量转发到Nginx
           "dest": 8080
       },
       {
           // VMess流量
           "path": "/vmess",
           "dest": 10001,
           "xver": 1
       },
       {
           // Trojan流量
           "path": "/trojan",
           "dest": 10002,
           "xver": 1
       }
     ]
```

```
    },
     "streamSettings": {
       "network": "tcp",
       "security": "tls",
       "tlsSettings": {
         "certificates": [...]
      }
    }
  }]
}
```

15.2.3 SNI分流

原理：

```
TLS握手时的SNI字段指定域名


根据SNI分流到不同服务



示例场景：
client.example.com  → V2Ray
www.example.com     → 正常网站
api.example.com     → API服务
```

Nginx stream模块实现：

```
stream {


    map $ssl_preread_server_name $backend {
        v2ray.example.com  127.0.0.1:10000;
        www.example.com    127.0.0.1:8080;
        default            127.0.0.1:8080;
    }



    server {
        listen 443;
        ssl_preread on;
```

```
        proxy_pass $backend;
    }
}
```

15.3 流量特征混淆

15.3.1 WebSocket混淆

特点：

```
WebSocket协议特征：
  • HTTP升级请求

    GET /path HTTP/1.1

    Upgrade: websocket
    Connection: Upgrade

  • 双向通信



  • 长连接

  • 广泛应用（聊天、推送等）





优势：
✓ 完全符合标准协议
✓ 防火墙通常放行
✓ 可通过CDN
✓ 难以识别为代理
```

V2Ray WebSocket配置：

```
{
```

```
  "streamSettings": {
    "network": "ws",
    "wsSettings": {
      "path": "/chat",  // 伪装成聊天应用
      "headers": {
        "Host": "your-domain.com",
        "User-Agent": "Mozilla/5.0..."  // 真实浏览器UA
     }
   },
    "security": "tls"
  }
}
```

15.3.2 HTTP/2混淆

HTTP/2特点：

```
优势：


✓ 多路复用
✓ 头部压缩
✓ 二进制分帧
✓ 现代网站广泛使用
✓ 流量特征真实



V2Ray HTTP/2配置：
{
  "streamSettings": {
    "network": "h2",
    "httpSettings": {
      "host": ["your-domain.com"],
      "path": "/api/v1/data"
   },
    "security": "tls"
  }
}
```

15.3.3 gRPC混淆

gRPC特点：

```
gRPC = Google RPC框架


✓ 基于HTTP/2
✓ 二进制协议
✓ 广泛用于微服务
✓ 流量特征真实
✓ CloudFlare原生支持



配置示例:
{
  "streamSettings": {
    "network": "grpc",
    "grpcSettings": {
      "serviceName": "UserService",  // 伪装成用户服务
      "multiMode": true
    },
    "security": "tls"
  }
}
```

15.3.4 mKCP伪装

mKCP伪装类型：

```
{


  "kcpSettings": {
    "header": {
      "type": "wechat-video"  // 伪装类型
    }
  }
}
```

支持的伪装类型：

```
none：不伪装


srtp：伪装成SRTP（实时传输协议）
utp：伪装成uTorrent传输
```

```
wechat-video：伪装成微信视频通话（推荐）
dtls：伪装成DTLS 1.2
wireguard：伪装成WireGuard



推荐:


  • 国内: wechat-video



  • 国际: dtls / srtp
```

15.4 防止DPI深度检测

15.4.1 DPI检测原理

深度包检测（Deep Packet Inspection）：

```
检测层次:
  • 端口检测

    检查目标端口
  • 协议识别

    分析协议特征
  • 特征匹配

    - 特定字节序列

      - TLS握手指纹
    - 流量统计特征


  • 行为分析

    - 流量模式
```

```
    - 时序特征
    - 包大小分布


  • 主动探测


    模拟客户端连接
```

15.4.2 TLS指纹对抗

TLS指纹识别：

```
TLS握手包含：
  • TLS版本




  • 加密套件列表

  • 扩展字段

  • 椭圆曲线类型




不同客户端组合不同：
Chrome TLS指纹 ≠ V2Ray默认指纹



DPI可识别：
"这个TLS握手来自代理工具，不是真实浏览器"
```

uTLS防指纹：

```
uTLS = 微型TLS库

模拟真实浏览器指纹
```

```
Xray配置:
{
  "tlsSettings": {
    "fingerprint": "chrome",  // 模拟Chrome
   // 或 "firefox", "safari", "edge"
    "serverName": "your-domain.com"
  }
}



效果:
TLS握手完全像Chrome浏览器
DPI无法区分
```

15.4.3 流量填充

Padding技术:

```
问题:

代理流量有特征:

  • 固定包大小

  • 特定时序模式



解决:
添加随机填充数据

  • 统一包大小

  • 随机延迟

  • 打乱时序
```

```
V2Ray Padding配置:
{
  "streamSettings": {
    "sockopt": {
      "tcpFastOpen": true,
      "tproxy": "off"
    }
  }
}
```

15.4.4 REALITY协议（新技术）

REALITY特点：

```
REALITY = 真实世界的TLS

✓ 不需要域名
✓ 不需要证书
✓ 完全模拟真实网站TLS
✓ 几乎不可能被识别



工作原理:

  • 选择一个真实大站（如microsoft.com）



  • 客户端TLS握手指向该站点

  • 服务器识别真实用户 vs 探测

  • 真实用户：建立代理连接

  • 探测者：转发到真实网站
```

```
配置示例:
{
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "...",
    "shortIds": [""]
  }
}
```

15.5 服务器安全加固

15.5.1 系统安全配置

最小权限原则:

```
# 不要用root运行代理服务
# 创建专用用户
useradd -r -s /bin/false v2ray
# 修改文件所有权
chown -R v2ray:v2ray /usr/local/etc/v2ray
chown v2ray:v2ray /usr/local/bin/v2ray
# 修改systemd服务
```

```
vi /etc/systemd/system/v2ray.service
[Service]
User=v2ray
Group=v2ray
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_BIND_SERVICE
```

禁用不必要服务：

```
# 查看运行的服务

systemctl list-units --type=service --state=running

# 禁用不需要的服务



systemctl disable bluetooth
systemctl disable cups
systemctl disable avahi-daemon

# 最小化安装：

# 只保留：

# - sshd

# - 代理服务
```

```
- nginx

- 必要的系统服务
```

文件权限加固：

```
# 敏感文件权限

chmod 600 /etc/v2ray/config.json
chmod 600 /etc/ssh/sshd_config
chmod 600 /root/.ssh/authorized_keys

日志目录

chmod 755 /var/log/v2ray
chmod 644 /var/log/v2ray/*.log

证书文件

chmod 644 /etc/letsencrypt/live/*/fullchain.pem
chmod 600 /etc/letsencrypt/live/*/privkey.pem
```

15.5.2 入侵检测

使用AIDE检测文件篡改：

```
# 安装AIDE

apt install aide -y
```

初始化数据库

```
aideinit
```

移动数据库

```
mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

检查文件完整性

```
aide --check
```

定期检查

```
crontab -e
0 2   * /usr/bin/aide --check | mail -s "AIDE Report" admin@example.com
```

监控系统调用：

```
# 安装auditd

apt install auditd -y
```

监控敏感文件

```
auditctl -w /etc/v2ray/config.json -p wa -k v2ray_config
auditctl -w /etc/passwd -p wa -k passwd_changes
auditctl -w /etc/shadow -p wa -k shadow_changes

# 查看审计日志

ausearch -k v2ray_config
```

15.5.3 自动化安全检查

定期安全扫描脚本：

```
#!/bin/bash
# security_check.sh

echo "=== 安全检查报告 $(date) ===" > /tmp/security_report.txt

# 1. 检查开放端口

echo -e "\n[开放端口]" >> /tmp/security_report.txt
netstat -tulnp >> /tmp/security_report.txt

# 2. 检查登录失败

echo -e "\n[登录失败（最近20条）]" >> /tmp/security_report.txt
grep "Failed password" /var/log/auth.log | tail -20 >> /tmp/security_report.txt
```

3. 检查Fail2Ban状态

```
echo -e "\n[Fail2Ban封禁列表]" >> /tmp/security_report.txt
fail2ban-client status sshd >> /tmp/security_report.txt
```

4. 检查系统更新

```
echo -e "\n[可用更新]" >> /tmp/security_report.txt
apt list --upgradable >> /tmp/security_report.txt
```

5. 检查磁盘使用

```
echo -e "\n[磁盘使用]" >> /tmp/security_report.txt
df -h >> /tmp/security_report.txt
```

6. 检查内存使用

```
echo -e "\n[内存使用]" >> /tmp/security_report.txt
free -h >> /tmp/security_report.txt
```

发送报告

```
mail -s "服务器安全检查报告" admin@example.com < /tmp/security_report.txt
```

15.5.4 备份与恢复

自动备份脚本：

```
#!/bin/bash
# backup.sh
# ==========================================
# 备份配置文件
# ==========================================
BACKUP_DIR="/root/backups"
DATE=$(date +%Y%m%d_%H%M%S)
# ==========================================
# 备份数据库
# ==========================================
mkdir -p $BACKUP_DIR
# ==========================================
# 备份证书
# ==========================================
tar -czf $BACKUP_DIR/config_$DATE.tar.gz \
  /etc/v2ray/ \
  /etc/nginx/ \
  /etc/ssh/ \
  /etc/x-ui/ 2>/dev/null
# ==========================================
# 备份证书
# ==========================================
if [ -f /etc/x-ui/x-ui.db ]; then
  cp /etc/x-ui/x-ui.db $BACKUP_DIR/x-ui_$DATE.db
fi
```

```
tar -czf $BACKUP_DIR/certs_$DATE.tar.gz \
  /etc/letsencrypt/ 2>/dev/null

# 删除30天前的备份



find $BACKUP_DIR -name "*.tar.gz" -mtime +30 -delete
find $BACKUP_DIR -name "*.db" -mtime +30 -delete

# 上传到远程（可选）

# rsync -av $BACKUP_DIR/ user@remote:/backups/



echo "备份完成: $DATE"
```

定时备份：

```
crontab -e
# 每天凌晨2点备份



0 2   * /root/scripts/backup.sh >> /var/log/backup.log 2>&1
```

本章小结

本章详细介绍了网络安全防护的各个方面：

核心要点：

- **常见攻击**：

- 主动探测：回落机制、密码验证

- DDoS：CloudFlare、流量清洗 - 暴力破解：强密码、限制尝试 - MITM：证书验证、HSTS - 流量分析：填充、混淆

- **端口伪装**：

- 使用443端口

- 端口复用技术 - SNI分流 - Fallback机制

- **流量混淆**：

- WebSocket伪装

- HTTP/2混淆 - gRPC传输 - mKCP伪装头

- **防DPI**：

- TLS指纹对抗

- uTLS技术 - 流量填充 - REALITY协议

- **服务器加固**：

- 最小权限原则

- 入侵检测 - 安全扫描 - 定期备份

安全建议：

```
必做：

✓ 使用TLS加密
✓ 配置Fallback
✓ 启用CloudFlare CDN
✓ 定期更新系统
✓ 定期备份配置



推荐：
✓ 使用REALITY/Trojan
✓ 配置入侵检测
✓ 监控异常流量
✓ 实施访问控制
```

```
高级:
✓ TLS指纹混淆
✓ 多层防护
✓ 流量分析对抗
```

安全层次:

```
基础安全(必须):
  • TLS加密
  • 强密码
  • 防火墙

中级安全(推荐):
  • 流量伪装
  • 回落机制
  • CDN保护

高级安全(进阶):
  • DPI对抗
  • 指纹混淆
```

- 多层代理

下一章我们将学习隐私保护的实践技巧。

实践任务：

- 配置Fallback机制
- 启用CloudFlare CDN
- 部署入侵检测
- 创建备份脚本
- 进行安全审计

第十六章 隐私保护实践

第16章 隐私保护实践

隐私保护是科学上网的重要目标。本章介绍如何防止各种隐私泄露，保护个人信息安全。

16.1 DNS泄漏防护

16.1.1 什么是DNS泄漏

DNS泄漏原理：

```
正常代理流程：

用户 → 代理 → 目标网站
     (全程加密)



DNS泄漏：
用户 → 本地DNS → ISP DNS服务器
     (明文查询)
     ↓
ISP看到你查询的域名
即使流量走代理，仍暴露访问意图



示例：
访问google.com


  • DNS查询google.com（ISP看到）



  • 通过代理访问（加密，ISP看不到）



结果：ISP知道你想访问Google
```

16.1.2 检测DNS泄漏

在线检测工具：

```
1. DNS Leak Test

   https://www.dnsleaktest.com

 • IP Leak

   https://ipleak.net
 • BrowserLeaks

   https://browserleaks.com/dns

检测方法：

 • 启用代理



 • 访问检测网站

 • 查看DNS服务器

 • 如果显示本地ISP DNS = 泄漏

 • 应该显示代理服务器DNS
```

手动检测：

```
# Windows

nslookup google.com
```

```
# 查看Server地址

# Linux/macOS

dig google.com

# 查看SERVER字段

# 或

nslookup google.com
```

16.1.3 防止DNS泄漏

方法1：代理DNS查询

```
Clash配置：

dns:
  enable: true
  enhanced-mode: fake-ip  # 或redir-host
  nameserver:
    - 223.5.5.5
    - 119.29.29.29
  fallback:
    - https://1.1.1.1/dns-query
    - https://8.8.8.8/dns-query


工作原理：
```

```
  • 国内域名 → 国内DNS


  • 国外域名 → 通过代理查询国外DNS
```

方法2:加密DNS(DoH/DoT)推荐

```
重要趋势(2025):

美国行政命令要求联邦政府强制使用加密DNS
标志着加密DNS成为网络安全标准


DNS over HTTPS (DoH):

  • DNS查询通过HTTPS加密


  • ISP无法看到查询内容

  • 浏览器原生支持




推荐DoH提供商:


隐私优先:

  • Cloudflare: https://1.1.1.1/dns-query

  ✓ 最快速度
```

```
  ✓ 不记录日志
  ✓ DNSSEC支持


  • Quad9: https://dns.quad9.net/dns-query


  ✓ 非营利组织


  ✓ 阻止恶意域名
  ✓ 强隐私保护


  • NextDNS: https://dns.nextdns.io


  ✓ 可自定义过滤


  ✓ 广告拦截
  ✓ 详细分析



国内DoH:


  • 阿里DNS: https://dns.alidns.com/dns-query



  • 腾讯DNS: https://doh.pub/dns-query


  • 360DNS: https://doh.360.cn/dns-query




Clash DoH配置:
dns:
  enable: true
  enhanced-mode: fake-ip
  nameserver:
    - https://dns.alidns.com/dns-query
    - https://doh.pub/dns-query
```

```
  fallback:
    - https://1.1.1.1/dns-query
    - https://dns.quad9.net/dns-query



DNS over TLS (DoT):


 • DNS查询通过TLS加密



 • 端口:853


 • 某些网络环境更稳定




dns:
  enable: true
  nameserver:
    - tls://dns.alidns.com
    - tls://1.1.1.1
  fallback:
    - tls://dns.quad9.net



DNS over QUIC (DoQ):


 • 新兴标准



 • 基于QUIC协议


 • 更快速度和更低延迟
```

方法3：修改系统DNS

```
Windows：
  • 控制面板 → 网络连接



  • 网络适配器 → 属性


  • IPv4 → 属性


  • 使用下面的DNS服务器:



    首选: 1.1.1.1


    备用: 8.8.8.8



macOS：
系统偏好设置 → 网络 → 高级 → DNS
添加:
1.1.1.1
8.8.8.8



Linux：
sudo vi /etc/resolv.conf
nameserver 1.1.1.1
nameserver 8.8.8.8
```

防止被覆盖

```
sudo chattr +i /etc/resolv.conf
```

方法4：强制DNS通过代理

```
V2Ray配置:
```

```
{
  "dns": {
    "servers": [
     {
        "address": "1.1.1.1",
        "port": 53,
        "domains": [
          "geosite:geolocation-!cn"
       ]
     },
     {
        "address": "223.5.5.5",
        "port": 53,
        "domains": [
          "geosite:cn"
       ]
     }
    ]
  },
  "outbounds": [
    {
      "protocol": "freedom",
      "settings": {},
      "tag": "direct"
    }
  ],
  "routing": {
    "rules": [
     {
        "type": "field",
        "outboundTag": "proxy",
        "network": "udp",
        "port": "53"
     }
    ]
  }
}
```

16.2 WebRTC泄漏预防

16.2.1 WebRTC泄漏原理

什么是WebRTC?

```
WebRTC = Web实时通信

用途:

  • 视频通话（Zoom、Google Meet）



  • 语音通话

  • P2P文件传输





问题:
WebRTC会暴露真实IP地址
即使使用代理/VPN
```

泄漏机制：

```
WebRTC建立连接时:
  • 收集本地IP地址

    - 内网IP

    - 公网IP

  • 通过STUN服务器

    获取NAT外部IP
  • 发送给对方

    建立P2P连接

结果:
网页JavaScript可读取真实IP
绕过代理/VPN
```

16.2.2 检测WebRTC泄漏

在线检测：

```
1. BrowserLeaks WebRTC

   https://browserleaks.com/webrtc

 • IP Leak

   https://ipleak.net
 • Perfect Privacy

   https://www.perfect-privacy.com/webrtc-leaktest/

检测结果:

 • 显示代理IP = 安全



 • 显示真实IP = 泄漏
```

手动检测：

```
<!-- 在浏览器控制台运行 -->

<script>
var pc = new RTCPeerConnection({iceServers:[]});
pc.createDataChannel('');
pc.createOffer().then(offer=>pc.setLocalDescription(offer));
pc.onicecandidate = function(ice){
  if(!ice || !ice.candidate) return;
  console.log(ice.candidate.candidate);
};
</script>
```

16.2.3 防止WebRTC泄漏

方法1:浏览器扩展(推荐)

```
Chrome/Edge(兼容Manifest V3):

  • WebRTC Leak Prevent


  ✓ 兼容Manifest V3


  ✓ 70,000+用户
  ✓ 主动维护


  • WebRTC Protect - Protect IP Leak


   ✓ Manifest V3兼容


  ✓ 100,000+用户
  ✓ 控制WebRTC流量路由


  • WebRTC Leak Shield


  ✓ 内置测试工具


  ✓ 自定义保护模式
   ✓ IPv4/IPv6支持


  • uBlock Origin(启用WebRTC保护)


  ✓ 综合隐私保护


  ✓ 广告拦截+WebRTC防护



Firefox:


  • WebRTC Protect(推荐)
```

```
  ✓ 完整WebRTC控制
  • Disable WebRTC




  • uBlock Origin




注意:
2025年后Chrome扩展必须兼容Manifest V3
旧版扩展可能已停止工作
```

方法2：浏览器设置

```
Firefox：
  • 地址栏输入: about:config




  • 搜索: media.peerconnection.enabled

  • 改为: false

  • 重启浏览器




Chrome（需扩展）：
Chrome不支持原生禁用
必须使用扩展



Edge（Chromium）：
edge://flags
搜索: WebRTC
禁用相关选项
```

方法3：使用Tor浏览器

```
Tor Browser特点:

✓ 默认禁用WebRTC
✓ 强制通过Tor网络
✓ 不会泄漏真实IP
✓ 最佳隐私保护
```

方法4：防火墙规则

```
# Linux - 阻止STUN服务器
# 添加iptables规则

iptables -A OUTPUT -p udp --dport 3478 -j DROP
iptables -A OUTPUT -p udp --dport 19302 -j DROP

# 保存规则

iptables-save > /etc/iptables/rules.v4

# Windows防火墙

# 创建出站规则阻止UDP 3478、19302端口
```

16.3 IPv6泄漏处理

16.3.1 IPv6泄漏问题

泄漏原因：

```
问题场景：

  • 系统同时启用IPv4和IPv6




  • 代理仅支持IPv4


  • 访问双栈网站（支持IPv4/IPv6）




泄漏过程：
网站同时提供IPv4和IPv6
浏览器优先使用IPv6
IPv6流量 → 不经过代理 → 直接访问
ISP看到IPv6流量



结果：
IPv4流量走代理（加密）
IPv6流量直连（明文）
隐私泄漏
```

16.3.2 检测IPv6泄漏

在线检测：

```
1. Test IPv6

   https://test-ipv6.com


  • IP Leak
```

```
  https://ipleak.net
- IPv6 Leak Test

  https://ipv6leak.com

检测方法:

- 启用代理


- 访问检测网站

- 查看IPv6地址

- 如果显示 = 泄漏

- 应该不显示或显示代理IPv6
```

16.3.3 防止IPv6泄漏

方法1：禁用IPv6（推荐）

```
# Linux
# 临时禁用


sysctl -w net.ipv6.conf.all.disable_ipv6=1
sysctl -w net.ipv6.conf.default.disable_ipv6=1

# 永久禁用
```

```
echo "net.ipv6.conf.all.disable_ipv6 = 1" >> /etc/sysctl.conf
echo "net.ipv6.conf.default.disable_ipv6 = 1" >> /etc/sysctl.conf
echo "net.ipv6.conf.lo.disable_ipv6 = 1" >> /etc/sysctl.conf
sysctl -p

# Windows

## 控制面板 → 网络连接 → 网络适配器属性

## 取消勾选：Internet 协议版本 6 (TCP/IPv6)

## 或PowerShell（管理员）

Disable-NetAdapterBinding -Name "*" -ComponentID ms_tcpip6

# macOS

networksetup -setv6off Wi-Fi
networksetup -setv6off Ethernet
```

方法2：代理IPv6流量

```
Clash配置:

ipv6: false  # 禁用IPv6
```

```
或
ipv6: true   # 启用IPv6代理（需节点支持）



V2Ray配置：
{
  "inbounds": [{
    "listen": "::",  // 监听IPv6
    "port": 1080
  }]
}



注意：


  • 需要节点支持IPv6



  • 大多数节点仅支持IPv4


  • 建议直接禁用IPv6
```

方法3：防火墙阻止IPv6

```
# Linux ip6tables


ip6tables -P INPUT DROP
ip6tables -P OUTPUT DROP
ip6tables -P FORWARD DROP

# 保存规则



ip6tables-save > /etc/iptables/rules.v6
```

```
Windows防火墙

创建规则阻止所有IPv6出站流量
```

16.4 匿名支付方式

16.4.1 传统支付的隐私问题

支付信息暴露：

```
支付宝/微信支付：

✗ 实名认证
✗ 绑定银行卡
✗ 交易记录
✗ 收款方信息
✗ 可追溯



信用卡：
✗ 个人信息
✗ 账单地址
✗ 交易记录
✗ 银行监控



PayPal：
✗ 绑定支付方式
✗ 交易历史
✗ 收款方可见姓名
```

16.4.2 增强隐私的支付方式

虚拟信用卡：

```
Privacy.com（美国）：
```

```
✓ 生成一次性虚拟卡号
✓ 限制金额
✓ 隐藏真实卡号
✗ 需要美国银行账户



Revolut（欧洲）：
✓ 虚拟卡
✓ 一次性卡号
✓ 控制支出
✗ 需要欧洲账户



国内方案：


  • 部分银行提供虚拟卡



  • 云闪付虚拟卡
```

预付卡/礼品卡：

```
优势：

✓ 无需身份验证
✓ 限定金额
✓ 用完即丢



类型：


  • Amazon礼品卡



  • Google Play礼品卡


  • Steam礼品卡
```

```
  • 预付费信用卡




注意:
[注意] 某些商家不接受
[注意] 可能有手续费
[注意] 余额可能过期
```

加密货币:

```
Bitcoin（比特币）:

✓ 去中心化
✓ 无需身份
[注意] 可追溯（链上透明）



Monero（门罗币）:
✓ 完全匿名
✓ 隐私保护
✓ 不可追溯
✗ 接受度较低



使用建议:

  • 通过去中心化交易所购买



  • 使用专用钱包

  • 不要KYC验证

  • 使用混币服务（比特币）

  • Tor网络访问
```

16.4.3 匿名支付最佳实践

购买VPS示例：

```
方案1：加密货币
  • 购买Monero/Bitcoin
  • 选择接受加密货币的VPS商家
  • 使用Tor浏览器访问
  • 临时邮箱注册
  • 加密货币支付
  • 完全匿名

方案2：礼品卡
  • 现金购买礼品卡
  • 使用Tor访问
  • 临时邮箱
  • 礼品卡支付
  • 相对匿名
```

```
方案3：虚拟信用卡

  • 申请虚拟卡

  • VPN/代理访问

  • 虚拟卡支付

  • 低度匿名
```

16.5 数字足迹清理

16.5.1 浏览器指纹防护

什么是浏览器指纹:

```
浏览器指纹(Browser Fingerprinting):

通过收集设备和浏览器信息创建唯一标识

收集的信息:

  • 屏幕分辨率

  • 操作系统版本

  • 浏览器版本

  • 安装的字体

  • Canvas指纹
```

```
  • WebGL渲染信息

  • 音频指纹

  • CPU核心数

  • 设备内存

  • 触摸支持

  • 时区设置

  • 语言设置




威胁:
即使使用隐私模式/代理/VPN
仍然可能被识别和追踪
```

Firefox 145反指纹技术

```
Firefox 145新增防护:

核心改进:
✓ 将可识别用户从65%降至20%
✓ 追踪减少70%
✓ 默认在隐私浏览模式启用
✓ ETP Strict模式支持



具体防护措施:

  • Canvas噪声注入

    - 向背景图像添加随机噪声

    - 防止Canvas指纹识别
```

```
• 字体限制

  - 阻止访问本地字体

  - 仅允许系统标准字体
  - 特殊语言例外（中文/阿拉伯语）

• 触摸支持混淆

  - 仅报告0、1或5个触摸点

  - 防止设备类型识别

• 屏幕分辨率模糊化

  - 限制报告的分辨率精度

  - 隐藏真实屏幕信息

• CPU信息限制

  - 限制报告的CPU核心数

  - 防止设备指纹


启用方法:

• 更新到Firefox 145+



• 设置 → 隐私与安全

• 增强型追踪保护 → 严格

• 或使用隐私浏览模式
```

```
注意:

  • 某些网站功能可能受影响

  • 可为特定网站禁用保护
```

其他浏览器指纹防护:

```
Brave浏览器:

✓ 默认启用指纹防护
✓ 每个会话随机化
✓ 基于Chromium


Tor Browser(最强):
✓ 最强指纹防护
✓ 所有用户相同指纹
✓ 完全匿名


LibreWolf:
✓ Firefox增强隐私版
✓ 默认隐私配置
✓ 移除遥测


Chrome/Edge:
✗ 原生防护较弱
需要扩展:

  • Privacy Badger
```

```
• uBlock Origin


• CanvasBlocker
```

测试指纹唯一性:

```
在线测试工具:

• AmIUnique


  https://www.amiunique.org


  - 全面指纹分析
  - 显示唯一性百分比


• Cover Your Tracks (EFF)


  https://coveryourtracks.eff.org


  - 电子前沿基金会工具
  - 详细防护建议


• BrowserLeaks


  https://browserleaks.com


  - 多项测试
   - Canvas/WebGL/音频指纹


• Fingerprint.com


  https://fingerprint.com/demo


  - 商业级指纹测试
  - 显示实际追踪效果
```

```
理想结果:

  • 唯一性 < 20%

  • 更通用的指纹

  • 难以追踪
```

16.5.2 浏览器足迹

清理浏览器数据:

```
定期清理:
  • 浏览历史

  • Cookie

  • 缓存

  • 下载记录

  • 表单数据

  • 密码（使用密码管理器）

Chrome：
设置 → 隐私和安全 → 清除浏览数据
时间范围：时间不限
勾选：所有项目
```

```
Firefox：
历史 → 清除最近的历史记录
时间范围：全部
勾选：所有项目



自动清理：
Firefox：
设置 → 隐私与安全
历史记录：Firefox关闭时清除历史记录
```

使用隐私模式：

```
Chrome：Ctrl+Shift+N


Firefox：Ctrl+Shift+P
Edge：Ctrl+Shift+N



特点：
✓ 不保存历史
✓ Cookie会话结束后删除
✓ 不保存表单
✗ 不能防止追踪
✗ ISP仍可看到
```

浏览器隔离：

```
工作浏览器：
  • Chrome(工作账号)



  • 保留历史和Cookie



私人浏览器：
```

```
  • Firefox(个人账号)



  • 定期清理

  • 新增:Firefox 145反指纹追踪




匿名浏览器:

  • Tor Browser



  • 完全隔离

  • 每次启动全新环境
```

16.5.2 系统级清理

Windows清理:

```
# 清除DNS缓存

ipconfig /flushdns

# 清除临时文件



cleanmgr
```

```
# 清除事件日志

wevtutil cl Application
wevtutil cl System
wevtutil cl Security

# 清除最近文档

del /q %APPDATA%\Microsoft\Windows\Recent\*
```

Linux清理：

```
# 清除bash历史

cat /dev/null > ~/.bash_history
history -c

# 清除系统日志

sudo journalctl --vacuum-time=1d

# 清除临时文件

rm -rf /tmp/*
rm -rf ~/.cache/*
```

```
# 清除DNS缓存

sudo systemd-resolve --flush-caches
```

macOS清理：

```
# 清除DNS缓存

sudo dscacheutil -flushcache
sudo killall -HUP mDNSResponder

# 清除bash历史

cat /dev/null > ~/.bash_history

# 清除系统日志

sudo rm -rf /var/log/*

# 清除缓存

rm -rf ~/Library/Caches/*
```

16.5.3 元数据清理

文件元数据：

```
文件包含的元数据:
  • 创建时间
  • 修改时间
  • 作者信息
  • GPS位置（照片）
  • 设备信息
  • 软件版本

清理工具:
  • ExifTool（跨平台）
  • MAT2（Linux）
  • metadata-remover（在线）

使用ExifTool：
exiftool -all= photo.jpg
exiftool -all= document.pdf
```

图片元数据清理：

```
# 安装ExifTool
```

Ubuntu

```
sudo apt install exiftool
```

macOS

```
brew install exiftool
```

查看元数据

```
exiftool image.jpg
```

删除所有元数据

```
exiftool -all= image.jpg
```

批量处理

```
exiftool -all= -r /path/to/photos/
```

保留基本信息，删除敏感信息

```
exiftool -GPS:all= -Author= -Copyright= image.jpg
```

16.5.4 数字足迹最小化

原则：

```
1. 最小化信息暴露

   - 不提供非必要信息
   - 使用假名
   - 虚拟身份

 • 使用一次性身份

   - 临时邮箱

   - 虚拟手机号
   - 一次性账号

 • 数据分隔

   - 不同用途不同身份

   - 不关联账号
   - 不重复使用密码

 • 定期清理

   - 删除旧账号

   - 清理历史数据
   - 撤销应用授权

 • 加密存储

   - 敏感文件加密

   - 使用加密盘
   - 安全删除
```

工具推荐：

```
临时邮箱:
  • Temp Mail
  • 10 Minute Mail
  • Guerrilla Mail

虚拟号码:
  • Google Voice (美国)
  • 接码平台 (临时)

密码管理:
  • Bitwarden
  • KeePass

加密工具:
  • VeraCrypt (磁盘加密)
```

```
  • 7-Zip（文件加密）

  • GPG（文件签名）




安全删除：

  • Eraser（Windows）



  • shred（Linux）

  • Secure Empty Trash（macOS）
```

本章小结

本章详细介绍了隐私保护的各个方面：

核心要点：

- **DNS泄漏：**

- 使用加密DNS（DoH/DoT）

- 强制DNS通过代理 - 检测和验证

- **WebRTC泄漏：**

- 浏览器扩展禁用

- 修改浏览器设置 - 使用Tor浏览器

- **IPv6泄漏：**

- 禁用IPv6（推荐）

- 代理IPv6流量 - 防火墙阻止

- **匿名支付：**

- 加密货币（Monero）

- 虚拟信用卡 - 预付卡/礼品卡

- **数字足迹**：

- 定期清理浏览器

- 删除元数据 - 使用隐私工具

隐私保护层次：

```
基础防护：


✓ 防止DNS泄漏
✓ 禁用WebRTC
✓ 禁用IPv6
✓ 使用隐私模式



中级防护：
✓ 加密DNS
✓ 浏览器隔离
✓ 定期清理数据
✓ 元数据清理



高级防护：
✓ Tor浏览器
✓ 加密货币支付
✓ 虚拟身份
✓ 数据加密
✓ 最小化足迹
```

下一章我们将学习审计与日志管理。

实践任务：

- 检测并修复DNS泄漏
- 禁用WebRTC
- 配置加密DNS
- 安装隐私保护扩展
- 清理数字足迹

第十七章 审计与日志管理

第17章 审计与日志管理

审计与日志管理是服务器运维的重要组成部分。合理的日志配置既能帮助诊断问题，又能保护用户隐私。

17.1 访问日志配置

17.1.1 为什么需要日志

日志的作用：

```
1. 故障诊断

   - 连接失败原因
   - 性能问题分析
   - 错误追踪

 • 安全监控

   - 异常访问检测

   - 攻击识别
   - 入侵发现

 • 流量统计

   - 使用量分析

   - 用户行为
   - 资源规划

 • 合规要求

   - 某些场景需要
```

```
  - 审计需求
```

日志的风险：

```
[注意] 隐私泄露
  • 访问记录
  • 用户IP
  • 访问时间
  • 目标网站

[注意] 法律风险
  • 日志可能被要求提供
  • 作为证据使用

[注意] 存储风险
  • 磁盘占用
  • 日志泄露
```

17.1.2 V2Ray/Xray日志配置

基础日志配置：

```
{

  "log": {
    "loglevel": "warning",
    "access": "/var/log/v2ray/access.log",
    "error": "/var/log/v2ray/error.log"
  }
}
```

日志级别说明：

```
none：
  • 不记录任何日志



  • 最佳隐私保护

  • 无法诊断问题





error：


  • 仅记录错误



  • 推荐用于生产环境

  • 隐私风险低
```

```
warning：

  • 记录警告和错误

  • 适度平衡
  • 一般推荐

info：

  • 记录详细信息

  • 包含连接信息
  • 有隐私风险

debug：

  • 最详细日志

  • 用于调试
  • 隐私风险高
  • 不要用于生产
```

隐私保护配置：

```
{

  "log": {
    "loglevel": "error",
    "access": "none",  // 不记录访问日志
    "error": "/var/log/v2ray/error.log"
  }
}
```

仅记录到内存：

```
{

  "log": {
    "loglevel": "warning",
    "access": "",  // 空字符串 = stdout
    "error": ""
  }
}
```

17.1.3 Nginx访问日志

默认日志格式：

```
http {

    log_format main '$remote_addr - $remote_user [$time_local] '
                    '"$request" $status $body_bytes_sent '
                    '"$http_referer" "$http_user_agent"';



    access_log /var/log/nginx/access.log main;
    error_log /var/log/nginx/error.log warn;
}
```

隐私保护日志格式：

```
# 匿名化IP地址

log_format privacy '$remote_addr_anonymized - [$time_local] '
                   '$request_method $status $body_bytes_sent';

# 使用map匿名化IP



map $remote_addr $remote_addr_anonymized {
    ~(?P<ip>\d+\.\d+\.\d+)\.\d+ $ip.0;
    ~(?P<ip>[^:]+:[^:]+): $ip::;
    default 0.0.0.0;
}

# 应用到server



server {
    access_log /var/log/nginx/access.log privacy;
}
```

禁用特定路径日志：

```
server {

   # 代理路径不记录日志
    location /v2ray {
        access_log off;
        error_log /dev/null;



        proxy_pass http://127.0.0.1:10000;
   }



   # 网站内容正常记录
    location / {
```

```
        access_log /var/log/nginx/web.log;
        root /var/www/html;
    }
}
```

17.1.4 日志轮转

logrotate配置：

```
# /etc/logrotate.d/v2ray

/var/log/v2ray/*.log {
    daily                # 每天轮转
    rotate 7             # 保留7天
    missingok           # 文件不存在不报错
    notifempty          # 空文件不轮转
    compress            # 压缩旧日志
    delaycompress       # 延迟压缩
    sharedscripts
    postrotate
        systemctl reload v2ray > /dev/null 2>&1 || true
    endscript
}
```

Nginx日志轮转：

```
# /etc/logrotate.d/nginx

/var/log/nginx/*.log {
    daily
    rotate 7
    missingok
    notifempty
    compress
    delaycompress
    sharedscripts
    postrotate
        [ -f /var/run/nginx.pid ] && kill -USR1 cat /var/run/nginx.pid
    endscript
}
```

立即执行轮转：

```
# 测试配置

logrotate -d /etc/logrotate.d/v2ray

# 强制轮转

logrotate -f /etc/logrotate.d/v2ray
```

17.2 流量分析工具

17.2.1 实时流量监控

vnStat监控：

```
# 安装vnStat

apt install vnstat -y

# 初始化接口

vnstat -i eth0 --create

# 启动服务

systemctl enable vnstat
systemctl start vnstat
```

查看统计

```
vnstat              # 总览
vnstat -h           # 小时统计
vnstat -d           # 日统计
vnstat -m           # 月统计
vnstat -l           # 实时监控
```

iftop实时监控：

```
# 安装

apt install iftop -y
```

运行

```
iftop -i eth0
```

参数说明

-n　不解析域名

-N　不解析端口

-P　显示端口

```
# -b    以字节显示

# 推荐用法（匿名）

iftop -i eth0 -n -N
```

nethogs进程监控：

```
# 安装

apt install nethogs -y

# 运行

nethogs eth0

# 显示每个进程的流量使用

# 适合识别异常流量
```

17.2.2 流量统计分析

使用X-UI面板：

```
X-UI内置功能：
```

```
✓ 每个用户流量统计
✓ 总流量统计
✓ 流量图表
✓ 到期提醒



访问:
http://your-ip:54321
面板 → 入站列表 → 流量统计
```

自定义流量统计脚本:

```
#!/bin/bash
# traffic_stats.sh



LOG_FILE="/var/log/v2ray/access.log"
OUTPUT="/tmp/traffic_report.txt"



echo "=== V2Ray流量统计 $(date) ===" > $OUTPUT
echo "" >> $OUTPUT

# 统计总连接数



TOTAL=$(grep "accepted" $LOG_FILE | wc -l)
echo "总连接数: $TOTAL" >> $OUTPUT

# 统计唯一IP数



UNIQUE_IPS=$(grep "accepted" $LOG_FILE | awk '{print $4}' | sort -u | wc -l)
echo "唯一IP数: $UNIQUE_IPS" >> $OUTPUT
```

```
# 统计Top 10 IP



echo -e "\nTop 10 IP地址:" >> $OUTPUT
grep "accepted" $LOG_FILE | awk '{print $4}' | sort | uniq -c | sort -rn | head -10 >> $OUTPUT

# 发送报告



mail -s "V2Ray流量报告" admin@example.com < $OUTPUT
```

17.2.3 带宽限制

iptables限速：

```
# 限制单个IP最大速率为10Mbps

iptables -A FORWARD -s 客户端IP -m limit --limit 10mbit/s -j ACCEPT
iptables -A FORWARD -s 客户端IP -j DROP

# 限制端口总带宽



iptables -A OUTPUT -p tcp --sport 443 -m hashlimit \
  --hashlimit-above 100mb/s \
  --hashlimit-mode srcip \
  --hashlimit-name proxy \
  -j DROP
```

tc流量控制：

```
#!/bin/bash
```

```
# 使用tc（Traffic Control）限速

INTERFACE="eth0"
DOWNLOAD_LIMIT="100mbit"
UPLOAD_LIMIT="100mbit"

# 清除现有规则

tc qdisc del dev $INTERFACE root 2>/dev/null

# 添加根队列

tc qdisc add dev $INTERFACE root handle 1: htb default 10

# 添加类（限制总带宽）

tc class add dev $INTERFACE parent 1: classid 1:1 htb rate $UPLOAD_LIMIT

# 添加默认类

tc class add dev $INTERFACE parent 1:1 classid 1:10 htb rate $UPLOAD_LIMIT

echo "带宽限制已设置: 上传 $UPLOAD_LIMIT"
```

V2Ray用户限速：

```
{

  "policy": {
    "levels": {
     "0": {
        "handshake": 4,
        "connIdle": 300,
        "uplinkOnly": 2,
        "downlinkOnly": 5,
        "statsUserUplink": true,
        "statsUserDownlink": true,
        "bufferSize": 10240
     }
    }
  }
}
```

17.3 异常检测机制

17.3.1 连接异常检测

异常连接模式：

```
正常用户：
  • 连接时间分散
  • 流量适中
  • 访问网站多样

异常模式：
  • 短时间大量连接
```

```
  • 流量异常巨大

  • 固定时间访问

  • 单一目标
```

检测脚本：

```
#!/bin/bash
# detect_abnormal.sh
# ═══════════════════════════════════════════════════════════
LOG="/var/log/v2ray/access.log"
ALERT_EMAIL="admin@example.com"

# 检查最近1小时连接数
# ═══════════════════════════════════════════════════════════
RECENT_CONN=$(grep "$(date -d '1 hour ago' '+%Y/%m/%d %H')" $LOG | wc -l)

if [ $RECENT_CONN -gt 10000 ]; then
    echo "警告: 最近1小时连接数异常 ($RECENT_CONN)" | \
    mail -s "V2Ray异常检测" $ALERT_EMAIL
fi

# 检查单IP连接数
# ═══════════════════════════════════════════════════════════
TOP_IP=$(tail -10000 $LOG | awk '{print $4}' | sort | uniq -c | sort -rn | head -1)
IP_COUNT=$(echo $TOP_IP | awk '{print $1}')
IP_ADDR=$(echo $TOP_IP | awk '{print $2}')
```

```
if [ $IP_COUNT -gt 1000 ]; then
    echo "警告: IP $IP_ADDR 连接数异常 ($IP_COUNT)" | \
    mail -s "V2Ray IP异常" $ALERT_EMAIL
fi

# 检查大流量用户

# 需要配合流量统计
```

17.3.2 攻击检测

端口扫描检测：

```
#!/bin/bash
# detect_portscan.sh

# 使用iptables recent模块检测端口扫描

# 60秒内连接超过10次视为扫描



iptables -A INPUT -p tcp --dport 22 -m state --state NEW -m recent --set
iptables -A INPUT -p tcp --dport 22 -m state --state NEW -m recent \
  --update --seconds 60 --hitcount 10 -j DROP

# 记录扫描IP
```

```
iptables -A INPUT -p tcp --dport 22 -m state --state NEW -m recent \
  --update --seconds 60 --hitcount 10 -j LOG --log-prefix "Port Scan: "
```

暴力破解检测（Fail2Ban）：

```
# /etc/fail2ban/filter.d/v2ray.conf

[Definition]
failregex = rejected.*from <HOST>
ignoreregex =

# /etc/fail2ban/jail.local



[v2ray]
enabled = true
port = 443
filter = v2ray
logpath = /var/log/v2ray/error.log
maxretry = 5
findtime = 600
bantime = 3600
```

17.3.3 自动响应机制

IP自动封禁：

```
#!/bin/bash
# auto_block.sh



LOG="/var/log/v2ray/error.log"
BLOCKED_IPS="/etc/v2ray/blocked_ips.txt"
```

```
# 查找最近的错误连接IP



grep "rejected" $LOG | tail -100 | awk '{print $NF}' | sort | uniq -c | sort -rn | \
while read count ip; do
    if [ $count -gt 10 ]; then
       # 检查是否已封禁
        if ! grep -q "$ip" $BLOCKED_IPS; then
            echo "封禁IP: $ip (失败次数: $count)"
            iptables -A INPUT -s $ip -j DROP
            echo "$ip" >> $BLOCKED_IPS
        fi
    fi
done
```

流量限制自动降级：

```
#!/bin/bash
# traffic_limit.sh

# 当总流量超过阈值时自动限速



THRESHOLD_GB=900  # 900GB
CURRENT_GB=$(vnstat --oneline | cut -d';' -f11)



if [ $CURRENT_GB -gt $THRESHOLD_GB ]; then
    echo "流量接近限额，启用限速"



   # 启用tc限速
    tc qdisc add dev eth0 root tbf rate 50mbit burst 32kbit latency 400ms
```

```
    # 发送通知
    echo "流量已达 ${CURRENT_GB}GB，已启用限速" | \
    mail -s "流量预警" admin@example.com
fi
```

17.4 日志隐私平衡

17.4.1 隐私优先配置

极致隐私方案：

```
{

  "log": {
    "loglevel": "none",  // 完全不记录日志
    "access": "none",
    "error": "none"
  }
}
```

适度隐私方案：

```
{

  "log": {
    "loglevel": "error",     // 仅记录错误
    "access": "none",        // 不记录访问
    "error": "/dev/null"     // 错误也不保存
  }
}
```

内存日志方案：

```
# 日志写入tmpfs（RAM disk）
# 重启后自动清除
```

```
# 创建内存目录

mkdir -p /var/log/v2ray-tmp
mount -t tmpfs -o size=50M tmpfs /var/log/v2ray-tmp

# V2Ray配置

{
  "log": {
    "loglevel": "warning",
    "access": "/var/log/v2ray-tmp/access.log",
    "error": "/var/log/v2ray-tmp/error.log"
  }
}

# /etc/fstab自动挂载

tmpfs /var/log/v2ray-tmp tmpfs size=50M 0 0
```

17.4.2 数据匿名化

IP地址匿名化：

```
#!/usr/bin/env python3
# anonymize_log.py

import re
import sys
```

```
def anonymize_ip(ip):
    """将IP地址最后一段替换为0"""
    if ':' in ip:  # IPv6
        parts = ip.split(':')
        return ':'.join(parts[:-1] + ['0'])
    else:  # IPv4
        parts = ip.split('.')
        return '.'.join(parts[:3] + ['0'])



def process_log(input_file, output_file):
    """处理日志文件"""
    ip_pattern = re.compile(r'\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}')



    with open(input_file, 'r') as fin, open(output_file, 'w') as fout:
        for line in fin:
          # 替换所有IP地址
            line = ip_pattern.sub(lambda m: anonymize_ip(m.group()), line)
            fout.write(line)



if __name__ == '__main__':
    process_log('/var/log/v2ray/access.log', '/tmp/access_anon.log')
```

时间戳模糊化：

```
def anonymize_timestamp(timestamp):


    """将时间戳精确到小时"""
    # 2024/01/15 14:23:45 → 2024/01/15 14:00:00
    return re.sub(r':\d{2}:\d{2}$', ':00:00', timestamp)
```

17.4.3 日志加密存储

使用加密文件系统：

```
# 创建加密容器
```

```
cryptsetup luksFormat /dev/sdb1
cryptsetup luksOpen /dev/sdb1 encrypted_logs

# 格式化

mkfs.ext4 /dev/mapper/encrypted_logs

# 挂载

mkdir -p /var/log/encrypted
mount /dev/mapper/encrypted_logs /var/log/encrypted

# 日志写入加密分区

{
  "log": {
    "access": "/var/log/encrypted/access.log",
    "error": "/var/log/encrypted/error.log"
  }
}
```

GPG加密日志：

```
#!/bin/bash
# encrypt_logs.sh

LOG_DIR="/var/log/v2ray"
ENCRYPTED_DIR="/backup/logs"
GPG_KEY="admin@example.com"
```

```
# 压缩并加密昨天的日志

DATE=$(date -d yesterday +%Y%m%d)

tar czf - $LOG_DIR/*-$DATE.log | \
  gpg --encrypt --recipient $GPG_KEY \
  -o $ENCRYPTED_DIR/logs-$DATE.tar.gz.gpg

# 删除原始日志

rm -f $LOG_DIR/*-$DATE.log

# 解密方法

# gpg --decrypt logs-20240115.tar.gz.gpg | tar xz
```

17.4.4 定期清理策略

自动清理脚本：

```
#!/bin/bash
# clean_logs.sh

LOG_DIR="/var/log/v2ray"
RETENTION_DAYS=7
```

```
# 删除7天前的日志

find $LOG_DIR -name "*.log" -mtime +$RETENTION_DAYS -delete

# 清空当前日志（保留文件）

truncate -s 0 $LOG_DIR/access.log
truncate -s 0 $LOG_DIR/error.log

echo "$(date): 日志已清理" >> /var/log/clean.log
```

定时任务：

```
crontab -e
# 每天凌晨4点清理日志

0 4   * /root/scripts/clean_logs.sh

# 每周日加密归档

0 3   0 /root/scripts/encrypt_logs.sh

# 每月1日完全清理
```

```
0 5 1   rm -rf /var/log/v2ray/* && systemctl restart v2ray
```

安全删除：

```
#!/bin/bash
# secure_delete.sh

# 使用shred安全删除日志



shred -vfz -n 3 /var/log/v2ray/access.log
shred -vfz -n 3 /var/log/v2ray/error.log

# 参数说明：

# -v 显示进度

# -f 强制改变权限

# -z 最后用零覆盖

# -n 3 覆盖3次

# 或使用wipe
```

```
wipe -rf /var/log/v2ray/
```

本章小结

本章介绍了审计与日志管理的核心内容：

核心要点：

- **访问日志配置**：

- 选择合适的日志级别

- 配置日志轮转 - 隐私保护配置 - 禁用敏感路径日志

- **流量分析工具**：

- vnStat流量统计

- iftop实时监控 - X-UI面板统计 - 带宽限制

- **异常检测机制**：

- 连接异常检测

- 端口扫描检测 - 暴力破解防护 - 自动响应机制

- **日志隐私平衡**：

- 隐私优先配置

- 数据匿名化 - 加密存储 - 定期清理

日志管理原则：

```
1. 最小化原则

   只记录必要信息


 • 时效性原则


   定期清理旧日志
 • 安全性原则
```

```
  加密敏感日志
- 隐私性原则

  匿名化用户数据
- 实用性原则

  保留诊断必需信息
```

推荐配置：

```
个人使用：

日志级别：none / error
访问日志：禁用
错误日志：内存或禁用
清理策略：重启清除



小规模共享：
日志级别：error
访问日志：禁用
错误日志：/dev/null
清理策略：每日清理



商业运营：
日志级别：warning
访问日志：匿名化
错误日志：加密存储
清理策略：7天保留
```

隐私与诊断平衡：

```
调试期间：
- 日志级别：debug



- 详细记录


- 问题解决后立即清理
```

```
正常运行：
  • 日志级别：error
  • 最小化记录
  • 定期清理

高隐私需求：
  • 日志级别：none
  • 完全禁用
  • 接受无法诊断
```

至此，第五部分"安全与隐私篇"全部完成！

实践任务：

- 配置合适的日志级别
- 设置日志轮转
- 部署异常检测脚本
- 实施日志加密
- 创建定期清理任务

第十八章 常见问题诊断

第18章 常见问题诊断

科学上网过程中难免遇到各种问题。本章提供系统化的故障排查方法，帮助快速定位和解决常见问题。

18.1 无法连接的排查步骤

18.1.1 问题分类

无法连接的常见表现：

```
1. 完全无法连接
   - 客户端显示：连接失败
   - 浏览器：无法访问网站

• 部分网站无法访问
  - 国内网站正常
  - 国外网站无法访问

• 时好时坏
  - 偶尔能连接
  - 经常断开

• 特定应用无法使用
  - 浏览器正常
  - 其他软件无法联网
```

18.1.2 系统化排查流程

第一步：检查客户端配置

```
Clash检查清单:

☐ 配置文件是否正确导入
☐ 代理模式是否启用（规则/全局）
☐ 节点是否选择
☐ 系统代理是否开启
☐ 端口是否冲突



验证方法:

  • Clash → General → System Proxy（开启）



  • Clash → Proxies → 选择可用节点

  • Clash → Logs → 查看错误信息



常见错误:
[禁止] "proxy handshake failed"
   → 节点配置错误或节点失效



[禁止] "timeout"
   → 网络连接超时



[禁止] "connection refused"
   → 端口被占用或防火墙阻止
```

第二步：验证节点可用性

```
# 1. 测试服务器IP可达性

ping 服务器IP

# 或（某些服务器禁ping）


curl -v telnet://服务器IP:端口

# 2. 测试端口连通性

# Windows


Test-NetConnection -ComputerName 服务器IP -Port 443

# Linux/macOS


nc -zv 服务器IP 443

# 或


telnet 服务器IP 443

# 3. 测试TLS握手
```

```
openssl s_client -connect 域名:443 -servername 域名



成功输出:
Verify return code: 0 (ok)



失败输出:
connect: Connection refused
或
SSL handshake failed
```

第三步：检查网络环境

```
本地网络检查:
  • 能否访问百度/淘宝等国内网站

    → 检查基础网络
  • DNS解析是否正常

    nslookup google.com
  • 防火墙是否阻止

    临时关闭防火墙测试
  • 杀毒软件是否拦截

    临时关闭测试

ISP限制检查:

  • 使用手机热点测试

    如果正常 → 宽带ISP限制
  • 更换DNS

    1.1.1.1 / 8.8.8.8
  • 尝试不同端口
```

```
    80/443/8443等
```

第四步：服务器端检查

```
# SSH连接服务器
# 1. 检查服务状态
systemctl status v2ray
systemctl status xray
systemctl status nginx

# 2. 查看服务日志
journalctl -u v2ray -n 50
tail -50 /var/log/v2ray/error.log

# 3. 检查端口监听
netstat -tulnp | grep 443
ss -tulnp | grep 443

# 4. 测试本地连接
curl -x socks5://127.0.0.1:1080 https://google.com

# 5. 检查防火墙
```

```
ufw status
iptables -L -n
```

18.1.3 具体故障案例

案例1：节点配置错误

```
现象:

Clash显示"proxy handshake failed"



原因:

  • UUID/密码错误



  • 加密方式不匹配

  • 传输协议配置错误

  • TLS设置错误




解决:

  • 对比客户端和服务器配置



  • 检查UUID是否一致

  • 验证alterId（VMess）

  • 确认TLS域名配置
```

```
服务器端V2Ray配置:
{
  "inbounds": [{
    "protocol": "vmess",
    "settings": {
      "clients": [{
        "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
        "alterId": 0
     }]
   }
  }]
}



客户端配置必须匹配:


 • uuid: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx



 • alterId: 0
```

案例2：端口被占用

```
现象:


服务启动失败
日志: bind: address already in use



检查:

# Linux
```

```
netstat -tulnp | grep :443
lsof -i :443

# 发现nginx占用了443端口



解决方案1: 更换V2Ray端口
{
  "inbounds": [{
    "port": 10000,  // 改为其他端口
    "listen": "127.0.0.1"
  }]
}



解决方案2: 使用端口复用（推荐）
配置Fallback让V2Ray和Nginx共用443
```

案例3：防火墙阻止

```
现象:

服务器端测试正常
客户端无法连接



检查:

# 服务器端



ufw status
iptables -L -n | grep 443



解决:
```

```
# UFW

ufw allow 443/tcp
ufw reload

# iptables

iptables -A INPUT -p tcp --dport 443 -j ACCEPT
iptables-save > /etc/iptables/rules.v4

# firewalld

firewall-cmd --permanent --add-port=443/tcp
firewall-cmd --reload
```

案例4：时间不同步

```
现象：

VMess连接失败
日志：invalid timestamp

原因：
VMess要求客户端和服务器时间差 < 90秒

检查：
```

客户端

```
date
```

服务器

```
date
```

解决:

服务器安装NTP

```
apt install chrony -y
systemctl enable chrony
systemctl start chrony
```

验证同步

```
chronyc tracking
```

Windows客户端

设置 → 时间和语言 → 自动设置时间

18.2 速度慢的优化方法

18.2.1 速度慢的原因分析

带宽瓶颈：

```
1. 服务器带宽不足

   - 共享带宽被占满
   - VPS商家限速

 • 本地带宽限制

   - 家庭宽带上行限制

   - ISP QoS限速

 • 线路拥堵

   - 国际出口拥堵

   - 高峰期缓慢
```

延迟问题：

```
1. 物理距离远

   - 美西延迟 ~150ms
   - 欧洲延迟 ~250ms
   - 亚洲延迟 ~50ms

 • 路由跳数多

   - 绕路严重

   - 中转节点多

 • 丢包率高
```

```
  - 网络不稳定

  - 线路质量差
```

18.2.2 速度测试方法

基础速度测试：

```
# 1. Speedtest测试
# 安装
curl -s https://packagecloud.io/install/repositories/ookla/speedtest-cli/script.deb.sh | sudo
apt install speedtest -y

# 测试
speedtest

# 指定服务器
speedtest --server-id=服务器ID

# 2. Fast.com (Netflix CDN)
# 浏览器访问
```

```
https://fast.com

# 命令行



npm install --global fast-cli
fast
```

代理速度测试：

```
# 通过代理下载文件测速
# Clash代理端口：7890

# Linux/macOS



export http_proxy=http://127.0.0.1:7890
export https_proxy=http://127.0.0.1:7890



curl -o /dev/null https://速度测试文件地址

# 或使用wget



wget -e use_proxy=yes \
     -e http_proxy=127.0.0.1:7890 \
     -O /dev/null \
     https://速度测试文件地址
```

```
# 测试文件来源

https://github.com/cloudflare/Cloudflare-CfSpeed/releases/latest/download/测试文件
```

节点延迟测试：

```
Clash内置测速：
  • Proxies → 节点

  • 点击闪电图标

  • 显示延迟（ms）

ClashX测速：
右键节点 → Benchmark

Clash for Windows：
Proxies → 右键节点 → Delay Test

或批量测速：
Proxies → 全选 → Speed Test
```

18.2.3 优化方案

优化1：启用BBR加速

```
# 检查是否已启用
```

```
lsmod | grep bbr
sysctl net.ipv4.tcp_congestion_control

# 启用BBR（见第9章）



echo "net.core.default_qdisc=fq" >> /etc/sysctl.conf
echo "net.ipv4.tcp_congestion_control=bbr" >> /etc/sysctl.conf
sysctl -p

# 验证



lsmod | grep bbr

# 应显示：tcp_bbr



效果：

  - 提升TCP吞吐量



  - 降低延迟

  - 改善弱网环境表现
```

优化2：调整MTU

```
# 检查当前MTU
```

```
ip link show eth0
ifconfig eth0
```

测试最佳MTU（Windows）

```
ping -f -l 1472 google.com
```

逐步减小数值直到不分片

设置MTU（Linux）

```
ip link set eth0 mtu 1400
```

永久设置

```
echo "MTU=1400" >> /etc/sysctl.conf
```

Windows

```
netsh interface ipv4 set subinterface "以太网" mtu=1400 store=persistent
```

效果：

- 减少分片

```
  • 提高传输效率

  • 适合弱网环境
```

优化3：使用CDN中转

```
CloudFlare CDN：
  • 域名接入CloudFlare

  • 开启CDN（橙色云朵）

  • V2Ray配置WebSocket+TLS

  • 客户端连接CDN域名

优势：
✓ 隐藏真实IP
✓ DDoS防护
✓ 就近接入
✓ 提升速度（某些地区）

配置示例：
{
  "streamSettings": {
    "network": "ws",
    "wsSettings": {
      "path": "/v2ray"
    },
    "security": "tls",
    "tlsSettings": {
      "serverName": "你的CF域名"
    }
  }
}
```

优化4：多路复用（Mux）

```
// Xray客户端配置

{
  "outbounds": [{
    "protocol": "vmess",
    "settings": {...},
    "mux": {
      "enabled": true,
      "concurrency": 8  // 并发连接数
   }
  }]
}



// V2Ray配置
{
  "mux": {
    "enabled": true,
    "concurrency": 8
  }
}



效果:
✓ 减少握手次数
✓ 复用TCP连接
✓ 提升并发性能



注意:
[注意] 不适用于UDP
[注意] 可能影响大文件下载
[注意] 某些协议不支持
```

优化5：更换传输协议

```
协议性能对比:
```

```
TCP（传统）
速度：★★★☆☆
稳定：★★★★★
延迟：一般
适用：通用场景



mKCP（KCP over UDP）
速度：★★★★★
稳定：★★★☆☆
延迟：低
适用：弱网、高丢包



QUIC
速度：★★★★☆
稳定：★★★★☆
延迟：低
适用：UDP友好网络



gRPC
速度：★★★★☆
稳定：★★★★☆
延迟：一般
适用：CDN场景



推荐：


  • 一般场景：TCP + TLS



  • 弱网环境：mKCP


  • CDN中转：gRPC / WebSocket
```

18.2.4 线路优化

选择优质线路：

```
线路类型：

  • 直连线路

    电信/联通/移动 → 目标服务器

    延迟：高
    稳定性：一般
    成本：低

  • CN2 GIA线路

    中国电信精品网

    延迟：低
    稳定性：高
    成本：高

  • IPLC/IEPL专线

    内网专线，不过GFW

    延迟：极低
    稳定性：极高
    成本：极高

  • 中转/中继

    国内VPS → 国外VPS

    延迟：中
    稳定性：高
    成本：中


选择建议：
```

```
  • 预算有限：普通线路 + BBR



  • 追求稳定：CN2 GIA

  • 极致体验：IPLC专线

  • 平衡方案：中转线路
```

搭建中转服务器：

```
# 使用iptables转发
# 国内VPS配置


# 1. 启用转发



echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf
sysctl -p

# 2. 配置转发规则


# 本地端口10000转发到国外服务器IP:443



iptables -t nat -A PREROUTING -p tcp --dport 10000 \
  -j DNAT --to-destination 国外IP:443
```

```
iptables -t nat -A POSTROUTING -p tcp -d 国外IP --dport 443 \
  -j SNAT --to-source 国内VPS_IP

# 3. 保存规则



iptables-save > /etc/iptables/rules.v4

# 客户端连接

# 服务器地址：国内VPS_IP

# 端口：10000
```

18.3 频繁断线解决方案

18.3.1 断线原因分析

网络层面：

```
1. NAT超时

   - 家用路由器NAT表满
   - 超时断开连接


 • ISP限制


   - QoS限制长连接


   - 检测到代理流量
```

```
  • 网络波动

    - WiFi信号不稳定

    - 移动网络切换

  • 服务器重启

    - 自动更新

    - 资源耗尽
```

协议层面：

```
1. TCP keepalive失效

    - 长时间无数据传输
    - 被中间设备断开

  • WebSocket超时

    - 代理服务器超时

    - CDN超时设置

  • TLS会话过期

    - Session timeout

    - 需要重新握手
```

18.3.2 客户端优化

Clash配置优化：

```
# 增强连接稳定性
```

```
tun:
  enable: true
  stack: system
  dns-hijack:
    - any:53
```

TCP并发

```
tcp-concurrent: true
```

设置超时

```
dial-timeout: 5000ms
```

V2Ray配置优化：

```
{

  "policy": {
    "levels": {
     "0": {
        "handshake": 4,
        "connIdle": 300,      // 连接空闲超时（秒）
        "uplinkOnly": 2,
        "downlinkOnly": 5,
        "bufferSize": 512
     }
    }
  },
  "outbounds": [{
    "protocol": "vmess",
    "streamSettings": {
      "sockopt": {
        "tcpKeepAliveInterval": 10,  // TCP保活间隔
        "tcpFastOpen": true
     }
```

```
    }
  }]
}
```

18.3.3 服务器端优化

系统TCP优化：

```
# /etc/sysctl.conf
# TCP KeepAlive设置
net.ipv4.tcp_keepalive_time = 600
net.ipv4.tcp_keepalive_intvl = 10
net.ipv4.tcp_keepalive_probes = 3

# 连接重用
net.ipv4.tcp_tw_reuse = 1

# 增加连接队列
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 8192

# 应用配置
sysctl -p
```

Nginx超时配置：

```
http {

    # 增加超时时间
     proxy_connect_timeout 60s;
     proxy_read_timeout 300s;
     proxy_send_timeout 300s;



     # WebSocket保活
     keepalive_timeout 300s;



     server {
         location /v2ray {
             proxy_pass http://127.0.0.1:10000;



           # 关键：WebSocket保活
             proxy_http_version 1.1;
             proxy_set_header Upgrade $http_upgrade;
             proxy_set_header Connection "upgrade";



           # 超时设置
             proxy_connect_timeout 60s;
             proxy_read_timeout 300s;
             proxy_send_timeout 300s;
        }
    }
}
```

监控和自动重启：

```
#!/bin/bash
# monitor.sh - 监控服务并自动重启
```

```
SERVICE="v2ray"



while true; do
    # 检查服务状态
    if ! systemctl is-active --quiet $SERVICE; then
        echo "$(date): $SERVICE 已停止，正在重启..." >> /var/log/monitor.log
        systemctl restart $SERVICE
    fi



    # 检查端口监听
    if ! netstat -tulnp | grep -q ":443.*LISTEN"; then
        echo "$(date): 端口443未监听，重启服务..." >> /var/log/monitor.log
        systemctl restart $SERVICE
    fi



    sleep 60
done
```

18.3.4 使用故障转移

Clash故障转移配置：

```
proxy-groups:


  - name: 自动切换
    type: fallback
    url: 'http://www.gstatic.com/generate_204'
    interval: 300
    proxies:
     - 节点1
     - 节点2
     - 节点3



  # 或使用负载均衡
  - name: 负载均衡
    type: load-balance
    url: 'http://www.gstatic.com/generate_204'
```

```
    interval: 300
    strategy: consistent-hashing  # 或 round-robin
    proxies:
     - 节点1
     - 节点2
     - 节点3
```

18.4 特定网站无法访问

18.4.1 问题分类

访问异常类型：

```
1. 完全无法打开

    - DNS解析失败
   - 连接被重置
   - 超时

 • 部分内容无法加载

   - 图片/视频无法显示

   - 资源加载失败
    - CDN被墙

 • 功能异常

   - 无法登录

   - 无法提交表单
    - WebSocket连接失败

 • 地区限制

   - 版权限制

   - 地理位置限制
```

18.4.2 DNS问题

DNS污染检测：

```
# 测试DNS解析
# 国内DNS
nslookup google.com 114.114.114.114
# 国外DNS
nslookup google.com 8.8.8.8
# 如果结果不一致 → DNS污染
# 使用DoH查询
curl -H 'accept: application/dns-json' \
  'https://1.1.1.1/dns-query?name=google.com&type=A'
```

解决方案：

```
# Clash配置强制DNS

dns:
  enable: true
  enhanced-mode: fake-ip
  fake-ip-range: 198.18.0.1/16
```

```
  fake-ip-filter:
    - '*.lan'
    - 'localhost.ptlogin2.qq.com'
  nameserver:
    - https://1.1.1.1/dns-query
    - https://8.8.8.8/dns-query
  fallback:
    - https://dns.google/dns-query

# 特定域名使用特定DNS



nameserver-policy:
  'www.google.com': 8.8.8.8
  '+.netflix.com': 1.1.1.1
```

18.4.3 分流规则问题

检查规则匹配：

```
# Clash日志模式

log-level: debug

# 查看规则匹配

# 日志会显示：

# [Rule] google.com match DOMAIN-SUFFIX → PROXY
```

```
# 常见问题：


- 规则顺序错误

  DIRECT规则在PROXY之前
- 域名拼写错误

  google.com 写成 gogle.com
- 缺少规则

  新网站未添加规则
# 解决：调整规则顺序



rules:
  - DOMAIN-SUFFIX,google.com,PROXY  # 具体规则在前
  - GEOIP,CN,DIRECT                 # 通用规则在后
  - MATCH,PROXY                      # 兜底规则最后
```

18.4.4 网站特殊要求

Netflix等流媒体：

```
问题：

显示"proxy detected"或地区不可用



原因：

- IP被识别为代理



- 不是原生IP
```

```
  • IP地区不匹配




解决:

  • 使用原生IP节点

    - AWS/GCP等云服务商

   - 家宽IP

  • 使用专门的流媒体节点

    - Netflix专用节点

   - 地区对应节点

  • DNS解锁

   某些服务商提供DNS解锁

Clash分流:
rules:
  - DOMAIN-SUFFIX,netflix.com,Netflix专用
  - DOMAIN-KEYWORD,netflix,Netflix专用
```

ChatGPT等AI服务:

```
问题:

"Access denied" / "Not available in your country"



原因:

  • 地区限制
```

```
  • IP风险评分高

  • 数据中心IP被屏蔽




解决:

  • 使用住宅IP节点



  • 避免使用亚洲IP

  • 使用美国/欧洲节点

  • 清除浏览器Cookie




推荐地区:
✓ 美国
✓ 英国
✓ 德国
✗ 香港
✗ 新加坡（部分可用）
```

18.5 客户端错误代码解析

18.5.1 Clash错误代码

常见错误及解决:

```
1. "dial tcp: lookup xxx: no such host"
```

```
  原因: DNS解析失败
  解决:
  - 检查DNS配置
  - 使用其他DNS
  - 检查域名是否正确


• "dial tcp: i/o timeout"


  原因: 连接超时

  解决:
  - 检查网络连接
  - 测试节点可用性
  - 增加超时时间


• "read: connection reset by peer"


  原因: 连接被重置

  解决:
  - 节点可能被墙
  - 检查服务器状态
  - 更换节点


• "proxy handshake failed"


  原因: 代理握手失败

  解决:
  - 检查UUID/密码
  - 验证加密方式
  - 确认协议配置


• "EOF"


  原因: 连接意外关闭
```

```
  解决:
  - 服务器可能重启
  - 检查Nginx配置
  - 查看服务器日志
```

18.5.2 V2Ray错误代码

核心错误信息:

```
1. "failed to handler mux client connection"

  原因: Mux配置问题
  解决:
  - 禁用Mux测试
  - 检查服务器支持
  - 减少并发数


 • "invalid user"


  原因: 用户验证失败

  解决:
  - 检查UUID
  - 验证alterId
  - 确认用户存在


 • "invalid timestamp"


  原因: 时间不同步

  解决:
  - 同步系统时间
  - 启用NTP
  - 时间差 < 90秒


 • "TLS handshake failed"


  原因: TLS握手失败
```

```
  解决:
  - 检查证书有效性
  - 验证域名配置
   - allowInsecure设置


• "transport: authentication handshake failed"


  原因: 认证握手失败


  解决:
  - 检查密钥/密码
  - 验证加密方式
  - 确认协议版本
```

18.5.3 系统错误代码

Windows错误:

```
1. 错误代码: 10061


   "Connection refused"
  原因: 目标端口未监听
  解决:
  - 检查代理端口设置
  - 确认服务已启动
  - 验证防火墙规则


• 错误代码: 10060


  "Connection timed out"


  原因: 连接超时
  解决:
  - 检查网络连接
  - 测试服务器可达性
  - 增加超时时间


• 错误代码: 10054


  "Connection reset"
```

```
   原因：连接被重置
   解决：
   - 节点可能失效
   - 检查防火墙
   - 更换节点
```

18.5.4 浏览器错误

Chrome错误代码：

```
1. ERR_PROXY_CONNECTION_FAILED

   代理连接失败
   解决：
   - 检查代理设置
   - 确认代理运行
   - 验证端口正确


 • ERR_TUNNEL_CONNECTION_FAILED

   隧道连接失败

   解决：
   - 代理服务器问题
   - 检查HTTPS代理
   - 测试节点状态


 • ERR_NAME_NOT_RESOLVED

   DNS解析失败

   解决：
   - 检查DNS设置
   - 清除DNS缓存
   - 使用其他DNS


 • ERR_TIMED_OUT

   连接超时
```

```
    解决:
    - 检查网络连接
    - 测试节点延迟
    - 更换节点



清除Chrome缓存:
chrome://net-internals/#sockets
→ Flush socket pools
chrome://net-internals/#dns
→ Clear host cache
```

本章小结

本章提供了系统化的故障排查方法:

核心要点:

- **无法连接排查**:

- 系统化排查流程

- 从客户端到服务器 - 逐层验证定位问题

- **速度优化**:

- 启用BBR加速

- 调整MTU - 使用CDN中转 - 多路复用 - 优化线路

- **频繁断线**:

- TCP KeepAlive

- 超时配置 - 故障转移 - 自动重启

- **特定网站问题**:

- DNS污染处理

- 分流规则调整 - 流媒体解锁 - 地区限制绕过

- **错误代码**:

- Clash错误解析

- V2Ray错误处理 - 系统错误理解 - 浏览器错误修复

故障排查原则：

```
1. 由近及远

   客户端 → 网络 → 服务器

 • 由简到繁

   基础检查 → 高级诊断
 • 逐层排除

   配置 → 网络 → 服务
 • 保留证据

   日志 → 截图 → 配置
 • 系统记录

   问题 → 原因 → 解决方案
```

快速诊断清单：

```
☐ 节点是否选择

☐ 系统代理是否开启
☐ 配置是否正确
☐ 服务是否运行
☐ 端口是否监听
☐ 防火墙是否放行
☐ 时间是否同步
☐ DNS是否正常
☐ 日志有无错误
☐ 能否ping通服务器
```

下一章我们将学习网络测试工具的使用。

实践任务：

- 掌握系统化排查流程
- 学会使用测速工具

- 配置故障转移
- 理解常见错误代码
- 建立问题记录习惯

第十九章 网络测试工具

第19章 网络测试工具

掌握网络测试工具是诊断问题的关键。本章介绍常用的网络测试工具及其使用方法。

19.1 Ping与Traceroute

19.1.1 Ping基础

Ping的作用：

```
功能：
  • 测试网络连通性
  • 检测延迟（RTT）
  • 检测丢包率
  • 初步诊断网络问题

原理：
发送ICMP Echo Request包
等待ICMP Echo Reply响应
计算往返时间

限制：
[注意] 某些服务器禁ping
[注意] ICMP可能被限速
[注意] 不能测试端口
[注意] 防火墙可能拦截
```

Ping命令使用：

```
# Linux/macOS基础用法

ping google.com

# 发送指定数量的包


ping -c 10 google.com

# 设置包大小（字节）


ping -s 1000 google.com

# 设置间隔时间（秒）


ping -i 0.5 google.com

# 持续ping并记录


ping google.com > ping.log &

# Windows用法


ping google.com
```

```
# 发送10个包

ping -n 10 google.com

# 持续ping

ping -t google.com

# 设置TTL

ping -i 64 google.com
```

结果分析：

```
输出示例：

64 bytes from 142.250.185.46: icmp_seq=1 ttl=116 time=35.2 ms
64 bytes from 142.250.185.46: icmp_seq=2 ttl=116 time=34.8 ms
64 bytes from 142.250.185.46: icmp_seq=3 ttl=116 time=35.5 ms


--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 34.8/35.2/35.5/0.3 ms


参数解读：

  • icmp_seq：序列号
```

```
  • ttl：生存时间（跳数限制）

  • time：往返时间（延迟）

  • packet loss：丢包率

  • rtt min/avg/max：最小/平均/最大延迟




健康指标:
✓ 丢包率 < 1%
✓ 延迟稳定（mdev小）
✓ TTL正常（>50）



异常情况:
[禁止] Request timeout：无响应
[禁止] Destination host unreachable：目标不可达
[禁止] 高丢包率: 网络不稳定
[禁止] 延迟波动大: 网络拥堵
```

19.1.2 Traceroute路由追踪

Traceroute作用：

```
功能:

  • 显示数据包路径



  • 识别网络瓶颈

  • 定位故障节点

  • 分析路由策略
```

```
原理:
逐步增加TTL值
每个路由器返回ICMP超时
收集路径信息
```

基础使用:

```
# Linux

traceroute google.com

# 指定最大跳数


traceroute -m 20 google.com

# 使用ICMP(默认UDP)


traceroute -I google.com

# TCP方式(端口80)


traceroute -T -p 80 google.com

# 不解析域名


traceroute -n google.com
```

```
# Windows (tracert)

tracert google.com

# 不解析域名

tracert -d google.com

# 指定最大跳数

tracert -h 20 google.com

# macOS

traceroute google.com

# 或

traceroute -I google.com
```

结果分析：

```
输出示例：

traceroute to google.com (142.250.185.46), 30 hops max, 60 byte packets
 1  192.168.1.1 (192.168.1.1)  1.234 ms  1.156 ms  1.089 ms
```

```
 2  10.0.0.1 (10.0.0.1)  5.432 ms  5.389 ms  5.301 ms
 3  202.97.33.1 (202.97.33.1)  8.765 ms  8.721 ms  8.678 ms
 4  202.97.50.253 (202.97.50.253)  15.234 ms  15.198 ms  15.167 ms
 5   *
 6  108.170.242.65 (108.170.242.65)  180.456 ms  180.423 ms  180.391 ms
 7  142.250.185.46 (142.250.185.46)  181.234 ms  181.201 ms  181.178 ms



分析要点:

  • 第一跳: 本地网关

    192.168.1.1 → 家用路由器

    延迟应 < 5ms

  • 第二跳: ISP接入

    10.0.0.1 → 运营商接入点

    延迟应 < 10ms

  • 中间跳: 骨干网

     202.97.x.x → 中国电信骨干网

    延迟逐步增加

  •   * : 不响应

    - 防火墙阻止ICMP

    - 路由器配置不响应
    - 正常现象

  • 突然延迟增大:

    如从15ms跳到180ms
```

```
  → 国际出口/海底光缆

• 最后一跳：目标服务器

  延迟应与ping相近
```

典型路由分析：

```
国内访问国外：

1  本地网关          1ms
2  ISP接入           5ms
3  城域网           10ms
4  省骨干网         15ms
5  国家骨干网       25ms
6  国际出口        150ms  ← 延迟剧增
7  海外ISP           180ms
8  目标服务器      185ms



绕路示例（北京访问香港）：
应该：北京 → 广州 → 香港（~40ms）
实际：北京 → 上海 → 日本 → 香港（~120ms）
→ 路由次优，需要优化线路



丢包节点识别：
 5  202.97.50.1    50% loss  ← 此节点有问题
 6  202.97.60.1    50% loss
 7  8.8.8.8        0% loss   ← 实际是上游问题
→ 从第一个高丢包节点开始排查
```

19.2 MTR路由测试

19.2.1 MTR简介

MTR优势：

```
MTR = My Traceroute
```

```
= Ping + Traceroute 结合



优势:
✓ 实时持续监控
✓ 显示每跳丢包率
✓ 显示每跳延迟统计
✓ 更准确的诊断



vs Traceroute:


  • Traceroute:单次快照



  • MTR:持续监控,统计分析
```

安装MTR:

```
# Ubuntu/Debian

sudo apt install mtr -y

# CentOS/RHEL


sudo yum install mtr -y

# macOS


brew install mtr
```

```
# Windows

# 下载WinMTR

# https://sourceforge.net/projects/winmtr/
```

19.2.2 MTR使用

基础命令：

```
# 图形界面（推荐新手）

mtr google.com

# 报告模式（生成统计报告）

mtr --report google.com

# 指定测试次数

mtr --report-cycles 100 google.com

# 不解析域名（更快）

mtr -n google.com
```

```
# TCP模式（测试端口）

mtr --tcp --port 443 google.com

# 输出到文件

mtr --report --report-cycles 100 google.com > mtr_report.txt

# JSON格式输出

mtr --json google.com

# CSV格式输出

mtr --csv google.com
```

图形界面操作：

```
运行: mtr google.com

界面显示:
                                My traceroute  [v0.95]
HOST: localhost                  Loss%   Snt   Last   Avg  Best  Wrst StDev

  • 192.168.1.1               0.0%   100    1.2   1.3   1.0   2.1   0.2

  • 10.0.0.1                  0.0%   100    5.4   5.6   5.2   7.8   0.5
```

```
  • 202.97.33.1                0.0%   100   12.3  12.5  11.8  15.6   0.8

  • ???                      100.0%  100    0.0   0.0   0.0   0.0   0.0

  • 108.170.242.65             5.0%   100  185.3 186.2 180.1 195.4   3.2

  • 142.250.185.46             0.0%   100  186.5 187.1 182.3 196.8   3.5




字段说明:

  • Loss%：丢包率



  • Snt：已发送包数

  • Last：最后一次延迟

  • Avg：平均延迟

  • Best：最佳延迟

  • Wrst：最差延迟

  • StDev：标准差（波动）




操作键:
d：切换显示模式
n：切换域名/IP显示
p：暂停/继续
q：退出
r：重置统计
```

19.2.3 结果分析

健康网络示例：

```
HOST                          Loss%   Snt   Avg
  • 192.168.1.1               0.0%   100   1.2



  • 10.0.0.1                  0.0%   100   5.4

  • 202.97.33.1               0.0%   100  12.3

  • 142.250.185.46            0.0%   100  35.6





特征：
✓ 无丢包（0%）
✓ 延迟稳定（StDev小）
✓ 延迟递增合理
```

问题网络识别：

```
场景1：中间节点丢包

HOST                          Loss%   Snt   Avg

  • 192.168.1.1               0.0%   100   1.2



  • 10.0.0.1                 15.0%   100   5.4  ← 问题节点

  • 202.97.33.1              15.0%   100  12.3

  • 目标                     15.0%   100  35.6
```

```
诊断：ISP接入点有问题，联系运营商


场景2：仅中间节点丢包
HOST                              Loss%   Snt   Avg

  • 192.168.1.1                0.0%   100   1.2


  • 10.0.0.1                  20.0%   100   5.4

  • 202.97.33.1                0.0%   100  12.3  ← 后续正常

  • 目标                       0.0%   100  35.6




诊断：第2跳限速/限制ICMP，不影响实际使用


场景3：延迟突增
HOST                              Loss%   Snt   Avg

  • 192.168.1.1                0.0%   100    1.2


  • 10.0.0.1                   0.0%   100    5.4

  • 国际出口                   0.0%   100  180.5  ← 延迟剧增

  • 目标                       0.0%   100  185.3
```

```
诊断：国际出口，正常现象；如延迟过高考虑换线路



场景4：高抖动
HOST                          Loss%   Snt   Avg  StDev


  • 192.168.1.1               0.0%   100   1.2   0.1



  • WiFi路由                   0.0%   100  15.4  25.3  ← 高抖动

  • 目标                      0.0%   100  35.6   5.2




诊断：WiFi不稳定，建议使用有线连接
```

19.3 SpeedTest速度测试

19.3.1 命令行SpeedTest

安装：

```
# 官方CLI
# Ubuntu/Debian
curl -s https://packagecloud.io/install/repositories/ookla/speedtest-cli/script.deb.sh | sudo
sudo apt install speedtest

# CentOS/RHEL
```

```
curl -s https://packagecloud.io/install/repositories/ookla/speedtest-cli/script.rpm.sh | sudo
sudo yum install speedtest

# macOS


brew install speedtest-cli

# 或使用Python版本


pip install speedtest-cli
```

基础使用：

```
# 自动选择最佳服务器

speedtest

# 列出附近服务器


speedtest --servers

# 指定服务器测试


speedtest --server-id=服务器ID
```

```
# 仅测试下载

speedtest --download

# 仅测试上传

speedtest --upload

# JSON格式输出

speedtest --json

# 简洁模式

speedtest --simple

# 分享结果（生成链接）

speedtest --share
```

输出示例：

```
Speedtest by Ookla

Server: China Telecom - Beijing (id = 5083)
ISP: China Telecom
```

```
Latency:     8.34 ms   (2.15 ms jitter)
Download:   95.67 Mbps (data used: 112.4 MB)
Upload:     28.45 Mbps (data used: 42.1 MB)
Packet Loss:     0.0%
Result URL: https://www.speedtest.net/result/c/xxxxx



分析:


  • Latency < 50ms：良好



  • Jitter < 10ms：稳定


  • Packet Loss < 1%：健康


  • 下载速度: 接近带宽上限为佳
```

19.3.2 代理速度测试

通过代理测速：

```
# 方法1: 设置环境变量


export http_proxy=http://127.0.0.1:7890
export https_proxy=http://127.0.0.1:7890
speedtest

# 方法2: 使用proxychains


# 安装



sudo apt install proxychains4
```

```
# 配置 /etc/proxychains4.conf


[ProxyList]
socks5 127.0.0.1 7890

# 使用


proxychains4 speedtest

# 对比直连和代理速度


echo "直连速度: "
speedtest --simple


echo "代理速度: "
export http_proxy=http://127.0.0.1:7890
speedtest --simple
unset http_proxy
```

Fast.com测速：

```
# 安装fast-cli

npm install --global fast-cli

# 测速
```

```
fast

# 上传下载都测



fast --upload

# JSON输出



fast --json

# 单位设置



fast --units bits  # 或 bytes



优势:
✓ Netflix CDN
✓ 测试流媒体速度
✓ 简单易用
```

19.3.3 批量测速脚本

多节点对比脚本:

```
#!/bin/bash
# speed_compare.sh
```

```
echo "=== 网络速度对比测试 ==="
echo ""
```

直连测速

```
echo "【直连测速】"
speedtest --simple
echo ""
```

代理1测速

```
echo "【代理1测速】"
export http_proxy=http://127.0.0.1:7890
export https_proxy=http://127.0.0.1:7890
speedtest --simple
unset http_proxy https_proxy
echo ""
```

代理2测速

```
echo "【代理2测速】"
export http_proxy=http://127.0.0.1:7891
export https_proxy=http://127.0.0.1:7891
speedtest --simple
unset http_proxy https_proxy
echo ""


echo "测试完成！"
```

19.4 端口连通性检测

19.4.1 Telnet测试

基础用法：

```
# 安装telnet
# Ubuntu
sudo apt install telnet
# CentOS
sudo yum install telnet
# 测试端口
telnet 服务器IP 443
# 成功输出：
Trying 1.2.3.4...
Connected to 1.2.3.4.
Escape character is '^]'.
# 失败输出：
```

```
telnet: Unable to connect to remote host: Connection refused

# 或


telnet: Unable to connect to remote host: Connection timed out

# 退出telnet


Ctrl + ]
quit
```

批量端口测试：

```
#!/bin/bash
# check_ports.sh


HOST="your-server.com"
PORTS=(22 80 443 8443 10000)


for port in "${PORTS[@]}"; do
    echo -n "Testing port $port... "
    timeout 3 bash -c "echo >/dev/tcp/$HOST/$port" 2>/dev/null
    if [ $? -eq 0 ]; then
        echo "✓ Open"
    else
        echo "✗ Closed"
    fi
done
```

19.4.2 Netcat（nc）测试

功能更强大的测试：

```
# 安装

sudo apt install netcat

# 测试TCP端口


nc -zv 服务器IP 443

# 输出：


Connection to 服务器IP 443 port [tcp/https] succeeded!

# 测试UDP端口


nc -zvu 服务器IP 53

# 扫描端口范围


nc -zv 服务器IP 1-1000

# 设置超时（秒）


nc -zv -w 3 服务器IP 443
```

```
# 保持连接（交互模式）

nc 服务器IP 80
GET / HTTP/1.1
Host: example.com

# 端口监听（服务器端）

nc -l 12345

# 连接测试（客户端）

nc 服务器IP 12345
```

高级用法：

```
# 测试HTTP响应

echo -e "GET / HTTP/1.1\nHost: google.com\n\n" | nc google.com 80

# 测试HTTPS握手

echo | openssl s_client -connect google.com:443 2>/dev/null | grep "Verify return code"

# 文件传输测试
```

```
# 接收端

nc -l 12345 > received_file

# 发送端

nc 目标IP 12345 < file_to_send
```

19.4.3 Nmap扫描

安装和使用：

```
# 安装

sudo apt install nmap

# 扫描单个端口

nmap -p 443 服务器IP

# 扫描多个端口

nmap -p 22,80,443 服务器IP

# 扫描端口范围
```

```
nmap -p 1-1000 服务器IP
```

快速扫描常用端口

```
nmap --top-ports 100 服务器IP
```

TCP SYN扫描（需root）

```
sudo nmap -sS 服务器IP
```

UDP扫描

```
sudo nmap -sU -p 53 服务器IP
```

服务版本检测

```
nmap -sV 服务器IP
```

操作系统检测

```
sudo nmap -O 服务器IP
```

详细输出

```
nmap -v 服务器IP
```

输出解读：

```
Starting Nmap 7.80 ( https://nmap.org )

Nmap scan report for example.com (1.2.3.4)
Host is up (0.035s latency).



PORT    STATE  SERVICE  VERSION
22/tcp  open   ssh      OpenSSH 8.2p1
80/tcp  open   http     nginx 1.18.0
443/tcp open   ssl/http nginx 1.18.0



状态说明:

  • open：端口开放



  • closed：端口关闭

  • filtered：被防火墙过滤

  • unfiltered：可达但状态未知





[注意] 注意:
仅扫描自己的服务器
未授权扫描可能违法
```

19.5 TLS握手测试

19.5.1 OpenSSL测试

基础TLS测试：

```
# 连接测试

openssl s_client -connect google.com:443

# 指定SNI


openssl s_client -connect IP:443 -servername domain.com

# 显示证书


openssl s_client -connect google.com:443 -showcerts

# 测试特定TLS版本


openssl s_client -connect google.com:443 -tls1_2
openssl s_client -connect google.com:443 -tls1_3

# 测试特定加密套件


openssl s_client -connect google.com:443 -cipher 'ECDHE-RSA-AES128-GCM-SHA256'
```

```
# 简洁输出

echo | openssl s_client -connect google.com:443 2>/dev/null | grep -E "subject|issuer|Verify"
```

证书信息查看：

```
# 查看证书详情

echo | openssl s_client -connect google.com:443 2>/dev/null | openssl x509 -noout -text

# 证书有效期

echo | openssl s_client -connect google.com:443 2>/dev/null | openssl x509 -noout -dates

# 证书颁发者

echo | openssl s_client -connect google.com:443 2>/dev/null | openssl x509 -noout -issuer

# 证书主体

echo | openssl s_client -connect google.com:443 2>/dev/null | openssl x509 -noout -subject

# 证书SAN（Subject Alternative Name）

echo | openssl s_client -connect google.com:443 2>/dev/null | openssl x509 -noout -ext subject
```

19.5.2 SSL Labs测试

在线测试工具：

```
1. SSL Labs Server Test

   https://www.ssllabs.com/ssltest/



功能：
✓ 全面的SSL/TLS分析
✓ 证书验证
✓ 协议支持检测
✓ 加密套件检测
✓ 漏洞扫描
✓ 评分（A+到F）



使用：

  • 输入域名



  • 等待扫描（1-2分钟）

  • 查看详细报告





重点关注：

  • Overall Rating：总评分



  • Certificate：证书状态

  • Protocol Support：协议支持
```

```
  • Cipher Suites：加密套件

  • Vulnerabilities：已知漏洞
```

命令行测试：

```
# 使用testssl.sh
# 下载


git clone https://github.com/drwetter/testssl.sh.git
cd testssl.sh

# 测试


./testssl.sh google.com

# 测试特定端口


./testssl.sh google.com:443

# 快速测试


./testssl.sh --fast google.com
```

```
# 仅测试协议

./testssl.sh --protocols google.com

# 仅测试证书

./testssl.sh --certinfo google.com

# JSON输出

./testssl.sh --jsonfile result.json google.com
```

19.5.3 证书链验证

完整验证：

```
# 获取证书链

echo | openssl s_client -connect google.com:443 -showcerts 2>/dev/null > cert_chain.pem

# 提取服务器证书

openssl x509 -in cert_chain.pem -out server_cert.pem

# 验证证书链
```

```
openssl verify -CAfile /etc/ssl/certs/ca-certificates.crt server_cert.pem

# 输出：


server_cert.pem: OK  # 验证成功

# 检查证书是否过期


openssl x509 -in server_cert.pem -noout -checkend 86400

# 输出：


Certificate will not expire  # 未过期

# 证书指纹


openssl x509 -in server_cert.pem -noout -fingerprint -sha256
```

Let's Encrypt证书检查：

```
# 检查证书颁发者

echo | openssl s_client -connect your-domain.com:443 2>/dev/null | \
  openssl x509 -noout -issuer
```

应显示：

```
issuer=C = US, O = Let's Encrypt, CN = R3
```

检查有效期

```
echo | openssl s_client -connect your-domain.com:443 2>/dev/null | \
  openssl x509 -noout -dates
```

输出：

```
notBefore=Jan  1 00:00:00 2024 GMT
notAfter=Apr  1 00:00:00 2024 GMT
```

计算剩余天数

```
echo | openssl s_client -connect your-domain.com:443 2>/dev/null | \
  openssl x509 -noout -enddate | cut -d= -f2 | xargs -I {} date -d {} +%s | \
  awk -v now=$(date +%s) '{print int(($1-now)/86400)" days"}'
```

本章小结

本章介绍了常用的网络测试工具：

核心工具：

- **Ping**：

- 基础连通性测试

- 延迟和丢包检测 - 快速初步诊断

- **Traceroute**：

- 路由路径追踪

- 识别网络瓶颈 - 定位故障节点

- **MTR**：

- Ping + Traceroute结合

- 持续监控统计 - 更准确的诊断

- **SpeedTest**：

- 带宽速度测试

- 延迟抖动测试 - 代理性能对比

- **端口检测**：

- Telnet：简单测试

- Netcat：功能强大 - Nmap：全面扫描

- **TLS测试**：

- OpenSSL：命令行测试

- SSL Labs：在线分析 - testssl.sh：自动化测试

工具选择指南：

```
快速检查连通性：

→ ping



查看网络路径：
→ traceroute



深入路由分析：
→ mtr



测试速度：
→ speedtest / fast
```

```
检查端口:
→ nc / telnet



扫描服务:
→ nmap



验证TLS：
→ openssl / testssl.sh



全面SSL分析:
→ SSL Labs
```

诊断流程：

```
1. Ping测试

   ↓

 • 能ping通?

   是 → 端口测试（nc/telnet）

   否 → 路由追踪（traceroute/mtr）
   ↓

 • 端口通?

   是 → TLS测试（openssl）

   否 → 防火墙/端口问题
   ↓

 • TLS正常?

   是 → 速度测试（speedtest）
```

```
  否 → 证书/配置问题
```

实用技巧：

```
# 一键诊断脚本

#!/bin/bash
HOST="your-server.com"
PORT=443



echo "=== 网络诊断报告 ==="
echo "目标: $HOST:$PORT"
echo ""



echo "【Ping测试】"
ping -c 5 $HOST
echo ""



echo "【端口测试】"
nc -zv -w 3 $HOST $PORT
echo ""



echo "【TLS测试】"
echo | openssl s_client -connect $HOST:$PORT 2>/dev/null | grep "Verify return code"
echo ""



echo "【MTR测试】"
mtr --report --report-cycles 10 $HOST
```

至此，第六部分"故障排查篇"全部完成！

实践任务：

- 熟练使用ping和traceroute
- 掌握MTR分析方法
- 进行速度对比测试
- 检测服务器端口状态
- 验证TLS证书配置

第二十章 不同场景应用

第20章 不同场景应用

不同使用场景对科学上网有不同的需求。本章针对各种典型场景提供最佳实践方案。

20.1 日常浏览场景

20.1.1 需求分析

日常浏览特点：

```
使用场景：
  • 访问Google搜索
  • 查看YouTube视频
  • 浏览Twitter/Facebook
  • 使用Gmail/Drive
  • 阅读技术文档

需求特征：
✓ 稳定性优先
✓ 延迟要求适中（<200ms）
✓ 带宽需求一般
✓ 长时间连接
✓ 多设备使用

性能要求：
```

```
  • 延迟：50-200ms


  • 速度：10-50Mbps

  • 稳定性：>99%

  • 丢包率：<1%
```

20.1.2 推荐配置

节点选择：

```
地区选择：
  • 香港（推荐）

    延迟：10-50ms

    优势：距离近、速度快
    适合：日常浏览、YouTube

  • 新加坡

    延迟：30-80ms

    优势：稳定、线路好
    适合：全面使用

  • 日本

    延迟：40-100ms

    优势：带宽大、稳定
    适合：下载、视频
```

```
  • 美西（洛杉矶）

    延迟：120-180ms

    优势：内容丰富
    适合：访问美国服务



线路选择：

  • 预算有限：普通线路



  • 追求速度：CN2 GIA

  • 极致体验：IPLC专线
```

Clash分流规则：

```
# 日常浏览优化配置

mode: rule



dns:
  enable: true
  enhanced-mode: fake-ip
  nameserver:
    - 223.5.5.5
    - 119.29.29.29
  fallback:
     - https://1.1.1.1/dns-query
     - https://8.8.8.8/dns-query



proxy-groups:
  - name: [新] 节点选择
```

```
    type: select
    proxies:
     - ♻️ 自动选择
     - 🇭🇰 香港节点
     - 🇸🇬 新加坡节点
      - DIRECT



  - name: ♻️ 自动选择
    type: url-test
    url: http://www.gstatic.com/generate_204
    interval: 300
    proxies:
     - 香港-1
     - 香港-2
     - 新加坡-1



  - name: [视频] YouTube
    type: select
    proxies:
     - 🇭🇰 香港节点
     - 🇸🇬 新加坡节点
     - [新] 节点选择



  - name: 🎬 Netflix
    type: select
    proxies:
     - 🇸🇬 新加坡节点
     - 🇭🇰 香港节点



rules:
  # Google服务
  - DOMAIN-SUFFIX,google.com,[新] 节点选择
  - DOMAIN-SUFFIX,googleapis.com,[新] 节点选择
  - DOMAIN-SUFFIX,gstatic.com,[新] 节点选择



  # YouTube
  - DOMAIN-SUFFIX,youtube.com,[视频] YouTube
  - DOMAIN-SUFFIX,googlevideo.com,[视频] YouTube
  - DOMAIN-SUFFIX,ytimg.com,[视频] YouTube
```

```
# 社交媒体
- DOMAIN-SUFFIX,twitter.com,[新] 节点选择
- DOMAIN-SUFFIX,facebook.com,[新] 节点选择
- DOMAIN-SUFFIX,instagram.com,[新] 节点选择



# 开发工具
- DOMAIN-SUFFIX,github.com,[新] 节点选择
- DOMAIN-SUFFIX,stackoverflow.com,[新] 节点选择



# 国内直连
- GEOIP,CN,DIRECT
- MATCH,[新] 节点选择
```

20.1.3 浏览器优化

Chrome插件推荐：

```
1. SwitchyOmega（代理切换）

   功能：
   - 自动切换代理
   - 规则匹配
   - 情景模式



   配置：
   - 自动切换：GFWList规则
   - 代理服务器：127.0.0.1:7890
   - 直连情景：国内网站


 • uBlock Origin（广告拦截）


   优势：


   - 轻量高效
   - 隐私保护
   - 提升速度
```

```
  • Tampermonkey（脚本管理）

    用途：

    - 网页优化
    - 功能增强
    - 自动化操作

  • Proxy SwitchyOmega备份规则

    定期导出配置备份
```

Firefox优化：

```
关于：about:config

性能优化：
network.http.max-connections: 900
network.http.max-persistent-connections-per-server: 10



隐私保护：
privacy.resistFingerprinting: true
privacy.trackingprotection.enabled: true



DNS优化：
network.trr.mode: 2  # DoH模式
network.trr.uri: https://1.1.1.1/dns-query
```

20.1.4 多设备同步

方案设计：

```
方案1：订阅链接共享
```

优势：
✓ 简单易用
✓ 自动更新
✓ 多设备同步

实现：

- 获取订阅链接
- 各设备导入
- 定期更新

方案2：旁路由方案
优势：
✓ 全设备覆盖
✓ 无需配置
✓ 集中管理

实现：

- 软路由/树莓派
- 安装OpenWrt/PassWall
- 配置透明代理
- 设备网关指向旁路由

```
方案3：家庭共享
家长账号订阅 → 分享给家庭成员
注意流量限制和设备数量
```

20.2 流媒体解锁

20.2.1 流媒体特点

主流平台需求：

```
Netflix：

地区：美国、日本、新加坡等
要求：原生IP、低延迟
速度：25Mbps（4K需求）
检测：严格的代理检测



Disney+：
地区：美国、香港、台湾等
要求：原生IP
速度：15Mbps
检测：中等



HBO Max：
地区：美国
要求：美国原生IP
速度：10Mbps
检测：较严格



YouTube Premium：
地区：印度（价格低）
要求：稳定IP
速度：50Mbps（4K）
检测：较宽松
```

```
Spotify：
地区：多国可用
要求：稳定连接
检测：较宽松
```

20.2.2 解锁方案

节点要求：

```
必须条件：

✓ 原生IP（Native IP）
✓ 非数据中心IP
✓ 住宅IP或广播IP
✓ 地区匹配



检测方法：

  • 访问流媒体网站测试



  • 使用检测工具



    https://www.netflix.com/title/81280792
  • IP质量检测

    https://ipinfo.io/

   检查：type字段
    - hosting：数据中心IP（[禁止]）
    - isp：ISP IP（✓）



购买建议：

  • 选择标注"解锁Netflix"的节点
```

```
  • 咨询服务商支持情况

  • 优先选择住宅IP节点
```

Clash流媒体配置：

```
proxy-groups:

  # Netflix专用组
  - name: 🎬 Netflix
    type: select
    proxies:
      - 🇺🇸 美国原生
      - 🇸🇬 新加坡原生
      - 🇯🇵 日本原生
      - 🇭🇰 香港原生



  # Disney+专用组
  - name: 🎥 Disney+
    type: select
    proxies:
      - 🇺🇸 美国原生
      - 🇭🇰 香港原生
      - 🇸🇬 新加坡原生



  # YouTube Premium
  - name: [视频] YouTube
    type: select
    proxies:
      - 🇮🇳 印度节点  # Premium价格低
      - 🇦🇷 阿根廷节点
      - [新] 节点选择



rules:
  # Netflix
  - DOMAIN-SUFFIX,netflix.com,🎬 Netflix
```

```
  - DOMAIN-SUFFIX,nflxvideo.net,🎬 Netflix
  - DOMAIN-SUFFIX,nflximg.net,🎬 Netflix
  - DOMAIN-SUFFIX,nflxext.com,🎬 Netflix



  # Disney+
  - DOMAIN-SUFFIX,disney.com,🎥 Disney+
  - DOMAIN-SUFFIX,disneyplus.com,🎥 Disney+
  - DOMAIN-SUFFIX,dssott.com,🎥 Disney+



  # YouTube
  - DOMAIN-SUFFIX,youtube.com,[视频] YouTube
  - DOMAIN-SUFFIX,googlevideo.com,[视频] YouTube



  # Spotify
  - DOMAIN-SUFFIX,spotify.com,[新] 节点选择
  - DOMAIN-SUFFIX,scdn.co,[新] 节点选择
```

20.2.3 DNS解锁

原理：

```
某些流媒体通过DNS判断地区

修改DNS解析结果实现解锁



优势：
✓ 无需原生IP
✓ 成本低
✓ 配置简单



限制：
[注意] 仅对部分平台有效
[注意] 可能不稳定
[注意] 有法律风险
```

实现方法：

```
# Clash DNS解锁配置

dns:
  enable: true
  enhanced-mode: fake-ip



  # 特定域名使用解锁DNS
  nameserver-policy:
    # Netflix解锁DNS
    '+.netflix.com': [解锁DNS服务器]
    '+.nflxvideo.net': [解锁DNS服务器]



注意:
需要购买DNS解锁服务
或使用提供此功能的机场
```

20.2.4 观看优化

画质设置：

```
Netflix：
  • 账户设置 → 播放设置



  • 数据使用量: 高（最多）

  • 需要Premium套餐（4K）





速度需求:

  • 标清（SD）: 3 Mbps
```

```
  • 高清（HD）：5 Mbps

  • 超高清（4K）：25 Mbps




YouTube：
设置 → 画质：4K（需Premium）
速度需求：

  • 1080p：5 Mbps



  • 4K：20 Mbps




Disney+：
自动调整，需稳定15Mbps以上
```

缓冲优化：

```
问题：频繁缓冲

解决：

  • 测试节点速度

    确保速度满足要求
  • 关闭其他占用带宽的应用

    下载、上传等
  • 降低画质
```

```
  先稳定观看再提升
- 更换节点

  选择延迟低、速度快的
- 使用有线连接

  WiFi不稳定时
```

20.3 游戏加速优化

20.3.1 游戏场景分析

游戏类型需求：

```
FPS游戏（如PUBG、Valorant）：

延迟：<50ms（越低越好）
稳定性：极高
丢包率：<0.1%
带宽：5-10Mbps



MOBA游戏（如LOL、Dota2）：
延迟：<80ms
稳定性：高
丢包率：<0.5%
带宽：3-5Mbps



MMO游戏：
延迟：<150ms
稳定性：中高
带宽：5-15Mbps



主机游戏联机：
延迟：<100ms
稳定性：高
带宽：10-30Mbps
```

```
下载游戏：
延迟：不敏感
带宽：越大越好
稳定性：中
```

20.3.2 游戏加速方案

协议选择：

```
推荐协议：
  • IPLC/IEPL专线

    延迟：最低

    稳定性：最高
    成本：高

  • CN2 GIA

    延迟：低

    稳定性：高
    成本：中高

  • WireGuard

    延迟：低

    性能：优秀
    成本：中


不推荐：
[禁止] WebSocket：延迟高
[禁止] 多层代理：延迟累加
[禁止] CDN中转：不稳定
```

游戏专用配置：

```
# Clash游戏模式配置

proxy-groups:
  - name: [游戏] 游戏加速
    type: url-test  # 自动选择延迟最低
    url: http://www.gstatic.com/generate_204
    interval: 60  # 频繁测试
    tolerance: 20  # 延迟差20ms内不切换
    proxies:
     - 香港IPLC-1
     - 香港IPLC-2
     - 日本CN2-1



rules:
  # Steam
  - DOMAIN-SUFFIX,steampowered.com,[游戏] 游戏加速
  - DOMAIN-SUFFIX,steamcontent.com,[游戏] 游戏加速



  # Epic
  - DOMAIN-SUFFIX,epicgames.com,[游戏] 游戏加速



  # PlayStation Network
  - DOMAIN-SUFFIX,playstation.net,[游戏] 游戏加速
  - DOMAIN-SUFFIX,playstation.com,[游戏] 游戏加速



  # Xbox Live
  - DOMAIN-SUFFIX,xboxlive.com,[游戏] 游戏加速



  # 游戏IP段（需具体配置）
  - IP-CIDR,游戏服务器IP段,[游戏] 游戏加速
```

20.3.3 平台特定优化

Steam优化：

```
下载优化:

  • Steam → 设置 → 下载



  • 下载地区: 选择代理地区

  • 限制带宽: 不限制

  • 允许后台下载





社区访问:

  • 代理Steam社区



  • 使用Steamcommunity 302项目




Clash规则:
rules:
  # Steam下载CDN (国内直连更快)
  - DOMAIN-SUFFIX,steamcontent.com,DIRECT
  # Steam商店和社区 (代理)
  - DOMAIN-SUFFIX,steampowered.com,[游戏] 游戏加速
  - DOMAIN-SUFFIX,steamcommunity.com,[游戏] 游戏加速
```

主机游戏优化:

```
PlayStation:

  • 设置网络连接
```

```
  • 自定义设置

  • DNS设置: 手动


    主DNS：8.8.8.8

    辅DNS：8.8.4.4

  • MTU设置: 1473


  • 代理服务器: 使用


    地址: 路由器IP

    端口: 代理端口


Xbox：

  • 网络设置 → 高级设置


  • DNS设置: 手动


    主DNS：8.8.8.8

    辅DNS：1.1.1.1

  • 代理: 通过路由器透明代理



Switch：
```

```
• 设置 → 互联网 → 互联网设置



• 选择WiFi → 更改设置

• DNS设置：手动



  主DNS：8.8.8.8

  辅DNS：1.1.1.1

• 代理：通过路由器
```

20.3.4 延迟优化技巧

系统优化：

```
# Linux游戏优化
# 1. 禁用不必要的服务



systemctl disable bluetooth
systemctl disable cups

# 2. 网络优化


# /etc/sysctl.conf
```

```
net.ipv4.tcp_fastopen = 3
net.ipv4.tcp_low_latency = 1
net.ipv4.tcp_no_metrics_save = 1
```

应用

```
sysctl -p
```

3. 使用性能CPU调速器

```
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
```

Windows优化

1. 禁用Nagle算法（注册表）

```
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
TcpAckFrequency = 1
TCPNoDelay = 1
```

2. 网络适配器设置

```
禁用：节能模式
禁用：流控制
禁用：校验和卸载
```

3. 优先级设置

任务管理器 → 游戏进程 → 高优先级

20.4 远程办公配置

20.4.1 需求特点

办公场景：

```
常用服务：
  • Google Workspace（Gmail、Drive、Docs）
  • Microsoft 365（Outlook、OneDrive）
  • Zoom/Teams视频会议
  • Slack/Discord团队沟通
  • GitHub代码托管
  • VPN企业连接

需求：
✓ 高稳定性（不能断线）
✓ 低延迟视频会议（<150ms）
✓ 大带宽文件传输
✓ 安全性高
✓ 多设备支持
```

20.4.2 推荐方案

节点配置：

```
主节点（工作时间）：
  • 地区：香港/新加坡
  • 线路：CN2 GIA/IPLC
  • 特点：稳定性极高

备用节点：
  • 地区：日本/美西
  • 自动故障转移
  • 确保不断线

Clash配置：
proxy-groups:
  - name: [办公] 办公专用
    type: fallback  # 故障转移
    url: http://www.gstatic.com/generate_204
    interval: 60
    proxies:
     - 香港-主节点
     - 新加坡-备用1
     - 日本-备用2
```

```
rules:
  # Google Workspace
  - DOMAIN-SUFFIX,google.com,[办公] 办公专用
  - DOMAIN-SUFFIX,googleapis.com,[办公] 办公专用
  - DOMAIN-SUFFIX,gstatic.com,[办公] 办公专用



  # Microsoft 365
  - DOMAIN-SUFFIX,office.com,[办公] 办公专用
  - DOMAIN-SUFFIX,microsoft.com,[办公] 办公专用
  - DOMAIN-SUFFIX,microsoftonline.com,[办公] 办公专用



  # Zoom
  - DOMAIN-SUFFIX,zoom.us,[办公] 办公专用



  # Slack
  - DOMAIN-SUFFIX,slack.com,[办公] 办公专用



  # GitHub
  - DOMAIN-SUFFIX,github.com,[办公] 办公专用
  - DOMAIN-SUFFIX,githubusercontent.com,[办公] 办公专用
```

20.4.3 视频会议优化

Zoom优化：

```
网络要求：

• 1对1视频：600 kbps - 1.8 Mbps



• 群组视频：1-3 Mbps


• 屏幕共享：50-150 kbps


• 高清视频：2.5-3 Mbps
```

```
优化设置：

  • Zoom → 设置 → 视频

    - 启用HD视频

    - 针对视频剪辑优化（仅演示时）

  • 音频设置

    - 自动调整麦克风

    - 抑制背景噪音：高

  • 高级设置

    - 使用双显示器（如有）

    - 全屏显示参与者视频


网络建议：
✓ 使用有线连接
✓ 关闭其他占用带宽的应用
✓ 选择延迟低的节点
✓ 提前5分钟测试连接
```

Teams优化：

```
带宽需求：
  • 音频：50 kbps



  • 视频720p：1.2 Mbps
```

```
  • 视频1080p：2.5 Mbps

  • 屏幕共享: 1.5 Mbps




Teams设置:

  • 设置 → 设备

    - 测试音频/视频

    - 选择最佳设备

  • 通话设置

    - 关闭传入视频

    - 禁用动画和特效


代理配置:
Teams需要代理以下域名:

  • teams.microsoft.com



  • *.teams.microsoft.com

  • *.skype.com
```

20.4.4 企业VPN叠加

场景:

```
公司要求：连接企业VPN

同时需要：科学上网访问外网



解决方案1：分流配置
企业VPN → 公司内网
科学上网 → 外网服务



Clash配置：
rules:
  # 公司内网IP（直连，走企业VPN）
  - IP-CIDR,10.0.0.0/8,DIRECT
  - IP-CIDR,172.16.0.0/12,DIRECT
  - IP-CIDR,192.168.0.0/16,DIRECT



  # 公司域名（直连）
  - DOMAIN-SUFFIX,company.com,DIRECT



  # 外网服务（代理）
  - DOMAIN-SUFFIX,google.com,PROXY
  - MATCH,PROXY



解决方案2：嵌套代理
客户端 → 科学上网 → 企业VPN → 公司网络
设置代理链



解决方案3：虚拟机隔离
主机：科学上网
虚拟机：企业VPN
避免冲突
```

20.5 开发者需求

20.5.1 开发场景

典型需求：

```
访问服务：
  • GitHub/GitLab代码托管
  • StackOverflow技术问答
  • Google搜索技术资料
  • Docker Hub拉取镜像
  • NPM/PyPI包管理
  • AWS/GCP云服务控制台
  • Vercel/Netlify部署平台

特点：
✓ 大量API请求
✓ 频繁Git操作
✓ 下载依赖包
✓ 访问文档
✓ 稳定持久连接
```

20.5.2 命令行代理

Git代理配置：

```
# HTTP代理
```

```
git config --global http.proxy http://127.0.0.1:7890
git config --global https.proxy http://127.0.0.1:7890

# 仅GitHub代理

git config --global http.https://github.com.proxy http://127.0.0.1:7890

# SSH代理（~/.ssh/config）

Host github.com
    User git
    ProxyCommand nc -X 5 -x 127.0.0.1:7890 %h %p

# 或使用connect-proxy

ProxyCommand connect -S 127.0.0.1:7890 %h %p

# 取消代理

git config --global --unset http.proxy
git config --global --unset https.proxy
```

包管理器代理：

```
# NPM

npm config set proxy http://127.0.0.1:7890
npm config set https-proxy http://127.0.0.1:7890
```

或使用环境变量

```
export HTTP_PROXY=http://127.0.0.1:7890
export HTTPS_PROXY=http://127.0.0.1:7890
```

Yarn

```
yarn config set proxy http://127.0.0.1:7890
yarn config set https-proxy http://127.0.0.1:7890
```

pip

```
pip install package --proxy http://127.0.0.1:7890
```

或配置文件（~/.pip/pip.conf）

```
[global]
proxy = http://127.0.0.1:7890
```

Homebrew（macOS）

```
export ALL_PROXY=socks5://127.0.0.1:7890
brew install package
```

```
# Docker

# ~/.docker/config.json



{
  "proxies": {
    "default": {
      "httpProxy": "http://127.0.0.1:7890",
      "httpsProxy": "http://127.0.0.1:7890"
    }
  }
}
```

20.5.3 开发工具配置

VSCode代理：

```
// settings.json

{
  "http.proxy": "http://127.0.0.1:7890",
  "http.proxyStrictSSL": false,



  // Git集成
  "git.autofetch": true,



  // 扩展市场
  "extensions.autoUpdate": true
}
```

JetBrains IDE（IDEA/PyCharm）：

```
Settings → Appearance & Behavior → System Settings → HTTP Proxy
```

```
选择: Manual proxy configuration
HTTP:
  Host: 127.0.0.1
  Port: 7890



SOCKS:
  Host: 127.0.0.1
  Port: 7890



勾选: Check connection
测试URL：https://www.google.com
```

终端代理：

```
# Bash/Zsh（~/.bashrc 或 ~/.zshrc）

export http_proxy=http://127.0.0.1:7890
export https_proxy=http://127.0.0.1:7890
export all_proxy=socks5://127.0.0.1:7890

# 仅当前会话



export http_proxy=http://127.0.0.1:7890

# 临时使用（单条命令）



http_proxy=http://127.0.0.1:7890 curl https://google.com

# 函数封装（添加到~/.bashrc）
```

```
proxy_on() {
    export http_proxy=http://127.0.0.1:7890
    export https_proxy=http://127.0.0.1:7890
    export all_proxy=socks5://127.0.0.1:7890
    echo "代理已开启"
}



proxy_off() {
    unset http_proxy
    unset https_proxy
    unset all_proxy
    echo "代理已关闭"
}

# 使用



proxy_on   # 开启
proxy_off  # 关闭
```

20.5.4 CI/CD代理

GitHub Actions代理：

```
# .github/workflows/build.yml

name: Build



on: [push]



jobs:
  build:
    runs-on: ubuntu-latest
```

```
    steps:
    - uses: actions/checkout@v3



    - name: Setup proxy
      run: |
        export http_proxy=${{ secrets.PROXY_URL }}
        export https_proxy=${{ secrets.PROXY_URL }}



    - name: Install dependencies
      run: npm install
      env:
        http_proxy: ${{ secrets.PROXY_URL }}
        https_proxy: ${{ secrets.PROXY_URL }}
```

GitLab CI代理：

```
# .gitlab-ci.yml

variables:
  http_proxy: "http://proxy-server:port"
  https_proxy: "http://proxy-server:port"



before_script:
  - export http_proxy=$http_proxy
  - export https_proxy=$https_proxy



build:
  script:
    - npm install
    - npm run build
```

本章小结

本章针对不同场景提供了最佳实践方案：

核心要点：

- **日常浏览**：

- 稳定性优先

- 智能分流规则 - 多设备同步 - 浏览器优化

- **流媒体解锁**：

- 原生IP节点

- 专用分流配置 - DNS解锁方案 - 画质优化

- **游戏加速**：

- 低延迟协议

- 自动选择最优 - 平台特定优化 - 系统调优

- **远程办公**：

- 故障转移机制

- 视频会议优化 - 企业VPN叠加 - 高稳定性保障

- **开发者需求**：

- 命令行代理

- 开发工具配置 - 包管理器设置 - CI/CD集成

场景选择矩阵：

| 场景 | 延迟要求 | 带宽需求 | 稳定性 | 推荐线路 |

| --- | --- | --- | --- | --- |
流媒体	中	高	高	原生IP节点
游戏	极低	中	极高	IPLC/CN2
办公	低	中高	极高	专线/CN2
开发	中	高	高	稳定线路

配置原则：

```
1. 按需选择

   不同场景不同配置


 • 分流优化


   精细化规则匹配
 • 故障转移


   备用节点保障
 • 性能优先


   选择最优线路
 • 成本平衡


   性能与价格权衡
```

下一章我们将学习成本优化策略。

实践任务：

- 配置适合自己的分流规则
- 测试流媒体解锁效果
- 优化游戏延迟
- 配置开发工具代理
- 建立备用节点机制

第二十一章 成本优化策略

第21章 成本优化策略

科学上网需要持续投入。本章介绍如何在保证质量的前提下优化成本。

21.1 机场服务对比

21.1.1 机场分类

按价格分类：

```
入门级（月付10-30元）：

特点：

  • 基础线路

  • 流量限制较多

  • 节点数量少

  • 稳定性一般

适合：偶尔使用、预算有限

中端级（月付30-80元）：
特点：

  • CN2/BGP线路

  • 流量充足（100GB-500GB）
```

- 节点较多
- 稳定性较好

适合：日常使用、轻度办公

高端级（月付80-200元）：
特点：

- IPLC/IEPL专线
- 大流量/不限流
- 节点众多
- 稳定性极高
- 流媒体解锁

适合：重度使用、专业需求

顶级（月付200元以上）：
特点：

- 全专线
- 不限流量
- 全球节点
- 7×24客服
- 企业级稳定性

```
适合：商业用途、团队使用
```

21.1.2 机场评估标准

关键指标：

```
1. 线路质量

    ✓ CN2 GIA优于CN2 GT
    ✓ IPLC/IEPL最优
   ✓ 普通线路次之



   测试方法：
   - 查看线路说明
   - 测试延迟和速度
   - 高峰期测试


 • 流量配额


   评估自身需求：


   - 日常浏览：50-100GB/月
   - 视频观看：200-500GB/月
   - 下载需求：500GB+/月



   计算方法：
    1080p视频约3GB/小时
    4K视频约7GB/小时


 • 节点数量与分布


   ✓ 香港/台湾/日本/新加坡


   ✓ 美国西海岸/东海岸
   ✓ 欧洲主要国家
```

```
  避免:
  [禁止] 仅单一地区
  [禁止] 冷门地区为主


• 稳定性


  考察:

  - 运营时长（>1年）
  - 用户评价
  - 售后服务
  - 公告更新频率



  警告信号:
  [禁止] 频繁跑路传闻
  [禁止] 长期无更新
  [禁止] 客服失联


• 附加功能


  加分项:

  ✓ 流媒体解锁
  ✓ 游戏加速
   ✓ ChatGPT支持
  ✓ 多设备支持
  ✓ 客户端支持完善


• 性价比


  计算公式:

  性价比 = (流量GB × 线路质量系数) / 价格



  线路质量系数:
  - 普通线路: 1.0
   - CN2 GT：1.5
   - CN2 GIA：2.0
   - IPLC：3.0
```

21.1.3 主流机场推荐

选择建议：

```
入门推荐：
  • 稳定但基础的服务
  • 适合尝试和轻度使用
  • 月付15-25元
  • 100GB流量
  • 基础CN2线路

特点：
✓ 价格实惠
✓ 门槛低
✓ 适合新手
✗ 高峰期可能慢
✗ 节点较少

中端推荐：
  • 平衡性价比
  • 适合日常重度使用
  • 月付50-80元
  • 300-500GB流量
```

```
  • CN2 GIA/部分IPLC




特点:
✓ 性能稳定
✓ 流量充足
✓ 节点丰富
✓ 流媒体解锁



高端推荐:


  • 极致体验



  • 适合专业需求


  • 月付150-300元


  • 不限流量


  • 全IPLC专线




特点:
✓ 延迟极低
✓ 速度极快
✓ 稳定性极高
✓ 全功能支持
✗ 价格较高
```

21.1.4 避坑指南

常见陷阱:

```
1. 超低价陷阱

   月付5-10元，宣称不限速



   真相:
   - 超售严重
   - 高峰期不可用
   - 随时跑路风险



   识别:
   价格远低于成本（IPLC成本至少50元/月）


 • 虚假宣传


   宣传:

   "100%解锁Netflix"
    "1Gbps带宽"
   "永久免费"



   现实:
   - 部分节点解锁
   - 实际速度远低
   - 免费节点不稳定



   验证:
   试用后再购买


 • 跑路风险


   特征:

   - 新机场大力促销
   - 年付优惠力度大
   - 无退款政策
```

```
  防范：
  - 月付为主
  - 避免长期套餐
  - 分散购买


• 过度超售


  表现：


  - 晚高峰速度骤降
  - 频繁断线
  - 节点经常满载



  识别：
  - 试用高峰期
  - 查看用户评价
  - 对比不同时段


• 隐私风险


  问题机场：


  - 记录用户访问
  - 出售用户数据
  - 被执法部门控制



  防范：
  - 选择知名机场
  - 避免使用真实信息
  - 加密敏感通信
```

21.2 自建vs购买分析

21.2.1 成本对比

自建成本：

VPS费用:

- 入门VPS：$5-10/月

- 中端VPS：$10-20/月

- 高端VPS：$20-50/月

- CN2 GIA：$30-100/月

时间成本:

- 初次搭建: 2-4小时

- 学习成本: 5-10小时

- 维护更新: 1-2小时/月

- 故障处理: 不定

总成本:
假设VPS $10/月
一年成本 = $120 + 时间成本

购买机场:

- 中端机场: 50-80元/月

```
  • 年付：600-960元

  • 无需维护

  • 多节点选择




一年成本 = 600-960元
```

成本分析：

```
自建优势：

✓ 独享带宽
✓ 完全掌控
✓ 隐私性高
✓ 学习技术



自建劣势：
✗ 单点故障
✗ 仅一个地区
✗ 需要技术
✗ 时间投入
✗ IP可能被封



购买优势：
✓ 多节点
✓ 故障转移
✓ 专业维护
✓ 开箱即用
✓ 客服支持



购买劣势：
✗ 共享带宽
```

```
✗ 隐私风险
✗ 跑路风险
✗ 价格较高
```

21.2.2 适用场景

推荐自建：

```
适合人群：

✓ 有技术基础
✓ 时间充足
✓ 追求隐私
✓ 流量需求大
✓ 预算有限但不怕麻烦


场景：

  • 开发者学习


  • 个人长期使用

  • 家庭共享

  • 特殊需求




注意：
需要备用方案
防止服务器故障
```

推荐购买：

```
适合人群：
```

```
✓ 无技术基础
✓ 时间宝贵
✓ 需要稳定性
✓ 多地区需求
✓ 不想折腾



场景：

  • 日常使用



  • 商业用途

  • 追求便利

  • 多设备多人




建议：
选择知名稳定机场
避免过度依赖单一服务
```

21.2.3 混合方案

最优组合：

```
方案1：自建+机场

自建：

  • 主力节点



  • 日常使用
```

- 流量密集任务

机场：

- 备用节点
- 多地区需求
- 流媒体解锁

成本：
自建VPS $10/月 + 入门机场 20元/月
总计：约100元/月

优势：
✓ 成本可控
✓ 双重保障
✓ 功能互补

方案2：多机场组合
主力机场（中端）：

- 日常使用
- 50元/月

```
备用机场（入门）：

  • 应急使用

  • 20元/月

总计：70元/月

优势：
✓ 互为备份
✓ 避免单点故障
✓ 灵活切换

方案3：按需自建
平时：使用机场
特殊需求：临时搭建VPS

  • 大量下载

  • 特定地区

  • 隐私要求高

成本：
月费50元 + 偶尔VPS费用
```

21.3 流量节省技巧

21.3.1 智能分流

精细化规则：

```
# Clash优化配置

rules:
  # 国内网站直连
  - DOMAIN-SUFFIX,baidu.com,DIRECT
  - DOMAIN-SUFFIX,taobao.com,DIRECT
  - DOMAIN-SUFFIX,qq.com,DIRECT
  - DOMAIN-SUFFIX,163.com,DIRECT
  - DOMAIN-SUFFIX,sina.com.cn,DIRECT


  # 国内CDN直连
  - DOMAIN-SUFFIX,cdn.jsdelivr.net,DIRECT
  - DOMAIN-SUFFIX,unpkg.com,DIRECT


  # 大文件下载直连（如果国内有镜像）
  - DOMAIN-SUFFIX,mirrors.tuna.tsinghua.edu.cn,DIRECT
  - DOMAIN-SUFFIX,mirrors.aliyun.com,DIRECT


  # 需要代理的才走代理
  - DOMAIN-SUFFIX,google.com,PROXY
  - DOMAIN-SUFFIX,youtube.com,PROXY
  - DOMAIN-SUFFIX,github.com,PROXY


  # GeoIP CN直连
  - GEOIP,CN,DIRECT


  # 其他走代理
  - MATCH,PROXY
```

```
效果:
节省50%以上流量
避免国内访问走代理
```

21.3.2 选择性代理

浏览器分离:

```
Chrome:
  • 用于国内网站



  • 不启用代理

  • 访问淘宝、B站等




Firefox/Edge:

  • 用于国外网站



  • 启用代理

  • 访问Google、YouTube




好处:
✓ 流量精确控制
✓ 避免误用
✓ 提高速度
```

应用分流：

```
需要代理：
  • Chrome（外网浏览器）
  • Telegram
  • Twitter客户端
  • Steam社区

不需要代理：
  • 微信
  • QQ
  • 国内音乐/视频APP
  • 国内游戏

Clash配置：
  • 程序规则模式
  • 指定应用走代理
  • 其他应用直连
```

21.3.3 流量监控

监控工具：

```
Clash：
  • Dashboard查看
  • 实时流量统计
  • 按连接查看

GlassWire（Windows）：
  • 详细流量分析
  • 按应用统计
  • 流量预警

NetWorx：
  • 跨平台
  • 流量报告
```

```
  • 配额管理




命令行监控:

# Linux



vnstat -l  # 实时
vnstat -d  # 每日
vnstat -m  # 每月

# macOS



nettop -m route
```

流量统计分析:

```
#!/bin/bash
# traffic_analysis.sh

# 获取Clash流量统计

curl -s http://127.0.0.1:9090/traffic | jq

# 计算今日使用
```

```
TODAY_UP=$(vnstat --oneline | cut -d';' -f4)
TODAY_DOWN=$(vnstat --oneline | cut -d';' -f5)



echo "今日上传: $TODAY_UP"
echo "今日下载: $TODAY_DOWN"

# 预警



LIMIT_GB=100
CURRENT_GB=$(vnstat --oneline | cut -d';' -f11)



if [ $CURRENT_GB -gt $LIMIT_GB ]; then
    echo "警告: 月流量已超过${LIMIT_GB}GB"
fi
```

21.3.4 减少浪费

常见浪费:

```
1. 自动更新走代理

   Windows更新
   App Store更新
  软件自动更新



  解决:
  - 设置更新时段
  - 手动更新
  - 更新时关闭代理


 • 云同步走代理


   OneDrive同步
```

```
  iCloud同步
  Dropbox同步



 解决:
 - 暂停自动同步
 - 选择性同步
 - 同步时关闭代理


• 后台应用


 自动下载

 推送通知
 数据同步



 解决:
 - 关闭不必要的后台
 - 限制应用联网
 - 定期检查流量


• 视频自动播放


  YouTube自动播放


  Twitter视频
  Facebook视频



 解决:
 - 禁用自动播放
 - 降低视频质量
  - WiFi环境观看
```

21.4 共享方案设计

21.4.1 家庭共享

方案1：订阅分享

适用场景：

- 2-5人家庭
- 同一网络
- 信任环境

实现方法：

- 购买多设备套餐
- 分享订阅链接
- 各设备独立导入

优势：
✓ 成本分摊
✓ 独立使用
✓ 灵活管理

成本：
多设备套餐：80元/月
家庭4人分摊：20元/人

```
注意：
[注意] 遵守机场设备限制
[注意] 避免同时在线过多
```

方案2：路由器共享

```
设备：
  • 软路由（推荐）
  • 刷OpenWrt的路由器
  • 树莓派

优势：
✓ 全设备覆盖
✓ 无需单独配置
✓ 集中管理
✓ 对家人透明

配置：
  • 安装PassWall/SSR Plus
  • 导入订阅
  • 配置分流规则
  • 设置为主路由或旁路由
```

```
成本：
软路由硬件：300-1000元（一次性）
订阅：50-80元/月



适合：


  • 技术能力较强



  • 家庭成员多


  • 长期使用
```

方案3：中继代理

```
主设备：
  • 运行Clash/V2Ray



  • 开启允许局域网连接




其他设备：


  • 代理地址：主设备IP



  • 代理端口：7890
```

```
Clash配置:
allow-lan: true
bind-address: '*'
port: 7890



注意:
[注意] 主设备需常开
[注意] 同一局域网
[注意] 流量经过主设备
```

21.4.2 朋友合租

合租机场:

```
模式:
  • 3-5人合租



  • 费用AA

  • 共享账号/订阅





套餐选择:


  • 不限设备



  • 大流量（500GB-1TB）

  • 价格100-200元/月

  • 人均20-50元/月
```

管理方式：

- 指定管理员

  负责续费和维护

- 费用分摊

  月初收取

  或年付分摊

- 使用规则

  避免滥用

  流量超标共同承担

风险：
[注意] 成员退出
[注意] 费用纠纷
[注意] 使用冲突
[注意] 隐私问题

自建共享：

方案：

- 合租VPS

- 共同搭建

- 费用分摊

```
成本:
CN2 GIA VPS：$50/月
5人分摊: $10/人（约70元）



配置:

  • 使用多用户面板（X-UI）



  • 每人独立账号

  • 流量单独统计

  • 可设置限速




优势:
✓ 成本低
✓ 独立账号
✓ 可控性强



劣势:
✗ 需要技术
✗ 单点故障
✗ 维护责任
```

21.4.3 商业共享

小规模销售:

```
[注意] 法律风险警告:
```

未经许可提供VPN服务违法
本节仅供技术学习

如果合规运营:

- 申请相关资质
- 使用正规服务器
- 遵守法律法规
- 不涉及违法内容

技术方案:

- X-UI多用户管理
- 流量统计
- 到期管理
- 自动化运维

定价策略:

- 根据成本和市场

```
  • 保持竞争力

  • 提供试用




风险:
[注意] 法律风险极高
[注意] 技术门槛高
[注意] 投入大
[注意] 强烈不建议
```

21.4.4 共享注意事项

安全隔离:

```
1. 账号隔离

    - 独立UUID/密码
    - 避免共用账号
    - 定期更换

  • 流量隔离

    - 独立统计

    - 限制配额
    - 避免超用

  • 隐私保护

    - 不记录日志

    - 加密传输
    - 最小化信息收集

  • 责任划分
```

```
  - 明确使用规则

  - 禁止违法用途
  - 各自负责
```

使用约定：

```
建议制定规则：

• 流量使用

  - 避免大量下载BT

  - 避免滥用
  - 超额自费

• 稳定性

  - 不频繁切换

  - 不暴力测试
  - 不攻击行为

• 隐私尊重

  - 不窥探他人

  - 不共享配置给外人
  - 保护共同利益

• 费用管理

  - 按时缴费

  - 透明账单
  - 合理分摊
```

本章小结

本章介绍了成本优化的各种策略：

核心要点：

- **机场服务对比**：

- 按需选择等级

- 评估关键指标 - 避免常见陷阱 - 性价比优先

- **自建vs购买**：

- 成本全面对比

- 适用场景分析 - 混合方案设计 - 风险收益权衡

- **流量节省**：

- 智能分流规则

- 选择性代理 - 流量监控 - 减少浪费

- **共享方案**：

- 家庭共享

- 朋友合租 - 共享注意事项 - 风险防范

成本优化策略：

```
省钱方案（月50元以内）：
  • 入门机场 + 精细分流
  • 自建VPS（普通线路）
  • 小规模合租

平衡方案（月50-100元）：
  • 中端机场
```

```
  • 自建CN2 + 备用机场
  • 家庭共享分摊




高端方案（月100-200元）：
  • 高端机场


  • 多机场组合
  • 自建专线 + 流媒体机场




极致方案（月200元以上）：
  • 顶级机场


  • 全IPLC专线
  • 多地自建 + 机场备份
```

选择原则：

```
1. 明确需求
```

```
    - 使用频率
    - 流量需求
    - 稳定性要求
    - 特殊功能


  • 量力而行


    - 根据预算


    - 避免过度消费
    - 注重性价比


  • 安全第一


    - 避免太便宜的


    - 选择稳定服务
    - 保护隐私


  • 灵活调整


    - 定期评估


    - 优化方案
    - 及时止损
```

避免误区：

```
[禁止] 过度追求低价

    质量和稳定性更重要



[禁止] 盲目囤货
    年付有跑路风险
```

```
[禁止] 忽视隐私
    选择可信服务商



[禁止] 违法共享
    商业销售有法律风险
```

下一章我们将学习合规使用建议。

实践任务：

- 评估当前成本
- 对比不同方案
- 优化分流规则
- 监控流量使用
- 制定长期策略

第二十二章 合规使用建议

第22章 合规使用建议

科学上网需要在法律框架内合理使用。本章介绍相关法律法规和使用边界。

22.1 法律法规认知

22.1.1 相关法律条文

主要法律依据：

```
《中华人民共和国计算机信息网络国际联网管理暂行规定》

第六条：
计算机信息网络直接进行国际联网，必须使用邮电部国家
公用电信网提供的国际出入口信道。
任何单位和个人不得自行建立或者使用其他信道进行国际联网。



《计算机信息网络国际联网安全保护管理办法》



第五条：
任何单位和个人不得利用国际联网制作、复制、查阅和
传播下列信息：
（一）煽动抗拒、破坏宪法和法律、行政法规实施的；
（二）煽动颠覆国家政权，推翻社会主义制度的；
（三）煽动分裂国家、破坏国家统一的；
（四）煽动民族仇恨、民族歧视，破坏民族团结的；
（五）捏造或者歪曲事实，散布谣言，扰乱社会秩序的；
（六）宣扬封建迷信、淫秽、色情、赌博、暴力、凶杀、
      恐怖，教唆犯罪的；
（七）公然侮辱他人或者捏造事实诽谤他人的；
（八）损害国家机关信誉的；
（九）其他违反宪法和法律、行政法规的。



《刑法》相关条款：
```

```
第285条：非法侵入计算机信息系统罪
第286条：破坏计算机信息系统罪
第287条：利用计算机实施犯罪



关键点理解：
[注意] "翻墙"本身在法律上属于灰色地带
[注意] 使用翻墙访问违法内容是明确违法的
[注意] 提供翻墙服务（VPN销售）未经许可违法
[注意] 个人学习和工作使用一般不被追究
```

22.1.2 执法实践

现实情况：

```
执法重点：

  • 严格打击


    ✗ 提供VPN服务（销售、架设）


    ✗ 传播违法信息
    ✗ 从事违法活动
    ✗ 大规模组织翻墙


  • 一般不追究


    ✓ 个人学习使用


    ✓ 查阅技术资料
    ✓ 正常工作需求
    ✓ 学术研究


  • 处罚案例


    - 销售VPN：行政处罚或刑事处罚


    - 搭建VPN服务器：行政处罚
    - 传播违法信息：依法追究
    - 单纯个人使用：极少追究
```

处罚形式：

- 警告

- 罚款（一般5000-15000元）

- 没收违法所得

- 刑事责任（严重情况）

注意：
[注意] 法律规定和执法实践存在差异
[注意] 不同地区执法尺度不同
[注意] 趋势可能随时变化
[注意] 不应存在侥幸心理

22.1.3 合法替代方案

正规渠道：

1. 教育科研网

   适用：高校师生、科研机构
   方式：教育网国际出口
   特点：
   ✓ 合法合规
   ✓ 学术资源丰富
   ✓ 速度较快
   限制：仅限学术用途

- 企业专线

  适用：企业、外资机构

```
  方式：申请MPLS VPN等
  特点：
  ✓ 完全合法
  ✓ 稳定可靠
  ✓ 技术支持
  限制：
  - 成本高昂
  - 需要企业资质
  - 仅限业务使用


• 官方国际业务


  运营商：中国电信、联通、移动


  产品：国际业务套餐
  特点：
  ✓ 合法合规
  ✓ 正规服务
  限制：
  - 可能较慢
  - 价格不菲
  - 功能受限


• 出国访问


  方式：出国时正常访问国际互联网


  特点：
  ✓ 完全合法
  ✓ 无限制
  限制：需要出国
```

22.2 使用场景边界

22.2.1 合理使用场景

正当用途：

```
✓ 学习和研究
```

```
  - 访问Google Scholar学术搜索
  - 查阅arXiv论文
  - 浏览国外大学课程资源
  - 学习最新技术文档
  - 观看教育视频



✓ 工作需要
  - 访问GitHub等开发平台
  - 使用Google Workspace办公
  - 查询技术资料
  - 客户沟通（国外客户）
  - 市场调研



✓ 正常娱乐
  - 观看YouTube视频
  - 使用Instagram等社交平台
  - 玩国外游戏
  - 听Spotify音乐
  - 看Netflix影视



✓ 信息获取
  - 浏览国外新闻
  - 了解不同观点
  - 学习外语
  - 文化交流



原则:
合法、正当、合理
不涉及违法内容
不从事违法活动
```

22.2.2 禁止使用场景

严禁行为:

```
✗ 政治敏感
```

```
  - 发布、传播危害国家安全的内容
  - 参与、组织非法政治活动
  - 煽动分裂、颠覆等行为
  - 散布谣言、虚假信息



✗ 违法犯罪
  - 传播淫秽色情内容
  - 从事网络赌博
  - 销售违禁品
  - 进行网络诈骗
  - 侵犯知识产权
  - 进行黑客攻击



✗ 侵权行为
  - 侵犯他人隐私
  - 网络暴力
  - 诽谤他人
  - 盗版侵权



✗ 商业违规
  - 未经许可提供VPN服务
  - 销售翻墙工具
  - 组织大规模翻墙
  - 从事非法交易



后果：
轻则：警告、罚款
重则：刑事责任、判刑
```

22.2.3 灰色地带

需要注意的场景：

```
谨慎对待：

• 社交媒体发言


  - 避免敏感话题
```

  - 注意言论尺度
  - 理性表达观点
  - 尊重法律法规

- 信息分享

  - 不转发未经证实的信息

  - 不传播违法内容
  - 注意信息来源
  - 甄别真假

- 群组交流

  - 谨慎加入敏感群组

  - 不参与违法讨论
  - 保持理性冷静
  - 及时退出可疑群组

- 下载内容

  - 避免下载盗版

  - 尊重版权
  - 注意内容合法性
  - 使用正版服务

建议：
遵守"三不"原则：

- 不信谣、不传谣

- 不参与违法活动

- 不触碰法律红线

22.3 风险防范意识

22.3.1 个人信息保护

隐私保护原则：

```
1. 最小化原则

   - 不提供真实身份信息
   - 使用化名/昵称
   - 避免暴露个人详情
   - 谨慎上传照片

 • 分隔原则

   - 国内外账号分离

   - 不同平台不同身份
   - 不关联真实身份
   - 独立密码管理

 • 匿名原则

   - 使用临时邮箱

   - 虚拟手机号
   - 匿名支付
    - VPN叠加Tor

 • 加密原则

   - 敏感通信加密

   - 使用端到端加密
   - 文件加密存储
   - 安全删除数据
```

实践：

- 不在社交媒体暴露真实身份
- 不使用真实照片做头像
- 不关联国内社交账号
- 定期更换身份信息

22.3.2 安全意识

防范措施：

技术安全：

- 使用加密协议
  - ✓ TLS/HTTPS
  - ✓ 现代加密算法
  - ✓ 安全的代理协议
- 防止泄露
  - ✓ DNS泄露防护
  - ✓ WebRTC禁用
  - ✓ IPv6泄露处理
  - ✓ 时区伪装
- 设备安全
  - ✓ 系统更新

```
  ✓ 安全软件
  ✓ 防火墙启用
  ✓ 全盘加密


行为安全:


  • 谨慎发言


    - 避免敏感话题


    - 不发表极端言论
    - 理性讨论
    - 尊重他人


  • 保护隐私


    - 不暴露位置


    - 不上传敏感信息
    - 注意照片EXIF
    - 谨慎社交


  • 甄别信息


    - 核实信息来源


    - 不轻信传言
    - 理性判断
    - 独立思考


  • 及时止损


    - 发现风险立即停止


    - 删除敏感内容
    - 更换账号
    - 必要时寻求帮助
```

22.3.3 服务商选择

可信度评估：

```
选择标准：

  • 运营历史

    ✓ 运营时间 > 1年

    ✓ 用户口碑良好
    ✓ 无重大安全事故
    ✓ 持续更新维护

  • 隐私政策

    ✓ 明确的隐私政策

    ✓ 不记录日志（No-log）
    ✓ 匿名注册支持
    ✓ 加密货币支付

  • 技术实力

    ✓ 使用现代协议

    ✓ 安全配置
    ✓ 及时更新
    ✓ 专业团队

  • 法律管辖

    ✓ 避免中国大陆注册

    ✓ 优先隐私友好国家
    ✓ 了解服务商背景
    ✓ 避免可疑来源


警告信号：
[禁止] 要求实名认证
[禁止] 过度收集信息
```

```
[禁止] 来源不明
[禁止] 价格异常低
[禁止] 强制推广
[禁止] 跑路传闻
```

22.3.4 应急预案

风险应对：

```
场景1：服务被封

应对：

  • 提前准备备用方案

  • 多个服务商分散风险

  • 保存配置备份

  • 快速切换

场景2：账号异常
应对：

  • 立即停止使用

  • 更换账号

  • 检查是否泄露

  • 必要时报告
```

场景3：收到警告
应对：

- 立即停止违规行为
- 删除相关内容
- 寻求法律咨询
- 配合调查

场景4：设备被查
建议：

- 不存储敏感信息
- 使用全盘加密
- 定期清理记录
- 了解自己的权利

预防措施：
✓ 建立应急联系方式
✓ 定期备份重要数据
✓ 分散风险
✓ 保持低调
✓ 遵守法律

22.4 责任与义务

22.4.1 个人责任

自我约束：

```
基本原则：
  • 守法底线
    - 遵守中国法律
    - 尊重当地法规
    - 不触碰红线
    - 理性使用
  • 道德约束
    - 不传播谣言
    - 不网络暴力
    - 不侵犯他人
    - 尊重版权
  • 理性思考
    - 独立判断
    - 批判性思维
    - 多方验证
    - 避免极端
  • 自我保护
    - 保护隐私
    - 防范风险
    - 谨言慎行
    - 及时止损
```

使用承诺：
我承诺：
☐ 仅用于合法用途
☐ 不从事违法活动
☐ 不传播违法内容
☐ 遵守法律法规
☐ 自负法律责任

22.4.2 社会责任

正面影响：

应该做到：

- 促进交流

  ✓ 跨文化理解

  ✓ 学习先进知识
  ✓ 开阔视野
  ✓ 文明对话

- 传播正能量

  ✓ 分享优质内容

  ✓ 理性表达观点
  ✓ 尊重不同意见
  ✓ 维护形象

- 提升自我

  ✓ 学习新知识

  ✓ 提升技能
  ✓ 个人成长
  ✓ 专业发展

- 守护底线

```
    ✓ 不被煽动


    ✓ 理性爱国
    ✓ 明辨是非
    ✓ 正面发声



避免：
[禁止] 被利用传播不实信息
[禁止] 参与对立情绪
[禁止] 损害国家形象
[禁止] 盲目崇外
```

22.4.3 技术伦理

开发者责任：

```
如果你是开发者：

  • 技术中立


    - 技术本身无罪


    - 但使用需负责
    - 考虑社会影响
    - 引导正确使用


  • 安全优先


    - 保护用户隐私


    - 不记录敏感信息
    - 使用安全技术
    - 及时修复漏洞


  • 拒绝作恶


    - 不用于违法目的
```

```
    - 不侵犯他人
    - 不损害社会
    - 符合伦理道德


  • 教育引导


    - 提供安全教育


    - 强调合法使用
    - 警示风险
    - 承担责任



开源精神:
✓ 分享知识
✓ 技术交流
✓ 推动进步
✗ 但不鼓励违法
```

22.4.4 建设性使用

正确姿势:

```
推荐做法:

  • 学习提升


    - 访问Google Scholar


    - 学习GitHub项目
    - 观看技术讲座
    - 参与开源社区


  • 工作效率


    - 使用Gmail/Drive


     - GitHub代码管理
     - Stack Overflow求助
    - 国际业务沟通
```

```
  • 视野拓展

    - 了解多元观点

    - 学习外国文化
    - 观看优质内容
    - 提升外语能力

  • 合理娱乐

     - YouTube优质频道

     - Netflix正版影视
     - Spotify听音乐
    - 正常社交互动


价值观:
✓ 兼听则明
✓ 独立思考
✓ 批判吸收
✓ 理性爱国
✓ 遵纪守法
```

本章小结

本章强调了合规使用的重要性:

核心要点:

- **法律法规**:

- 了解相关法律

- 明确执法实践 - 知晓法律风险 - 寻求合法途径

- **使用边界**:

- 区分合理用途

- 避免违法行为 - 谨慎灰色地带 - 遵守底线

- **风险防范**：

- 保护个人隐私

- 提高安全意识 - 选择可信服务 - 准备应急预案

- **责任义务**：

- 承担个人责任

- 履行社会责任 - 遵守技术伦理 - 建设性使用

基本原则：

```
法律层面：

✓ 遵守中国法律
✓ 不触碰红线
✓ 了解法律风险
✓ 寻求合法途径


道德层面：
✓ 不传播谣言
✓ 不侵犯他人
✓ 不网络暴力
✓ 尊重版权


安全层面：
✓ 保护隐私
✓ 防范风险
✓ 谨言慎行
✓ 理性使用


价值层面：
✓ 独立思考
✓ 兼听则明
✓ 批判吸收
✓ 建设性使用
```

使用建议：

推荐：

✓ 学习和研究
✓ 工作需要
✓ 正当娱乐
✓ 视野拓展

谨慎：
[注意] 政治话题
[注意] 敏感内容
[注意] 未证实信息
[注意] 争议讨论

禁止：
✗ 违法犯罪
✗ 传播违禁内容
✗ 侵犯他人
✗ 提供非法服务

最后提醒：

[注意] 重要声明：

- 本书内容仅供技术学习和研究使用
- 使用科学上网工具需遵守当地法律法规
- 用户对自己的行为承担全部法律责任
- 作者不承担因使用本书内容产生的任何责任
- 建议优先使用合法合规的网络访问方式

记住：

- 技术是中性的

- 使用需要负责

- 法律底线不可触碰

- 理性使用，自我保护

全书总结：

通过本书学习，你应该掌握：

技术知识：
✓ 网络基础原理
✓ 代理协议详解
✓ 客户端配置
✓ 服务器搭建
✓ 安全防护
✓ 故障排查

实践能力：
✓ 选择合适方案
✓ 配置使用工具
✓ 优化性能
✓ 解决问题
✓ 保护隐私
✓ 成本控制

正确认知：
✓ 法律意识
✓ 安全意识
✓ 风险防范
✓ 责任义务
✓ 理性使用
✓ 自我保护

愿你：

- 安全使用
- 合法合规
- 理性思考
- 不断学习
- 提升自我

全书完

感谢阅读本书！

科学上网是一门技术，更是一种能力。 希望本书能帮助你：

- 掌握必要的技术知识
- 建立正确的使用观念
- 培养理性的思维方式
- 在合法合规的前提下
- 更好地利用互联网资源
- 实现个人成长和发展

记住： **技术服务于人，人要遵守法律。 独立思考，理性判断，自我提升。**

祝你学习愉快，使用顺利！

附录A 术语表

附录A：术语表

常用技术术语中英对照

基础概念

中文	英文	缩写	说明
科学上网	-	-	绕过网络访问限制的技术统称
翻墙	-	-	科学上网的俗称
代理	Proxy	-	代理服务器，中转网络请求
虚拟私人网络	Virtual Private Network	VPN	加密的虚拟网络连接
防火墙	Firewall	-	网络安全防护系统
防火长城	Great Firewall	GFW	中国网络防火墙系统

网络协议

中文	英文	缩写	说明
超文本传输协议	Hypertext Transfer Protocol	HTTP	网页传输协议
安全超文本传输协议	HTTP Secure	HTTPS	加密的HTTP协议
传输控制协议	Transmission Control Protocol	TCP	可靠的传输协议
用户数据报协议	User Datagram Protocol	UDP	快速但不可靠的传输协议
互联网协议	Internet Protocol	IP	网络层协议
域名系统	Domain Name System	DNS	域名解析系统
传输层安全	Transport Layer Security	TLS	加密传输协议
安全套接字层	Secure Sockets Layer	SSL	TLS的前身

代理协议

中文	英文	缩写	说明
影梭	Shadowsocks	SS	轻量级代理协议
影梭R	ShadowsocksR	SSR	SS的增强版本
-	VMess	-	V2Ray的原创协议
-	VLESS	-	轻量级的VMess
-	Trojan	-	模仿HTTPS的协议
-	Reality	-	无需域名证书的TLS协议(2023+)
-	Hysteria	-	基于QUIC的高性能代理
-	Hysteria 2	-	Hysteria的第二代(2024+)
-	WireGuard	WG	现代VPN协议
快速UDP互联网连接	Quick UDP Internet Connections	QUIC	Google开发的协议
-	SOCKS5	-	通用代理协议

加密技术

中文	英文	缩写	说明
高级加密标准	Advanced Encryption Standard	AES	对称加密算法
-	ChaCha20	-	流加密算法
-	Poly1305	-	消息认证码
带关联数据的认证加密	Authenticated Encryption with Associated Data	AEAD	认证加密模式
伽罗瓦计数器模式	Galois/Counter Mode	GCM	AEAD模式之一
椭圆曲线加密	Elliptic Curve Cryptography	ECC	非对称加密
-	RSA	-	非对称加密算法

客户端工具

中文	英文	缩写	说明
-	Clash	-	跨平台代理客户端
-	Clash Verge	-	Clash现代化GUI客户端
-	Clash Verge Rev	-	Clash Verge继续版本
-	V2RayN	-	Windows V2Ray客户端
-	ClashX	-	macOS Clash客户端
-	Shadowrocket	-	iOS代理客户端（小火箭）
-	Surge	-	iOS/macOS高级网络工具
-	Quantumult X	-	iOS代理客户端（圈X）
-	Qv2ray	-	跨平台V2Ray客户端
-	v2rayNG	-	Android V2Ray客户端
-	NekoBox	-	Android多协议客户端

服务器相关

中文	英文	缩写	说明
虚拟专用服务器	Virtual Private Server	VPS	虚拟服务器
基础设施即服务	Infrastructure as a Service	IaaS	云服务模式
安全外壳协议	Secure Shell	SSH	远程登录协议
中国电信CN2网络	ChinaNet Next Carrier Network	CN2	电信优质网络
CN2全球互联精品网	CN2 Global Internet Access	CN2 GIA	CN2最高级别
国际私有租用线路	International Private Leased Circuit	IPLC	国际专线
国际以太网专线	International Ethernet Private Line	IEPL	以太网专线
-	X-UI	-	多用户管理面板
-	3X-UI	-	X-UI增强版(支持Reality)
-	mack-a	-	V2Ray/Xray一键脚本
-	Xray-core	-	V2Ray分支,支持XTLS

网络优化

中文	英文	缩写	说明
瓶颈带宽和往返时间	Bottleneck Bandwidth and Round-trip time	BBR	Google的TCP拥塞控制算法
多路复用	Multiplexing	Mux	复用单个连接
内容分发网络	Content Delivery Network	CDN	内容分发网络
最大传输单元	Maximum Transmission Unit	MTU	数据包最大尺寸
往返时间	Round-Trip Time	RTT	数据往返延迟
服务质量	Quality of Service	QoS	网络服务质量

配置相关

中文	英文	缩写	说明
-	YAML	-	配置文件格式
-	JSON	-	数据交换格式
通用资源标识符	Uniform Resource Identifier	URI	资源标识符
统一资源定位符	Uniform Resource Locator	URL	网址
服务器名称指示	Server Name Indication	SNI	TLS扩展
无类别域间路由	Classless Inter-Domain Routing	CIDR	IP地址表示法

安全与隐私

中文	英文	缩写	说明
深度包检测	Deep Packet Inspection	DPI	流量深度分析
网络地址转换	Network Address Translation	NAT	地址转换技术
中间人攻击	Man-in-the-Middle	MITM	拦截通信攻击
分布式拒绝服务	Distributed Denial of Service	DDoS	分布式攻击
洋葱路由	The Onion Router	Tor	匿名网络
DNS over HTTPS	DNS over HTTPS	DoH	加密DNS查询
DNS over TLS	DNS over TLS	DoT	TLS加密DNS
DNS over QUIC	DNS over QUIC	DoQ	QUIC加密DNS(新兴)
网络实时通信	Web Real-Time Communication	WebRTC	浏览器实时通信
浏览器指纹	Browser Fingerprinting	-	通过浏览器特征追踪用户
增强追踪保护	Enhanced Tracking Protection	ETP	Firefox隐私保护功能

其他术语

中文	英文	缩写	说明
机场	Airport	-	代理服务提供商俗称
节点	Node	-	代理服务器
订阅	Subscription	-	节点订阅链接
分流	Routing/Split	-	智能路由选择
直连	Direct	-	不通过代理
全局	Global	-	所有流量走代理
规则	Rule	-	分流规则
回落	Fallback	-	伪装机制，错误请求转发
流控	Flow Control	-	VLESS流控（如xtls-rprx-vision）
延迟	Latency/Ping	-	网络延迟
带宽	Bandwidth	-	网络传输速率
流量	Traffic	-	数据传输量
丢包	Packet Loss	-	数据包丢失
跳数	Hops	-	路由跳转次数

技术缩写速查

```
常用缩写快速查询:

网络基础:
IP, TCP, UDP, HTTP, HTTPS, DNS, TLS, SSL, NAT



代理协议:
SS, SSR, VMess, VLESS, Reality, Hysteria 2, SOCKS5, WireGuard (WG)



加密算法:
AES, ChaCha20, RSA, ECC, AEAD, GCM
```

```
服务器:
VPS, SSH, CN2, IPLC, IEPL, CDN



优化技术:
BBR, Mux, MTU, RTT, QoS



安全相关:
DPI, MITM, DDoS, DoH, DoT, DoQ, WebRTC, ETP



其他:
GFW, VPN, URL, URI, SNI, CIDR
```

使用提示：

- 建议收藏本术语表，遇到不懂的术语时查阅
- 英文缩写通常在技术文档中使用
- 中文术语更常用于日常交流
- 某些术语（如"机场"）为社区俗称，非正式术语

附录B 配置模板

第10章 VPS选购指南

自建代理服务器需要租用VPS（Virtual Private Server，虚拟专用服务器）。本章将指导你如何选择合适的VPS供应商和配置。

10.1 VPS供应商对比

10.1.1 主流VPS供应商

国际大厂：

1. Vultr

```
优势：

✓ 全球18+机房
✓ 按小时计费，随时销毁
✓ 支持支付宝/微信支付
✓ 新用户送$100（有条件）
✓ 开通快速（1分钟）


配置与价格：
$6/月  - 1核 1G 内存 25G SSD 1TB流量
$12/月 - 1核 2G 内存 55G SSD 2TB流量
$24/月 - 2核 4G 内存 80G SSD 3TB流量


推荐机房：

  • 东京（延迟低，但易被墙）


  • 洛杉矶（性价比高）

  • 新加坡（稳定）
```

```
官网: https://www.vultr.com
```

2. DigitalOcean

```
优势:

✓ 老牌稳定
✓ 文档详细
✓ 社区活跃
✓ 界面友好
✗ 价格稍贵
✗ 需信用卡验证


配置与价格:
$6/月  - 1核 1G 内存 25G SSD 1TB流量
$12/月 - 1核 2G 内存 50G SSD 2TB流量
$24/月 - 2核 4G 内存 80G SSD 4TB流量


推荐机房:

  • 旧金山


  • 新加坡

  • 纽约


官网: https://www.digitalocean.com
```

3. Linode

```
优势:


✓ 性能优秀
✓ 稳定可靠
✓ 客服响应快
✓ 免费备份



配置与价格:
$5/月  - 1核 1G 内存 25G SSD 1TB流量
$10/月 - 1核 2G 内存 50G SSD 2TB流量
$20/月 - 2核 4G 内存 80G SSD 4TB流量



推荐机房:


  • 东京



  • 弗里蒙特


  • 新加坡




官网: https://www.linode.com
```

4. Bandwagon（搬瓦工）

```
特点:


✓ 针对中国优化
✓ CN2 GIA线路
✓ 支持支付宝
✗ 价格较贵
✗ 经常缺货
```

```
配置与价格:
$49.99/年 - 1核 1G 内存 20G SSD 1TB流量（CN2）
$169.99/年 - 1核 1G 内存 20G SSD 1TB流量（CN2 GIA）



推荐机房:


  • 洛杉矶 CN2 GIA




官网: https://bandwagonhost.com
```

5. Hostwinds

```
优势:


✓ 不限流量
✓ 支持支付宝
✓ 性价比高
✓ 可随时换IP（$3/次）



配置与价格:
$4.99/月 - 1核 1G 内存 30G SSD 无限流量
$8.99/月 - 1核 2G 内存 50G SSD 无限流量



推荐机房:


  • 西雅图



  • 达拉斯
```

```
官网: https://www.hostwinds.com
```

6. AWS Lightsail

```
优势:

✓ 亚马逊出品
✓ 稳定可靠
✓ 与AWS生态集成
✗ 配置复杂
✗ 价格不透明



配置与价格:
$3.5/月 - 1核 512M 内存 20G SSD 1TB流量
$5/月   - 1核 1G 内存 40G SSD 2TB流量
$10/月  - 1核 2G 内存 60G SSD 3TB流量



推荐机房:

  • 东京



  • 首尔

  • 新加坡





官网: https://aws.amazon.com/lightsail/
```

10.1.2 供应商选择建议

对比表:

供应商	价格	性能	稳定性	易用性	支付方式	推荐度
Vultr	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	支付宝/信用卡	[推荐][推荐][推荐][推荐][推荐]
DigitalOcean	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	信用卡	[推荐][推荐][推荐][推荐]
Linode	[推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	信用卡/PayPal	[推荐][推荐][推荐][推荐]
搬瓦工	[推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐]	支付宝	[推荐][推荐][推荐]
Hostwinds	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]	[推荐][推荐][推荐]	支付宝/信用卡	[推荐][推荐][推荐][推荐]
AWS	[推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐][推荐][推荐][推荐]	[推荐][推荐]	信用卡	[推荐][推荐][推荐]

场景推荐：

```
新手入门：

首选：Vultr
理由：支付方便、开通快、按小时计费、易上手



追求性能：
首选：Linode / DigitalOcean
理由：性能稳定、口碑好、文档齐全
```

```
追求性价比：
首选：Hostwinds
理由：不限流量、价格低、可换IP



中国优化：
首选：搬瓦工 CN2 GIA
理由：专线直连、速度快、延迟低
注意：价格贵、经常缺货



企业级需求：
首选：AWS Lightsail
理由：稳定可靠、可扩展性强
```

10.1.3 注册与购买流程

Vultr注册购买示例：

```
步骤1：注册账号

  • 访问 https://www.vultr.com



  • 点击 Create Account

  • 填写邮箱和密码

  • 验证邮箱





步骤2：充值


  • 登录后台
```

```
  • Billing → Make Payment

  • 选择支付方式:



    - Alipay（支付宝）

    - WeChat Pay（微信支付）
    - Credit Card（信用卡）
    - PayPal

  • 充值金额（建议$10起）




步骤3: 部署服务器

  • Products → Deploy New Server



  • 选择服务器类型: Cloud Compute

  • 选择机房位置（Server Location）



    推荐: Tokyo、Los Angeles、Singapore
  • 选择操作系统（Server Type）

    推荐: Ubuntu 20.04 LTS / Debian 11
  • 选择配置（Server Size）

    推荐: $6/月（1GB内存）
  • 禁用IPv6（可选）



  • Server Hostname & Label（自定义名称）

  • Deploy Now（立即部署）
```

```
步骤4：等待开通

  • 通常1-3分钟完成

  • 状态变为Running即可使用

  • 记录IP地址、用户名、密码
```

注意事项：

```
[注意] IP被墙问题
  • 新开VPS的IP可能已被墙

  • 开通后立即测试连通性

  • 如果被墙，立即删除重新开通

  • Vultr可随时删除服务器，按小时计费


[注意] 账号安全

  • 使用强密码

  • 启用两步验证（2FA）
```

```
  • 不要分享账号信息




[注意] 付款安全

  • 优先使用支付宝/微信


  • 信用卡注意盗刷风险

  • 定期检查账单
```

10.2 地域选择策略

10.2.1 地理位置与延迟

距离与延迟关系：

```
物理定律：

光速 ≈ 300,000 km/s
光纤中实际速度 ≈ 200,000 km/s



理论延迟计算：
北京 → 东京（约2100km）
理论延迟 = 2100 / 200,000 × 2 ≈ 21ms



实际延迟（包含路由跳转）：
北京 → 东京：30-60ms
北京 → 洛杉矶：150-200ms
北京 → 纽约：200-250ms
北京 → 欧洲：250-350ms
```

中国大陆用户延迟参考：

机房位置	延迟范围	适用场景	推荐度
香港	5-30ms	游戏、实时通信	[推荐][推荐][推荐][推荐][推荐]
日本	30-80ms	日常使用、游戏	[推荐][推荐][推荐][推荐][推荐]
韩国	30-80ms	游戏、流媒体	[推荐][推荐][推荐][推荐]
新加坡	50-100ms	平衡之选	[推荐][推荐][推荐][推荐][推荐]
美西	130-180ms	日常使用	[推荐][推荐][推荐][推荐]
美东	180-230ms	流媒体	[推荐][推荐][推荐]
欧洲	250-350ms	特定服务	[推荐][推荐]

10.2.2 线路类型

电信/联通/移动线路区别：

普通线路（163骨干网）：

特点：

- 价格便宜
- 高峰期拥堵
- 绕路严重
- 丢包率高

适合：

- 预算有限

```
  • 低峰期使用

  • 非高要求用户
```

CN2线路：

```
CN2 GT（Global Transit）：
  • 部分路由走CN2

  • 价格适中

  • 高峰期表现尚可

CN2 GIA（Global Internet Access）：

  • 全程CN2网络

  • 价格昂贵

  • 速度快、延迟低

  • 高峰期稳定

对比：
普通线路：$5-10/月
CN2 GT：   $10-20/月
CN2 GIA：  $30-100/月
```

BGP多线：

```
特点：


✓ 自动选择最优路由
✓ 电信/联通/移动都快
✓ 智能切换
✗ 价格较贵



推荐：
搬瓦工 CN2 GIA
```

10.2.3 机房推荐

按需求选择：

游戏加速：

```
首选：香港、日本、韩国


延迟要求：< 100ms
推荐机房：


  • Vultr 东京



  • Linode 东京


  • Vultr 新加坡
```

日常使用：

```
首选：日本、新加坡、美西


延迟要求：< 200ms
推荐机房：
```

```
  • Vultr 东京/洛杉矶



  • DigitalOcean 旧金山

  • Linode 弗里蒙特
```

流媒体解锁：

```
Netflix：美国原生IP

Disney+：美国、日本
HBO Max：美国



推荐机房:

  • DigitalOcean 纽约



  • Vultr 洛杉矶

  • AWS 美西
```

特定服务：

```
ChatGPT/OpenAI：
  • 支持大部分美国、欧洲、日本IP



  • 避免: 香港、印度、俄罗斯
```

```
Google：

  • 任何地区都可以

  • 推荐：新加坡、日本

GitHub：

  • 任何地区都可以

  • 推荐：美国、新加坡
```

10.2.4 IP被墙检测

测试方法：

```
# 本地ping测试

ping your-vps-ip

# 如果超时，可能被墙

# 进一步确认：
```

```
# 1. 使用在线工具

https://www.ping.pe/your-vps-ip
https://tools.ipip.net/ping.php

# 2. 使用国外服务器测试

# 如果国外能ping通，国内不通 = 被墙

# 3. TCP端口测试

telnet your-vps-ip 22

# 4. HTTP测试

curl -I http://your-vps-ip
```

IP被墙后的处理：

```
方案1：更换IP（推荐）

Vultr/DigitalOcean：

  • 删除当前服务器

  • 重新部署
```

- 免费获得新IP

Hostwinds：

- 付费更换IP（$3/次）

方案2：使用CDN中转

- 配置CloudFlare CDN

- 隐藏真实IP
- 参考第9章CDN配置

方案3：换服务商/机房

- 选择被墙概率低的机房

- 避免热门IP段

10.3 配置需求分析

10.3.1 个人使用配置

单人使用：

```
基础配置（$5-6/月）：

CPU：1核
内存：1GB
硬盘：20-25GB SSD
流量：500GB-1TB/月
带宽：1Gbps



适合：
✓ 日常浏览
✓ 偶尔看视频
✓ 轻度使用



性能：

  • 同时连接：1-2设备



  • 视频：720P流畅

  • 下载：~5-10MB/s
```

重度使用（$10-12/月）：

```
推荐配置：

CPU：1-2核
内存：2GB
硬盘：50GB SSD
流量：2TB/月
带宽：1Gbps
```

```
适合：
✓ 高频使用
✓ 看高清视频
✓ 大文件下载


性能：

  • 同时连接：3-5设备


  • 视频：1080P流畅

  • 下载：~10-20MB/s
```

10.3.2 多人共享配置

2-5人共享：

```
推荐配置（$20/月）：

CPU：2核
内存：4GB
硬盘：80GB SSD
流量：3-4TB/月
带宽：1Gbps


管理：

  • 使用X-UI面板


  • 设置流量限制

  • 分配不同端口
```

5-10人共享：

```
推荐配置（$40/月）：


CPU：4核
内存：8GB
硬盘：160GB SSD
流量：5-6TB/月
带宽：1Gbps



注意：
[注意] 合理分配流量
[注意] 设置限速策略
[注意] 监控使用情况
[注意] 注意服务商TOS
```

10.3.3 流量计算

流量消耗参考：

```
网页浏览：


普通网页：~1-3MB/页
图片多的网页：~5-10MB/页
估算：100页/天 × 5MB ≈ 500MB/天 ≈ 15GB/月



视频观看：
YouTube 720P：~1.5GB/小时
YouTube 1080P：~3GB/小时
Netflix HD：~3GB/小时
Netflix 4K：~7GB/小时
估算：2小时/天 × 3GB ≈ 6GB/天 ≈ 180GB/月



文件下载：
软件/游戏：偶尔10-50GB
估算：20GB/月
```

```
总计：
轻度：15 + 60 + 0 ≈ 75GB/月（500GB套餐足够）
中度：15 + 180 + 10 ≈ 205GB/月（500GB-1TB）
重度：30 + 360 + 50 ≈ 440GB/月（1TB+）
```

安全余量：

```
建议选择流量为实际需求的2-3倍：

实际需求：200GB/月
推荐选择：500GB-1TB套餐



原因：
✓ 避免超额费用
✓ 预留升级空间
✓ 应对突发需求
```

10.3.4 性能测试

VPS性能测试脚本：

```
# 综合性能测试（推荐）

wget -qO- bench.sh | bash

# 或


curl -Lso- bench.sh | bash

# 测试内容：


  - CPU性能
```

- 内存性能
- 硬盘I/O速度
- 网络速度（全球多节点）

Superbench脚本

```
wget -qO- git.io/superbench.sh | bash
```

或

```
curl -Lso- git.io/superbench.sh | bash
```

测试内容更详细：

- 系统信息
- CPU性能（单核/多核）
- 内存测试
- 硬盘测试（顺序/随机读写）
- 网络测试（上传/下载）

解读测试结果：

```
CPU性能:


单核跑分 > 1000: 优秀
单核跑分 500-1000: 良好
单核跑分 < 500: 较弱



硬盘I/O：
> 500MB/s：SSD，优秀
100-500MB/s：SSD，良好
< 100MB/s：HDD，不推荐



网络速度（中国测试节点）：
> 50MB/s：优秀
20-50MB/s：良好
10-20MB/s：一般
< 10MB/s：较差
```

10.4 付款方式与隐私保护

10.4.1 付款方式对比

支付宝/微信支付：

```
优势:


✓ 方便快捷
✓ 国人友好
✓ 无需信用卡
✓ 汇率透明



劣势:
✗ 实名认证
✗ 可追溯
✗ 部分商家不支持
```

```
隐私等级: [推荐][推荐]


支持商家:

  • Vultr ✓



  • Hostwinds ✓

  • 搬瓦工 ✓

  • DigitalOcean ✗

  • Linode ✗
```

信用卡:

```
优势:

✓ 广泛接受
✓ 自动续费
✓ 有消费保护



劣势:
✗ 需要验证
✗ 可能被盗刷
✗ 有额度限制



隐私等级: [推荐][推荐]
```

```
注意事项：
[注意] 使用虚拟信用卡
[注意] 设置消费提醒
[注意] 定期检查账单
```

PayPal：

```
优势：

✓ 国际通用
✓ 买家保护
✓ 相对安全


劣势：
✗ 需绑定银行卡/信用卡
✗ 可能冻结账户
✗ 有手续费


隐私等级：[推荐][推荐][推荐]


支持商家：

  • 几乎所有国际商家
```

加密货币：

```
优势：

✓ 匿名性强
✓ 不可追溯
✓ 去中心化


劣势：
✗ 操作复杂
```

```
✗ 汇率波动
✗ 支持商家少
✗ 不可退款



隐私等级：[推荐][推荐][推荐][推荐][推荐]



支持商家：


  • 部分小众VPS商家



  • 需要寻找
```

10.4.2 隐私保护建议

注册信息：

```
建议做法：


✓ 使用临时邮箱（注册后绑定常用邮箱）
✓ 不填写真实姓名（如允许）
✓ 地址使用生成器
✓ 使用VPN/代理注册



临时邮箱推荐：


  • https://temp-mail.org



  • https://10minutemail.com


  • https://guerrillamail.com
```

```
地址生成器：

  • https://www.fakeaddressgenerator.com
```

付款隐私：

```
降低追溯性：
  • 使用虚拟信用卡
    - Privacy.com（美国）
    - Revolut（欧洲）
  • 使用礼品卡
    - Amazon礼品卡
    - Google Play礼品卡（部分商家）
  • 使用加密货币
    - Bitcoin（追溯性较强）
    - Monero（隐私性强）

注意：
[注意] 不要过度追求匿名
[注意] 平衡隐私与便利性
[注意] 遵守当地法律
```

10.4.3 账号安全

密码安全：

```
强密码原则：

✓ 至少16位
✓ 包含大小写字母、数字、符号
✓ 不使用个人信息
✓ 每个服务使用不同密码



密码管理器推荐：

  • Bitwarden（开源、免费）



  • 1Password（功能强大）

  • KeePass（本地存储）





示例强密码：
Kx9#mP2$vL4@nQ8!wR5
```

两步验证（2FA）：

```
启用2FA：
  • VPS后台 → Security Settings



  • Enable Two-Factor Authentication

  • 使用认证器应用扫描二维码

  • 保存备用码
```

```
认证器应用推荐:

  • Google Authenticator

  • Microsoft Authenticator

  • Authy（支持多设备同步）

重要:
[注意] 备份恢复码
[注意] 多设备备份
[注意] 不要截图保存
```

10.5 IP质量检测

10.5.1 IP类型识别

住宅IP vs 数据中心IP：

```
住宅IP（Residential IP）：
  • 来自ISP分配给家庭用户

  • 信誉度高

  • 不易被封

  • 价格昂贵

  • VPS很少提供
```

```
数据中心IP（Datacenter IP）：

  • 来自数据中心

  • 信誉度一般

  • 容易被识别

  • 价格便宜

  • VPS常用




检测工具:
https://ipinfo.io/your-ip
查看 "org" 字段，如果是ISP名称 = 住宅IP
```

10.5.2 IP黑名单检测

检测IP是否被列入黑名单：

```
在线检测工具:
  • MXToolbox

    https://mxtoolbox.com/blacklists.aspx
  • MultiRBL

    http://multirbl.valli.org/
  • WhatIsMyIPAddress

    https://whatismyipaddress.com/blacklist-check
```

```
检测内容:

  • Spamhaus



  • Barracuda

  • SORBS

  • 等100+黑名单数据库




判断:
✓ Clean：干净，可用
[注意] Listed：被列入黑名单，可能影响邮件等服务
```

10.5.3 流媒体解锁检测

Netflix解锁检测:

```
# 使用脚本检测

bash <(curl -L -s check.unlock.media)

# 或



wget -O nf https://github.com/sjlleo/netflix-verify/releases/download/v3.1.0/nf_linux_amd64 &&

# 结果解读:
```

```
✓ 原生IP：完全解锁Netflix
[注意] 非原生IP：可能无法观看
✗ 不支持: 无法使用Netflix

# 手动测试



curl -I https://www.netflix.com

# 查看返回的地区信息
```

其他流媒体：

```
# Disney+ 检测

curl -I https://www.disneyplus.com

# YouTube Premium检测

# 登录后查看是否能使用Premium功能

# HBO Max检测



curl -I https://play.hbomax.com
```

10.5.4 端口封锁检测

常用端口检测：

```
# 在VPS上运行（需先安装nc）

apt install netcat -y

# 监听测试端口


nc -l 1234

# 在本地测试连通性


telnet your-vps-ip 1234

# 或


nc -zv your-vps-ip 1234

# 常见被封端口：


25    - SMTP（几乎都被封）
465   - SMTPS
587   - 邮件提交
1194  - OpenVPN
1723  - PPTP VPN
```

在线端口检测：

```
工具：
```

```
https://www.yougetsignal.com/tools/open-ports/
https://canyouseeme.org/



测试常用代理端口:
443  - HTTPS（推荐）
80   - HTTP
8080 - 备用HTTP
1080 - SOCKS
```

10.5.5 IP质量综合评分

IP质量评估清单：

```
✓ Ping延迟


  < 100ms：优秀
  100-200ms：良好
  > 200ms：一般



✓ 丢包率
  0%: 完美
  < 1%: 优秀
  1-5%: 可接受
  > 5%: 较差



✓ 路由跳数
  < 15跳: 优秀
  15-25跳: 一般
  > 25跳: 绕路严重



✓ IP类型
  住宅IP：[推荐][推荐][推荐][推荐][推荐]
  数据中心IP：[推荐][推荐][推荐]
```

```
✓ 黑名单状态
  Clean：[推荐][推荐][推荐][推荐][推荐]
  Listed：[推荐][推荐]



✓ 流媒体解锁
  Netflix原生: [推荐][推荐][推荐][推荐][推荐]
  部分解锁: [推荐][推荐][推荐]
  不解锁: [推荐]



✓ 端口开放
  全部开放: [推荐][推荐][推荐][推荐][推荐]
  部分限制: [推荐][推荐][推荐]
```

综合测试脚本：

```
# 融合怪测试（推荐）


bash <(curl -L -s https://raw.githubusercontent.com/spiritLHLS/ecs/main/ecs.sh)

# 测试项目：


  - 系统信息



  - CPU性能

  - 内存测试

  - 硬盘速度

  - 网络测速

  - 流媒体解锁

  - IP质量检测
```

- 路由跟踪

等待测试完成（约5-10分钟）

获得详细的VPS性能报告

本章小结

本章详细介绍了VPS选购的各个方面：

核心要点：

- **供应商选择**：

- Vultr：新手友好、支付方便

- DigitalOcean/Linode：性能稳定 - 搬瓦工：中国优化、CN2线路 - Hostwinds：性价比高、不限流量

- **地域选择**：

- 游戏：香港、日本（低延迟）

- 日常：日本、新加坡、美西 - 流媒体：美国原生IP - 特定服务：按需选择

- **配置需求**：

- 个人使用：1GB内存，500GB-1TB流量

- 重度使用：2GB内存，2TB流量 - 多人共享：按人数倍增

- **付款隐私**：

- 支付宝/微信：方便但可追溯

- 信用卡：广泛接受 - PayPal：国际通用 - 加密货币：匿名性强

- **IP质量**：

- 延迟测试

- 黑名单检测 - 流媒体解锁 - 端口开放情况

选购建议：

```
新手推荐方案：


供应商：Vultr
机房：东京/洛杉矶/新加坡
配置：$6/月（1GB内存、1TB流量）
付款：支付宝
理由：简单、便宜、够用



进阶方案：
供应商：搬瓦工
机房：洛杉矶 CN2 GIA
配置：$50/年起
付款：支付宝
理由：速度快、稳定、针对中国优化



预算充足：
供应商：Linode / DigitalOcean
机房：按需选择
配置：$10-20/月
付款：信用卡/PayPal
理由：性能好、稳定性高、服务优质
```

下一章我们将学习服务器环境配置。

实践任务：

- 注册VPS供应商账号
- 测试不同机房延迟
- 购买并开通VPS
- 进行IP质量检测
- 记录服务器信息

附录C 资源推荐

附录C：资源推荐

本附录整理了科学上网相关的官方文档、工具下载、社区资源等信息。

一、官方文档链接

代理协议文档

V2Ray / Xray

- V2Ray 官方文档：https://www.v2ray.com/
- V2Ray 白话文教程：https://guide.v2fly.org/
- Xray 官方文档：https://xtls.github.io/
- Project X 社区：https://github.com/XTLS
- Reality 协议文档：https://github.com/XTLS/REALITY

Shadowsocks

- Shadowsocks 官网：https://shadowsocks.org/
- Shadowsocks-Rust：https://github.com/shadowsocks/shadowsocks-rust
- Shadowsocks-libev：https://github.com/shadowsocks/shadowsocks-libev

Trojan

- Trojan 官方：https://trojan-gfw.github.io/trojan/
- Trojan-Go：https://github.com/p4gefau1t/trojan-go

Hysteria

- Hysteria 2 官方：https://v2.hysteria.network/
- GitHub 仓库：https://github.com/apernet/hysteria
- 中文文档：https://v2.hysteria.network/zh/

Clash

- Clash 官方：https://github.com/Dreamacro/clash (已归档)
- Clash Meta：https://github.com/MetaCubeX/mihomo

- Clash Premium：https://github.com/Dreamacro/clash/releases/tag/premium (已停更)

WireGuard

- WireGuard 官网：https://www.wireguard.com/
- WireGuard 文档：https://www.wireguard.com/quickstart/

技术标准文档

RFC 文档

- RFC 1928 (SOCKS5)：https://www.rfc-editor.org/rfc/rfc1928
- RFC 2818 (HTTPS)：https://www.rfc-editor.org/rfc/rfc2818
- RFC 8446 (TLS 1.3)：https://www.rfc-editor.org/rfc/rfc8446
- RFC 9000 (QUIC)：https://www.rfc-editor.org/rfc/rfc9000

加密标准

- NIST 加密标准：https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines
- TLS 最佳实践：https://wiki.mozilla.org/Security/Server_Side_TLS

二、客户端工具下载

Windows 平台

Clash Verge Rev（2025推荐）

- 官方仓库：https://github.com/clash-verge-rev/clash-verge-rev
- 下载地址：https://github.com/clash-verge-rev/clash-verge-rev/releases
- 特点：现代化界面，持续维护，支持Clash Meta内核

Clash for Windows

- 官方仓库：https://github.com/Fndroid/clash_for_windows_pkg
- 状态：已停止维护(2023年11月)
- 备注：建议迁移到Clash Verge Rev

V2RayN

- 官方仓库：https://github.com/2dust/v2rayN
- 下载地址：https://github.com/2dust/v2rayN/releases
- 支持：V2Ray、Xray、Shadowsocks、Hysteria 2

Qv2ray

- 官方仓库：https://github.com/Qv2ray/Qv2ray
- 状态：项目已停止维护
- 备注：不推荐新用户使用

macOS 平台

Clash Verge Rev（2025推荐）

- 官方仓库：https://github.com/clash-verge-rev/clash-verge-rev
- 下载地址：https://github.com/clash-verge-rev/clash-verge-rev/releases
- 特点：现代化界面，跨平台一致体验

ClashX

- 官方仓库：https://github.com/yichengchen/clashX
- 下载地址：https://github.com/yichengchen/clashX/releases
- 状态：维护缓慢

ClashX Pro

- 官方仓库：https://install.appcenter.ms/users/clashx/apps/clashx-pro/distribution_groups/public
- 功能：增强版本，支持TUN模式

V2RayX

- 官方仓库：https://github.com/Cenmrev/V2RayX
- 状态：项目已停止维护

Surge

- 官网：https://nssurge.com/
- App Store：搜索 "Surge"
- 注意：付费软件

Linux 平台

Clash

- Clash for Linux：https://github.com/Dreamacro/clash/releases
- 配置教程：https://github.com/Dreamacro/clash/wiki

V2Ray / Xray

- V2Ray 安装脚本：https://github.com/v2fly/fhs-install-v2ray
- Xray 安装脚本：https://github.com/XTLS/Xray-install

Qv2ray

- 同 Windows 版本

Android 平台

Clash Meta for Android（2025推荐）

- 官方仓库：https://github.com/MetaCubeX/ClashMetaForAndroid
- 下载地址：https://github.com/MetaCubeX/ClashMetaForAndroid/releases
- 特点：支持Meta内核，功能强大

Clash for Android

- 官方仓库：https://github.com/Kr328/ClashForAndroid
- Google Play：搜索 "Clash for Android"
- 状态：项目已归档(2023)

V2RayNG

- 官方仓库：https://github.com/2dust/v2rayNG
- Google Play：搜索 "V2RayNG"
- 支持：Xray、Reality、Hysteria 2

NekoBox

- 官方仓库：https://github.com/MatsuriDayo/NekoBoxForAndroid
- 下载地址：https://github.com/MatsuriDayo/NekoBoxForAndroid/releases
- 特点：支持多种协议，现代化界面

SagerNet

- 官方仓库：https://github.com/SagerNet/SagerNet
- 状态：项目已停止维护

Surfboard

- Google Play：搜索 "Surfboard"
- 特点：支持 Surge 配置

iOS 平台

Shadowrocket（小火箭）

- App Store：搜索 "Shadowrocket"
- 价格：$2.99（需美区账号）
- 推荐：功能强大，易用

Surge

- App Store：搜索 "Surge"
- 价格：$49.99（专业版）
- 特点：最强大的网络工具

Quantumult X（圈X）

- App Store：搜索 "Quantumult X"
- 价格：$7.99
- 特点：高级规则支持

Loon

- App Store：搜索 "Loon"
- 价格：$5.99
- 特点：类似 Surge

路由器固件

OpenWrt

- 官网：https://openwrt.org/
- 下载：https://downloads.openwrt.org/
- PassWall：https://github.com/xiaorouji/openwrt-passwall
- SSR Plus：https://github.com/fw876/helloworld

梅林固件

- 官网：https://www.asuswrt-merlin.net/
- Koolshare：https://koolshare.cn/

三、服务器工具

一键脚本

V2Ray / Xray

- mack-a 脚本：https://github.com/mack-a/v2ray-agent
- 233boy 脚本：https://github.com/233boy/v2ray
- Xray 官方脚本：https://github.com/XTLS/Xray-install
- Reality 一键脚本：https://github.com/zxcvos/Xray-script

Hysteria 2

- 官方安装文档：https://v2.hysteria.network/docs/getting-started/Installation/
- 一键脚本：https://github.com/apernet/hysteria/blob/master/install_server.sh

Trojan

- Trojan-Go 脚本：https://github.com/p4gefau1t/trojan-go
- Jrohy 脚本：https://github.com/Jrohy/trojan

WireGuard

- angristan 脚本：https://github.com/angristan/wireguard-install
- Nyr 脚本：https://github.com/Nyr/wireguard-install

面板工具

X-UI

- 官方仓库：https://github.com/vaxilu/x-ui
- 状态：项目已停止维护
- 建议：使用3X-UI替代

3X-UI

- 官方仓库：https://github.com/MHSanaei/3x-ui
- 下载地址：https://github.com/MHSanaei/3x-ui/releases
- 功能：Web 管理界面，多用户管理，支持Reality

V2-UI

- 官方仓库：https://github.com/sprov065/v2-ui
- 注意：项目已停止维护，建议使用 3X-UI

V2board

- 官方仓库：https://github.com/v2board/v2board
- 功能：机场管理面板

证书工具

acme.sh

- 官方仓库：https://github.com/acmesh-official/acme.sh
- 文档：https://github.com/acmesh-official/acme.sh/wiki

Certbot

- 官网：https://certbot.eff.org/
- 文档：https://eff-certbot.readthedocs.io/

四、辅助工具

订阅转换

在线转换服务

- Subscription Converter：https://sub-web.netlify.app/
- ACL4SSR 在线转换：https://acl4ssr-sub.github.io/

自建转换服务

- subconverter：https://github.com/tindy2013/subconverter
- sub-web：https://github.com/CareyWang/sub-web

规则集资源

GeoIP / GeoSite

- V2Ray 规则：https://github.com/v2fly/domain-list-community
- Loyalsoldier：https://github.com/Loyalsoldier/v2ray-rules-dat
- Chocolate4u：https://github.com/Chocolate4U/Iran-v2ray-rules

Clash 规则

- ACL4SSR：https://github.com/ACL4SSR/ACL4SSR
- ConnersHua：https://github.com/DivineEngine/Profiles

- Clash Meta 规则：https://github.com/MetaCubeX/meta-rules-dat

隐私保护规则

- AdGuard 规则：https://github.com/AdguardTeam/AdguardFilters
- EasyList：https://easylist.to/
- Anti-AD：https://anti-ad.net/

测速工具

在线测速

- Speedtest：https://www.speedtest.net/
- Fast.com：https://fast.com/
- LibreSpeed：https://librespeed.org/

节点测试

- CloudFlare Speed Test：https://speed.cloudflare.com/
- Netflix 检测：https://www.netflix.com/title/81280792

IP 检测

- IPInfo：https://ipinfo.io/
- IP-API：https://ip-api.com/
- IPLeak：https://ipleak.net/
- BrowserLeaks：https://browserleaks.com/

DNS 泄漏检测

- DNS Leak Test：https://www.dnsleaktest.com/
- DNS Check：https://dnscheck.tools/

WebRTC 泄漏检测

- BrowserLeaks WebRTC：https://browserleaks.com/webrtc
- Perfect Privacy：https://www.perfect-privacy.com/webrtc-leaktest/

浏览器指纹检测

- AmIUnique：https://www.amiunique.org/
- Cover Your Tracks (EFF)：https://coveryourtracks.eff.org/
- Fingerprint.com：https://fingerprint.com/demo/

五、社区论坛资源

中文社区

综合论坛

- V2EX：https://www.v2ex.com/
- 恩山论坛：https://www.right.com.cn/forum/
- 全球主机交流论坛：https://hostloc.com/

专业社区

- NodeSeek：https://www.nodeseek.com/
- HostLoc：https://hostloc.com/
- LowendTalk：https://lowendtalk.com/

Telegram 群组

- V2Ray 官方群：@projectv2ray
- Clash 中文群：@ClashCN
- 机场观察：@jichangbaoliao

英文社区

Reddit

- r/VPN：https://www.reddit.com/r/VPN/
- r/privacy：https://www.reddit.com/r/privacy/
- r/Shadowsocks：https://www.reddit.com/r/Shadowsocks/

GitHub

- V2Ray：https://github.com/v2ray
- Clash：https://github.com/Dreamacro/clash
- Shadowsocks：https://github.com/shadowsocks

六、VPS 服务商

主流服务商

国际大厂

- Vultr：https://www.vultr.com/
- DigitalOcean：https://www.digitalocean.com/
- Linode：https://www.linode.com/
- AWS Lightsail：https://aws.amazon.com/lightsail/
- Google Cloud：https://cloud.google.com/

性价比选择

- RackNerd：https://www.racknerd.com/
- BandwagonHost（搬瓦工）：https://bandwagonhost.com/
- HostWinds：https://www.hostwinds.com/

CN2 线路

- GigsGigsCloud：https://www.gigsgigscloud.com/
- DMIT：https://www.dmit.io/
- Akkocloud：https://akkocloud.com/

域名注册

推荐服务商

- Cloudflare：https://www.cloudflare.com/
- NameSilo：https://www.namesilo.com/
- Namecheap：https://www.namecheap.com/
- GoDaddy：https://www.godaddy.com/

七、学习资源

视频教程

YouTube 频道

- 不良林：https://www.youtube.com/@bulianglin
- 老李秃噜：https://www.youtube.com/@laolitulu

- TechHolic：https://www.youtube.com/@techholic

B站 UP主

- 搜索关键词："V2Ray教程"、"Clash教程"、"科学上网"

博客文章

技术博客

- Project V 官方博客：https://www.v2ray.com/
- ToutyRater：https://toutyrater.github.io/
- V2Fly 用户指南：https://guide.v2fly.org/

电子书

推荐书籍

- 《V2Ray 完全使用手册》
- 《网络协议详解》
- 《加密技术基础》

八、工具对比表

客户端推荐对比

平台	推荐工具	优点	缺点	价格
Windows	Clash Verge Rev	现代化、持续维护	-	免费
macOS	Clash Verge Rev	跨平台一致	-	免费
Linux	Clash Meta	轻量、功能强	配置复杂	免费
Android	NekoBox	多协议支持	-	免费
iOS	Shadowrocket	性价比高	需美区账号	$2.99

VPS 推荐对比

服务商	地区	价格/月	带宽	推荐度
Vultr	全球	$5起	1TB	[推荐][推荐][推荐][推荐]
DigitalOcean	全球	$5起	1TB	[推荐][推荐][推荐][推荐]
搬瓦工	美国	$50起	1TB	[推荐][推荐][推荐][推荐][推荐]
DMIT	美国	$15起	500GB	[推荐][推荐][推荐][推荐][推荐]

[注意] 使用提示

- **安全性**：优先从官方渠道下载工具
- **更新**：定期检查工具更新
- **备份**：重要配置请及时备份
- **合规**：遵守当地法律法规
- **隐私**：注意保护个人信息

免责声明：

- 本附录仅提供技术资源链接
- 请用户自行判断资源的安全性
- 使用任何工具和服务的风险由用户自负
- 建议优先选择知名度高、口碑好的资源

最后更新：2025年1月 注：链接可能随时变化，请以实际访问为准

重要提示：

- [注意] Clash for Windows 已于2023年11月停止维护，建议迁移到 Clash Verge Rev
- [注意] Clash for Android 已于2023年归档，建议使用 Clash Meta for Android
- [完成] 推荐协议：VLESS + Reality（无需域名）、Hysteria 2（高性能）
- [完成] 推荐使用3X-UI面板管理Reality节点

附录D 参考文献

附录D：参考文献

本附录列出了编写本书时参考的技术文档、RFC标准、开源项目等资料。

一、RFC 标准文档

核心网络协议

- **RFC 791** - Internet Protocol (IP)

https://www.rfc-editor.org/rfc/rfc791

- **RFC 793** - Transmission Control Protocol (TCP)

https://www.rfc-editor.org/rfc/rfc793

- **RFC 768** - User Datagram Protocol (UDP)

https://www.rfc-editor.org/rfc/rfc768

- **RFC 1035** - Domain Names (DNS)

https://www.rfc-editor.org/rfc/rfc1035

HTTP 相关

- **RFC 2616** - HTTP/1.1

https://www.rfc-editor.org/rfc/rfc2616

- **RFC 7540** - HTTP/2

https://www.rfc-editor.org/rfc/rfc7540

- **RFC 6455** - WebSocket Protocol

https://www.rfc-editor.org/rfc/rfc6455

代理与安全

- **RFC 1928** - SOCKS Protocol Version 5

https://www.rfc-editor.org/rfc/rfc1928

- **RFC 2818** - HTTP Over TLS (HTTPS)

https://www.rfc-editor.org/rfc/rfc2818

- **RFC 5246** - TLS Protocol Version 1.2

https://www.rfc-editor.org/rfc/rfc5246

- **RFC 8446** - TLS Protocol Version 1.3

https://www.rfc-editor.org/rfc/rfc8446

- **RFC 7539** - ChaCha20 and Poly1305

https://www.rfc-editor.org/rfc/rfc7539

现代协议

- **RFC 9000** - QUIC Protocol

https://www.rfc-editor.org/rfc/rfc9000

- **RFC 9114** - HTTP/3

https://www.rfc-editor.org/rfc/rfc9114

- **RFC 8484** - DNS over HTTPS (DoH)

https://www.rfc-editor.org/rfc/rfc8484

- **RFC 7858** - DNS over TLS (DoT)

https://www.rfc-editor.org/rfc/rfc7858

- **RFC 9250** - DNS over Dedicated QUIC (DoQ)

https://www.rfc-editor.org/rfc/rfc9250

二、官方技术文档

代理协议

- **V2Ray 官方文档**

https://www.v2ray.com/

https://guide.v2fly.org/

- **Xray 文档**

https://xtls.github.io/

- **Reality 协议文档** (2023+)

https://github.com/XTLS/REALITY

作者：RPRX

- **Hysteria 2 文档** (2024+)

https://v2.hysteria.network/

https://github.com/apernet/hysteria

- **Shadowsocks 文档**

https://shadowsocks.org/

- **Trojan 协议**

https://trojan-gfw.github.io/trojan/

- **Clash 文档**

https://github.com/Dreamacro/clash/wiki

注：原项目已归档

- **Clash Meta (mihomo) 文档**

https://wiki.metacubex.one/

- **WireGuard 文档**

https://www.wireguard.com/

三、开源项目

核心项目

- **V2Ray-Core**

https://github.com/v2fly/v2ray-core

许可：MIT License

- **Xray-Core**

https://github.com/XTLS/Xray-core

许可：MPL-2.0 License

- **Reality** (2023+)

https://github.com/XTLS/REALITY

作者：RPRX 许可：MPL-2.0 License

- **Hysteria 2** (2024+)

https://github.com/apernet/hysteria

许可：MIT License

- **Clash**

https://github.com/Dreamacro/clash

许可：GPL-3.0 License 状态：已归档(2023)

- **Clash Meta (mihomo)**

https://github.com/MetaCubeX/mihomo

许可：GPL-3.0 License

- **Shadowsocks-Rust**

https://github.com/shadowsocks/shadowsocks-rust

许可：MIT License

- **Trojan-Go**

https://github.com/p4gefau1t/trojan-go

许可：GPL-3.0 License

- **WireGuard**

https://git.zx2c4.com/wireguard-linux

许可：GPL-2.0 License

客户端项目

- **Clash for Windows**

https://github.com/Fndroid/clash_for_windows_pkg

状态：已停止维护(2023.11)

- **Clash Verge Rev** (2024+)

https://github.com/clash-verge-rev/clash-verge-rev

许可：GPL-3.0 License

- **V2RayN**

https://github.com/2dust/v2rayN

持续维护中

- **ClashX**

https://github.com/yichengchen/clashX

- **Clash Meta for Android**

https://github.com/MetaCubeX/ClashMetaForAndroid

- **NekoBox for Android** (2024+)

https://github.com/MatsuriDayo/NekoBoxForAndroid

- **Qv2ray**

https://github.com/Qv2ray/Qv2ray

状态：已停止维护

工具项目

- **acme.sh** (证书工具)

https://github.com/acmesh-official/acme.sh

- **subconverter** (订阅转换)

https://github.com/tindy2013/subconverter

- **X-UI** (面板)

https://github.com/vaxilu/x-ui

状态：已停止维护

- **3X-UI** (面板，2024+)

https://github.com/MHSanaei/3x-ui

支持Reality协议

四、技术白皮书与设计文档

- **Shadowsocks 设计文档**

https://shadowsocks.org/en/spec/Protocol.html

- **V2Ray 开发者文档**

https://www.v2ray.com/developer/

- **Trojan 协议设计**

https://trojan-gfw.github.io/trojan/protocol

- **Reality 协议技术原理** (2023)

作者：RPRX

https://github.com/XTLS/REALITY#readme

- **Hysteria 2 协议设计** (2024)

https://v2.hysteria.network/docs/advanced/Full-Server-Config/

- **WireGuard 白皮书**

作者：Jason A. Donenfeld

https://www.wireguard.com/papers/wireguard.pdf

- **BBR 拥塞控制算法**

作者：Neal Cardwell, Google Team

https://queue.acm.org/detail.cfm?id=3022184

- **XTLS Vision 技术文档** (2022)

https://github.com/XTLS/Xray-core/discussions/1295

五、学术研究

网络审查研究

- **"How the Great Firewall of China is Blocking Tor"**

USENIX FOCI 2012

- **"Empirical Analysis of Internet Filtering in China"**

University of Cambridge

加密与安全

- **NIST SP 800-38D** - AES-GCM 加密模式

https://csrc.nist.gov/publications/detail/sp/800-38d/final

- **FIPS 197** - AES 标准

https://csrc.nist.gov/publications/detail/fips/197/final

- **NIST SP 800-81 Revision 3** - DNS Security

https://csrc.nist.gov/publications/detail/sp/800-81/3/final

六、技术博客与教程

- **V2Ray 白话文教程**

https://toutyrater.github.io/

作者：ToutyRater

- **233boy 博客**

https://233boy.com/

- **秋水逸冰**

https://teddysun.com/

- **荒岛博客**

https://lala.im/

七、安全与隐私最佳实践

- **Mozilla SSL Configuration**

https://ssl-config.mozilla.org/

- **OWASP TLS Cheat Sheet**

https://cheatsheetseries.owasp.org/

- **NIST Cryptographic Standards**

https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines

- **EFF - Cover Your Tracks** (浏览器指纹防护)

https://coveryourtracks.eff.org/

- **Mozilla Firefox Enhanced Tracking Protection**

https://support.mozilla.org/kb/enhanced-tracking-protection-firefox-desktop

- **US Executive Order on DNS Encryption**

"Strengthening and Promoting Innovation in the Nation's Cybersecurity"

八、相关书籍

网络协议

- **《TCP/IP详解 卷1：协议》**

作者：W. Richard Stevens

ISBN：978-7-111-07566-0

- **《HTTP权威指南》**

作者：David Gourley, Brian Totty

ISBN：978-7-115-28188-0

网络安全

- **《密码学原理与实践》**

作者：Douglas R. Stinson

ISBN：978-7-121-13386-0

- **《应用密码学》**

作者：Bruce Schneier

ISBN：978-7-111-07393-2

Linux 系统

- **《鸟哥的Linux私房菜》**

作者：鸟哥

ISBN：978-7-115-22626-3

九、法律法规

中国相关法律

- **《中华人民共和国网络安全法》**

实施时间：2017年6月1日

- **《计算机信息网络国际联网管理暂行规定》**

发布机构：国务院，1996年

- **《中华人民共和国刑法》相关条款**

第285、286、287条

国际法规

- **GDPR** - 欧盟数据保护法规

实施时间：2018年5月25日

- **CCPA** - 加州消费者隐私法

实施时间：2020年1月1日

[重点] 引用说明

本书在编写过程中参考了以上资料，在此表示感谢。

引用格式：

- RFC文档：`[RFC编号] 标题，URL`
- 开源项目：`项目名，GitHub URL，许可证`
- 学术论文：`作者．"标题"．会议/期刊，年份`
- 书籍：`作者．书名．出版社，ISBN`

致歉说明：

由于本书涉及内容广泛，编写过程中可能参考了诸多网络文章、博客、教程等资料。尽管我们已尽力列出所有引用来源，但仍可能存在疏漏或遗漏标注的情况。如果您发现本书引用了您的作品但未在此处列出，我们深表歉意，请通过反馈渠道联系我们，我们会及时补充标注。

[注意] 版权说明

- 所有参考文献版权归原作者所有
- RFC文档为公共领域标准
- 开源项目遵循各自许可证
- 引用内容仅供学习研究使用
- 请遵守相关版权和许可协议

💬 补充与完善

如果您发现本书引用了某些资料但未在此处列出，或者您认为有重要的参考资料应当添加到本附录中，欢迎通过以下方式反馈：

- 📧 发送邮件说明文献信息
- 💬 在社区讨论中提出建议
- 🐛 提交问题报告并注明文献来源

我们会认真审核并将有价值的参考文献添加到本附录中，以便为更多读者提供完整的学习资源。

提交格式建议：

- 文献类型（RFC/开源项目/博客/书籍等）
- 标题或项目名称
- 作者或维护者
- 链接地址（如有）
- 简要说明其价值

最后更新：2025年1月 注：链接可能随时变化，建议访问官方渠道获取最新信息